

C# — leerboek

Programming Advanced · van tekst tot database · 34 hoofdstukken · met
voorspelvragen om jezelf te testen

Inhoudsopgave

C# — leerboek	Error! Bookmark not defined.
Hoe je dit gebruikt	4
Inhoud	6
Leerwijzer per les	8
Keuzegids	9
Deel 1 — De basis	11
1. Tekst	12
2. Getallen en omzetten	27
3. Datum en tijd	36
4. Beslissen en herhalen	45
Deel 2 — Verzamelingen	52
De verzamelingen naast elkaar	52
5. Array	53
6. List	69
7. Dictionary	85
8. HashSet	100
9. Queue	112
10. Stack	121
Deel 3 — Zoeken en rekenen	129
11. LINQ	130
12. IEnumerable	142
Deel 4 — Eigen types	149
13. class	150
14. record	157
15. struct	162
16. Referentie tegenover waarde	166
17. Omgaan met niets	169
18. enum	175
19. interface	180
20. Overerving en abstracte klassen	184
21. Generics	193
22. Types herkennen	199
23. Uitzonderingen	202
24. static, const en readonly	211
Deel 5 — In je API	218
25. Attributen en routes	219
26. De helpers van een controller	224

27. Program.cs en dependency injection	228
28. Logging	233
29. Asynchroon werken	237
30. JSON en bestanden	242
Deel 6 — Data	246
31. EF Core: model en database	247
32. EF Core: opvragen en bewaren.....	255
33. DTO's en Mapster	266
34. Dapper.....	272
Veelgemaakte verwarringen.....	280
Index	286

Hoe je dit gebruikt

Dit leerboek brengt twee dingen samen die je nodig hebt om vlot in C# te programmeren: de **bouwstenen** — wat een `List`, een `Dictionary` of een `record` is — en de **methodes** — wat je ermee kan dóen. Ze staan in een volgorde die van eenvoudig naar ingewikkeld gaat.

Elk hoofdstuk is ook volledig op zich. Bij een verzameling of een type staat niet enkel wat het zelf kan, maar ook wat het kan **via iets anders**: via LINQ, via een `foreach`, via een omzetting of via `object`. Zo mis je niets als je één hoofdstuk apart opzoekt. Wordt iets pas in een later hoofdstuk volledig uitgelegd, dan staat er een verwijzing naar dat hoofdstuk.

Er zijn twee soorten hoofdstukken.

- Een **bouwsteen** volgt altijd dezelfde stappen: wat het is, hoe het eruitziet, wat je ermee kan, wat het ook kan via iets anders, waar je op let, wanneer wel en wanneer niet, en de veelgemaakte fouten.
- Een **methodehoofdstuk** is geordend op wat je wil doen: *ik wil een stuk uit een tekst halen, ik wil één item uit een lijst*. Per vraag staan de methodes die ervoor bestaan, en welke je in welk geval neemt.

Elk hoofdstuk eindigt met **Voorspel de uitkomst**. Lees de code, beslis wat er op het scherm komt, en kijk pas dan naar het antwoord, op de laatste pagina van het hoofdstuk. Uitleg lezen voelt als begrijpen, maar pas als je een uitkomst juist voorspelt, weet je of het zit. Heb je er één fout, lees dan de uitleg bij het antwoord en probeer de code zelf uit in een lege consoletoepassing.

Kom je later een naam tegen en wil je weten wat hij doet, gebruik dan de index achteraan.

Volg je de lessen van Programming Advanced, dan zegt de [leerwijzer](#) welke hoofdstukken bij elke les horen.

Kleuren en labels

Label of kleur	Betekenis
METHODES	een hoofdstuk geordend op wat je wil doen
VERZAMELING	een bouwsteen die meerdere dingen bewaart
EIGEN TYPE	een bouwsteen die je zelf maakt
CONCEPT	een idee dat door alle hoofdstukken heen loopt
IN JE API	wat je nodig hebt zodra je een API bouwt
DATA	werken met een database
methode	de naam van een methode of eigenschap
resultaat	wat eruit komt
Exception	het programma crasht, of compileert niet

Label of kleur	Betekenis
Wel / Niet	wanneer je een bouwsteen kiest, en wanneer niet

OPMERKING

De voorbeelden zijn losse regels over fruit, steden, leerlingen, dieren, recepten en verkeerslichten, en hier en daar een onderdeel of een ventilator. Het zijn nooit stukken uit Bouwcontrole of uit je schooloefeningen. Welke bouwsteen je in je eigen projecten gebruikt, beslis je zelf — dit leerboek geeft je de kennis om die keuze te kunnen maken.

OPMERKING

De hoofdletters doen ertoe. In C# is het `Find`, niet `find`, en `typeof`, niet `TypeOf`. Methodes beginnen met een hoofdletter, sleutelwoorden van de taal met een kleine letter. Kleine letters vooraan zijn meestal JavaScript.

Inhoud

Deel	Hoofdstuk	Wat je leert
Vooraan	Leerwijzer per les	welke hoofdstukken bij welke les horen
	Keuzegids	welke bouwsteen je neemt, en waar je iets vindt
1. De basis	1. Tekst	lengte, hoofdletters, knippen, zoeken, vergelijken
	2. Getallen en omzetten	tekst naar getal, afronden, delen met rest
	3. Datum en tijd	datums maken, rekenen, tonen
	4. Beslissen en herhalen	if, switch, for, while
2. Verzamelingen	5. Array	een rij met vaste lengte
	6. List	een lijst die groeit en krimpt
	7. Dictionary	opzoeken op een sleutel
	8. HashSet	een verzameling zonder dubbels
	9. Queue	wie eerst komt, eerst geholpen
	10. Stack	het laatste eerst terug
3. Zoeken en rekenen	11. LINQ	zoeken, filteren, tellen en sorteren in elke verzameling
	12. IEnumerable	wat LINQ eigenlijk teruggeeft
4. Eigen types	13. class	een bouwplan voor je eigen objecten
	14. record	een pakketje gegevens dat draait om de inhoud
	15. struct	een klein pakketje dat echt gekopieerd wordt
	16. Referentie tegenover waarde	het belangrijkste idee van dit leerboek
	17. Omgaan met niets	null, ??, ?. en getallen die mogen ontbreken
	18. enum	een vaste lijst benoemde keuzes
	19. interface	een contract: wat iets moet kunnen
	20. Overerving en abstracte klassen	verder bouwen op een andere klasse, abstract, override
	21. Generics	een type met een open plaats, zoals List<T>
	22. Types herkennen	is, as, typeof, nameof
	23. Uitzonderingen	try, catch, throw: fouten opvangen en melden
	24. static, const en readonly	wat bij de klasse hoort, en wat niet meer verandert
5. In je API	25. Attributen en routes	welke URL bij welke methode uitkomt
	26. De helpers van een controller	Ok, NotFound, BadRequest en de rest
	27. Program.cs en dependency injection	opstarten, diensten registreren en opvragen
	28. Logging	meldingen in het terminalvenster
	29. Asynchroon werken	async, await, Task
	30. JSON en bestanden	lezen, schrijven, omzetten

Deel	Hoofdstuk	Wat je leert
6. Data	31. EF Core: model en database	tabellen, relaties en migraties
	32. EF Core: opvragen en bewaren	ophalen, toevoegen, wijzigen, Include
	33. DTO's en Mapster	enkel doorgeven wat de client nodig heeft
	34. Dapper	zelf SQL schrijven, met een micro-ORM
Achteraan	Veelgemaakte verwarringen	tweeënveertig paren die op elkaar lijken
	Index	alles alfabetisch, met elk hoofdstuk waar het voorkomt

Leerwijzer per les

Welke hoofdstukken horen bij welke les van Programming Advanced? Lees ze vóór of naast de les. Deel 1 heb je in elke les nodig; de tabel noemt de hoofdstukken die in die les nieuw zijn of het meest terugkomen.

Les	Onderwerp	Hoofdstukken
01	Introductie	13, 20, 25, 26, 27
02	HTTP Verbs	6, 11, 17, 24, 25, 26
03	Repositories	17, 19, 24, 27, 28
04	Entity Framework Core	20, 27, 31
05	Asynchroon programmeren	12, 23, 29, 32
06	Relaties	2, 3, 17, 31
07	Generics & Unit of Work	17, 19, 20, 21, 32
08	Include statements	7, 13, 32, 34
09	DTO's	11, 14, 33

OPMERKING

De tabel loopt tot les 09. Komt er een les bij, zoek dan de namen die je erin tegenkomt op in de index achteraan.

Keuzegids

Welke bouwsteen neem ik?

Ik wil...	Neem	Hoofdstuk
een bedrag of prijs, exact tot op de cent	<code>decimal</code>	2
een datum of een tijdstip	<code>DateTime</code>	3
een vast aantal dingen dat nooit verandert	<code>array</code>	5
een aantal dingen waar ik bijvoeg en afhaal	<code>List</code>	6
iets terugvinden op een naam, code of nummer	<code>Dictionary</code>	7
zeker zijn dat er geen dubbels in zitten	<code>HashSet</code>	8
dingen afhandelen in de volgorde waarin ze binnenkwamen	<code>Queue</code>	9
het laatst toegevoegde als eerste terugnemen	<code>Stack</code>	10
een verzameling doorgeven zonder te zeggen welke soort	<code>IEnumerable</code>	12
een eigen ding met gegevens én gedrag	<code>class</code>	13
een pakketje gegevens dat gelijk is zodra de inhoud gelijk is	<code>record</code>	14
een klein pakketje dat bij kopiëren echt gekopieerd wordt	<code>struct</code>	15
een vaste lijst benoemde keuzes	<code>enum</code>	18
afspreken wat iets moet kunnen, zonder te zeggen hoe	<code>interface</code>	19
verwante klassen die gegevens en uitgewerkte methodes delen	<code>abstract class</code>	20
één stuk code dat werkt voor elk type dat je invult	generics, <code><T></code>	21
iets dat bij de klasse hoort, niet bij één object	<code>static</code>	24
een waarde die na het aanmaken nooit meer verandert	<code>const</code> of <code>readonly</code>	24
gegevens bewaren in een database, zonder zelf SQL te schrijven	EF Core	31
aan de client enkel geven wat hij nodig heeft	een DTO	33
zelf de SQL schrijven voor een snelle leesvraag	Dapper	34

Waar vind ik hoe ik...

Ik wil...	Hoofdstuk
een stuk uit een tekst halen, of tekst opsplitsen	1
twee teksten vergelijken zonder op hoofdletters te letten	1
tekst omzetten naar een getal zonder te crashen	2
iets verdelen: hoeveel per groep, hoeveel over	2
met datums rekenen, of een datum tonen	3
kiezen tussen meerdere mogelijkheden	4
iets een aantal keer herhalen	4
iets toevoegen aan of verwijderen uit een lijst	6
alles zien wat een verzameling kan, ook via LINQ	5 tot 10
weten welke lus op welke verzameling werkt, en wat ik mag veranderen terwijl ze loopt	4
per soort tellen hoeveel er van zijn	7
één item uit een verzameling halen	11
weten of er iets in zit dat voldoet	11
filteren, sorteren, optellen, het grootste vinden	11
zelf een reeks getallen maken, met of zonder stap	11
begrijpen waarom mijn resultaat plots veranderd is	12
begrijpen waarom mijn lijst na een methode veranderd is	16
een verzameling kopiëren	16
iets doen met een waarde die <code>null</code> kan zijn	17
begrijpen wat <code>= default!</code> achter een eigenschap doet	17
weten welke methodes <code>null</code> geven in plaats van te crashen	17
weten hoe mijn eigen klasse zich gedraagt in een lijst, een <code>HashSet</code> of een <code>Dictionary</code>	13, 14
mijn eigen klasse sorteerbaar maken	19
een methode van een basisklasse anders laten werken	20
één klasse schrijven die voor elk type werkt	21
nagaan van welk type een object is	22
enkel de objecten van één type uit een lijst halen	22
een fout opvangen, zodat mijn programma niet stopt	23
zelf een fout melden vanuit een methode	23
weten welke methode welke uitzondering geeft	23
een lijst bewaren tussen twee verzoeken	24
een lijst alleen-lezen maken	24
een methode aan een URL koppelen	25

Ik wil...	Hoofdstuk
een waarde uit de URL in mijn methode krijgen	25
de juiste statuscode teruggeven in een API	26
mijn repository aan mijn controller geven	27
iets melden in het terminalvenster terwijl de API draait	28
een methode asynchroon maken	29
een JSON-bestand inlezen	30
weten hoe een verzameling of een enum eruitziet in JSON	30
een tabel maken voor mijn klasse	31
een relatie tussen twee tabellen vastleggen	31
mijn database bijwerken na een wijziging aan een klasse	31
gegevens ophalen, toevoegen, wijzigen of verwijderen in de database	32
gerelateerde gegevens meeladen	32
weten welke LINQ-methodes EF Core kent, en welke async-versies er zijn	32
een entiteit omzetten naar een DTO	33

Deel 1 — De basis

Tekst, getallen, datums en beslissingen heb je in elk ander hoofdstuk nodig. Daarom komen ze eerst.

1. Tekst

METHODES

Een tekst heet in C# een `string`. Alles in dit hoofdstuk werkt op een `string`. Eén ding vooraf, omdat het overal terugkomt: **een tekst verandert nooit**. Elke methode hieronder geeft een **nieuwe** tekst terug en laat de oude zoals ze was.

Hoe lang is het, is het leeg?

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Length</code>	aantal tekens; een eigenschap, geen methode, dus zonder haakjes	<code>"moederbord".Length</code>	<code>10</code>
<code>string.IsNullOrEmpty</code>	is het null of een lege tekst?	<code>string.IsNullOrEmpty("")</code>	<code>true</code>
<code>string.IsNullOrWhiteSpace</code>	is het null, leeg, of enkel spaties, tabs of regeleindes?	<code>string.IsNullOrWhiteSpace(" ")</code>	<code>true</code>
<code>string.Empty</code>	een lege tekst, hetzelfde als <code>""</code>	<code>string.Empty == ""</code>	<code>true</code>

Voor invoer die van een gebruiker komt neem je bijna altijd `IsNullOrWhiteSpace`. Iemand die een spatie intikt heeft niets ingevuld, maar `IsEmpty` vindt dat wel een waarde.

LET OP

`Length` op `null` crasht met een `NullReferenceException`. `naam?.Length` geeft dan `null` in plaats van te crashen. Zie hoofdstuk 17.

Hoofdletters, kleine letters, spaties weg

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToUpper</code>	alles hoofdletters	<code>"mb-b650".ToUpper()</code>	<code>"MB-B650"</code>
<code>ToLower</code>	alles kleine letters	<code>"MB-B650".ToLower()</code>	<code>"mb-b650"</code>
<code>ToUpperInvariant</code> , <code>ToLowerInvariant</code>	hetzelfde, maar los van de taal van de computer	<code>"mb-b650".ToUpperInvariant()</code>	<code>"MB-B650"</code>
<code>Trim</code>	spaties vooraan en achteraan weg	<code>" ATX ".Trim()</code>	<code>"ATX"</code>
<code>TrimStart</code>	enkel vooraan	<code>" ATX".TrimStart()</code>	<code>"ATX"</code>
<code>TrimEnd</code>	enkel achteraan	<code>"ATX ".TrimEnd()</code>	<code>"ATX"</code>
<code>Trim</code>	andere tekens weg dan spaties	<code>"--ATX--".Trim('-')</code>	<code>"ATX"</code>
<code>TrimStart</code>	voorlooppnullen weg	<code>"007".TrimStart('0')</code>	<code>"7"</code>
<code>PadLeft</code>	vul aan tot een lengte, vooraan	<code>"7".PadLeft(3, '0')</code>	<code>"007"</code>
<code>PadRight</code>	vul aan tot een lengte, achteraan	<code>"AM5".PadRight(6, '.')</code>	<code>"AM5..."</code>

Deze methodes veranderen je tekst niet, ze geven een nieuwe terug. `naam.Trim()` op een regel alleen doet niets; je moet het resultaat ergens opvangen.

`Trim` haalt niet enkel spaties weg, maar ook tabs en regeleindes. `PadLeft` op een tekst die al lang genoeg is, doet niets: `"appel".PadLeft(3)` blijft `"appel"`.

Eén hoofdletter vooraan bestaat niet als methode. Je bouwt het zelf: `tekst[0..1].ToUpper() + tekst[1..]` maakt van `"appel"` `"Appel"` — maar crasht op een lege tekst.

Een stuk uit een tekst halen

Methode	Wat het doet	Voorbeeld	Resultaat
Substring	vanaf een positie, eventueel een aantal tekens	<code>"moederbord".Substring(6)</code>	<code>"bord"</code>
Substring	met lengte erbij	<code>"moederbord".Substring(0, 6)</code>	<code>"moeder"</code>
[..]	hetzelfde, met de bereikoperator	<code>"moederbord"[0..6]</code>	<code>"moeder"</code>
[^..]	de laatste n tekens	<code>"LZ-2026-8820"[^4..]</code>	<code>"8820"</code>
[i]	één teken op positie i; geeft een <code>char</code>	<code>"Gent"[0]</code>	<code>'G'</code>
[^1]	het laatste teken	<code>"kiwi"[^1]</code>	<code>'i'</code>
Split	knip op een teken, geeft een array	<code>"ATX,mATX,ITX".Split(',')</code>	<code>["ATX", "mATX", "ITX"]</code>
Split	knip op meerdere tekens	<code>"appel;peer,kiwi".Split(';',' ','')</code>	<code>["appel", "peer", "kiwi"]</code>
Split	knip op een stuk tekst	<code>"appel en peer en kiwi".Split(" en ")</code>	<code>["appel", "peer", "kiwi"]</code>
Split	lege stukken weglaten	<code>"rood;groen;;blauw".Split(';',' ', StringSplitOptions.RemoveEmptyEntries)</code>	<code>["rood", "groen", "blauw"]</code>
Split	spaties rond elk stuk weg	<code>"appel , peer , kiwi".Split(',',' ', StringSplitOptions.TrimEntries)</code>	<code>["appel", "peer", "kiwi"]</code>
Split	hoogstens n stukken	<code>"naam=Jan=Peeters".Split('=',' ', 2)</code>	<code>["naam", "Jan=Peeters"]</code>
string.Join	plak stukken aan elkaar met een scheider	<code>string.Join(" / ", delen)</code>	<code>"ATX / mATX / ITX"</code>
string.Concat	plak stukken aan elkaar zonder scheider	<code>string.Concat("ATX", "-", "AM5")</code>	<code>"ATX-AM5"</code>
Replace	vervang alle voorkomens	<code>"MB-B650-01".Replace("-", "")</code>	<code>"MBB65001"</code>
Replace	zonder op hoofdletters te letten	<code>"Peer en peer".Replace("PEER", "peer", StringComparison.OrdinalIgnoreCase)</code>	<code>"peer en peer"</code>

Methode	Wat het doet	Voorbeeld	Resultaat
Remove	haal weg vanaf een positie, eventueel een aantal tekens	<code>"moederbord".Remove(6)</code>	<code>"moeder"</code>
Insert	voeg in op een positie	<code>"moederbord".Insert(6, "-")</code>	<code>"moeder-bord"</code>

Je kan beide opties samen meegeven: `StringSplitOptions.RemoveEmptyEntries` | `StringSplitOptions.TrimEntries`.

LET OP

`Substring`, `Remove`, `Insert` en `[..]` lopen stuk zodra je buiten de tekst grijpt. `"ATX".Substring(0, 5)` en `"ATX"[0..5]` geven geen lege tekst maar een **`ArgumentOutOfRangeException`**. Controleer eerst de `Length`, of gebruik `Split` wanneer je niet weet hoe lang iets is.

Zit er iets in, en waar?

Methode	Wat het doet	Voorbeeld	Resultaat
Contains	zit het erin, ja of nee	<code>"MB-B650-01".Contains("650")</code>	<code>true</code>
Contains	zonder op hoofdletters te letten	<code>"MB-B650".Contains("b650", StringComparison.OrdinalIgnoreCase)</code>	<code>true</code>
StartsWith	begint het ermee	<code>"MB-B650".StartsWith("MB")</code>	<code>true</code>
EndsWith	eindigt het erop	<code>"MB-B650-01".EndsWith("01")</code>	<code>true</code>
IndexOf	positie van het eerste voorkomen	<code>"MB-B650-01".IndexOf("-")</code>	<code>2</code>
IndexOf	niet gevonden	<code>"MB-B650".IndexOf("x")</code>	<code>-1</code>
IndexOf	zoek pas vanaf een positie	<code>"MB-B650-01".IndexOf('-', 3)</code>	<code>7</code>
LastIndexOf	positie van het laatste voorkomen	<code>"MB-B650-01".LastIndexOf("-")</code>	<code>7</code>
IndexOfAny, LastIndexOfAny	eerste of laatste van meerdere tekens	<code>"MB-B650-01".IndexOfAny(new[] { '5', '0' })</code>	<code>5</code>

`Contains`, `StartsWith`, `EndsWith` en `IndexOf` werken met een stuk tekst (`"650"`) én met één teken (`'-'`), en ze aanvaarden allemaal een `StringComparison` als laatste waarde.

`Contains` als je enkel wil weten óf het erin zit. `IndexOf` als je daarna nog iets met de plaats wil doen, bijvoorbeeld alles ervoor of erna afsplitsen.

LET OP

`IndexOf` geeft `-1` terug als er niets gevonden is, geen `null`. Vergeet je dat te controleren en steek je die `-1` in een `Substring`, dan loopt je code stuk op een plek die er onschuldig uitziet.

Tekst vergelijken

Methode	Wat het doet	Voorbeeld	Resultaat
<code>==</code>	letterlijk gelijk, hoofdlettergevoelig	<code>"AM5" == "am5"</code>	<code>false</code>
<code>!=</code>	verschillend	<code>"AM5" != "am5"</code>	<code>true</code>
<code>Equals</code>	idem, maar je kan de regels meegeven	<code>"AM5".Equals("am5", StringComparison.OrdinalIgnoreCase)</code>	<code>true</code>
<code>string.Equals</code>	idem, en crasht niet als een van beide <code>null</code> is	<code>string.Equals(naam, "am5", StringComparison.OrdinalIgnoreCase)</code>	<code>true</code> of <code>false</code>
<code>CompareTo</code>	welke komt eerst in de sortering	<code>"b".CompareTo("a")</code>	<code>1</code> : b komt na a
<code>string.Compare</code>	idem, met regels en zonder te crashen op <code>null</code>	<code>string.Compare("appel", "Banaan")</code>	<code>-1</code> : appel komt eerst

Wil je twee sockets vergelijken die de ene keer met hoofdletters en de andere keer zonder binnenkomen, dan is `Equals` met `OrdinalIgnoreCase` het nette antwoord. `ToUpper()` op allebei werkt ook, maar maakt twee nieuwe teksten om er één vergelijking mee te doen.

`CompareTo` en `string.Compare` geven een getal: **negatief** als de eerste vooraan komt, **0** als ze gelijk zijn, **positief** als de eerste achteraan komt. Reken niet op 1 of -1: met `StringComparison.Ordinal` kan het ook 31 zijn.

<code>StringComparison</code>	Hoofdletters tellen	Sorteert zoals
<code>Ordinal</code>	ja	de codes van de tekens: alle hoofdletters vóór alle kleine letters
<code>OrdinalIgnoreCase</code>	nee	idem, zonder hoofdletters
<code>CurrentCulture</code>	ja	een woordenboek in de taal van de computer
<code>CurrentCultureIgnoreCase</code>	nee	idem, zonder hoofdletters
<code>InvariantCulture</code> , <code>InvariantCultureIgnoreCase</code>	ja / nee	een woordenboek, los van de taal van de computer

Voor codes, sleutels en wachtwoorden neem je `Ordinal` of `OrdinalIgnoreCase`. Voor namen die je alfabetisch aan een mens toont, de `Culture`-varianten.

Tekst opbouwen met waarden erin

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>\$"..."</code>	zet waarden rechtstreeks in de tekst	<code>\$"Socket {socket} past"</code>	<code>"Socket AM5 past"</code>
<code>\${x:0.00}"</code>	met een opmaak voor getallen	<code>\${prijs:0.00} euro"</code>	<code>"19.50 euro"</code> of <code>"19,50 euro"</code> , zie hoofdstuk 2
<code>\${x,5}"</code>	uitlijnen op een breedte	<code>\${getal,5}"</code>	<code>"[7]"</code>
<code>+</code>	plakken	<code>"Socket " + socket</code>	<code>"Socket AM5"</code>

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>+=</code>	iets achteraan bijplakken	<code>tekst += "!"</code>	<code>tekst</code> is een nieuwe tekst met ! erachter
<code>string.Format</code>	oudere vorm met plaatshouders	<code>string.Format("Socket {0}", socket)</code>	"Socket AM5"
<code>new string</code>	een teken een aantal keer herhalen	<code>new string('-', 5)</code>	"-----"

De `$"..."-`vorm is wat je overal zal tegenkomen. Je mag er ook berekeningen in zetten: `$"Nog {sloten - modules} vrij"`.

Speciale tekens schrijf je met een backslash: `\` is een aanhalingsteken, `\n` een nieuwe regel, `\t` een tab, `\\` een backslash. Zet je `@` voor de tekst, dan telt de backslash niet meer als speciaal: `@ "C:\data\fruit.txt"`. Drie aanhalingstekens (`"""..."""`) maken een tekst waarin je gewone aanhalingstekens mag zetten.

Tekst in stukken opbouwen: StringBuilder

Elke `+=` maakt een nieuwe tekst. In een lus die duizend keer draait, zijn dat duizend teksten. Een `StringBuilder` bouwt in één stuk op en maakt pas op het einde een tekst.

```
var sb = new StringBuilder(); // using System.Text;
foreach (int i in new[] { 1, 2, 3 })
{
    sb.Append(i);
    sb.Append(';');
}
string resultaat = sb.ToString(); // "1;2;3;"
```

Methode	Wat het doet	Voorbeeld	Resultaat
Append	plak achteraan bij; tekst, teken of getal	<code>sb.Append("appel")</code>	de builder bevat "appel"
AppendLine	plak bij en begin een nieuwe regel	<code>sb.AppendLine("regel 1")</code>	"regel 1" en een regeleinde
AppendJoin	plak een reeks bij, met een scheider	<code>sb.AppendJoin(", ", fruit)</code>	"appel, peer, kiwi"
Insert, Remove, Replace	zoals bij een tekst, maar in de builder zelf	<code>sb.Insert(0, "> ")</code>	"> " staat vooraan
Length, [i]	lengte, en een teken lezen of vervangen	<code>sb[0] = 'X'</code>	het eerste teken is nu X
Clear	maak leeg	<code>sb.Clear()</code>	Length is 0
ToString	maak er een tekst van	<code>sb.ToString()</code>	een string

Losse tekens: char

Een tekst bestaat uit tekens. Eén teken is een `char` en staat tussen **enkele** aanhalingstekens: `'a'`. `"kiwi"[0]` geeft een `char`, geen `string`.

Methode	Wat het doet	Voorbeeld	Resultaat
<code>char.IsDigit</code>	is het een cijfer	<code>char.IsDigit('7')</code>	<code>true</code>
<code>char.IsLetter</code>	is het een letter, ook met accent	<code>char.IsLetter('é')</code>	<code>true</code>
<code>char.IsLetterOrDigit</code>	letter of cijfer	<code>char.IsLetterOrDigit('-')</code>	<code>false</code>
<code>char.IsWhiteSpace</code>	spatie, tab of regeleinde	<code>char.IsWhiteSpace('\t')</code>	<code>true</code>
<code>char.IsUpper</code> , <code>char.IsLower</code>	hoofdletter, kleine letter	<code>char.IsUpper('A')</code>	<code>true</code> ; bij een cijfer allebei <code>false</code>
<code>char.IsPunctuation</code>	leesteken	<code>char.IsPunctuation('!')</code>	<code>true</code>
<code>char.IsAsciiDigit</code> , <code>char.IsAsciiLetter</code>	enkel 0–9, of enkel a–z en A–Z	<code>char.IsAsciiLetter('é')</code>	<code>false</code>
<code>char.ToUpper</code> , <code>char.ToLower</code>	één teken als hoofdletter of kleine letter	<code>char.ToUpper('a')</code>	<code>'A'</code>
<code>char.GetNumericValue</code>	het cijfer als getal	<code>char.GetNumericValue('7')</code>	7, als <code>double</code> ; bij een letter <code>-1</code>
<code>'7' - '0'</code>	het cijfer als <code>int</code> , de korte manier	<code>'7' - '0'</code>	7

LET OP

Met tekens rekenen geeft getallen, geen tekst. `'7' + 1` is `56`: de code van het teken 7, plus 1. En `'a' + 'b'` is `195`. Wil je tekst, zorg dan dat er een `string` bij staat: `"a" + "b"` is `"ab"`.

Tekst en getallen

Een getal uit een tekst halen doe je met `TryParse`. [Hoofdstuk 2](#) gaat er dieper op in; de kern:

Methode	Wat het doet	Voorbeeld	Resultaat
<code>int.TryParse</code>	probeer om te zetten, zonder te crashen	<code>int.TryParse("120", out int aantal)</code>	<code>true</code> , aantal is 120; bij <code>"abc"</code> <code>false</code> en 0
<code>int.Parse</code>	zet om, of crasht	<code>int.Parse("abc")</code>	<code>FormatException</code>
<code>ToString</code>	een getal als tekst	<code>120.ToString()</code>	<code>"120"</code>

`TryParse` aanvaardt spaties rond het getal (`" 42 "` lukt), maar geen letters erachter (`"12abc"` lukt niet).

Wat het ook kan, via iets anders

Een tekst is ook een **reeks tekens**: je kan hem teken per teken doorlopen. In C# is een `string` een `IEnumerable<char>` ([hoofdstuk 12](#)). Daardoor werkt alles wat met een reeks werkt ook op een tekst.

Bron	Waarom het op een tekst werkt	Volledige uitleg
LINQ	een tekst is een reeks tekens	hoofdstuk 11
<code>foreach</code>	doorloopt de tekst teken per teken	hoofdstuk 4
<code>switch</code> en <code>is</code>	vergelijken met vaste teksten en patronen	hoofdstuk 4
andere verzamelingen	een lijst of set van de tekens	hoofdstukken 5 tot 8
wat elk object kan	een tekst is ook een object	hoofdstuk 13

Wat je over LINQ op een tekst moet weten:

- LINQ werkt **teken per teken**. `"kiwi".Count()` telt 4 tekens.
- Het resultaat is **geen tekst** maar een reeks tekens. `"kiwi".Reverse()` geeft geen `"iwik"`. Maak er weer tekst van met `string.Concat(...)` of `new string(...ToArray())`.
- LINQ werkt enkel met `using System.Linq`; bovenaan. In een nieuw project staat die er automatisch bij.

Tekens zoeken en nagaan

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
First , FirstOrDefault	het eerste teken (dat voldoet)	<code>"abc123".First(char.IsDigit)</code>	<code>'1'</code>	<code>[0]</code> , <code>IndexOfAny</code>
Last , LastOrDefault	het laatste teken (dat voldoet)	<code>"kiwi".Last()</code>	<code>'i'</code>	<code>[^1]</code>
Single , SingleOrDefault	het enige teken dat voldoet	<code>"ab1c".Single(char.IsDigit)</code>	<code>'1'</code>	—
ElementAt	het teken op een positie	<code>"kiwi".ElementAt(2)</code>	<code>'w'</code>	<code>[2]</code>
Any	is er een teken dat voldoet	<code>"abc123".Any(char.IsDigit)</code>	<code>true</code>	—
All	voldoen alle tekens	<code>"1234".All(char.IsDigit)</code>	<code>true</code>	—
Contains	zit dit teken erin	<code>"kiwi".Contains('w')</code>	<code>true</code>	<code>Contains</code> : de tekst heeft een eigen versie
SequenceEqual	dezelfde tekens in dezelfde volgorde	<code>"kiwi".SequenceEqual("kiwi")</code>	<code>true</code>	<code>==</code>

LET OP

- `FirstOrDefault` op een tekst geeft bij niets geen `null`, maar het lege teken `'\0'`.
- `"".All(char.IsDigit)` is `true`: een lege tekst heeft geen enkel teken dat géén cijfer is. Wil je weten of iets een getal is, controleer dan ook dat het niet leeg is, of gebruik `int.TryParse`.

Tekens kiezen, tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
Where	enkel de tekens die voldoen	<code>string.Concat("a1b2c3".Where(char.IsDigit))</code>	<code>"123"</code>
Select	maak van elk teken iets anders	<code>"123".Select(c => c - '0')</code>	<code>1, 2, 3</code> als getallen
Distinct	elk teken één keer	<code>string.Concat("banaan".Distinct())</code>	<code>"ban"</code>
Take, Skip, TakeLast, SkipLast	de eerste of laatste n tekens	<code>"kiwi".TakeLast(2)</code>	<code>'w', 'i'</code>
TakeWhile, SkipWhile	zolang de voorwaarde klopt	<code>"123abc".TakeWhile(char.IsDigit)</code>	<code>'1', '2', '3'</code>
Chunk	verdeel in stukken van n tekens	<code>"abcdefg".Chunk(3)</code>	<code>[a, b, c], [d, e, f], [g]</code>
Count	hoeveel tekens voldoen	<code>"abc123".Count(char.IsDigit)</code>	<code>3</code>
Count	hoe vaak komt een teken voor	<code>"banaan".Count(c => c == 'a')</code>	<code>3</code>
Sum	tel de cijfers op	<code>"1234".Sum(c => c - '0')</code>	<code>10</code>
Min, Max	het kleinste of grootste teken	<code>"kiwi".Max()</code>	<code>'w'</code>
CountBy	tel per teken	<code>"banaan".CountBy(c => c)</code>	<code>b 1, a 3, n 2</code>
GroupBy	groepeer de tekens	<code>"banaan".GroupBy(c => c)</code>	<code>b: 1, a: 3, n: 2</code>

LET OP

`"1234".Sum(c => c)` geeft niet 10 maar de som van de **codes** van de tekens. Trek er `'0'` af, zoals in de tabel.

Ordenen en combineren

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
Reverse	de tekens in omgekeerde volgorde	<code>string.Concat("kiwi".Reverse())</code>	<code>"iwik"</code>

Methode	Wat het doet	Voorbeeld	Resultaat
Order, OrderDescending	de tekens gesorteerd	<code>string.Concat("kiwi".Order())</code>	"iikw"
Concat, Union, Intersect, Except	combineer de tekens van twee teksten	<code>string.Concat("appel".Intersect("peer"))</code>	"pe": de letters die in beide zitten
Append, Prepend	één teken achteraan of vooraan	<code>string.Concat("kiwi".Append('!'))</code>	"kiwi!"
Zip	koppel de tekens van twee teksten	<code>"abc".Zip("123")</code>	(a, 1), (b, 2), (c, 3)

Tekens sorteren gebeurt op hun **code**: alle hoofdletters komen vóór alle kleine letters. `"bAaB".Order()` geeft A, B, a, b. Teksten in een lijst sorteren gebeurt wél zoals in een woordenboek: zie *Let* op hieronder.

Tekens vastleggen en doorlopen

Komt van LINQ, de taal zelf en de andere verzamelingen.

Schrijfwijze	Wat het doet	Resultaat
<code>"kiwi".ToArray()</code>	een array van de tekens (eigen methode)	'k', 'i', 'w', 'i'
<code>"kiwi".ToList(), "kiwi".ToArray()</code>	een lijst of array van de tekens (LINQ)	4 tekens
<code>"banaan".ToHashSet(), new HashSet<char>("banaan")</code>	de verschillende tekens	'b', 'a', 'n'
<code>new string(tekens)</code>	van een array tekens weer een tekst maken	<code>new string(new[] { 'k', 'i' })</code> is "ki"
<code>string.Concat(reeks)</code>	van een reeks tekens weer een tekst maken	<code>string.Concat("kiwi".Reverse())</code> is "iwik"
<code>foreach (char c in "kiwi")</code>	elk teken één keer	k, i, w, i
<code>for (int i = 0; i < tekst.Length; i++)</code>	elke positie, met <code>tekst[i]</code>	0, 1, 2, 3
<code>switch (fruit) { case "kiwi": ... }</code>	kiezen op een vaste tekst	zie hoofdstuk 4

BELANGRIJK

`tekst[0] = 'K';` compileert niet: een tekst kan je niet veranderen, ook niet één teken. Maak een nieuwe tekst, of gebruik een `StringBuilder`, waar `sb[0] = 'K'` wel mag.

Wat elk object kan

Komt van `object`. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
ToString	de tekst zelf	<code>"kiwi".ToString()</code>	"kiwi"

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Equals / ==</code>	dezelfde inhoud	<code>"kiwi" == new string(new[] { 'k', 'i', 'w', 'i' })</code>	<code>true</code>
<code>GetType</code>	welk type	<code>"kiwi".GetType().Name</code>	<code>String</code>
<code>GetHashCode</code>	een getal per tekst; gelijke teksten geven hetzelfde getal	<code>"kiwi".GetHashCode()</code>	een getal

Een tekst is de grote uitzondering: `==` vergelijkt bij een `string` de **inhoud**, terwijl het bij een lijst, een array of een gewone klasse vraagt of het om **hetzelfde object** gaat. Pas op: staat een van beide als `object` getypeerd, dan vergelijkt `==` weer op referentie.

Bestaat ook, maar kom je in deze cursus niet tegen

`Normalize`, `IsNormalized`, `EnumerateRunes`, `GetPinnableReference`, `AsSpan`, `CopyTo` (tekens naar een array), `ReplaceLineEndings`, `string.Create`, `string.Intern`, `CompareOrdinal`, `char.IsSymbol`, `char.IsNumber`, `char.IsControl`, `TextInfo.ToTitleCase` (elk woord met een hoofdletter), reguliere expressies (`Regex`), en uit LINQ `AsParallel` en `Cast`.

Let op bij tekst

Op de tekst zelf	Via LINQ	Het verschil
<code>Length</code>	<code>Count()</code>	Hetzelfde getal. <code>Length</code> zonder haakjes is meteen bekend; <code>Count()</code> telt. Een voorwaarde kan enkel met LINQ: <code>Count(char.IsDigit)</code> .
<code>Contains("650")</code>	<code>Contains('6')</code>	De tekst zoekt een stuk tekst of een teken. LINQ zoekt enkel één teken.
<code>[0]</code>	<code>First()</code>	Hetzelfde teken. Op een lege tekst crashen ze allebei, met een andere uitzondering.
<code>[1..3]</code>	<code>Skip(1).Take(2)</code>	<code>[1..3]</code> geeft meteen een tekst , en crasht buiten de tekst. LINQ geeft een reeks tekens en crasht nooit.
<code>ToUpper()</code>	<code>Select(char.ToUpper)</code>	<code>ToUpper</code> geeft een tekst. <code>Select</code> geeft een reeks tekens.
<code>ToCharArray()</code>	<code>ToArray()</code>	Hetzelfde resultaat: een <code>char[]</code> .

LET OP

- Elke methode geeft een **nieuwe** tekst. `naam.Trim()`; op een regel alleen doet niets.
- `string.Join("-", "kiwi")` geeft `"kiwi"`, niet `"k-i-w-i"`: C# ziet de tekst als één waarde. Wil je de tekens scheiden, geef dan de tekens mee: `string.Join("-", "kiwi".ToCharArray())`.
- `Console.WriteLine("kiwi".Reverse())` toont geen omgekeerde tekst maar de naam van een type. Zet er `string.Concat(...)` rond.
- `"appel" == "Appel"` is `false`. Vergelijk je zonder hoofdletters, gebruik dan `StringComparison.OrdinalIgnoreCase`, niet twee keer `ToUpper()`.
- Teksten in een lijst sorteren (`Order`, `Sort`) gaat zoals in een woordenboek: `appel`, `Banaan`, `citroen`. Met `StringComparer.Ordinal` krijg je de volgorde van de codes: `Banaan`, `appel`, `citroen`. Losse **tekens** sorteren altijd op hun code.
- `ToUpper` en `ToLower` volgen de taal van de computer. Op een Turkse computer wordt `"i".ToUpper()` een `İ` met een punt. Voor codes en sleutels neem je `ToUpperInvariant`.
- Sommige tekens tellen voor twee: een emoji heeft `Length 2`.

Voorspel de uitkomst

Vraag 1

```
string s = "  Gent  ";  
s.Trim();  
Console.WriteLine("[ " + s + " ]");
```

[Toon het antwoord ›](#)

Vraag 2

```
string code = "ABC-123";  
Console.WriteLine(code.Substring(code.IndexOf("-") + 1));
```

[Toon het antwoord ›](#)

Vraag 3

```
Console.WriteLine("appel" == "Appel");  
Console.WriteLine("appel".Equals("Appel", StringComparison.OrdinalIgnoreCase));
```

[Toon het antwoord ›](#)

Vraag 4

```
string[] delen = "rood;groen;;blauw".Split(';');  
Console.WriteLine(delen.Length);
```

[Toon het antwoord ›](#)

Vraag 5

```
string s = "kiwi";  
var omgekeerd = s.Reverse();  
Console.WriteLine(s);  
Console.WriteLine(string.Concat(omgekeerd));
```

[Toon het antwoord ›](#)

Vraag 6

```
string code = "A1B2C3";  
Console.WriteLine(code.Count(char.IsDigit));  
Console.WriteLine(code.Any(char.IsLower));
```

[Toon het antwoord ›](#)

Vraag 7

```
Console.WriteLine('7' + 1);  
Console.WriteLine('7' - '0' + 1);
```

[Toon het antwoord ›](#)

Vraag 8

```
string[] delen = " a , , b ".Split(',', StringSplitOptions.RemoveEmptyEntries);  
Console.WriteLine(delen.Length);
```

[Toon het antwoord ›](#)

Vraag 9

```
Console.WriteLine(string.Join("-", "kiwi"));  
Console.WriteLine(string.Join("-", "kiwi".ToCharArray()));
```

[Toon het antwoord ›](#)

Vraag 10

```
string s = "Gent";  
Console.WriteLine(s[1..3]);  
Console.WriteLine(s[^1]);  
Console.WriteLine(s[1..9]);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

[Gent]

`Trim` verandert de tekst niet. Het geeft een nieuwe tekst terug, en die werd hier nergens opgevangen. Wil je dat het werkt, dan schrijf je `s = s.Trim();`.

[← Terug naar de vraag](#)

VRAAG 2

123

Het streepje staat op positie 3. Eén verder is positie 4, en vanaf daar neemt `Substring` alles tot het einde. Dit is het patroon om alles ná een scheidingsteken te krijgen.

[← Terug naar de vraag](#)

VRAAG 3

False en daarna True

`==` let op hoofdletters. Met `OrdinalIgnoreCase` zeg je uitdrukkelijk dat het niet mag.

[← Terug naar de vraag](#)

VRAAG 4

4

Tussen de twee puntkomma's staat niets, en dat niets telt ook mee als stuk: een lege tekst. De vier stukken zijn rood, groen, een lege tekst en blauw.

[← Terug naar de vraag](#)

VRAAG 5

kiwi en daarna iwik

`Reverse` is LINQ: de tekst zelf verandert niet, en het resultaat is een reeks tekens. `string.Concat` maakt daar weer een tekst van.

[← Terug naar de vraag](#)

VRAAG 6

3 en daarna False

Drie tekens zijn een cijfer: 1, 2 en 3. Kleine letters zitten er niet in: A, B en C zijn hoofdletters.

[← Terug naar de vraag](#)

VRAAG 7

56 en daarna 8

Met een teken rekenen gebeurt met zijn code, en de code van '7' is 55. Trek je er de code van '0' (48) van af, dan hou je het cijfer 7 over.

[Terug naar de vraag](#)

VRAAG 8

3

Het middelste stuk is een spatie, geen lege tekst, dus `RemoveEmptyEntries` laat het staan. Met `RemoveEmptyEntries` | `TrimEntries` wordt eerst de spatie weggehaald, en dan pas het lege stuk: dan blijven er 2.

[Terug naar de vraag](#)

VRAAG 9

kiwi en daarna k-i-w-i

Een tekst alleen is voor `string.Join` één waarde: er valt niets te scheiden. Een array van tekens zijn vier waarden.

[Terug naar de vraag](#)

VRAAG 10

en, daarna `t`, en dan `ArgumentOutOfRangeException` — het programma crasht.

`1..3` neemt de posities 1 en 2. `^1` is het laatste teken. `1..9` gaat voorbij het einde van de tekst, en daar crasht het bereik.

[Terug naar de vraag](#)

2. Getallen en omzetten

METHODES

Tekst omzetten naar een getal

Methode	Wat het doet	Voorbeeld	Resultaat
<code>int.Parse</code>	zet om, of loopt stuk	<code>int.Parse("120")</code>	120
<code>int.Parse</code>	bij onzin	<code>int.Parse("abc")</code>	uitzondering
<code>int.TryParse</code>	probeert om te zetten, zegt of het lukte	<code>int.TryParse("120", out int n)</code>	true, n is 120
<code>int.TryParse</code>	bij onzin	<code>int.TryParse("abc", out int n)</code>	false, n is 0
<code>double.Parse</code>	hetzelfde voor kommagetallen	<code>double.Parse("2.5")</code>	hangt af van je taalinstelling — zie hieronder
<code>decimal.Parse</code> , <code>decimal.TryParse</code>	hetzelfde voor een decimal	<code>decimal.TryParse("19,99", out decimal p)</code>	true op een Belgische computer
<code>Parse</code> met cultuur	lees altijd met een punt als decimaalteken	<code>double.Parse("2.5", CultureInfo.InvariantCulture)</code>	2.5, op elke computer
<code>Convert.ToInt32</code>	zet om; bij null geeft het 0 in plaats van te crashen	<code>Convert.ToInt32("12")</code>	12

`Parse` en `TryParse` aanvaarden spaties rond het getal (" 42 " lukt), maar geen letters en, zonder extra instelling, geen scheidingsteken voor duizendtallen: `int.Parse("1.000")` geeft een

FormatException.

BELANGRIJK

De regel is eenvoudig: komt de tekst van een gebruiker, gebruik `TryParse`. Komt ze uit je eigen code en weet je zeker dat het een getal is, dan mag `Parse`. Een `Parse` op gebruikersinvoer is een crash die op je zit te wachten.

LET OP

`double.Parse` leest een kommagetal volgens de taalinstelling van je computer. Op een Belgische computer is de komma het decimaalteken en de punt het teken voor duizendtallen, zoals in 1.000. `double.Parse("2.5")` geeft daar 25 — zonder foutmelding. Komt je getal uit een bestand of een API, waar altijd een punt staat, geef dan `CultureInfo.InvariantCulture` mee: `double.Parse("2.5", CultureInfo.InvariantCulture)`.

Omgekeerd toont `Console.WriteLine(2.5)` op jouw computer 2,5. In dit leerboek staan kommagetallen in de schrijfwijze van C# zelf, met een punt.

Een getal als tekst tonen

Methode	Wat het doet	Voorbeeld	Resultaat
---------	--------------	-----------	-----------

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToString</code>	het getal als tekst	<code>120.ToString()</code>	"120"
<code>ToString("0.00")</code>	altijd twee cijfers na de komma	<code>19.5m.ToString("0.00")</code>	"19,50" op een Belgische computer
<code>ToString("N2")</code>	met scheiding voor duizendtallen	<code>1234.5m.ToString("N2")</code>	"1.234,50"
<code>ToString("C")</code>	als bedrag, met munteenheid	<code>19.5m.ToString("C")</code>	"€ 19,50"
<code>ToString("P0")</code>	als percentage	<code>0.25.ToString("P0")</code>	"25%"
<code>ToString("D3")</code>	geheel getal, aangevuld met nullen	<code>7.ToString("D3")</code>	"007"
<code>ToString</code> met cultuur	met een punt, los van de computer	<code>2.5.ToString(CultureInfo.InvariantCulture)</code>	"2.5"

Dezelfde opmaak werkt in een tekst met `$: "${prijs:0.00} euro"`. Zie [hoofdstuk 1](#).

Rekenen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Math.Max</code>	de grootste van twee	<code>Math.Max(3, 7)</code>	7
<code>Math.Min</code>	de kleinste van twee	<code>Math.Min(3, 7)</code>	3
<code>Math.Abs</code>	zonder minteken	<code>Math.Abs(-5)</code>	5
<code>Math.Round</code>	afroonden	<code>Math.Round(2.4)</code>	2
<code>Math.Ceiling</code>	altijd naar boven	<code>Math.Ceiling(2.1)</code>	3
<code>Math.Floor</code>	altijd naar beneden	<code>Math.Floor(2.9)</code>	2
<code>Math.Pow</code>	tot de macht	<code>Math.Pow(2, 3)</code>	8
<code>/</code>	delen van gehele getallen	<code>7 / 2</code>	3
<code>%</code>	de rest na het delen	<code>7 % 2</code>	1
<code>Math.Round</code>	afroonden op een aantal cijfers	<code>Math.Round(19.987m, 2)</code>	19.99
<code>Math.Round</code>	schools afronden	<code>Math.Round(2.5, MidpointRounding.AwayFromZero)</code>	3
<code>Math.Truncate</code>	knip alles na de komma weg	<code>Math.Truncate(-2.7)</code>	-2
<code>Math.Sqrt</code>	vierkantswortel	<code>Math.Sqrt(16)</code>	4
<code>Math.Clamp</code>	hou binnen een minimum en maximum	<code>Math.Clamp(15, 0, 10)</code>	10
<code>Math.Sign</code>	negatief, nul of positief	<code>Math.Sign(-3)</code>	-1
<code>++, --, +=</code>	één erbij, één eraf, iets erbij	<code>teller += 5</code>	teller is 5 groter

`Math.Floor(-2.7)` is `-3` en `Math.Truncate(-2.7)` is `-2`: `Floor` gaat altijd naar beneden, `Truncate` knipt gewoon af. Bij positieve getallen is er geen verschil.

`Math.Pow` en `Math.Sqrt` geven altijd een `double` terug, ook bij gehele getallen.

LET OP

Twee dingen die bijna iedereen één keer verkeerd doet. `7 / 2` geeft `3` en niet `3.5`, want twee gehele getallen delen geeft een geheel getal. En `Math.Round(2.5)` geeft `2`, niet `3` — C# rondt een halve eenheid naar het dichtstbijzijnde even getal. Wil je het schoolse afronden, dan geef je `MidpointRounding.AwayFromZero` mee.

Het deelteken en het restteken samen zijn precies wat je nodig hebt om iets te verdelen: hoeveel per groep, en hoeveel blijven er over.

Grenzen, en omzetten tussen soorten getallen

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
<code>int.MaxValue</code> , <code>int.MinValue</code>	het grootste en kleinste <code>int</code>	<code>int.MaxValue</code>	<code>2147483647</code>
<code>long</code>	een geheel getal voor grotere waarden	<code>long aantal = 3_000_000_000;</code>	compileert; als <code>int</code> niet
<code>(int)</code>	een kommagetal omzetten naar een geheel getal	<code>(int)2.9</code>	<code>2</code> : alles na de komma valt weg
<code>Convert.ToInt32</code>	idem, maar afronden	<code>Convert.ToInt32(2.9)</code>	<code>3</code> ; <code>Convert.ToInt32(2.5)</code> is <code>2</code>
<code>(double)</code>	een geheel getal als kommagetal	<code>10 / (double)4</code>	<code>2.5</code>
<code>checked</code>	crash bij overlopen in plaats van stil verder te tellen	<code>checked(int.MaxValue + teller)</code>	<code>OverflowException</code>

LET OP

- Een `int` die over zijn maximum gaat, crasht niet maar springt naar het **minimum**: `int.MaxValue + 1` met een variabele wordt `-2147483648`.
- Delen door nul: met gehele getallen en `decimal` crasht het met een `DivideByZeroException`. Met een `double` niet: `1.0 / 0` is `Infinity`, en `0.0 / 0` is `NaN` ("geen getal").
- `(double)(10 / 4)` is `2`: eerst wordt geheel gedeeld, pas daarna omgezet. Zet om **vóór** je deelt.

Kommagetallen: double tegenover decimal

C# heeft twee soorten kommagetallen die je vaak tegenkomt. Een `double` is snel, maar bewaart sommige getallen net niet exact. Een `decimal` is trager, maar rekent tot op de cent juist. Vanaf les 06 staat een prijs in de cursus als `decimal`.

Vorm	Wat het doet	Voorbeeld	Resultaat
------	--------------	-----------	-----------

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>double</code>	snel kommagetal, kan een heel klein beetje afwijken	<code>0.1 + 0.2 == 0.3</code>	<code>false</code>
<code>decimal</code>	exact kommagetal, voor geld en prijzen	<code>0.1m + 0.2m == 0.3m</code>	<code>true</code>
<code>m</code> achter een getal	maakt er een <code>decimal</code> van	<code>decimal prijs = 19.99m;</code>	compileert
zonder <code>m</code>	een kommagetal zonder letter is een <code>double</code>	<code>decimal prijs = 19.99;</code>	compileert niet
<code>(double)</code>	zet een <code>decimal</code> om naar een <code>double</code>	<code>(double)19.99m</code>	<code>19.99</code>
<code>Math.Round</code>	afronden op een aantal cijfers na de komma	<code>Math.Round(19.987m, 2)</code>	<code>19.99</code>

LET OP

Een `decimal` en een `double` mag je niet zomaar door elkaar gebruiken. `prijs * 1.21` compileert niet als `prijs` een `decimal` is, want `1.21` is een `double`. Schrijf `prijs * 1.21m`.

Een willekeurig getal

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Random.Shared.Next</code>	een geheel getal vanaf het eerste, tot vóór het tweede	<code>Random.Shared.Next(1, 7)</code>	een getal van 1 tot en met 6, zoals een dobbelsteen
<code>Random.Shared.NextDouble</code>	een kommagetal van 0 tot vóór 1	<code>Random.Shared.NextDouble()</code>	bv. <code>0.7314</code>

Het tweede getal bij `Next` doet zelf niet mee: `Next(1, 7)` geeft nooit 7.

Wat het ook kan, via iets anders

Een getal is een **waarde**, geen object met methodes zoals een lijst: `int`, `double` en `decimal` zijn structs (hoofdstuk 15). Wat je ermee kan, komt van drie plaatsen: de reketekens, de klasse `Math`, en wat elk object kan. Zodra je met **meerdere** getallen werkt, komt LINQ erbij.

Bron	Wat het geeft	Volledige uitleg
LINQ op een reeks getallen	<code>Sum</code> , <code>Average</code> , <code>Min</code> , <code>Max</code> , <code>Order</code> , <code>Distinct</code> , ...	hoofdstuk 11
<code>Enumerable.Range</code>	een reeks opeenvolgende getallen	hoofdstuk 11
vergelijken	<code><</code> , <code>></code> , <code>==</code> , <code>CompareTo</code>	hoofdstuk 4
tekst met <code>\$</code>	getallen opmaken in een tekst	hoofdstuk 1
wat elk object kan	<code>ToString</code> , <code>Equals</code> , <code>GetType</code>	hoofdstuk 13

Rekenen met een reeks getallen

Komt van LINQ. In de voorbeelden is `getallen` de lijst `5, 8, 3, 8, 1`.

Methode	Wat het doet	Voorbeeld	Resultaat
Sum	tel alles op	<code>getallen.Sum()</code>	25
Average	het gemiddelde, altijd als <code>double</code>	<code>getallen.Average()</code>	5
Min, Max	kleinste en grootste	<code>getallen.Max()</code>	8
Count	hoeveel voldoen	<code>getallen.Count(g => g % 2 == 0)</code>	2: de even getallen
Order, OrderDescending	gesorteerd	<code>getallen.Order()</code>	1, 3, 5, 8, 8
Distinct	elk getal één keer	<code>getallen.Distinct()</code>	5, 8, 3, 1
Enumerable.Range	een reeks maken	<code>Enumerable.Range(1, 10).Sum()</code>	55

Het gemiddelde van gehele getallen is een `double`: `new List<int> { 1, 2 }.Average()` geeft 1.5, ook al zijn het gehele getallen. Bij een lijst `decimal` is het gemiddelde een `decimal`.

Vergelijken en wat elk object kan

Methode	Wat het doet	Voorbeeld	Resultaat
CompareTo	welke is kleiner	<code>3.CompareTo(7)</code>	-1: negatief, want 3 is kleiner
Equals / ==	dezelfde waarde	<code>3 == 3</code>	<code>true</code>
GetType	welk type	<code>3.GetType().Name</code>	<code>Int32</code> ; 2.5 is <code>Double</code> , 2.5m is <code>Decimal</code>
ToString	als tekst	<code>3.ToString()</code>	"3"

Bij getallen vergelijkt `==` de **waarde**: twee getallen zijn gelijk als ze even groot zijn. Dat is anders dan bij een lijst of een object van een klasse.

Bestaat ook, maar kom je in deze cursus niet tegen

`float` (een minder nauwkeurige `double`), `short`, `byte`, `uint`, `ulong`, `BigInteger` (willekeurig grote getallen), `Math.Log`, `Math.Sin` en hun familie, `Math.PI`, `int.IsEvenInteger`, `int.Clamp` en de andere statische methodes die sinds .NET 7 rechtstreeks op `int`, `double` en `decimal` zitten.

Let op bij getallen

LET OP

- Twee gehele getallen delen geeft een geheel getal: `7 / 2` is `3`. Ook `-7 / 2` wordt naar nul toe afgekapt: `-3`, en `-7 % 2` is `-1`.
- `Math.Round` en `Convert.ToInt32` ronden een halve eenheid af naar het **even** getal: `Math.Round(2.5)` is `2`. (`int`) rondt helemaal niet af, maar knipt.
- `Parse` en `ToString` volgen de taal van de computer. Op een Belgische computer is de komma het decimaalteken: `double.Parse("2.5")` geeft `25`, en `2.5.ToString()` geeft `"2,5"`.
- `decimal` voor geld, `double` voor metingen. Ze door elkaar gebruiken compileert niet zonder omzetting.
- `getallen.Sum()` op een lijst `int` die samen boven `int.MaxValue` uitkomt, crasht met een **OverflowException**. Gewone optelling met `+` springt stil naar het minimum.

Voorspel de uitkomst

Vraag 1

```
Console.WriteLine(7 / 2);  
Console.WriteLine(7 % 2);
```

[Toon het antwoord ›](#)

Vraag 2

```
bool gelukt = int.TryParse("12a", out int getal);  
Console.WriteLine(gelukt);  
Console.WriteLine(getal);
```

[Toon het antwoord ›](#)

Vraag 3

```
Console.WriteLine(Math.Round(3.5));  
Console.WriteLine(Math.Round(4.5));
```

[Toon het antwoord ›](#)

Vraag 4

Op een computer met Belgische taalinstelling:

```
double d = double.Parse("2.5");  
Console.WriteLine(d);
```

[Toon het antwoord ›](#)

Vraag 5

```
Console.WriteLine(0.1 + 0.2 == 0.3);  
Console.WriteLine(0.1m + 0.2m == 0.3m);
```

[Toon het antwoord ›](#)

Vraag 6

```
decimal prijs = 10m;  
var metBtw = prijs * 1.21;
```

[Toon het antwoord ›](#)

Vraag 7

```
Console.WriteLine((int)2.9);  
Console.WriteLine(Convert.ToInt32(2.9));
```

[Toon het antwoord ›](#)

Vraag 8

```
int a = 10;  
int b = 4;  
Console.WriteLine((double)(a / b));  
Console.WriteLine((double)a / b);
```

[Toon het antwoord ›](#)

Vraag 9

```
var getallen = new List<int> { 1, 2 };  
Console.WriteLine(getallen.Average());  
Console.WriteLine(getallen.Sum() / getallen.Count);
```

[Toon het antwoord ›](#)

Vraag 10

```
int teller = int.MaxValue;  
teller++;  
Console.WriteLine(teller < 0);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

3 en daarna 1

Twee gehele getallen delen geeft een geheel getal: 2 past 3 keer in 7. Wat er overblijft, 1, krijg je met %.

[Terug naar de vraag](#)

VRAAG 2

False en daarna 0

"12a" is geen getal. TryParse crasht niet, geeft false terug, en zet getal op nul.

[Terug naar de vraag](#)

VRAAG 3

4 en daarna 4

Geen tikfout. C# rondt een halve eenheid af naar het dichtstbijzijnde **even** getal: 3,5 wordt 4, en 4,5 wordt ook 4. Wil je dat 4,5 een 5 wordt, dan geef je MidpointRounding.AwayFromZero mee.

[Terug naar de vraag](#)

VRAAG 4

25

Op een Belgische computer is de punt het teken voor duizendtallen, zoals in 1.000. double.Parse leest "2.5" dus als vijfentwintig, zonder foutmelding. Met double.Parse("2.5", CultureInfo.InvariantCulture) krijg je wel twee en een half. Zie de waarschuwing hierboven.

[Terug naar de vraag](#)

VRAAG 5

False en daarna True

Een double bewaart 0,1 en 0,2 niet exact, en de som ligt een heel klein beetje naast 0,3. Een decimal rekent met tientallen, zoals jij op papier, en komt wel precies op 0,3 uit. Daarom is een prijs een decimal.

[Terug naar de vraag](#)

VRAAG 6

Compileert niet.

1.21 zonder letter is een double, en C# weigert een decimal met een double te vermenigvuldigen. Het zou niet weten welk van de twee soorten het resultaat moet worden. Met 1.21m lukt het wel.

[Terug naar de vraag](#)

VRAAG 7

2 en daarna 3

`(int)` knipt alles na de komma weg. `Convert.ToInt32` rondt af.

[Terug naar de vraag](#)

VRAAG 8

2 en daarna 2.5

In de eerste regel wordt eerst geheel gedeeld: $10 / 4$ is 2. Pas daarna wordt die 2 een `double`. In de tweede regel is `a` al een `double` vóór het delen. Op een Belgische computer toont de tweede regel 2,5.

[Terug naar de vraag](#)

VRAAG 9

1.5 en daarna 1

`Average` geeft altijd een `double`. Zelf delen met twee gehele getallen geeft een geheel getal.

[Terug naar de vraag](#)

VRAAG 10

True

Een `int` kan niet groter dan `int.MaxValue`. Eén erbij springt naar het kleinste `int`, een negatief getal, zonder foutmelding.

[Terug naar de vraag](#)

3. Datum en tijd

METHODES

OPMERKING

In les 06 en 09 krijgen je modellen een datum, zoals de dag waarop iets aangemaakt werd. Daarvoor gebruik je `DateTime`.

Een datum maken

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>new DateTime(j, m, d)</code>	een vaste datum	<code>new DateTime(2026, 10, 3)</code>	3 oktober 2026, om middernacht
<code>new DateTime(j, m, d, u, min, s)</code>	een vaste datum met uur	<code>new DateTime(2026, 10, 3, 14, 30, 0)</code>	3 oktober 2026, 14.30 uur
<code>DateTime.Now</code>	nu, op de klok van de computer	<code>DateTime.Now</code>	het huidige moment
<code>DateTime.Today</code>	vandaag, om middernacht	<code>DateTime.Today</code>	de datum van vandaag, zonder uur
<code>DateTime.UtcNow</code>	nu, in wereltijd	<code>DateTime.UtcNow</code>	in België één of twee uur vroeger dan <code>Now</code>
<code>DateOnly</code>	enkel een datum, zonder uur	<code>new DateOnly(2026, 10, 3)</code>	3 oktober 2026
<code>TimeOnly</code>	enkel een uur, zonder datum	<code>new TimeOnly(14, 30)</code>	14.30 uur
<code>DateTime.MinValue</code> , <code>DateTime.MaxValue</code>	de vroegste en laatste mogelijke datum	<code>DateTime.MinValue</code>	1 januari van het jaar 1
<code>DateTime.DaysInMonth</code>	hoeveel dagen heeft die maand	<code>DateTime.DaysInMonth(2026, 2)</code>	28
<code>DateTime.IsLeapYear</code>	is het een schrikkeljaar	<code>DateTime.IsLeapYear(2028)</code>	true

Maanden tellen hier vanaf 1, niet vanaf 0 zoals de posities in een array. Januari is 1, december 12.

Een stuk eruit halen

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>Year</code>	het jaar	<code>d.Year</code>	2026
<code>Month</code>	de maand, van 1 tot 12	<code>d.Month</code>	10
<code>Day</code>	de dag van de maand	<code>d.Day</code>	3
<code>Hour</code> , <code>Minute</code>	het uur en de minuut	<code>d.Hour</code>	14
<code>DayOfWeek</code>	de dag van de week, als enum	<code>d.DayOfWeek</code>	Saturday

Vorm	Wat het doet	Voorbeeld	Resultaat
Date	dezelfde dag, met het uur op middernacht	<code>d.Date</code>	3 oktober 2026, 00.00 uur
Second, Millisecond	seconden en milliseconden	<code>d.Second</code>	0
DayOfYear	de hoeveelste dag van het jaar	<code>d.DayOfYear</code>	276
TimeOfDay	enkel het uur, als <code>TimeSpan</code>	<code>d.TimeOfDay</code>	14:30:00
Kind	lokale tijd, wereltijd, of niet gezegd	<code>d.Kind</code>	<code>Unspecified</code> ; <code>DateTime.Now</code> is Local, <code>DateTime.UtcNow</code> is Utc

`DayOfWeek` is een enum zoals in hoofdstuk 18, met Engelse namen: `Monday` tot en met `Sunday`. Dat blijft zo, ook op een Nederlandstalige computer.

Rekenen met datums

Vorm	Wat het doet	Voorbeeld	Resultaat
AddDays	een aantal dagen erbij, of eraf met een minteken	<code>d.AddDays(7)</code>	een week later
AddMonths	een aantal maanden erbij	<code>d.AddMonths(1)</code>	een maand later
AddHours	een aantal uren erbij	<code>d.AddHours(-2)</code>	twee uur vroeger
-	het verschil tussen twee datums	<code>eind - begin</code>	een <code>TimeSpan</code>
Days	het aantal hele dagen in een <code>TimeSpan</code>	<code>(eind - begin).Days</code>	14
TotalHours	het verschil in uren, met komma	<code>(eind - begin).TotalHours</code>	336
<, >, ==	vergelijken: welke komt eerst	<code>begin < eind</code>	true
AddYears, AddMinutes, AddSeconds	jaren, minuten, seconden erbij	<code>d.AddMinutes(45)</code>	15.15 uur
Add	een <code>TimeSpan</code> erbij	<code>d.Add(TimeSpan.FromHours(2))</code>	16.30 uur
Subtract	hetzelfde als -	<code>d.Subtract(begin)</code>	een <code>TimeSpan</code>
CompareTo	negatief, 0 of positief, zoals bij tekst	<code>d.CompareTo(new DateTime(2026, 1, 1))</code>	1: d komt later

Net als bij tekst veranderen deze methodes de datum zelf niet. `d.AddDays(1)` op een regel alleen doet niets; je moet het resultaat opvangen.

Een tijdsduur: TimeSpan

Het verschil tussen twee datums is een `TimeSpan`: een duur, geen moment. In de voorbeelden is `duur` het verschil tussen 1 oktober 0.00 uur en 15 oktober 6.00 uur.

Vorm	Wat het doet	Voorbeeld	Resultaat
Days, Hours, Minutes	de delen : hele dagen, dan de uren die overblijven ...	<code>duur.Days</code> , <code>duur.Hours</code>	14 en 6
TotalDays, TotalHours, TotalMinutes	de hele duur in één eenheid, met komma	<code>duur.TotalHours</code>	342
TimeSpan.FromMinutes, FromHours, FromDays	een duur maken	<code>TimeSpan.FromMinutes(90)</code>	01:30:00
new TimeSpan	een duur uit uren, minuten, seconden	<code>new TimeSpan(1, 30, 0).TotalMinutes</code>	90
TimeSpan.Zero	een duur van niets	<code>TimeSpan.Zero</code>	00:00:00
Duration	zonder minteken	<code>(vroeg - laat).Duration()</code>	een positieve duur
<, >, +, -	vergelijken en optellen	<code>TimeSpan.FromHours(1) < TimeSpan.FromMinutes(90)</code>	true

LET OP

Hours is niet hetzelfde als TotalHours. Bij 14 dagen en 6 uur is Hours 6 — enkel de uren bovenop de hele dagen — en TotalHours 342. En een verschil met de latere datum eerst is negatief: `(1 januari - 3 januari).Days` is -2.

Enkel een datum of enkel een uur: DateOnly en TimeOnly

Vorm	Wat het doet	Voorbeeld	Resultaat
DateOnly.FromDateTime	het datumdeel van een DateTime	<code>DateOnly.FromDateTime(d)</code>	3 oktober 2026
AddDays, AddMonths, AddYears	zoals bij DateTime	<code>new DateOnly(2026, 10, 3).AddDays(30)</code>	2 november 2026
DayNumber	dagnummer; zo reken je het verschil in dagen	<code>eind.DayNumber - begin.DayNumber</code>	het aantal dagen ertussen
ToDateTime	maak er weer een DateTime van, met een uur	<code>dag.ToDateTime(new TimeOnly(9, 0))</code>	3 oktober 2026, 9.00 uur
TimeOnly.AddMinutes, AddHours	rekenen met een uur; na middernacht begint het opnieuw	<code>new TimeOnly(14, 30).AddHours(10)</code>	00:30
IsBetween	ligt dit uur tussen twee andere	<code>uur.IsBetween(new TimeOnly(9, 0), new TimeOnly(17, 0))</code>	true voor 14.30

Twee DateOnly's van elkaar aftrekken met - kan niet. Gebruik DayNumber.

Tonen en inlezen

Vorm	Wat het doet	Voorbeeld	Resultaat
------	--------------	-----------	-----------

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>ToString("yyyy-MM-dd")</code>	toon in een vaste vorm	<code>d.ToString("yyyy-MM-dd")</code>	"2026-10-03"
<code>ToString("HH:mm")</code>	enkel het uur	<code>d.ToString("HH:mm")</code>	"14:30"
<code>DateTime.TryParse</code>	probeer een tekst om te zetten, volgens de taalinstelling	<code>DateTime.TryParse("03/10/2026", out DateTime d)</code>	op een Belgische computer: 3 oktober
<code>DateTime.ParseExact</code>	zet om volgens een vorm die jij opgeeft	<code>DateTime.ParseExact("2026-10-03", "yyyy-MM-dd", CultureInfo.InvariantCulture)</code>	3 oktober 2026
<code>ToString("dddd")</code>	de naam van de dag, in de taal van de computer	<code>d.ToString("dddd")</code>	"zaterdag" op een Nederlandstalige computer
<code>ToString("d MMMM yyyy")</code>	met de naam van de maand	<code>d.ToString("d MMMM yyyy")</code>	"3 oktober 2026"
<code>ToString("d"), ToShortDateString</code>	de korte datum van de taalinstelling	<code>d.ToString("d")</code>	"3/10/2026" op een Belgische computer
<code>DateTime.TryParseExact</code>	probeer om te zetten volgens jouw vorm, zonder te crashen	<code>DateTime.TryParseExact("03-10-2026", "dd-MM-yyyy", CultureInfo.InvariantCulture, DateTimeStyles.None, out DateTime d)</code>	true

LET OP

In een opmaak betekent `MM` maand en `mm` minuten. `HH` is het uur van 0 tot 23, `hh` het uur van 1 tot 12. Eén verkeerde hoofdletter geeft een datum die er geloofwaardig uitziet en toch fout is.

En net als bij `double.Parse` in hoofdstuk 2 leest `DateTime.TryParse` volgens de taalinstelling. "03/10/2026" is in België 3 oktober, in de Verenigde Staten 10 maart. Ken je de vorm, gebruik dan `ParseExact`.

Wat het ook kan, via iets anders

Een `DateTime` is, net als een getal, een **waarde** (struct, hoofdstuk 15). Twee datums met hetzelfde moment zijn gelijk. Wat er via iets anders bij komt:

Bron	Wat het geeft	Volledige uitleg
LINQ op een reeks datums	Min, Max, Order, Where, GroupBy	hoofdstuk 11
<code>DayOfWeek</code> is een enum	vergelijken, switch, omzetten naar een getal	hoofdstuk 18
vergelijken	<, >, ==, CompareTo	hoofdstuk 4
tekst met \$	<code>\$"{d:dd/MM/yyyy}"</code>	hoofdstuk 1
wat elk object kan	<code>ToString</code> , <code>Equals</code> , <code>GetType</code>	hoofdstuk 13

Rekenen met een reeks datums

Komt van LINQ. In de voorbeelden is `datums` de lijst 1 maart 2026, 24 december 2025 en 3 oktober 2026.

Methode	Wat het doet	Voorbeeld	Resultaat
Max, Min	de laatste of vroegste datum	<code>datums.Max()</code>	3 oktober 2026
Order, OrderDescending	gesorteerd, van vroeg naar laat	<code>datums.Order()</code>	24 december 2025, 1 maart 2026, 3 oktober 2026
Count, Where	datums die voldoen	<code>datums.Count(d => d.Year == 2026)</code>	2
GroupBy	groeperen per jaar, maand of dag	<code>datums.GroupBy(d => d.Year)</code>	2026: 2 datums, 2025: 1
Contains	zit dit moment erin	<code>datums.Contains(new DateTime(2026, 3, 1))</code>	true
MaxBy, OrderBy	met objecten: op hun datum	<code>bestellingen.OrderBy(b => b.Datum)</code>	oudste bestelling eerst

LET OP

`Sum` en `Average` bestaan niet voor datums: `datums.Average()` compileert niet. Een reeks `TimeSpan` optellen doe je met `duren.Aggregate(TimeSpan.Zero, (a, b) => a + b)`.

Vergelijken en wat elk object kan

Methode	Voorbeeld	Resultaat
<code>==</code>	<code>new DateTime(2026, 10, 3) == new DateTime(2026, 10, 3, 14, 30, 0)</code>	false: middernacht is niet 14.30 uur
<code>Date + ==</code>	<code>d.Date == new DateTime(2026, 10, 3)</code>	true: enkel de dag vergeleken
<code>GetType</code>	<code>d.GetType().Name</code>	DateTime
<code>ToString</code>	<code>d.ToString()</code>	volgens de taalinstelling, bv. 3/10/2026 14:30:00
<code>(int) op DayOfWeek</code>	<code>(int)d.DayOfWeek</code>	6; zondag is 0, niet 7

Bestaat ook, maar kom je in deze cursus niet tegen

`DateTimeOffset` (een moment mét tijdzone), `TimeZoneInfo` (omrekenen tussen tijdzones), `Calendar` en `ISOWeek` (weeknummers), `DateTime.SpecifyKind`, `ToLocalTime`, `ToUniversalTime`.

Let op bij datums

LET OP

- Een `DateTime` die je niet invult, is niet leeg maar 1 januari van het jaar 1. Ook `FirstOrDefault` op een lege lijst datums geeft die datum, geen `null`. Wil je dat een datum kan ontbreken, gebruik dan `DateTime?` (hoofdstuk 17).
- Zondag is dag 0 van `DayOfWeek`, niet 7. Wie op maandag als eerste dag rekent, moet daar rekening mee houden.
- `DateTime.Now` is de lokale tijd van de computer; `DateTime.UtcNow` de wereltijd. Een PostgreSQL-database met EF Core aanvaardt in een kolom met tijdzone enkel wereltijd (`Kind` is `Utc`): een `DateTime.Now` bewaren daar crasht. Gebruik `DateTime.UtcNow`, of zet de kolom op een datum zonder tijdzone. Zie hoofdstuk 31.
- `==` vergelijkt tot op de tick: twee keer `DateTime.Now` na elkaar is bijna nooit gelijk. Vergelijk met `.Date` als je enkel de dag bedoelt.

Voorspel de uitkomst

Vraag 1

```
var d = new DateTime(2026, 1, 31);
Console.WriteLine(d.AddMonths(1).Day);
```

Toon het antwoord ›

Vraag 2

```
var begin = new DateTime(2026, 10, 1);
var eind = new DateTime(2026, 10, 15);
Console.WriteLine((eind - begin).Days);
```

Toon het antwoord ›

Vraag 3

```
var d = new DateTime(2026, 10, 3);
d.AddDays(1);
Console.WriteLine(d.Day);
```

Toon het antwoord ›

Vraag 4

```
var d = new DateTime(2026, 10, 3, 14, 5, 0);
Console.WriteLine(d.ToString("yyyy-mm-dd"));
```

Toon het antwoord ›

Vraag 5

```
var d = new DateTime(2026, 2, 29);
```

[Toon het antwoord ›](#)

Vraag 6

```
var begin = new DateTime(2026, 10, 1);  
var eind = new DateTime(2026, 10, 15, 6, 0, 0);  
var duur = eind - begin;  
Console.WriteLine(duur.Hours);  
Console.WriteLine(duur.TotalHours);
```

[Toon het antwoord ›](#)

Vraag 7

```
var datums = new List<DateTime>();  
var eerste = datums.FirstOrDefault();  
Console.WriteLine(eerste.Year);
```

[Toon het antwoord ›](#)

Vraag 8

```
var d = new DateTime(2026, 10, 4); // een zondag  
Console.WriteLine((int)d.DayOfWeek);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

28

Februari 2026 heeft geen 31ste. `AddMonths` blijft dan op de laatste dag van die maand staan, in plaats van door te schuiven naar maart.

[Terug naar de vraag](#)

VRAAG 2

14

Van de eerste tot de vijftiende zijn het veertien dagen. Het verschil tussen twee datums is een `TimeSpan`, en `Days` geeft daarvan het aantal hele dagen.

[Terug naar de vraag](#)

VRAAG 3

3

`AddDays` geeft een nieuwe datum terug en laat `d` met rust, net zoals `Trim` bij tekst. Wil je dat het werkt, dan schrijf je `d = d.AddDays(1);`.

[Terug naar de vraag](#)

VRAAG 4

2026-05-03

`mm` met kleine letters zijn de minuten, en dat is hier 5. Voor de maand had er `MM` moeten staan.

[Terug naar de vraag](#)

VRAAG 5

`ArgumentOutOfRangeException` — het programma crasht.

2026 is geen schrikkeljaar, dus 29 februari bestaat niet. Het compileert wel, want de compiler rekent geen kalenders na. De fout komt pas als de regel draait.

[Terug naar de vraag](#)

VRAAG 6

6 en daarna 342

`Hours` geeft enkel de uren bovenop de 14 hele dagen. `TotalHours` zet de hele duur om naar uren: 14 maal 24 plus 6.

[Terug naar de vraag](#)

VRAAG 7

1

Een `DateTime` kan niet `null` zijn. `FirstOrDefault` op een lege lijst geeft de standaardwaarde: 1 januari van het jaar 1.

[◀ Terug naar de vraag](#)

VRAAG 8

0

In `DayOfWeek` is zondag de eerste dag, met nummer 0. Maandag is 1, zaterdag 6.

[◀ Terug naar de vraag](#)

4. Beslissen en herhalen

METHODES

OPMERKING

Ken je dit al uit een andere taal, lees dan vooral de tabellen en de waarschuwingen. Een paar dingen werken in C# net iets anders, zoals de switch-expressie.

Vergelijken en combineren

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>==</code>	is gelijk aan	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	is niet gelijk aan	<code>5 != 3</code>	<code>true</code>
<code><, <=</code>	kleiner dan, kleiner dan of gelijk aan	<code>3 <= 3</code>	<code>true</code>
<code>>, >=</code>	groter dan, groter dan of gelijk aan	<code>2 > 3</code>	<code>false</code>
<code>&&</code>	en: allebei waar	<code>leeftijd >= 3 && leeftijd <= 17</code>	<code>true</code> voor 3 tot en met 17
<code> </code>	of: minstens één waar	<code>dag == "za" dag == "zo"</code>	<code>true</code> in het weekend
<code>!</code>	niet: draait om	<code>!gevonden</code>	<code>true</code> als er niets gevonden is

`&&` en `||` stoppen zodra het antwoord vastligt. Bij `s != null && s.Length > 3` wordt de lengte enkel opgevraagd als `s` niet `null` is. Dat is geen toeval: zo schrijf je een controle die niet crasht.

Iets doen als...

```
int leeftijd = 15;

if (leeftijd < 3)
{
    Console.WriteLine("Gratis");
}
else if (leeftijd < 18)
{
    Console.WriteLine("Kindertarief");
}
else
{
    Console.WriteLine("Volle prijs");
}
```

C# loopt de voorwaarden van boven naar onder af en neemt de **eerste** die klopt. De rest wordt niet meer bekeken. De volgorde van je `else if` bepaalt dus het antwoord.

Voor een keuze tussen twee waarden bestaat er een korte vorm:

```
string tekst = leeftijd >= 18 ? "volwassen" : "minderjarig";
```

Lees het als: *klopt de voorwaarde? Dan het eerste, anders het tweede.*

Kiezen uit veel mogelijkheden: switch

Vergelijk je één waarde met een reeks vaste mogelijkheden, dan is een `switch` overzichtelijker dan een lange rij `else if`:

```
string dag = "za";

switch (dag)
{
    case "za":
    case "zo":
        Console.WriteLine("Weekend");
        break;
    case "wo":
        Console.WriteLine("Halve dag");
        break;
    default:
        Console.WriteLine("Weekdag");
        break;
}
```

Moet er gewoon een waarde uitkomen, dan is de **switch-expressie** korter. Ze werkt ook met grenzen:

```
int graden = 25;

string advies = graden switch
{
    < 10 => "Neem een jas",
    < 20 => "Neem een trui",
    _ => "Een t-shirt volstaat"
};
```

De `_` betekent *al de rest*. Ook hier geldt: de eerste die past, wint.

Herhalen

```
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(i);           // 0, 1, 2
}

int teller = 10;
while (teller > 0)
{
    teller -= 3;                    // 7, 4, 1, -2: dan stopt het
}

foreach (string naam in namen)
{
    // ...
}
```

```
Console.WriteLine(naam);    // elk item één keer
}
```

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
if / else if / else	doe iets als een voorwaarde klopt	<code>if (x > 0) { ... }</code>	de eerste die klopt, wint
? :	kies tussen twee waarden	<code>x > 0 ? "plus" : "min"</code>	"plus" of "min"
switch	voer een blok uit per mogelijkheid	<code>case "za": ... break;</code>	elk blok eindigt met break
switch-expressie	geef een waarde per mogelijkheid	<code>x switch { 1 => "een", _ => "veel" }</code>	"een" of "veel"
< 10 =>	een grens in een switch-expressie	<code>< 10 => "koud"</code>	alles onder 10
and, or	grenzen combineren in een switch-expressie	<code>>= 3 and <= 17 => "kind"</code>	3 tot en met 17
_ =>	al de rest	<code>_ => "onbekend"</code>	wat nergens anders past
for	herhaal een vast aantal keer	<code>for (int i = 0; i < 5; i++)</code>	i gaat van 0 tot 4
while	herhaal zolang iets klopt	<code>while (teller > 0)</code>	misschien nul keer
do ... while	herhaal, en kijk pas achteraf	<code>do { ... } while (x > 0);</code>	minstens één keer
foreach	elk item van een verzameling	<code>foreach (var n in namen)</code>	elk item één keer
break	stop de lus nu	<code>if (gevonden) break;</code>	de lus is voorbij
continue	sla de rest van deze ronde over	<code>if (x < 0) continue;</code>	door naar de volgende ronde

Wat het ook kan, via iets anders

Welke lus op welke verzameling

Verzameling	foreach	for met [i]	iets veranderen terwijl foreach loopt
tekst (hoofdstuk 1)	elk teken, als char	ja	kan niet: een tekst verander je nooit
array (hoofdstuk 5)	ja	ja	<code>getallen[0] = 9</code> mag; de lus ziet de nieuwe waarde
List (hoofdstuk 6)	ja	ja	Add, Remove en zelfs <code>lijst[0] = 9</code> geven InvalidOperationException
Dictionary (hoofdstuk 7)	elk paar, als <code>KeyValuePair</code>	nee	Add crasht; Remove en een waarde overschrijven mogen
HashSet (hoofdstuk 8)	ja	nee	Add van iets nieuws crasht; Remove mag
Queue, Stack (hoofdstuk 9, 10)	ja; een stapel van boven naar onder	nee	Enqueue, Dequeue, Push en Pop crashen
een resultaat van LINQ (hoofdstuk 11, 12)	ja	nee	—

De variabele van een `foreach` zelf kan je nooit een nieuwe waarde geven: `foreach (var x in getallen) { x = 0; }` compileert niet. Moet je aanpassen, gebruik dan een `for` met `[i]`. Moet je uit een lijst verwijderen terwijl je ze overloopt, loop dan achteruit: `for (int i = lijst.Count - 1; i >= 0; i--)`, of gebruik `RemoveAll` uit hoofdstuk 6.

Lussen die LINQ voor je schrijft

Komt van LINQ. Veel lussen met een `if` erin bestaan al als één methode.

In plaats van een lus met ...	Kan ook met	Resultaat
een <code>if</code> en een teller	<code>getallen.Count(x => x > 4)</code>	het aantal
een <code>if</code> en <code>break</code> zodra je iets vindt	<code>getallen.Any(x => x > 4)</code> of <code>FirstOrDefault(x => x > 4)</code>	<code>true/false</code> , of het eerste
een nieuwe lijst die je vult	<code>getallen.Where(x => x > 4).ToList()</code>	een nieuwe lijst
een som die je bijhoudt	<code>getallen.Sum()</code>	de som
<code>for (int i = 1; i <= 5; i++)</code> om getallen te maken	<code>Enumerable.Range(1, 5)</code>	1 tot en met 5
een eigen teller voor de plaats in een <code>foreach</code>	<code>foreach (var (i, x) in lijst.Index())</code>	plaats en item samen

Zie hoofdstuk 11 voor alle methodes. Een `switch` werkt ook op een enum (hoofdstuk 18), en `is` kijkt van welk type iets is (hoofdstuk 22).

Veelgemaakte fouten

LET OP

- `=` geeft een waarde, `==` vergelijkt. `if (x = 5)` compileert in C# niet, want een toewijzing is geen ja of nee.
- Een `switch`-expressie zonder `_` compileert met een waarschuwing. Past de waarde nergens, dan crasht ze terwijl het programma draait.
- `for (int i = 0; i <= lijst.Count; i++)` loopt één keer te veel. De laatste positie is `Count - 1`, dus het is `<`, niet `<=`.
- Een `while` waarvan de voorwaarde nooit vals wordt, stopt nooit. Kijk na of er in de lus iets verandert aan wat je controleert.
- Teksten vergelijken met `==` let op hoofdletters. Komt een waarde van een gebruiker, denk dan aan `Equals` met `OrdinalIgnoreCase` uit hoofdstuk 1.

Voorspel de uitkomst

Vraag 1

```
int leeftijd = 70;

if (leeftijd >= 18)
{
```



```
        Console.WriteLine("volwassen");
    }
    else if (leeftijd >= 65)
    {
        Console.WriteLine("senior");
    }
    else
    {
        Console.WriteLine("kind");
    }
}
```

[Toon het antwoord ›](#)

Vraag 2

```
int graden = 15;

string advies = graden switch
{
    < 10 => "jas",
    < 20 => "trui",
    _ => "t-shirt"
};

Console.WriteLine(advies);
```

[Toon het antwoord ›](#)

Vraag 3

```
for (int i = 1; i <= 5; i++)
{
    if (i == 2)
    {
        continue;
    }

    if (i == 4)
    {
        break;
    }

    Console.Write(i);
}
```

[Toon het antwoord ›](#)

Vraag 4

```
int n = 10;

do
{
```

```
    Console.WriteLine(n);  
}  
while (n < 5);
```

[Toon het antwoord ›](#)

Vraag 5

```
int x = 5;  
  
string woord = x switch  
{  
    1 => "een",  
    2 => "twee"  
};  
  
Console.WriteLine(woord);
```

[Toon het antwoord ›](#)

Vraag 6

```
int[] reeks = { 1, 2, 3 };  
foreach (int x in reeks) { reeks[0] = 9; }  
Console.WriteLine(reeks[0]);  
  
var lijst = new List<int> { 1, 2, 3 };  
foreach (int x in lijst) { lijst[0] = 9; }  
Console.WriteLine(lijst[0]);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

volwassen

70 is groter dan 18, dus de eerste voorwaarde klopt al. De tweede wordt niet meer bekeken, ook al zou die ook kloppen. Wil je dat senioren apart behandeld worden, dan moet de controle op 65 eerst komen.

[Terug naar de vraag](#)

VRAAG 2

trui

15 is niet kleiner dan 10, maar wel kleiner dan 20. De tweede regel past als eerste.

[Terug naar de vraag](#)

VRAAG 3

13

Bij 1 wordt er geschreven. Bij 2 slaat `continue` de rest over. Bij 3 wordt er weer geschreven. Bij 4 stopt `break` de hele lus, dus 4 en 5 komen er nooit. `Console.WriteLine` zet alles op één regel.

[Terug naar de vraag](#)

VRAAG 4

10

Een `do-lus` voert het blok eerst uit en kijkt pas daarna naar de voorwaarde. 10 is niet kleiner dan 5, dus het blijft bij die ene keer. Met een gewone `while` was er niets op het scherm gekomen.

[Terug naar de vraag](#)

VRAAG 5

`SwitchExpressionException` — het programma crasht.

Het compileert, met een waarschuwing dat niet elke waarde gedekt is. Als het programma draait en `x` is 5, past er geen enkele regel, en dan weet C# niet wat `woord` moet worden. Een laatste regel met `_` voorkomt dat.

[Terug naar de vraag](#)

VRAAG 6

9, en daarna **`InvalidOperationException`** — het programma crasht in de tweede lus.

Een array mag je aanpassen terwijl je hem overloopt. Een `List` houdt bij of ze veranderd is, en dan stopt de `foreach`. Ook een waarde vervangen telt als veranderen.

[Terug naar de vraag](#)

Deel 2 — Verzamelingen

Zes manieren om meerdere dingen samen te bewaren. Ze verschillen in wat ze toelaten en in hoe snel je iets terugvindt.

De verzamelingen naast elkaar

	array	List	Dictionary	HashSet	Queue	Stack
Grootte	vast	groeit	groeit	groeit	groeit	groeit
Volgorde bewaard	ja	ja	niet gegarandeerd	niet gegarandeerd	ja, eerst in eerst uit	ja, laatst in eerst uit
Dubbels	mag	mag	sleutels nee, waarden wel	nee	mag	mag
Opvragen op	positie	positie	sleutel	—	enkel vooraan	enkel bovenaan
Snel nagaan of iets erin zit	nee	nee	ja, op sleutel	ja	nee	nee
Hoeveel erin zitten	Length	Count	Count	Count	Count	Count
foreach	ja	ja	ja, per paar	ja	ja, van voor naar achter	ja, van boven naar onder
for met [i]	ja	ja	nee	nee	nee	nee
Aanpassen tijdens een foreach	een waarde vervangen mag	crasht	Add crasht; Remove en overschrijven mogen	Add crasht; Remove mag	crasht	crasht
LINQ	ja	ja	ja, op de paren	ja	ja	ja

Die laatste rij verdient uitleg. Een `List` met tienduizend items moet bij `Contains` in het slechtste geval alle tienduizend nakijken. Een `Dictionary` of `HashSet` weet meteen waar hij moet kijken, hoe groot hij ook is. Bij tien items merk je dat niet. Bij een miljoen wel.

Elke verzameling kan ook alles van LINQ uit [hoofdstuk 11](#). Elk hoofdstuk hieronder toont daarom eerst wat de verzameling zelf kan, en daarna wat ze kan via iets anders, met de voorbeelden op dezelfde gegevens. Zo lees je per verzameling alles op één plaats.

5. Array

VERZAMELING

Wat het is

Een rij vakjes met een vaste lengte. Zoals een eierdoos: zes plaatsen, niet meer en niet minder. Je kan een ei vervangen, maar je kan er geen zevende plaats bij maken.

Hoe het eruitziet

```
string[] dagen = { "ma", "di", "wo", "do", "vr" };

Console.WriteLine(dagen[0]);           // ma
Console.WriteLine(dagen.Length);       // 5

dagen[4] = "vrijdag";                  // vervangen mag

int[] scores = new int[3];              // drie vakjes, allemaal 0

foreach (string dag in dagen)
{
    Console.WriteLine(dag);
}
```

Je zal ook de kortere schrijfwijze `int[] getallen = [1, 2, 3];` tegenkomen. Die doet hetzelfde.

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens deze drie arrays, zodat je elk resultaat kan narekenen.

```
string[] fruit    = { "appel", "peer", "kiwi" };
int[]   getallen  = { 5, 8, 3, 8, 1 };
Fan[]   fans      =
{
    new Fan { Id = 1, Merk = "NZXT", Prijs = 24.99m },
    new Fan { Id = 2, Merk = "Noctua", Prijs = 29.99m },
    new Fan { Id = 3, Merk = "NZXT", Prijs = 89.99m },
    new Fan { Id = 4, Merk = "Arctic", Prijs = 19.99m }
};
```

`Fan` is een klasse met drie eigenschappen. Hoe je zelf zo'n klasse maakt, staat in [hoofdstuk 13](#).

Wat je ermee kan

Een array heeft zelf maar een handvol eigenschappen en methodes. Het meeste werk doet de klasse `Array`: je schrijft niet `getallen.Sort()` maar `Array.Sort(getallen)`. De array is een bouwsteen van de taal zelf, ouder dan `List`, en de hulpmethodes zitten daarom apart: je geeft de array mee tussen de haakjes.

Op de array zelf

Methode	Wat het doet	Voorbeeld	Resultaat
Length	hoeveel vakjes; een eigenschap	<code>getallen.Length</code>	5
[i]	lees of vervang het vakje op positie i	<code>fruit[1]</code>	peer
CopyTo	kopieer alles naar een andere array, vanaf een positie	<code>fruit.CopyTo(doe1, 2)</code>	doe1 met vijf vakjes wordt null , null , appel, peer, kiwi
Clone	maak een kopie	<code>(int[])getallen.Clone()</code>	een nieuwe array 5, 8, 3, 8, 1
Rank	hoeveel dimensies	<code>getallen.Rank</code>	1
GetLength	hoeveel vakjes in één dimensie	<code>getallen.GetLength(0)</code>	5
GetUpperBound	de laatste positie in één dimensie	<code>getallen.GetUpperBound(0)</code>	4
GetLowerBound	de eerste positie in één dimensie	<code>getallen.GetLowerBound(0)</code>	0

`Clone` geeft een object terug. Daarom staat er `(int[])` voor: je zegt de compiler dat het een array van getallen is.

Zoeken, via de klasse Array

Methode	Wat het doet	Voorbeeld	Resultaat
Array.IndexOf	op welke positie staat het eerst	<code>Array.IndexOf(getallen, 8)</code>	1, of -1
Array.LastIndexOf	op welke positie staat het laatst	<code>Array.LastIndexOf(getallen, 8)</code>	3, of -1
Array.Exists	is er minstens één die voldoet	<code>Array.Exists(fans, f => f.Prijs > 50)</code>	true
Array.TrueForAll	voldoen ze allemaal	<code>Array.TrueForAll(fans, f => f.Prijs > 20)</code>	false: de Arctic kost minder
Array.Find	de eerste die voldoet	<code>Array.Find(fans, f => f.Id == 3)</code>	de fan met Id 3, of null
Array.FindLast	de laatste die voldoet	<code>Array.FindLast(fans, f => f.Merk == "NZXT")</code>	de fan met Id 3
Array.FindAll	allemaal die voldoen	<code>Array.FindAll(fans, f => f.Merk == "NZXT")</code>	een nieuwe array met fan 1 en 3
Array.FindIndex	positie van de eerste die voldoet	<code>Array.FindIndex(fans, f => f.Id == 3)</code>	2, of -1
Array.FindLastIndex	positie van de laatste die voldoet	<code>Array.FindLastIndex(fans, f => f.Merk == "NZXT")</code>	2, of -1
Array.BinarySearch	zoek snel in een gesorteerde array	<code>Array.BinarySearch(gesorteerd, 8)</code>	de positie, of een negatief getal

Ordenen, vullen en kopiëren, via de klasse Array

Methode	Wat het doet	Voorbeeld	Resultaat
---------	--------------	-----------	-----------

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Array.Sort</code>	sorteer de array zelf	<code>Array.Sort(getallen)</code>	1, 3, 5, 8, 8
<code>Array.Sort</code>	sorteer op iets dat je zelf kiest	<code>Array.Sort(fans, (a, b) => a.Prijs.CompareTo(b.Prijs))</code>	goedkoopste eerst
<code>Array.Sort</code>	sorteer twee arrays samen, op de eerste	<code>Array.Sort(scores, namen)</code>	de namen staan in de volgorde van hun score
<code>Array.Reverse</code>	keer de array zelf om	<code>Array.Reverse(getallen)</code>	1, 8, 3, 8, 5
<code>Array.Fill</code>	zet overal dezelfde waarde	<code>Array.Fill(scores, 10)</code>	10, 10, 10
<code>Array.Clear</code>	zet alles terug op de standaardwaarde	<code>Array.Clear(getallen)</code>	0, 0, 0, 0, 0; de lengte blijft 5
<code>Array.Copy</code>	kopieer een aantal vakjes naar een andere array	<code>Array.Copy(getallen, doel, 3)</code>	doel begint met 5, 8, 3
<code>Array.Resize</code>	maak een nieuwe array met een andere lengte	<code>Array.Resize(ref fruit, 5)</code>	appel, peer, kiwi, null, null
<code>Array.ConvertAll</code>	maak van elk item iets anders	<code>Array.ConvertAll(fans, f => f.Merk)</code>	een nieuwe <code>string[]</code> met de merken
<code>Array.ForEach</code>	doe iets met elk item	<code>Array.ForEach(fruit, f => Console.WriteLine(f))</code>	drie regels op het scherm
<code>Array.AsReadOnly</code>	een versie die niemand kan veranderen	<code>Array.AsReadOnly(fruit)</code>	vervangen kan niet meer
<code>Array.Empty</code>	een lege array	<code>Array.Empty<int>()</code>	een array met lengte 0

`Array.Sort`, `Array.Reverse`, `Array.Fill` en `Array.Clear` veranderen de array die je meegeeft. Ze geven niets terug.

BELANGRIJK

`Array.Resize` maakt de array niet groter: dat kan niet. Hij maakt een **nieuwe** array met de nieuwe lengte, kopieert de inhoud, en laat je variabele naar die nieuwe array wijzen. Daarom staat er `ref` voor. Moet je vaak iets bijvoegen, neem dan een `List`.

OPMERKING

In .NET 10 mag je sommige van deze methodes ook rechtstreeks op de array schrijven, zoals `getallen.Sort()` of `getallen.IndexOf(8)`. Ze komen dan uit `MemoryExtensions`, de hulpmethodes voor `Span`, en doen hetzelfde als `Array.Sort` en `Array.IndexOf`. In de cursus en in de meeste voorbeelden zie je de schrijfwijze met `Array..`

Meerdere dimensies

Een array kan ook een rooster zijn: rijen en kolommen.

```
int[,] rooster = new int[2, 3];    // 2 rijen, 3 kolommen, allemaal 0
rooster[1, 2] = 7;
```

```

Console.WriteLine(rooster.Rank);           // 2
Console.WriteLine(rooster.Length);        // 6 – alle vakjes samen
Console.WriteLine(rooster.GetLength(0));   // 2 – rijen
Console.WriteLine(rooster.GetLength(1));   // 3 – kolommen

int[][] rijen = new int[2][];             // een array van arrays
rijen[0] = new int[] { 1, 2 };
rijen[1] = new int[] { 3, 4, 5 };         // elke rij mag een andere lengte hebben
Console.WriteLine(rijen[1][2]);           // 5

```

`Length` telt bij een rooster alle vakjes samen. Wil je het aantal rijen of kolommen, dan heb je `GetLength` nodig.

Wat het ook kan, via iets anders

Een array kan veel meer dan de methodes hierboven, omdat ze ook iets anders ís. Ze is een **reeks**: iets wat je één voor één kan doorlopen. In C# heet dat `IEnumerable` ([hoofdstuk 12](#)). Alles wat met een reeks werkt, werkt dus ook met een array.

Bron	Waarom het op een array werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
<code>foreach</code>	<code>foreach</code> doorloopt elke reeks	hoofdstuk 4
<code>^</code> en <code>..</code> tussen vierkante haken	een array heeft een lengte en posities	hieronder
<code>string.Join</code>	wil een reeks teksten of waarden	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 6 tot 10
wat elk object kan	een array is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je array nooit**. Ze geeft een nieuw resultaat.
- Dat resultaat is meestal **geen array**, maar een reeks. Wil je er weer een array van, zet er dan `.ToArray()` achter. Een lijst krijg je met `.ToList()`.
- `Where`, `Select`, `OrderBy` en hun familie rekenen pas op het moment dat je het resultaat gebruikt. Verandert je array tussendoor, dan rekenen ze met de nieuwe inhoud.
- LINQ werkt enkel met `using System.Linq;` bovenaan. In een nieuw project staat die er automatisch bij.

OPMERKING

`Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken. In oudere voorbeelden op internet kom je ze niet tegen.

Eén item nemen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
First	de eerste die voldoet	<code>fans.First(f => f.Merk == "NZXT")</code>	fan 1; niets gevonden geeft een InvalidOperationException	—
FirstOrDefault	de eerste die voldoet, of niets	<code>fans.FirstOrDefault(f => f.Id == 9)</code>	<code>null</code>	<code>Array.Find</code>
Single	de enige die voldoet	<code>fans.Single(f => f.Id == 2)</code>	fan 2; nul of twee gevonden geeft een InvalidOperationException	—
SingleOrDefault	de enige die voldoet, of niets	<code>fans.SingleOrDefault(f => f.Merk == "NZXT")</code>	InvalidOperationException: er zijn er twee	—
Last	de laatste die voldoet	<code>fans.Last(f => f.Merk == "NZXT")</code>	fan 3	—
LastOrDefault	de laatste die voldoet, of niets	<code>fans.LastOrDefault(f => f.Prijs > 100)</code>	<code>null</code>	<code>Array.FindLast</code>
ElementAt	het item op een positie	<code>fans.ElementAt(1)</code>	fan 2	<code>fans[1]</code>
ElementAtOrDefault	het item op een positie, of niets	<code>fans.ElementAtOrDefault(9)</code>	<code>null</code>	—

Neem `FirstOrDefault` wanneer het normaal is dat er niets gevonden wordt, zoals een `Id` dat niet bestaat. Neem `SingleOrDefault` wanneer er hoogstens één mag zijn: de uitzondering bij twee is dan een alarm over je gegevens. Neem `First` alleen wanneer je zeker weet dat er iets is.

Weten of iets erin zit

Komt van *LINQ*.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
Any	is er minstens één	<code>fans.Any()</code>	<code>true</code>	<code>fans.Length > 0</code>

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
Any	is er minstens één die voldoet	<code>fans.Any(f => f.Prijs > 50)</code>	true	<code>Array.Exists</code>
All	voldoen ze allemaal	<code>fans.All(f => f.Prijs > 20)</code>	false	<code>Array.TrueForAll</code>
Contains	zit deze waarde erin	<code>getallen.Contains(3)</code>	true	<code>Array.IndexOf(...)</code> <code>>= 0</code>
SequenceEqual	hebben twee reeksen dezelfde inhoud, in dezelfde volgorde	<code>getallen.SequenceEqual(new[] { 5, 8, 3, 8, 1 })</code>	true	—

LET OP

All op een lege array geeft **true**: er is geen enkel item dat níet voldoet. Controleer eerst of er wel iets in zit.

Een deel nemen

Komt van *LINQ*. Het resultaat is telkens een nieuwe reeks; je array blijft zoals ze was.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
Where	enkel wat voldoet	<code>fans.Where(f => f.Merk == "NZXT")</code>	fan 1 en 3	<code>Array.FindAll</code>
Select	maak van elk item iets anders	<code>fans.Select(f => f.Merk)</code>	NZXT, Noctua, NZXT, Arctic	<code>Array.ConvertAll</code>
SelectMany	plat een lijst van lijsten	<code>klassen.SelectMany(k => k.Leerlingen)</code>	alle leerlingen samen	—
Distinct	gooi dubbels weg	<code>getallen.Distinct()</code>	5, 8, 3, 1	—
DistinctBy	één item per waarde van iets	<code>fans.DistinctBy(f => f.Merk)</code>	fan 1, 2 en 4	—
Take	de eerste n	<code>getallen.Take(2)</code>	5, 8	<code>getallen[..2]</code>
Skip	sla de eerste n over	<code>getallen.Skip(2)</code>	3, 8, 1	<code>getallen[2..]</code>
TakeLast	de laatste n	<code>getallen.TakeLast(2)</code>	8, 1	<code>getallen[^2..]</code>
SkipLast	alles behalve de laatste n	<code>getallen.SkipLast(2)</code>	5, 8, 3	<code>getallen[..^2]</code>
TakeWhile	neem zolang de voorwaarde klopt, stop bij de eerste die niet klopt	<code>getallen.TakeWhile(g => g > 2)</code>	5, 8, 3, 8	—
SkipWhile	sla over zolang de voorwaarde klopt	<code>getallen.SkipWhile(g => g > 2)</code>	1	—
Chunk	verdeel in stukken van n	<code>getallen.Chunk(2)</code>	[5, 8], [3, 8], [1], elk stuk een array	—

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
OfType	enkel de items van een bepaald type	<code>onderdelen.OfType<Processor>()</code>	enkel de processors	—
Index	elk item samen met zijn positie	<code>fruit.Index()</code>	<code>(0, appel), (1, peer), (2, kiwi)</code>	—

`OfType` heb je nodig bij overerving: een array van een basisklasse waar verschillende soorten in zitten. Zie [hoofdstuk 20](#).

Tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
Count	hoeveel er zijn	<code>fans.Count()</code>	4	<code>Length</code>
Count	hoeveel er voldoen	<code>fans.Count(f => f.Prijs > 25)</code>	2	—
Sum	tel op	<code>fans.Sum(f => f.Prijs)</code>	164.96	—
Min	de kleinste waarde	<code>fans.Min(f => f.Prijs)</code>	19.99	—
Max	de grootste waarde	<code>fans.Max(f => f.Prijs)</code>	89.99	—
Average	het gemiddelde	<code>fans.Average(f => f.Prijs)</code>	41.24	—
MinBy	het item met de kleinste waarde	<code>fans.MinBy(f => f.Prijs)</code>	fan 4	—
MaxBy	het item met de grootste waarde	<code>fans.MaxBy(f => f.Prijs)</code>	fan 3	—
CountBy	tel per waarde van iets	<code>fans.CountBy(f => f.Merk)</code>	NZXT 2, Noctua 1, Arctic 1	—
Aggregate	voeg alles samen met een eigen regel	<code>fruit.Aggregate((a, b) => a + " - " + b)</code>	appel-peer-kiwi	—
AggregateBy	voeg samen per waarde van iets	<code>fans.AggregateBy(f => f.Merk, 0m, (som, f) => som + f.Prijs)</code>	NZXT 114.98, Noctua 29.99, Arctic 19.99	—

`Max` geeft het getal, `MaxBy` geeft het item. `Sum`, `Average`, `Min` en `Max` op een array getallen hebben geen voorwaarde nodig: `getallen.Sum()` geeft 25.

LET OP

`Average`, `Min` en `Max` op een lege array van getallen geven een `InvalidOperationException`. `Sum` geeft dan 0. Bij een lege array van teksten of objecten geven `Min` en `Max` null.

Ordenen en groeperen

Komt van LINQ. Je array blijft in haar oude volgorde; je krijgt een nieuwe, geordende reeks.

Methode	Wat het doet	Voorbeeld	Resultaat	Via de klasse Array
Order	sorteer de waarden oplopend	<code>getallen.Order()</code>	1, 3, 5, 8, 8	<code>Array.Sort</code>
OrderDescending	sorteer de waarden aflopend	<code>getallen.OrderDescending()</code>	8, 8, 5, 3, 1	—
OrderBy	sorteer oplopend op iets	<code>fans.OrderBy(f => f.Prijs)</code>	fan 4, 1, 2, 3	<code>Array.Sort</code> met een eigen regel
OrderByDescending	sorteer aflopend op iets	<code>fans.OrderByDescending(f => f.Prijs)</code>	fan 3, 2, 1, 4	—
ThenBy	tweede sorteersleutel, oplopend	<code>fans.OrderBy(f => f.Merk).ThenBy(f => f.Prijs)</code>	per merk, en binnen een merk de goedkoopste eerst	—
ThenByDescending	tweede sorteersleutel, aflopend	<code>fans.OrderBy(f => f.Merk).ThenByDescending(f => f.Prijs)</code>	per merk, en binnen een merk de duurste eerst	—
Reverse	omgekeerde volgorde, in een nieuwe reeks	<code>getallen.Reverse()</code>	1, 8, 3, 8, 5; getallen zelf blijft 5, 8, 3, 8, 1	<code>Array.Reverse</code>
Shuffle	willekeurige volgorde	<code>fruit.Shuffle()</code>	elke keer anders	—
GroupBy	maak groepen	<code>fans.GroupBy(f => f.Merk)</code>	drie groepen: NZXT met 2 fans, Noctua en Arctic met elk 1	—
ToLookup	maak groepen, meteen vastgelegd	<code>fans.ToLookup(f => f.Merk)</code>	dezelfde groepen; <code>opzoek["NZXT"]</code> geeft fan 1 en 3	—

LET OP

`getallen.Reverse()` en `Array.Reverse(getallen)` lijken op elkaar, maar doen iets anders. `getallen.Reverse()` is LINQ: je krijgt een nieuwe, omgekeerde reeks en je array blijft zoals ze was. `Array.Reverse(getallen)` draait de array zelf om en geeft niets terug. Bij een `List` is het net omgekeerd: daar draait `lijst.Reverse()` de lijst zelf om (hoofdstuk 6).

Twée reeksen combineren

Komt van LINQ. In de voorbeelden is `a` de array 1, 2, 3 en `b` de array 3, 4.

Methode	Wat het doet	Voorbeeld	Resultaat
Concat	plak de tweede achter de eerste	<code>a.Concat(b)</code>	1, 2, 3, 3, 4
Union	alles uit beide, zonder dubbels	<code>a.Union(b)</code>	1, 2, 3, 4

Methode	Wat het doet	Voorbeeld	Resultaat
Intersect	wat in beide zit	<code>a.Intersect(b)</code>	3
Except	wat in de eerste zit en niet in de tweede	<code>a.Except(b)</code>	1, 2
UnionBy, IntersectBy, ExceptBy	hetzelfde, maar vergeleken op één eigenschap	<code>fans.ExceptBy(new[] { "NZXT" }, f => f.Merk)</code>	fan 2 en 4
Append	één item achteraan, in het resultaat	<code>a.Append(9)</code>	1, 2, 3, 9; a blijft drie lang
Prepend	één item vooraan, in het resultaat	<code>a.Prepend(0)</code>	0, 1, 2, 3
Zip	koppel de eerste aan de eerste, de tweede aan de tweede ...	<code>fruit.Zip(getallen)</code>	(appel, 5), (peer, 8), (kiwi, 3)
Join	koppel items uit twee reeksen met dezelfde sleutel	<code>fans.Join(merken, f => f.Merk, m => m.Naam, (f, m) => ...)</code>	enkel de paren die bij elkaar horen
LeftJoin	zoals Join , maar elk item uit de eerste blijft, ook zonder partner	<code>fans.LeftJoin(merken, ...)</code>	alle fans, met of zonder merk
RightJoin	zoals Join , maar elk item uit de tweede blijft	<code>fans.RightJoin(merken, ...)</code>	alle merken, met of zonder fan
GroupJoin	koppel aan elk item uit de eerste de groep die erbij hoort	<code>merken.GroupJoin(fans, ...)</code>	elk merk met zijn fans

Een array kan niet groeien, maar met `Append`, `Concat` en `.ToArray()` erachter maak je een **nieuwe**, langere array: `getallen = getallen.Append(9).ToArray();`. `Zip` stopt bij de kortste van de twee: de 8 en de 1 uit `getallen` vallen weg.

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
ToArray	maak er een nieuwe array van	<code>getallen.Where(g => g > 4).ToArray()</code>	een <code>int[]</code> met 5, 8, 8
ToList	maak er een <code>List</code> van, die wél kan groeien	<code>fans.ToList()</code>	een <code>List<Fan></code>
ToHashSet	maak er een <code>HashSet</code> van: dubbels vallen weg	<code>getallen.ToHashSet()</code>	5, 8, 3, 1
ToDictionary	maak er een opzoektabel van	<code>fans.ToDictionary(f => f.Id)</code>	<code>tabel[3]</code> geeft fan 3
ToLookup	maak er groepen van	<code>fans.ToLookup(f => f.Merk)</code>	zie hierboven

LET OP

ToDictionary crasht met een **ArgumentException** als twee items dezelfde sleutel hebben.
fans.ToDictionary(f => f.Merk) crasht dus: NZXT komt twee keer voor.

Doorlopen en posities

Komt van de taal C# zelf.

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
foreach	elk item één keer	foreach (string f in fruit) { ... }	appel, peer, kiwi
for met Length	elke positie	for (int i = 0; i < fruit.Length; i++) { ... }	positie 0, 1, 2
[i]	het item op positie i	fruit[0]	appel
[i] =	vervang het item op positie i	fruit[0] = "mango"	de appel is een mango
[^1]	tellen vanaf achteren: het laatste	fruit[^1]	kiwi
[1..3]	een stuk, van positie 1 tot vóór 3	fruit[1..3]	een nieuwe array peer, kiwi
[..2]	van het begin tot vóór 2	fruit[..2]	een nieuwe array appel, peer
[2..]	van positie 2 tot het einde	getallen[2..]	een nieuwe array 3, 8, 1
[.. a, .. b]	een nieuwe array uit twee arrays	int[] samen =[.. a, .. b];	1, 2, 3, 3, 4

foreach en for staan volledig in [hoofdstuk 4](#). In een foreach kan je een vakje niet vervangen; daarvoor heb je for en [i] = nodig.

Omzetten naar iets anders

Komt van de andere verzamelingen en van string.

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
string.Join	zet alles achter elkaar in één tekst	string.Join(", ", fruit)	appel, peer, kiwi
new List<T>(array)	een lijst, die wel kan groeien	new List<string>(fruit)	een lijst met dezelfde inhoud
new HashSet<T>(array)	een HashSet: dubbels vallen weg	new HashSet<int>(getallen)	5, 8, 3, 1
new Queue<T>(array)	een wachtrij, in dezelfde volgorde	new Queue<string>(fruit).Dequeue()	appel
new Stack<T>(array)	een stapel: het laatste ligt bovenop	new Stack<string>(fruit).Pop()	kiwi

De lijst, de `HashSet`, de wachtrij en de stapel staan in de hoofdstukken 6, 8, 9 en 10.

Wat elk object kan

Komt van *object*: elke array is ook een object. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToString</code>	het object als tekst	<code>fruit.ToString()</code>	<code>System.String[]</code>
<code>Equals</code> / <code>==</code>	is dit dezelfde array	<code>getallen == new[] { 5, 8, 3, 8, 1 }</code>	<code>false</code>
<code>GetType</code>	welk type is dit	<code>fruit.GetType().Name</code>	<code>String[]</code>
<code>GetHashCode</code>	een getal per object, voor opzoektabellen	<code>fruit.GetHashCode()</code>	een getal; zelden zelf nodig

`ToString` geeft de naam van het type, niet de inhoud. Wil je de inhoud zien, gebruik dan `string.Join`. En `==` vergelijkt of het om dezelfde array gaat, niet of de inhoud gelijk is. Voor de inhoud is er `SequenceEqual`.

Bestaat ook, maar kom je in deze cursus niet tegen

`Array.Copy` met een beginpositie in beide arrays, `Array.ConstrainedCopy`, `Array.CreateInstance`, `GetValue` en `SetValue` (een vakje lezen of vervangen zonder `[ ]`), `Array.MaxLength`, de methodes voor `Span` (`AsSpan`, en via `MemoryExtensions` onder meer `StartsWith` en `EndsWith` op een array), en uit LINQ: `AsParallel`, `Cast`, `DefaultIfEmpty`, `LongCount`, `ToImmutableArray`, `ToFrozenSet` en hun familie.

Let op bij een array

Een array heeft voor veel LINQ-methodes een tegenhanger in de klasse `Array`. Ze lijken op elkaar, maar zijn niet altijd hetzelfde.

Klasse Array	LINQ	Het verschil
<code>Length</code>	<code>Count()</code>	Hetzelfde getal. <code>Length</code> is een eigenschap van de array; <code>Count()</code> met haakjes is LINQ. Een voorwaarde meegeven kan enkel met LINQ: <code>Count(f => ...)</code> . Een array heeft geen <code>Count</code> zonder haakjes.
<code>Array.Find</code>	<code>FirstOrDefault</code>	Hetzelfde resultaat. <code>FirstOrDefault</code> werkt op elke verzameling, ook op een databasequery.
<code>Array.Exists</code>	<code>Any</code>	Hetzelfde resultaat.
<code>Array.TrueForAll</code>	<code>All</code>	Hetzelfde resultaat.
<code>Array.FindAll</code>	<code>Where</code>	<code>Array.FindAll</code> geeft meteen een nieuwe array . <code>Where</code> geeft een reeks die pas rekent als je ze gebruikt.
<code>Array.ConvertAll</code>	<code>Select</code>	Zelfde verschil: meteen een array tegenover een reeks.
<code>Array.Sort</code>	<code>Order</code> , <code>OrderBy</code>	<code>Array.Sort</code> verandert je array en geeft niets terug. <code>Order</code> en <code>OrderBy</code> laten je array zoals ze is en geven een nieuwe reeks.
<code>Array.Reverse</code>	<code>Reverse()</code>	<code>Array.Reverse(getallen)</code> draait de array zelf om. <code>getallen.Reverse()</code> is LINQ en geeft een nieuwe reeks.

Klasse Array	LINQ	Het verschil
<code>Array.IndexOf(...)</code> <code>>= 0</code>	<code>Contains</code>	Hetzelfde resultaat; <code>Contains</code> leest makkelijker.
<code>[1..3]</code>	<code>Skip(1).Take(2)</code>	<code>[1..3]</code> geeft meteen een nieuwe array, en crasht als het stuk buiten de array valt. <code>Skip</code> en <code>Take</code> crashen nooit: ze geven minder terug.

LET OP

- Het resultaat van `Where`, `Select` of `OrderBy` is geen array. `int[] groot = getallen.Where(g => g > 4);` compileert niet. Zet er `.ToArray()` achter.
- `int[] b = a;` maakt **geen kopie**. `a` en `b` zijn dan twee namen voor dezelfde array. Een kopie maak je met `.ToArray()`, `Clone()` of `[.. a]`.
- `Contains`, `Array.IndexOf` en `Distinct` vergelijken items met `Equals`. Bij een array van een `class` betekent dat: is het **hetzelfde object**, niet een object met dezelfde inhoud. Bij een `record` wordt de inhoud vergeleken. Zie [hoofdstuk 13](#) en [hoofdstuk 14](#).
- `FirstOrDefault`, `Array.Find` en hun familie geven bij een array van getallen geen `null` maar `0`, en bij een array van `bool` geen `null` maar `false`.
- LINQ werkt niet rechtstreeks op een rooster (`int[,]`): `rooster.Sum()` compileert niet. Schrijf `rooster.Cast<int>().Sum()`. Bij een array van arrays (`int[][]`) werkt LINQ wel, per rij.

Wanneer wel, wanneer niet

Wel — de hoeveelheid ligt vast en verandert nooit: de dagen van de week, de maanden, de vakjes van een bordspel.

Niet — zodra je iets wil bijvoegen of weghalen. Een array heeft geen `Add` en geen `Remove`.

Veelgemaakte fouten

LET OP

- De eerste positie is `0`, dus de laatste is `Length - 1`. Vraag je `dagen[5]` op in een array van vijf, dan crasht je programma.
- Het is `Length`, niet `Count`. Bij een `List` is het net omgekeerd.
- `new int[3]` geeft drie nullen. `new string[3]` geeft drie keer `null`, geen drie lege teksten.
- `Console.WriteLine(array)` toont de naam van het type, niet de inhoud.

Voorspel de uitkomst

Vraag 1

```
int[] a = new int[3];
Console.WriteLine(a[1]);
```


[Toon het antwoord ›](#)

Vraag 2

```
string[] s = { "a", "b", "c" };  
Console.WriteLine(s[s.Length - 1]);
```

[Toon het antwoord ›](#)

Vraag 3

```
string[] s = { "a", "b", "c" };  
Console.WriteLine(s[3]);
```

[Toon het antwoord ›](#)

Vraag 4

```
int[] a = { 1, 2, 3 };  
int[] b = a;  
b[0] = 99;  
Console.WriteLine(a[0]);
```

[Toon het antwoord ›](#)

Vraag 5

```
int[] a = { 3, 1, 2 };  
var gesorteerd = a.Order();  
Console.WriteLine(a[0]);  
Array.Sort(a);  
Console.WriteLine(a[0]);
```

[Toon het antwoord ›](#)

Vraag 6

```
string[] s = { "a", "b" };  
Array.Resize(ref s, 4);  
Console.WriteLine(s.Length);  
Console.WriteLine(s[3] == null);
```

[Toon het antwoord ›](#)

Vraag 7

```
int[] a = { 1, 2, 3 };  
int[] b = { 1, 2, 3 };  
Console.WriteLine(a == b);
```

```
Console.WriteLine(a.SequenceEqual(b));  
Console.WriteLine(a);
```

[Toon het antwoord ›](#)

Vraag 8

```
int[] a = { 10, 20, 30, 40 };  
int[] stuk = a[1..3];  
stuk[0] = 99;  
Console.WriteLine(a[1]);  
Console.WriteLine(a[^1]);
```

[Toon het antwoord ›](#)

Vraag 9

```
int[] a = { 5, 8, 3 };  
int[] b = a.Append(9).ToArray();  
Console.WriteLine(a.Length);  
Console.WriteLine(b.Length);
```

[Toon het antwoord ›](#)

Vraag 10

```
int[,] rooster = new int[2, 3];  
Console.WriteLine(rooster.Length);  
Console.WriteLine(rooster.GetLength(1));
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

0

Een nieuwe array van getallen is gevuld met de standaardwaarde, en die is voor een `int` gelijk aan nul.

[Terug naar de vraag](#)

VRAAG 2

C

`Length` is 3, dus `Length - 1` is 2, en positie 2 is het derde vakje.

[Terug naar de vraag](#)

VRAAG 3

`IndexOutOfRangeException` — het programma crasht.

Er zijn drie vakjes: 0, 1 en 2. Vakje 3 bestaat niet.

[Terug naar de vraag](#)

VRAAG 4

99

Dit is de belangrijkste vraag van dit hoofdstuk. `b = a` maakt geen kopie van de array. Het geeft dezelfde array een tweede naam. Verander je iets via `b`, dan zie je dat ook via `a`, want het is één en dezelfde eierdoos. Zie ook [hoofdstuk 16](#).

[Terug naar de vraag](#)

VRAAG 5

3 en daarna 1

Order is LINQ: de array blijft 3, 1, 2, het gesorteerde zit in `gesorteerd`. `Array.Sort` sorteert de array zelf.

[Terug naar de vraag](#)

VRAAG 6

4 en daarna `True`

`Array.Resize` maakte een nieuwe array van vier vakjes en kopieerde de twee teksten. De twee nieuwe vakjes zijn leeg: `null`.

[Terug naar de vraag](#)

VRAAG 7

`False`, daarna `True`, en dan `System.Int32[]`

`==` vraagt of `a` en `b` dezelfde array zijn: dat zijn ze niet. `SequenceEqual` vergelijkt de inhoud. En `Console.WriteLine` toont de naam van het type: `Int32` is de echte naam van `int`.

[Terug naar de vraag](#)

VRAAG 8

20 en daarna 40

`a[1..3]` maakt een **nieuwe** array met 20 en 30. Die veranderen raakt `a` niet. `^1` is het laatste vakje.

[Terug naar de vraag](#)

VRAAG 9

3 en daarna 4

Een array kan niet groeien. `Append` geeft een nieuwe reeks, `ToArray` maakt daar een nieuwe array van. `a` blijft drie vakjes lang.

[Terug naar de vraag](#)

VRAAG 10

6 en daarna 3

`Length` telt alle vakjes van het rooster: 2 rijen maal 3 kolommen. `GetLength(1)` geeft de lengte van de tweede dimensie: de kolommen.

[Terug naar de vraag](#)

6. List

VERZAMELING

Wat het is

Een array die kan groeien en krimpen. Zoals een boodschappenlijstje: je schrijft erbij, je streept door, en de volgorde blijft zoals je ze opschreef.

Tussen de punthaken staat wat erin mag. `List<string>` is een lijst van teksten, `List<int>` een lijst van getallen, `List<Leerling>` een lijst van leerlingen. Iets anders erin steken compileert niet.

Hoe het eruitziet

```
List<string> fruit = new List<string>();
fruit.Add("appel");
fruit.Add("peer");
fruit.Add("kiwi");

Console.WriteLine(fruit.Count);    // 3
Console.WriteLine(fruit[1]);       // peer

fruit.Remove("peer");
Console.WriteLine(fruit[1]);       // kiwi – alles schuift een plaats op

fruit.Insert(0, "banaan");         // vooraan bijzetten

List<string> kort = new List<string> { "appel", "peer" }; // meteen gevuld
```

Je zal ook de kortere schrijfwijze `List<string> kort = ["appel", "peer"];` tegenkomen. Die doet hetzelfde.

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens deze drie lijsten, zodat je elk resultaat kan narekenen.

```
List<string> fruit    = new List<string> { "appel", "peer", "kiwi" };
List<int>    getallen = new List<int> { 5, 8, 3, 8, 1 };
List<Fan>    fans     = new List<Fan>
{
    new Fan { Id = 1, Merk = "NZXT", Prijs = 24.99m },
    new Fan { Id = 2, Merk = "Noctua", Prijs = 29.99m },
    new Fan { Id = 3, Merk = "NZXT", Prijs = 89.99m },
    new Fan { Id = 4, Merk = "Arctic", Prijs = 19.99m }
};
```

`Fan` is een klasse met drie eigenschappen. Hoe je zelf zo'n klasse maakt, staat in [hoofdstuk 13](#).

Wat je ermee kan

Dit zijn de methodes die op een `List<T>` zelf zitten. Ze werken enkel op een lijst, niet op elke verzameling. Wat er daarnaast nog werkt, staat in het volgende deel, *Wat het ook kan, via iets anders*.

Toevoegen en verwijderen

Methode	Wat het doet	Voorbeeld	Resultaat
Add	zet er één achteraan bij	<code>fans.Add(nieuweFan)</code>	de lijst is één langer
AddRange	zet er meerdere achteraan bij	<code>fans.AddRange(extraFans)</code>	de lijst groeit
Insert	zet er één op een vaste plaats	<code>fans.Insert(0, nieuweFan)</code>	staat nu vooraan
InsertRange	zet er meerdere op een vaste plaats	<code>fruit.InsertRange(1, extraFruit)</code>	ze staan vanaf positie 1
Remove	haal dit ene item eruit	<code>fans.Remove(fan)</code>	<code>true</code> als het erin zat
RemoveAt	haal weg op positie	<code>fans.RemoveAt(0)</code>	de eerste is weg
RemoveRange	haal een aantal weg vanaf een positie	<code>getallen.RemoveRange(1, 2)</code>	5, 8, 1
RemoveAll	haal alles weg dat voldoet	<code>fans.RemoveAll(f => f.Merk == "NZXT")</code>	2: er zijn er twee weg
Clear	maak leeg	<code>fans.Clear()</code>	de lijst is leeg

`Remove` wil het item zelf, `RemoveAt` wil een positie, `RemoveAll` wil een voorwaarde. Drie namen die op elkaar lijken en drie verschillende dingen willen.

Erin zoeken

Methode	Wat het doet	Voorbeeld	Resultaat
Count	hoeveel zitten erin; een eigenschap	<code>fans.Count</code>	4
Contains	zit dit exacte item erin	<code>getallen.Contains(3)</code>	<code>true</code>
IndexOf	op welke positie staat het eerst	<code>getallen.IndexOf(8)</code>	1, of -1
LastIndexOf	op welke positie staat het laatst	<code>getallen.LastIndexOf(8)</code>	3, of -1
Exists	is er minstens één die voldoet	<code>fans.Exists(f => f.Prijs > 50)</code>	<code>true</code>
TrueForAll	voldoen ze allemaal	<code>fans.TrueForAll(f => f.Prijs > 20)</code>	<code>false</code> : de Arctic kost minder
Find	geef de eerste die voldoet	<code>fans.Find(f => f.Id == 3)</code>	de fan met Id 3, of <code>null</code>
FindLast	geef de laatste die voldoet	<code>fans.FindLast(f => f.Merk == "NZXT")</code>	de fan met Id 3
FindAll	geef allemaal die voldoen	<code>fans.FindAll(f => f.Merk == "NZXT")</code>	een nieuwe lijst met fan 1 en 3
FindIndex	op welke positie staat de eerste die voldoet	<code>fans.FindIndex(f => f.Id == 3)</code>	2, of -1

Methode	Wat het doet	Voorbeeld	Resultaat
FindLastIndex	op welke positie staat de laatste die voldoet	<code>fans.FindLastIndex(f => f.Merk == "NZXT")</code>	2, of -1
BinarySearch	zoek snel in een gesorteerde lijst	<code>gesorteerd.BinarySearch(8)</code>	de positie, of een negatief getal

Een stuk nemen of omzetten

Methode	Wat het doet	Voorbeeld	Resultaat
GetRange	een kopie van een stuk: vanaf positie, zoveel stuks	<code>getallen.GetRange(1, 3)</code>	een nieuwe lijst 8, 3, 8
Slice	hetzelfde als <code>GetRange</code>	<code>getallen.Slice(1, 3)</code>	een nieuwe lijst 8, 3, 8
ConvertAll	maak van elk item iets anders	<code>fans.ConvertAll(f => f.Merk)</code>	een nieuwe <code>List<string></code> met de merken
ForEach	doe iets met elk item	<code>fruit.ForEach(f => Console.WriteLine(f))</code>	drie regels op het scherm
ToArray	maak er een array van	<code>fans.ToArray()</code>	een array
CopyTo	kopieer naar een bestaande array	<code>fruit.CopyTo(doel)</code>	<code>doel</code> bevat de drie stuks fruit
AsReadOnly	een versie die niemand kan veranderen	<code>fans.AsReadOnly()</code>	geen <code>Add</code> of <code>Remove</code> meer mogelijk

Ordenen

Methode	Wat het doet	Voorbeeld	Resultaat
Sort	sorteer de lijst zelf	<code>getallen.Sort()</code>	1, 3, 5, 8, 8
Sort	sorteer op iets dat je zelf kiest	<code>fans.Sort((a, b) => a.Prijs.CompareTo(b.Prijs))</code>	goedkoopste eerst
Reverse	keer de lijst zelf om	<code>getallen.Reverse()</code>	1, 8, 3, 8, 5

BELANGRIJK

`Sort` en `Reverse` veranderen je lijst zelf en geven niets terug. De LINQ-methodes hieronder doen het omgekeerde: die laten je lijst met rust en geven een nieuw resultaat. Dat verschil verklaart waarom `lijst.Sort()` werkt en `var x = lijst.Sort()` niet compileert.

Hoeveel plaats er gereserveerd is

Een lijst houdt achter de schermen meer plaats vrij dan ze gebruikt, zodat niet elke `Add` alles moet verhuizen. Je hebt dit zelden nodig.

Methode	Wat het doet	Voorbeeld	Resultaat
Capacity	hoeveel er passen voor de lijst moet groeien; een eigenschap	<code>fruit.Capacity</code>	minstens 3
EnsureCapacity	reserveer alvast plaats	<code>fruit.EnsureCapacity(100)</code>	er passen er minstens 100

Methode	Wat het doet	Voorbeeld	Resultaat
<code>TrimExcess</code>	geef de vrije plaats terug	<code>fruit.TrimExcess()</code>	Capacity zakt tot Count, als er genoeg vrij was

Wat het ook kan, via iets anders

Een lijst kan veel meer dan de methodes hierboven, omdat ze ook iets anders is. Ze is een **reeks**: iets wat je één voor één kan doorlopen. In C# heet dat `IEnumerable` ([hoofdstuk 12](#)). Alles wat met een reeks werkt, werkt dus ook met een lijst.

Bron	Waarom het op een lijst werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
<code>foreach</code>	<code>foreach</code> doorloopt elke reeks	hoofdstuk 4
<code>^</code> en <code>..</code> tussen vierkante haken	een lijst heeft een <code>Count</code> , posities en <code>Slice</code>	hieronder
<code>string.Join</code>	wil een reeks teksten of waarden	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 5 , 7 tot 10
wat elk object kan	een lijst is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je lijst nooit**. Ze geeft een nieuw resultaat.
- Dat resultaat is meestal **geen** `List`, maar een reeks. Wil je er weer een lijst van, zet er dan `.ToList()` achter.
- `Where`, `Select`, `OrderBy` en hun familie rekenen pas op het moment dat je het resultaat gebruikt. Verandert je lijst tussendoor, dan rekenen ze met de nieuwe inhoud. Vraag 8 onderaan toont het.
- LINQ werkt enkel met `using System.Linq;` bovenaan. In een nieuw project staat die er automatisch bij.

OPMERKING

`Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken. In oudere voorbeelden op internet kom je ze niet tegen.

Eén item nemen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
<code>First</code>	de eerste die voldoet	<code>fans.First(f => f.Merk == "NZXT")</code>	fan 1; niets gevonden geeft een <code>InvalidOperationException</code>	—

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
FirstOrDefault	de eerste die voldoet, of niets	<code>fans.FirstOrDefault(f => f.Id == 9)</code>	<code>null</code>	<code>Find</code>
Single	de enige die voldoet	<code>fans.Single(f => f.Id == 2)</code>	fan 2; nul of twee gevonden geeft een <code>InvalidOperationException</code>	—
SingleOrDefault	de enige die voldoet, of niets	<code>fans.SingleOrDefault(f => f.Merk == "NZXT")</code>	<code>InvalidOperationException</code> : er zijn er twee	—
Last	de laatste die voldoet	<code>fans.Last(f => f.Merk == "NZXT")</code>	fan 3	—
LastOrDefault	de laatste die voldoet, of niets	<code>fans.LastOrDefault(f => f.Prijs > 100)</code>	<code>null</code>	<code>FindLast</code>
ElementAt	het item op een positie	<code>fans.ElementAt(1)</code>	fan 2	<code>fans[1]</code>
ElementAtOrDefault	het item op een positie, of niets	<code>fans.ElementAtOrDefault(9)</code>	<code>null</code>	—

Neem `FirstOrDefault` wanneer het normaal is dat er niets gevonden wordt, zoals een `Id` dat niet bestaat. Neem `SingleOrDefault` wanneer er hoogstens één mag zijn: de uitzondering bij twee is dan een alarm over je gegevens. Neem `First` alleen wanneer je zeker weet dat er iets is.

Weten of iets erin zit

Komt van *LINQ*.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
Any	is er minstens één	<code>fans.Any()</code>	<code>true</code>	<code>Count > 0</code>
Any	is er minstens één die voldoet	<code>fans.Any(f => f.Prijs > 50)</code>	<code>true</code>	<code>Exists</code>
All	voldoen ze allemaal	<code>fans.All(f => f.Prijs > 20)</code>	<code>false</code>	<code>TrueForAll</code>
Contains	zit deze waarde erin	<code>getallen.Contains(3)</code>	<code>true</code>	<code>Contains</code> : dezelfde

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
SequenceEqual	hebben twee reeksen dezelfde inhoud, in dezelfde volgorde	<code>getallen.SequenceEqual(new List<int> { 5, 8, 3, 8, 1 })</code>	<code>true</code>	—

LET OP

`All` op een lege lijst geeft `true`: er is geen enkel item dat níet voldoet. Controleer eerst of er wel iets in zit.

Een deel nemen

Komt van *LINQ*. Het resultaat is telkens een nieuwe reeks; je lijst blijft zoals ze was.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
Where	enkel wat voldoet	<code>fans.Where(f => f.Merk == "NZXT")</code>	fan 1 en 3	<code>FindAll</code>
Select	maak van elk item iets anders	<code>fans.Select(f => f.Merk)</code>	NZXT, Noctua, NZXT, Arctic	<code>ConvertAll</code>
SelectMany	plat een lijst van lijsten	<code>klassen.SelectMany(k => k.Leerlingen)</code>	alle leerlingen samen	—
Distinct	gooi dubbels weg	<code>getallen.Distinct()</code>	5, 8, 3, 1	—
DistinctBy	één item per waarde van iets	<code>fans.DistinctBy(f => f.Merk)</code>	fan 1, 2 en 4	—
Take	de eerste n	<code>getallen.Take(2)</code>	5, 8	<code>GetRange(0, 2)</code>
Skip	sla de eerste n over	<code>getallen.Skip(2)</code>	3, 8, 1	—
TakeLast	de laatste n	<code>getallen.TakeLast(2)</code>	8, 1	—
SkipLast	alles behalve de laatste n	<code>getallen.SkipLast(2)</code>	5, 8, 3	—
TakeWhile	neem zolang de voorwaarde klopt, stop bij de eerste die niet klopt	<code>getallen.TakeWhile(g => g > 2)</code>	5, 8, 3, 8	—
SkipWhile	sla over zolang de voorwaarde klopt	<code>getallen.SkipWhile(g => g > 2)</code>	1	—
Chunk	verdeel in stukken van n	<code>getallen.Chunk(2)</code>	[5, 8], [3, 8], [1]	—
OfType	enkel de items van een bepaald type	<code>onderdelen.OfType<Processor>()</code>	enkel de processors	—
Index	elk item samen met zijn positie	<code>fruit.Index()</code>	(0, appel), (1, peer), (2, kiwi)	—

`OfType` heb je nodig bij overerving: een lijst van een basisklasse waar verschillende soorten in zitten. Zie [hoofdstuk 20](#).

Tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
Count	hoeveel er zijn	<code>fans.Count()</code>	4	Count, zonder haakjes
Count	hoeveel er voldoen	<code>fans.Count(f => f.Prijs > 25)</code>	2	—
Sum	tel op	<code>fans.Sum(f => f.Prijs)</code>	164.96	—
Min	de kleinste waarde	<code>fans.Min(f => f.Prijs)</code>	19.99	—
Max	de grootste waarde	<code>fans.Max(f => f.Prijs)</code>	89.99	—
Average	het gemiddelde	<code>fans.Average(f => f.Prijs)</code>	41.24	—
MinBy	het item met de kleinste waarde	<code>fans.MinBy(f => f.Prijs)</code>	fan 4	—
MaxBy	het item met de grootste waarde	<code>fans.MaxBy(f => f.Prijs)</code>	fan 3	—
CountBy	tel per waarde van iets	<code>fans.CountBy(f => f.Merk)</code>	NZXT 2, Noctua 1, Arctic 1	—
Aggregate	voeg alles samen met een eigen regel	<code>fruit.Aggregate((a, b) => a + " - " + b)</code>	appel-peer-kiwi	—
AggregateBy	voeg samen per waarde van iets	<code>fans.AggregateBy(f => f.Merk, 0m, (som, f) => som + f.Prijs)</code>	NZXT 114.98, Noctua 29.99, Arctic 19.99	—

Max geeft het getal, **MaxBy** geeft het item. **Sum**, **Average**, **Min** en **Max** op een lijst getallen hebben geen voorwaarde nodig: `getallen.Sum()` geeft 25.

LET OP

Average, **Min** en **Max** op een lege lijst van getallen geven een **InvalidOperationException**. **Sum** geeft dan 0. Bij een lege lijst van teksten of objecten geven **Min** en **Max** null.

Ordenen en groeperen

Komt van LINQ. Je lijst blijft in haar oude volgorde; je krijgt een nieuwe, geordende reeks.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
Order	sorteer de waarden oplopend	<code>getallen.Order()</code>	1, 3, 5, 8, 8	Sort
OrderDescending	sorteer de waarden aflopend	<code>getallen.OrderDescending()</code>	8, 8, 5, 3, 1	—

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
OrderBy	sorteer oplopend op iets	<code>fans.OrderBy(f => f.Prijs)</code>	fan 4, 1, 2, 3	Sort met een eigen regel
OrderByDescending	sorteer aflopend op iets	<code>fans.OrderByDescending(f => f.Prijs)</code>	fan 3, 2, 1, 4	—
ThenBy	tweede sorteersleutel, oplopend	<code>fans.OrderBy(f => f.Merk).ThenBy(f => f.Prijs)</code>	per merk, en binnen een merk de goedkoopste eerst	—
ThenByDescending	tweede sorteersleutel, aflopend	<code>fans.OrderBy(f => f.Merk).ThenByDescending(f => f.Prijs)</code>	per merk, en binnen een merk de duurste eerst	—
Reverse	omgekeerde volgorde	<code>getallen.AsEnumerable().Reverse()</code>	1, 8, 3, 8, 5	Reverse: zie de waarschuwing
Shuffle	willekeurige volgorde	<code>fruit.Shuffle()</code>	elke keer anders	—
GroupBy	maak groepen	<code>fans.GroupBy(f => f.Merk)</code>	drie groepen: NZXT met 2 fans, Noctua en Arctic met elk 1	—
ToLookup	maak groepen, meteen vastgelegd	<code>fans.ToLookup(f => f.Merk)</code>	dezelfde groepen; <code>opzoek["NZXT"]</code> geeft fan 1 en 3	—

LET OP

`getallen.Reverse()` roept altijd de `Reverse` van de lijst zelf op: die draait je lijst om en geeft niets terug. Wil je de LINQ-versie, die een nieuwe reeks geeft en je lijst laat zoals ze is, schrijf dan `getallen.AsEnumerable().Reverse().var x = getallen.Reverse();` compileert niet.

Twée reeksen combineren

Komt van LINQ. In de voorbeelden is **a** de lijst 1, 2, 3 en **b** de lijst 3, 4.

Methode	Wat het doet	Voorbeeld	Resultaat
Concat	plak de tweede achter de eerste	<code>a.Concat(b)</code>	1, 2, 3, 3, 4
Union	alles uit beide, zonder dubbels	<code>a.Union(b)</code>	1, 2, 3, 4
Intersect	wat in beide zit	<code>a.Intersect(b)</code>	3
Except	wat in de eerste zit en niet in de tweede	<code>a.Except(b)</code>	1, 2

Methode	Wat het doet	Voorbeeld	Resultaat
UnionBy , IntersectBy , ExceptBy	hetzelfde, maar vergeleken op één eigenschap	<code>fans.ExceptBy(new[] { "NZXT" }, f => f.Merk)</code>	fan 2 en 4
Append	één item achteraan, in het resultaat	<code>a.Append(9)</code>	1, 2, 3, 9
Prepend	één item vooraan, in het resultaat	<code>a.Prepend(0)</code>	0, 1, 2, 3
Zip	koppel de eerste aan de eerste, de tweede aan de tweede ...	<code>fruit.Zip(getallen)</code>	(appel, 5), (peer, 8), (kiwi, 3)
Join	koppel items uit twee reeksen met dezelfde sleutel	<code>fans.Join(merken, f => f.Merk, m => m.Naam, (f, m) => ...)</code>	enkel de paren die bij elkaar horen
LeftJoin	zoals <code>Join</code> , maar elk item uit de eerste blijft, ook zonder partner	<code>fans.LeftJoin(merken, ...)</code>	alle fans, met of zonder merk
RightJoin	zoals <code>Join</code> , maar elk item uit de tweede blijft	<code>fans.RightJoin(merken, ...)</code>	alle merken, met of zonder fan
GroupJoin	koppel aan elk item uit de eerste de groep die erbij hoort	<code>merken.GroupJoin(fans, ...)</code>	elk merk met zijn fans

`Append` en `Prepend` lijken op `Add` en `Insert`, maar veranderen je lijst niet. `Zip` stopt bij de kortste van de twee: de 8 en de 1 uit `getallen` vallen weg.

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
ToList	maak er een nieuwe lijst van	<code>fans.Where(f => f.Prijs > 25).ToList()</code>	een <code>List<Fan></code> met fan 2 en 3
ToArray	maak er een array van	<code>getallen.ToArray()</code>	een array; de lijst heeft ook een eigen <code>ToArray</code>
ToHashSet	maak er een <code>HashSet</code> van: dubbels vallen weg	<code>getallen.ToHashSet()</code>	5, 8, 3, 1
ToDictionary	maak er een opzoektabel van	<code>fans.ToDictionary(f => f.Id)</code>	<code>tabel[3]</code> geeft fan 3
ToLookup	maak er groepen van	<code>fans.ToLookup(f => f.Merk)</code>	zie hierboven

LET OP

`ToDictionary` crasht met een **ArgumentException** als twee items dezelfde sleutel hebben.
`fans.ToDictionary(f => f.Merk)` crasht dus: NZXT komt twee keer voor.

Doorlopen en posities

Komt van de taal C# zelf.

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
<code>foreach</code>	elk item één keer	<code>foreach (string f in fruit) { ... }</code>	appel, peer, kiwi
<code>for</code> met <code>Count</code>	elke positie	<code>for (int i = 0; i < fruit.Count; i++) { ... }</code>	positie 0, 1, 2
<code>[i]</code>	het item op positie i	<code>fruit[0]</code>	appel
<code>[i] =</code>	vervang het item op positie i	<code>fruit[0] = "mango"</code>	de appel is een mango
<code>[^1]</code>	tellen vanaf achteren: het laatste	<code>fruit[^1]</code>	kiwi
<code>[1..3]</code>	een stuk, van positie 1 tot vóór 3	<code>fruit[1..3]</code>	een nieuwe lijst peer, kiwi
<code>[..2]</code>	van het begin tot vóór 2	<code>fruit[..2]</code>	een nieuwe lijst appel, peer
<code>[.. a, .. b]</code>	een nieuwe lijst uit twee lijsten	<code>List<int> samen = [.. a, .. b];</code>	1, 2, 3, 3, 4

`foreach` en `for` staan volledig in [hoofdstuk 4](#).

Omzetten naar iets anders

Komt van de andere verzamelingen en van `string`.

Schrijfwijze	Wat het doet	Voorbeeld	Resultaat
<code>string.Join</code>	zet alles achter elkaar in één tekst	<code>string.Join(", ", fruit)</code>	appel, peer, kiwi
<code>new List<T>(lijst)</code>	een kopie	<code>new List<string>(fruit)</code>	een nieuwe lijst met dezelfde inhoud
<code>new HashSet<T>(lijst)</code>	een HashSet: dubbels vallen weg	<code>new HashSet<int>(getallen)</code>	5, 8, 3, 1
<code>new Queue<T>(lijst)</code>	een wachtrij, in dezelfde volgorde	<code>new Queue<string>(fruit).Dequeue()</code>	appel
<code>new Stack<T>(lijst)</code>	een stapel: het laatste ligt bovenop	<code>new Stack<string>(fruit).Pop()</code>	kiwi

De `HashSet`, de wachtrij en de stapel staan in de hoofdstukken [8](#), [9](#) en [10](#).

Wat elk object kan

Komt van `object`: elke lijst is ook een object. Zie [hoofdstuk 13](#).

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToString</code>	het object als tekst	<code>fruit.ToString()</code>	<code>System.Collections.Generic.List`1[System.String]</code>

Methode	Wat het doet	Voorbeeld	Resultaat
Equals / <code>==</code>	is dit dezelfde lijst	<code>fruit == new List<string> { "appel", "peer", "kiwi" }</code>	<code>false</code>
GetType	welk type is dit	<code>fruit.GetType().Name</code>	<code>List`1</code>
GetHashCode	een getal per object, voor opzoektabelen	<code>fruit.GetHashCode()</code>	een getal; zelden zelf nodig

`ToString` geeft de naam van het type, niet de inhoud. Wil je de inhoud zien, gebruik dan `string.Join`. En `==` vergelijkt of het om dezelfde lijst gaat, niet of de inhoud gelijk is. Voor de inhoud is er `SequenceEqual`.

Bestaat ook, maar kom je in deze cursus niet tegen

`AsParallel` (verwerken op meerdere processorkernen), `Cast` (elk item omzetten naar een type, en crashen als dat niet lukt), `DefaultIfEmpty`, `LongCount`, `TryGetNonEnumeratedCount`, `AsQueryable`, `ToAsyncEnumerable`, de methodes die een onveranderlijke kopie maken (`ToImmutableList`, `ToFrozenSet` en hun familie), en de methodes voor XML.

Let op bij een lijst

Een lijst heeft voor veel LINQ-methodes een eigen tegenhanger. Ze lijken op elkaar, maar zijn niet altijd hetzelfde.

Eigen methode	LINQ	Het verschil
<code>Count</code>	<code>Count()</code>	Hetzelfde getal. <code>Count</code> zonder haakjes is een eigenschap van de lijst en meteen bekend; <code>Count()</code> met haakjes is LINQ. Een voorwaarde meegeven kan enkel met LINQ: <code>Count(f => ...)</code> .
<code>Find</code>	<code>FirstOrDefault</code>	Hetzelfde resultaat. <code>FirstOrDefault</code> werkt op elke verzameling, ook op een databasequery; <code>Find</code> niet. Daarom gebruikt de cursus bijna altijd <code>FirstOrDefault</code> .
<code>FindLast</code>	<code>LastOrDefault</code>	Hetzelfde resultaat.
<code>Exists</code>	<code>Any</code>	Hetzelfde resultaat.
<code>TrueForAll</code>	<code>All</code>	Hetzelfde resultaat.
<code>FindAll</code>	<code>Where</code>	<code>FindAll</code> geeft meteen een nieuwe <code>List</code> . <code>Where</code> geeft een reeks die pas rekt als je ze gebruikt, en rekt dan met wat er op dat moment in de lijst zit.
<code>ConvertAll</code>	<code>Select</code>	Zelfde verschil: <code>ConvertAll</code> geeft meteen een <code>List</code> , <code>Select</code> een reeks.
<code>GetRange</code>	<code>Skip + Take</code>	<code>GetRange</code> geeft meteen een kopie als <code>List</code> , en crasht als het stuk buiten de lijst valt. <code>Skip</code> en <code>Take</code> crashen nooit: ze geven minder terug.
<code>Sort</code>	<code>Order</code> , <code>OrderBy</code>	<code>Sort</code> verandert je lijst en geeft niets terug. <code>Order</code> en <code>OrderBy</code> laten je lijst zoals ze is en geven een nieuwe reeks.
<code>Reverse()</code>	<code>Reverse()</code>	Zelfde naam. Op een lijst wint altijd de eigen <code>Reverse</code> : die draait je lijst om en geeft niets terug. Voor de LINQ-versie: <code>AsEnumerable().Reverse()</code> .

Eigen methode	LINQ	Het verschil
ForEach	foreach	ForEach is een methode van de lijst; foreach werkt op elke reeks. Je mag in geen van beide de lijst zelf veranderen.
Contains	Contains	Hetzelfde. Bij een lijst van objecten vergelijken ze met Equals: zie hieronder.

LET OP

- Het resultaat van Where, Select of OrderBy is geen List. `List<Fan> duur = fans.Where(f => f.Prijs > 50);` compileert niet. Zet er `.ToList()` achter.
- Contains, IndexOf, Remove en Distinct vergelijken items met Equals. Bij een lijst van een class betekent dat: is het **hetzelfde object**, niet een object met dezelfde inhoud. Een nieuwe Fan met Id 1, merk NZXT en prijs 24.99 zit volgens Contains dus níet in de lijst. Bij een record wel, want een record vergelijkt op inhoud. Zie hoofdstuk 13 en hoofdstuk 14.
- FirstOrDefault, LastOrDefault, SingleOrDefault en ElementOrDefault geven bij een lijst van getallen geen null maar 0, en bij een lijst van bool geen null maar false. Bij een lijst van objecten of teksten is het wel null.

Wanneer wel, wanneer niet

Wel — de standaardkeuze voor "een aantal dingen". Twijfel je, begin dan met een List.

Niet — wanneer je telkens op dezelfde eigenschap zoekt (neem een Dictionary), of wanneer dubbels verboden zijn (neem een HashSet).

Veelgemaakte fouten

LET OP

- Het is Count, niet Length.
- Na Remove of RemoveAt schuift alles erachter een plaats op. Een positie die je vóór het verwijderen onthield, wijst daarna naar iets anders.
- Je mag een lijst niet veranderen terwijl je er met foreach doorheen loopt. Dat crasht, ook als je code er onschuldig uitziet.
- `List<string> lijst;` zonder new is geen lege lijst maar niets. `lijst.Add` crasht dan.
- `Console.WriteLine(lijst)` toont de naam van het type, niet de inhoud.

Voorspel de uitkomst

Vraag 1

```
var l = new List<int> { 10, 20, 30 };
l.RemoveAt(0);
Console.WriteLine(l[0]);
```


[Toon het antwoord ›](#)

Vraag 2

```
var l = new List<string> { "a", "b", "a" };  
l.Remove("a");  
Console.WriteLine(l.Count);
```

[Toon het antwoord ›](#)

Vraag 3

```
var l = new List<int> { 1, 2, 3 };  
foreach (int x in l)  
{  
    if (x == 2)  
    {  
        l.Remove(x);  
    }  
}
```

[Toon het antwoord ›](#)

Vraag 4

```
var l = new List<string> { "kiwi", "appel", "peer" };  
l.Sort();  
Console.WriteLine(l[0]);
```

[Toon het antwoord ›](#)

Vraag 5

```
var l = new List<int> { 1, 2, 3 };  
var omgekeerd = l.OrderByDescending(x => x);  
Console.WriteLine(l[0]);  
l.Reverse();  
Console.WriteLine(l[0]);
```

[Toon het antwoord ›](#)

Vraag 6

```
var l = new List<string> { "appel", "peer" };  
Console.WriteLine(l);  
Console.WriteLine(string.Join(" en ", l));
```

[Toon het antwoord ›](#)

Vraag 7

```
var a = new List<int> { 1, 2 };
var b = new List<int> { 1, 2 };
Console.WriteLine(a == b);
Console.WriteLine(a.SequenceEqual(b));
```

[Toon het antwoord ›](#)

Vraag 8

```
var l = new List<int> { 1, 2, 3 };
var metFindAll = l.FindAll(x => x > 1);
var metWhere = l.Where(x => x > 1);
l.Add(4);
Console.WriteLine(metFindAll.Count);
Console.WriteLine(metWhere.Count());
```

[Toon het antwoord ›](#)

Vraag 9

```
var l = new List<string> { "a", "b", "c", "d" };
Console.WriteLine(l[^1]);
Console.WriteLine(l[1..3].Count);
```

[Toon het antwoord ›](#)

Vraag 10

```
var getallen = new List<int>();
Console.WriteLine(getallen.Sum());
Console.WriteLine(getallen.FirstOrDefault());
Console.WriteLine(getallen.Max());
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

20

De 10 op positie 0 is weg. De 20 schuift op naar positie 0.

[Terug naar de vraag](#)

VRAAG 2

2

Remove haalt enkel het **eerste** voorkomen weg. De lijst is nu b, a. Wil je ze allemaal weg, dan heb je RemoveAll nodig.

[Terug naar de vraag](#)

VRAAG 3

InvalidOperationException — het programma crasht.

foreach merkt dat de lijst veranderd is terwijl hij erdoor liep, en weigert verder te gaan. Wil je tijdens het doorlopen verwijderen, gebruik dan RemoveAll met een voorwaarde, of loop door een kopie.

[Terug naar de vraag](#)

VRAAG 4

appel

Sort zet de lijst zelf op alfabetische volgorde. Het geeft niets terug, het verandert de lijst.

[Terug naar de vraag](#)

VRAAG 5

1 en daarna 3

OrderByDescending is LINQ: je lijst blijft 1, 2, 3, het omgekeerde zit in omgekeerd. Reverse is een methode van de lijst zelf: die draait de lijst om.

[Terug naar de vraag](#)

VRAAG 6

System.Collections.Generic.List`1[System.String] en daarna appel en peer

Console.WriteLine roept ToString op, en een lijst geeft daar de naam van haar type. string.Join zet de inhoud achter elkaar.

[Terug naar de vraag](#)

VRAAG 7

`False` en daarna `True`

`==` vraagt of `a` en `b` dezelfde lijst zijn. Het zijn twee lijsten die toevallig hetzelfde bevatten. `SequenceEqual` vergelijkt de inhoud, item per item.

[Terug naar de vraag](#)

VRAAG 8

2 en daarna 3

`FindAll` maakte meteen een nieuwe lijst met 2 en 3. Die verandert niet meer. `Where` rekent pas bij `Count()`, en dan zit de 4 er al in. Meer hierover in [hoofdstuk 12](#).

[Terug naar de vraag](#)

VRAAG 9

`d` en daarna 2

`^1` telt vanaf achteren: het laatste item. `1..3` neemt de posities 1 en 2, dus `b` en `c`: de 3 zelf hoort er niet meer bij.

[Terug naar de vraag](#)

VRAAG 10

`0`, daarna `0`, en dan `InvalidOperationException` — het programma crasht.

Een lege som is 0. `FirstOrDefault` vindt niets en geeft de standaardwaarde van een `int`: ook 0. Maar de grootste waarde van niets bestaat niet, dus `Max` crasht.

[Terug naar de vraag](#)

7. Dictionary

VERZAMELING

Wat het is

Een woordenboek. Je zoekt niet op paginanummer maar op het woord zelf. Elk woord staat er maar één keer in, en bij elk woord hoort een uitleg.

In C# heet het woord de **sleutel** en de uitleg de **waarde**. Tussen de punthaken staan ze allebei: `Dictionary<string, int>` heeft teksten als sleutel en getallen als waarde.

Hoe het eruitziet

```
Dictionary<string, string> hoofdsteden = new Dictionary<string, string>();

hoofdsteden.Add("België", "Brussel");
hoofdsteden["Frankrijk"] = "Parijs";           // toevoegen, of overschrijven

Console.WriteLine(hoofdsteden["België"]);      // Brussel
Console.WriteLine(hoofdsteden.Count);          // 2

if (hoofdsteden.TryGetValue("Spanje", out string? stad))
{
    Console.WriteLine(stad);
}
else
{
    Console.WriteLine("Onbekend land");        // dit wordt getoond
}

foreach (KeyValuePair<string, string> paar in hoofdsteden)
{
    Console.WriteLine($"{paar.Key}: {paar.Value}");
}
```

Bij het doorlopen krijg je telkens een paar terug: `.Key` is de sleutel, `.Value` de waarde. Zo'n paar heet een `KeyValuePair`. Onthoud die naam: alles wat je hieronder via LINQ doet, doe je op die paren.

Het patroon dat je het vaakst zal gebruiken: tellen

```
string[] dieren = { "kat", "hond", "kat", "vis", "kat" };
Dictionary<string, int> telling = new Dictionary<string, int>();

foreach (string dier in dieren)
{
    if (telling.ContainsKey(dier))
    {
        telling[dier]++;
    }
}
```

```

        else
        {
            telling[dier] = 1;
        }
    }

    Console.WriteLine(telling["kat"]);    // 3
    Console.WriteLine(telling["vis"]);    // 1

```

Zodra je ergens *hoeveel van elk* moet weten, is dit het antwoord. Het werkt voor woorden, letters, kleuren, stemmen — alles waar je per soort wil tellen.

Hetzelfde kan korter met `GetValueOrDefault`, uit de tabel hieronder:

```

foreach (string dier in dieren)
{
    telling[dier] = telling.GetValueOrDefault(dier) + 1;
}

```

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens dit woordenboek met ticketprijzen, zodat je elk resultaat kan narekenen.

```

Dictionary<string, int> prijzen = new Dictionary<string, int>
{
    ["dag"] = 65,
    ["combi"] = 120,
    ["vip"] = 210
};

```

Wat je ermee kan

Dit zijn de methodes en eigenschappen die op een `Dictionary` zelf zitten.

Toevoegen, veranderen en verwijderen

Methode	Wat het doet	Voorbeeld	Resultaat
Add	voeg toe, of loopt stuk bij een dubbele sleutel	<code>prijzen.Add("kind", 30)</code>	staat erin; <code>prijzen.Add("dag", 70)</code> geeft een ArgumentException
TryAdd	voeg toe als de sleutel vrij is	<code>prijzen.TryAdd("dag", 70)</code>	false : dag blijft 65
[sleutel] =	voeg toe, of overschrijf	<code>prijzen["dag"] = 70</code>	dag is nu 70
[sleutel] +=	verander een bestaande waarde	<code>prijzen["dag"] += 5</code>	70 ; bij een onbestaande sleutel een KeyNotFoundException
Remove	haal die sleutel weg	<code>prijzen.Remove("vip")</code>	true ; nog eens geeft false
Remove	haal weg en geef de oude waarde	<code>prijzen.Remove("vip", out int oud)</code>	true , oud is 210

Methode	Wat het doet	Voorbeeld	Resultaat
Clear	maak leeg	prijzen.Clear()	Count is 0

Opzoeken

Methode	Wat het doet	Voorbeeld	Resultaat
[sleutel]	haal op, of loopt stuk	prijzen["dag"]	65; prijzen["kind"] geeft een KeyNotFoundException
TryGetValue	haal op en zeg of het lukte	prijzen.TryGetValue("vip", out int p)	true, p is 210
GetValueOrDefault	haal op, of geef de standaardwaarde	prijzen.GetValueOrDefault("kind")	0
GetValueOrDefault	haal op, of geef wat jij kiest	prijzen.GetValueOrDefault("kind", -1)	-1
ContainsKey	bestaat die sleutel	prijzen.ContainsKey("vip")	true
ContainsValue	komt die waarde ergens voor	prijzen.ContainsValue(120)	true
Count	hoeveel paren; een eigenschap	prijzen.Count	3
Keys	alle sleutels	prijzen.Keys	dag, combi, vip
Values	alle waarden	prijzen.Values	65, 120, 210
Comparer	hoe sleutels vergeleken worden	prijzen.Comparer	de standaardvergelijking, of wat je bij het aanmaken meegaf

Dezelfde verhouding als bij `Parse` en `TryParse`: de vierkante haken lopen stuk op een sleutel die niet bestaat, `TryGetValue` en `GetValueOrDefault` niet. Weet je niet zeker of de sleutel er is, neem dan een van die twee.

`ContainsKey` is snel, hoe groot het woordenboek ook is. `ContainsValue` moet alle waarden aflopen: een woordenboek zoekt snel op het woord, niet op de uitleg.

Aanmaken en plaats reserveren

Schrijfwijze	Wat het doet	Resultaat
<code>new Dictionary<string, int>(StringComparer.OrdinalIgnoreCase)</code>	sleutels zonder onderscheid tussen hoofdletters en kleine letters	"Kat" en "kat" zijn dezelfde sleutel
<code>new Dictionary<string, int>(andereTabel)</code>	een kopie van een ander woordenboek	een nieuw woordenboek met dezelfde paren
<code>prijzen.Capacity</code>	hoeveel er passen voor het woordenboek moet groeien	minstens Count
<code>prijzen.EnsureCapacity(100)</code>	reserveer alvast plaats	er passen er minstens 100
<code>prijzen.TrimExcess()</code>	geef de vrije plaats terug	zelden nodig

Schrijfwijze	Wat het doet	Resultaat
prijzen.AsReadOnly()	een versie die niemand kan veranderen	geen Add, geen Remove

Twee manieren om toe te voegen

Schrijfwijze	Sleutel bestaat nog niet	Sleutel bestaat al
d.Add(sleutel, waarde)	voegt toe	crasht
d.TryAdd(sleutel, waarde)	voegt toe, geeft true	doet niets, geeft false
d[sleutel] = waarde	voegt toe	overschrijft

Kies bewust. `Add` is een bewaker: hij zegt je dat er iets dubbel binnenkomt. De vierkante haken zijn soepel: ze overschrijven zonder te klagen.

Bij het aanmaken geldt hetzelfde. `new Dictionary<string, int> { { "a", 1 }, { "a", 2 } }` gebruikt `Add` en crasht. `new Dictionary<string, int> { ["a"] = 1, ["a"] = 2 }` gebruikt de vierkante haken: `a` wordt 2.

Wat het ook kan, via iets anders

Een woordenboek is ook een **reeks**: een reeks **paren**. Elk paar is een `KeyValuePair` met een `.Key` en een `.Value`. Daardoor werkt alles wat met een reeks werkt ook op een woordenboek — maar je werkt dan telkens met paren, niet met losse waarden. Daarnaast zijn `Keys` en `Values` elk een eigen reeks.

Bron	Waarom het op een woordenboek werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks, hier een reeks paren	hoofdstuk 11
LINQ op <code>Keys</code> of <code>Values</code>	elk een reeks van losse sleutels of waarden	hoofdstuk 11
<code>foreach</code>	doorloopt de paren	hoofdstuk 4
<code>string.Join</code>	zet sleutels, waarden of paren in één tekst	hoofdstuk 1
andere verzamelingen	een lijst of set van de sleutels of waarden	hoofdstukken 6 en 8
wat elk object kan	een woordenboek is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je woordenboek nooit**. Ze geeft een nieuw resultaat.
- Dat resultaat is **geen woordenboek** maar een reeks paren. Wil je er weer een woordenboek van, zet er dan `.ToDictionary()` achter.
- Wie op de waarden wil werken, schrijft `p => p.Value`; wie op de sleutels wil werken, `p => p.Key`. Of je werkt meteen op `prijzen.Values` of `prijzen.Keys`.
- LINQ werkt enkel met `using System.Linq`; bovenaan. In een nieuw project staat die er automatisch bij.

OPMERKING

CountBy en AggregateBy bestaan sinds .NET 9, Shuffle, LeftJoin en RightJoin sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken.

Eén paar nemen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
First	het eerste paar dat voldoet	prijzen.First(p => p.Value > 100)	[combi, 120]; niets gevonden geeft een InvalidOperationException
FirstOrDefault	het eerste paar dat voldoet, of een leeg paar	prijzen.FirstOrDefault(p => p.Value > 500)	[, 0]: Key is null, Value is 0
Single	het enige paar dat voldoet	prijzen.Single(p => p.Value > 200)	[vip, 210]
SingleOrDefault	het enige paar, of een leeg paar	prijzen.SingleOrDefault(p => p.Value > 100)	InvalidOperationException : er zijn er twee
Last	het laatste paar	prijzen.Last()	[vip, 210]
LastOrDefault	het laatste dat voldoet, of een leeg paar	prijzen.LastOrDefault(p => p.Value < 0)	[, 0]
ElementAt	het paar op een positie	prijzen.ElementAt(1)	[combi, 120]
ElementAtOrDefault	het paar op een positie, of een leeg paar	prijzen.ElementAtOrDefault(9)	[, 0]

LET OP

FirstOrDefault en de andere ...OrDefault-methodes geven bij een woordenboek **nooit** null. Een KeyValuePair is een waarde, geen object: niets gevonden geeft een leeg paar, met null of 0 als sleutel en de standaardwaarde als waarde. if (paar == null) werkt dus niet. Controleer paar.Key == null, of gebruik Any vooraf. Zoek je op de sleutel, gebruik dan gewoon TryGetValue.

Weten of iets erin zit

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
Any	is er minstens één paar	prijzen.Any()	true	Count > 0
Any	is er een paar dat voldoet	prijzen.Any(p => p.Value > 200)	true	—
All	voldoen ze allemaal	prijzen.All(p => p.Value >= 65)	true	—
Contains	zit dit exacte paar erin	prijzen.Contains(new KeyValuePair<string, int>("dag", 65))	true; met 70 false	—
Keys.Contains	bestaat die sleutel	prijzen.Keys.Contains("vip")	true	ContainsKey
Values.Contains	komt die waarde voor	prijzen.Values.Contains(120)	true	ContainsValue

Een deel nemen

Komt van LINQ. Het resultaat is een reeks paren, geen woordenboek.

Methode	Wat het doet	Voorbeeld	Resultaat
Where	enkel de paren die voldoen	prijzen.Where(p => p.Value > 100)	[combi, 120], [vip, 210]
Select	maak van elk paar iets anders	prijzen.Select(p => \$"{p.Key}: {p.Value}")	dag: 65, combi: 120, vip: 210
SelectMany	plat de lijsten in de waarden	klassen.SelectMany(p => p.Value)	alle leerlingen van alle klassen samen
Distinct	gooi dubbels weg, hier op de waarden	telling.Values.Distinct()	3, 1
Take, Skip	de eerste n, of alles behalve de eerste n	prijzen.Take(2)	[dag, 65], [combi, 120]
TakeWhile, SkipWhile	neem of sla over zolang de voorwaarde klopt	prijzen.TakeWhile(p => p.Value < 200)	[dag, 65], [combi, 120]
Chunk	verdeel in stukken van n paren	prijzen.Chunk(2)	twee stukken: twee paren, en één paar

Take, Skip, TakeWhile en Chunk volgen de volgorde waarin je het woordenboek doorloopt. Die volgorde is niet gegarandeerd; zie *Let op* hieronder.

Tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
Count	hoeveel paren voldoen	prijzen.Count(p => p.Value > 100)	2
Sum	tel de waarden op	prijzen.Sum(p => p.Value) of prijzen.Values.Sum()	395

Methode	Wat het doet	Voorbeeld	Resultaat
Min, Max	de kleinste of grootste waarde	<code>prijzen.Values.Max()</code>	210
Average	het gemiddelde	<code>prijzen.Values.Average()</code>	131.666...
MinBy, MaxBy	het paar met de kleinste of grootste waarde	<code>prijzen.MaxBy(p => p.Value)</code>	[vip, 210]
MaxBy(...).Key	de sleutel met de grootste waarde	<code>telling.MaxBy(p => p.Value).Key</code>	kat
CountBy	tel per waarde van iets	<code>dieren.CountBy(d => d)</code>	[kat, 3], [hond, 1], [vis, 1]: het telpatroon in één regel
Aggregate, AggregateBy	voeg samen met een eigen regel, eventueel per sleutel	<code>fans.AggregateBy(f => f.Merk, 0m, (som, f) => som + f.Prijs)</code>	de totale prijs per merk

Welk dier komt het vaakst voor? is `telling.MaxBy(p => p.Value).Key`. Met `Max` krijg je enkel het getal 3, met `MaxBy` het paar, en met `.Key` erachter de naam.

Ordenen en groeperen

Komt van LINQ. Een woordenboek heeft zelf geen volgorde om te sorteren. Je krijgt een geordende reeks paren.

Methode	Wat het doet	Voorbeeld	Resultaat
OrderBy	sorteer oplopend op iets	<code>prijzen.OrderBy(p => p.Key)</code>	[combi, 120], [dag, 65], [vip, 210]
OrderByDescending	sorteer aflopend	<code>prijzen.OrderByDescending(p => p.Value)</code>	[vip, 210], [combi, 120], [dag, 65]
ThenBy, ThenByDescending	tweede sorteersleutel	<code>telling.OrderByDescending(p => p.Value).ThenBy(p => p.Key)</code>	[kat, 3], [hond, 1], [vis, 1]
Order, OrderDescending	sorteer de waarden zelf	<code>prijzen.Keys.Order()</code>	combi, dag, vip
Reverse	omgekeerde volgorde	<code>prijzen.Reverse()</code>	[vip, 210], [combi, 120], [dag, 65]
Shuffle	willekeurige volgorde	<code>prijzen.Shuffle()</code>	elke keer anders
GroupBy	maak groepen	<code>telling.GroupBy(p => p.Value)</code>	groep 3: kat; groep 1: hond en vis
ToLookup	maak groepen, meteen vastgelegd	<code>telling.ToLookup(p => p.Value, p => p.Key)</code>	opzoek[1] geeft hond, vis

LET OP

`prijzen.Order()` zonder sleutel crasht met een **InvalidOperationException**: C# weet niet hoe het twee paren moet vergelijken. Zeg altijd waarop: `OrderBy(p => p.Key)` of `OrderBy(p => p.Value)`.

Twee reeksen combineren

Komt van LINQ. In de voorbeelden is `extra` een tweede woordenboek met `kind = 30` en `dag = 70`.

Methode	Wat het doet	Voorbeeld	Resultaat
Concat	plak de paren achter elkaar	<code>prijzen.Concat(extra)</code>	vijf paren, dag twee keer
UnionBy	alles uit beide, één paar per sleutel	<code>prijzen.UnionBy(extra, p => p.Key)</code>	<code>[dag, 65],</code> <code>[combi, 120],</code> <code>[vip, 210],</code> <code>[kind, 30]</code>
IntersectBy	de paren waarvan de sleutel ook in de tweede zit	<code>prijzen.IntersectBy(extra.Keys, p => p.Key)</code>	<code>[dag, 65]</code>
ExceptBy	de paren waarvan de sleutel niet in de tweede zit	<code>prijzen.ExceptBy(extra.Keys, p => p.Key)</code>	<code>[combi, 120],</code> <code>[vip, 210]</code>
Union, Intersect, Except	hetzelfde op hele paren	<code>prijzen.Keys.Except(extra.Keys)</code>	<code>combi, vip</code>
Append, Prepend	één paar achteraan of vooraan, in het resultaat	<code>prijzen.Append(new KeyValuePair<string, int>("kind", 30))</code>	vier paren; prijzen blijft drie
Zip	koppel twee reeksen per positie	<code>prijzen.Keys.Zip(prijzen.Values)</code>	<code>(dag, 65),</code> <code>(combi, 120),</code> <code>(vip, 210)</code>
Join, LeftJoin, RightJoin, GroupJoin	koppel items uit twee reeksen met dezelfde sleutel	<code>bestellingen.Join(prijzen, b => b.Ticket, p => p.Key, (b, p) => p.Value)</code>	de prijs per bestelling

Een `Join` met een woordenboek is zelden nodig: met `prijzen[b.Ticket]` zoek je de prijs meteen op.

LET OP

`prijzen.Concat(extra).ToDictionary()` crasht met een **ArgumentException**: dag komt twee keer voor. Neem eerst `UnionBy(..., p => p.Key)`: dan wint het eerste woordenboek.

Een woordenboek maken, of er iets anders van maken

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
---------	--------------	-----------	-----------

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToDictionary</code>	maak van paren weer een woordenboek	<code>prijzen.Where(p => p.Value > 100).ToDictionary()</code>	een Dictionary met combi en vip
<code>ToDictionary</code>	maak een woordenboek uit een andere reeks	<code>fruit.ToDictionary(f => f, f => f.Length)</code>	<code>[appel, 5], [peer, 4], [kiwi, 4]</code>
<code>ToDictionary</code>	met een eigen vergelijking van de sleutels	<code>woorden.ToDictionary(w => w, w => 0, StringComparer.OrdinalIgnoreCase)</code>	sleutels zonder onderscheid in hoofdletters
<code>GroupBy + ToDictionary</code>	per sleutel een lijst	<code>fans.GroupBy(f => f.Merk).ToDictionary(g => g.Key, g => g.ToList())</code>	<code>tabel["NZXT"]</code> geeft twee fans
<code>ToList</code>	een lijst van paren	<code>prijzen.ToList()</code>	<code>lijst[0]</code> is <code>[dag, 65]</code>
<code>ToArray</code>	een array van paren	<code>prijzen.ToArray()</code>	een <code>KeyValuePair<string, int>[]</code>
<code>Keys.ToList</code> , <code>Values.ToList</code>	een lijst van de sleutels of waarden	<code>prijzen.Values.ToList()</code>	<code>65, 120, 210</code>
<code>Values.ToHashSet</code>	de verschillende waarden	<code>telling.Values.ToHashSet()</code>	<code>3, 1</code>

Doorlopen

Komt van de taal C# zelf.

Schrijfwijze	Wat het doet	Resultaat
<code>foreach (KeyValuePair<string, int> p in prijzen)</code>	elk paar	<code>p.Key</code> en <code>p.Value</code>
<code>foreach (var (ticket, prijs) in prijzen)</code>	elk paar, meteen in twee variabelen	<code>ticket</code> is de sleutel, <code>prijs</code> de waarde
<code>foreach (string ticket in prijzen.Keys)</code>	elke sleutel	<code>dag, combi, vip</code>
<code>foreach (int prijs in prijzen.Values)</code>	elke waarde	<code>65, 120, 210</code>

Een woordenboek heeft **geen positie** zoals een lijst: `prijzen[0]` zoekt naar de **sleutel** 0, niet naar het eerste paar. Bij `string`-sleutels compileert dat niet; bij `int`-sleutels geeft het een **KeyNotFoundException** als er geen sleutel 0 is. Een paar op positie vraag je met `ElementAt`, maar zie *Let op*.

BELANGRIJK

Tijdens een `foreach` mag je **geen paar bijvoegen**: dat crasht met een **InvalidOperationException**. Een bestaande waarde overschrijven (`prijzen[p.Key] = ...`) en een paar weghalen mag wel. Moet je toch toevoegen, loop dan door een kopie: `foreach (string t in prijzen.Keys.ToList())`.

Omzetten naar iets anders

Komt van de andere verzamelingen en van `string`.

Schrijfwijze	Wat het doet	Resultaat
<code>string.Join(", ", prijzen.Keys)</code>	de sleutels in één tekst	<code>dag, combi, vip</code>
<code>string.Join(", ", prijzen)</code>	de paren in één tekst	<code>[dag, 65], [combi, 120], [vip, 210]</code>
<code>new List<string>(prijzen.Keys)</code>	een lijst van de sleutels	<code>dag, combi, vip</code>
<code>new HashSet<int>(telling.Values)</code>	de verschillende waarden	<code>3, 1</code>
<code>new Dictionary<string, int>(prijzen)</code>	een kopie	een nieuw woordenboek met dezelfde paren

Wat elk object kan

Komt van `object`: elk woordenboek is ook een object. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToString</code>	het object als tekst	<code>prijzen.ToString()</code>	de naam van het type, <code>System.Collections.Generic.Dictionary`2[...]</code>
<code>Equals</code> / <code>==</code>	is dit hetzelfde woordenboek	twee woordenboeken met dezelfde paren	<code>false</code>
<code>GetType</code> , <code>GetHashCode</code>	type en hashcode	<code>prijzen.GetType().Name</code>	<code>Dictionary`2</code>

Een paar toont zich wel netjes: `new KeyValuePair<string, int>("dag", 65).ToString()` geeft `[dag, 65]`.

Bestaat ook, maar kom je in deze cursus niet tegen

`SortedDictionary` (een woordenboek dat altijd op sleutel gesorteerd blijft), `OrderedDictionary<TKey, TValue>` (houdt de volgorde van toevoegen bij en heeft posities, sinds .NET 9), `ConcurrentDictionary` (voor meerdere taken tegelijk), `ReadOnlyDictionary`, `GetAlternateLookup`, en uit LINQ: `AsParallel`, `Cast`, `ToFrozenDictionary`, `ToImmutableDictionary`.

Let op bij een woordenboek

Op het woordenboek zelf	Via LINQ of iets anders	Het verschil
<code>ContainsKey</code>	<code>Keys.Contains, Any(p => p.Key == ...)</code>	Hetzelfde resultaat, maar <code>ContainsKey</code> is snel en <code>Any</code> loopt alle paren af.
<code>ContainsValue</code>	<code>Values.Contains, Any(p => p.Value == ...)</code>	Hetzelfde resultaat; alle drie lopen ze alles af.
<code>TryGetValue</code>	<code>FirstOrDefault(p => p.Key == ...)</code>	<code>TryGetValue</code> zoekt snel en zegt of het lukte. <code>FirstOrDefault</code> loopt alles af en geeft bij niets een leeg paar, geen null.
<code>Count</code>	<code>Count()</code>	Hetzelfde getal. Een voorwaarde kan enkel met LINQ: <code>Count(p => p.Value > 100)</code> .

Op het woordenboek zelf	Via LINQ of iets anders	Het verschil
<code>Remove</code>	<code>Where(p => ...)</code>	<code>Remove</code> haalt echt iets weg. <code>Where</code> geeft een nieuwe reeks paren en laat het woordenboek zoals het is.

LET OP

- **De volgorde is niet gegarandeerd.** Vandaag krijg je de paren meestal terug in de volgorde waarin je ze toevoegde, maar na een `Remove` en een nieuwe `Add` kan dat veranderen. `First`, `Last`, `ElementAt`, `Take`, `Skip` en `Reverse` hangen van die volgorde af. Heb je een volgorde nodig, zet er dan eerst een `OrderBy` voor.
- LINQ geeft **paren**, geen woordenboek. `Dictionary<string, int> duur = prijzen.Where(p => p.Value > 100);` compileert niet. Zet er `.ToDictionary()` achter.
- `...OrDefault` geeft een **leeg paar**, nooit `null`.
- Een `null`-sleutel kan niet: `prijzen[null]` en `Add(null, ...)` geven een **`ArgumentNullException`**. Een `null`-waarde mag wel.
- Een kopie met `new Dictionary<string, int>(tabel)` of `ToDictionary()` neemt de `StringComparer.OrdinalIgnoreCase` van het origineel **niet** mee. Geef hem opnieuw mee als je hem nodig hebt.
- Gebruik je een object van een `class` als **sleutel**, dan vergelijkt het woordenboek op referentie: een nieuw object met dezelfde inhoud vindt het niet. Een `record` als sleutel vergelijkt op inhoud. Zie [hoofdstuk 14](#).

Wanneer wel, wanneer niet

Wel — je zoekt telkens op één en dezelfde eigenschap: een naam, een code, een nummer. En je wil dat snel doen, hoe groot de verzameling ook is.

Niet — wanneer de volgorde ertoe doet, of wanneer je afwisselend op verschillende eigenschappen zoekt. Een woordenboek zoekt snel op het woord, maar niet op de uitleg.

Veelgemaakte fouten

LET OP

- `hoofdsteden["Spanje"]` lezen terwijl Spanje er niet in staat, crasht. Twijfel je, gebruik dan `TryGetValue`, `GetValueOrDefault` of eerst `ContainsKey`.
- Sleutels zijn hoofdlettergevoelig. `"Kat"` en `"kat"` zijn twee verschillende sleutels. Wil je dat niet, geef dan `StringComparer.OrdinalIgnoreCase` mee bij het aanmaken.
- Reken niet op de volgorde bij het doorlopen. Vandaag komen ze toevallig terug in de volgorde waarin je ze toevoegde, maar dat is geen belofte.
- `telling[dier]++` op een sleutel die er nog niet is, crasht: er is nog niets om bij op te tellen.

Voorspel de uitkomst

Vraag 1

```
var d = new Dictionary<string, int>();  
d["a"] = 1;  
d["a"] = 2;  
Console.WriteLine(d["a"]);  
Console.WriteLine(d.Count);
```

[Toon het antwoord ›](#)

Vraag 2

```
var d = new Dictionary<string, int>();  
d.Add("a", 1);  
d.Add("a", 2);
```

[Toon het antwoord ›](#)

Vraag 3

```
var d = new Dictionary<string, int> { ["Kat"] = 4 };  
Console.WriteLine(d.ContainsKey("kat"));
```

[Toon het antwoord ›](#)

Vraag 4

```
var d = new Dictionary<string, int> { ["x"] = 5 };  
bool gevonden = d.TryGetValue("y", out int waarde);  
Console.WriteLine(gevonden);  
Console.WriteLine(waarde);
```

[Toon het antwoord ›](#)

Vraag 5

```
var d = new Dictionary<string, int> { ["dag"] = 65, ["vip"] = 210 };  
var paar = d.FirstOrDefault(p => p.Value > 500);  
Console.WriteLine(paar.Key == null);  
Console.WriteLine(paar.Value);
```

[Toon het antwoord ›](#)

Vraag 6


```
var telling = new Dictionary<string, int> { ["kat"] = 3, ["hond"] = 1 };  
Console.WriteLine(telling.Values.Max());  
Console.WriteLine(telling.MaxBy(p => p.Value).Key);
```

[Toon het antwoord ›](#)

Vraag 7

```
var d = new Dictionary<string, int> { ["a"] = 1, ["b"] = 2 };  
foreach (var p in d)  
{  
    d["c"] = 3;  
}
```

[Toon het antwoord ›](#)

Vraag 8

```
var d = new Dictionary<string, int>(StringComparer.OrdinalIgnoreCase) { ["Kat"] = 4 };  
var kopie = new Dictionary<string, int>(d);  
Console.WriteLine(d.ContainsKey("kat"));  
Console.WriteLine(kopie.ContainsKey("kat"));
```

[Toon het antwoord ›](#)

Vraag 9

```
var d = new Dictionary<string, int> { ["dag"] = 65, ["vip"] = 210 };  
var duur = d.Where(p => p.Value > 100).ToDictionary();  
Console.WriteLine(duur.Count);  
Console.WriteLine(d.Count);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

2 en daarna 1

De tweede regel overschrijft de eerste. Er is nog altijd maar één sleutel.

[Terug naar de vraag](#)

VRAAG 2

ArgumentException — het programma crasht.

Add weigert een sleutel die er al in zit. Dat is het verschil met de vierkante haken uit vraag 1.

[Terug naar de vraag](#)

VRAAG 3

False

"Kat" met hoofdletter en "kat" zonder zijn voor een gewone Dictionary twee verschillende sleutels.

[Terug naar de vraag](#)

VRAAG 4

False en daarna 0

De sleutel "y" bestaat niet. TryGetValue crasht niet maar geeft false terug, en zet waarde op de standaardwaarde van een int: nul.

[Terug naar de vraag](#)

VRAAG 5

True en daarna 0

Geen enkel paar voldoet. FirstOrDefault geeft dan geen null, maar een leeg paar: de sleutel is null, de waarde 0.

[Terug naar de vraag](#)

VRAAG 6

3 en daarna kat

Max op de waarden geeft het hoogste getal. MaxBy geeft het paar met dat getal, en .Key de sleutel ervan.

[Terug naar de vraag](#)

VRAAG 7

InvalidOperationException — het programma crasht.

d["c"] = 3 voegt een nieuwe sleutel toe terwijl de foreach loopt. Een bestaande waarde overschrijven had wel gemogen; iets bijvoegen niet.

[Terug naar de vraag](#)

VRAAG 8

True en daarna False

Het origineel negeert hoofdletters. De kopie kreeg de paren mee, maar niet die manier van vergelijken: daar is "kat" weer een andere sleutel dan "Kat".

[← Terug naar de vraag](#)

VRAAG 9

1 en daarna 2

Where geeft de paren boven 100, ToDictionary maakt er een nieuw woordenboek van met enkel vip. Het origineel verandert niet.

[← Terug naar de vraag](#)

8. HashSet

VERZAMELING

Wat het is

Een verzameling waarin alles maar één keer voorkomt. Zoals een stickeralbum: een dubbele sticker plak je niet opnieuw in, je weet gewoon dat je hem al hebt.

Hoe het eruitziet

```
HashSet<string> bezocht = new HashSet<string>();

bezocht.Add("Gent");
bezocht.Add("Brugge");
bool nieuw = bezocht.Add("Gent");           // false – stond er al in

Console.WriteLine(bezocht.Count);           // 2
Console.WriteLine(bezocht.Contains("Brugge")); // True
```

Een `HashSet` kan ook rekenen met verzamelingen:

Methode	Wat er overblijft
<code>a.UnionWith(b)</code>	alles wat in a óf b zit
<code>a.IntersectWith(b)</code>	enkel wat in a én b zit
<code>a.ExceptWith(b)</code>	wat in a zit maar niet in b

Die drie veranderen `a` zelf.

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens deze verzamelingen, zodat je elk resultaat kan narekenen.

```
HashSet<string> steden = new HashSet<string> { "Gent", "Brugge", "Antwerpen", "Leuven" };
HashSet<int> a = new HashSet<int> { 1, 2, 3, 4 };
HashSet<int> b = new HashSet<int> { 3, 4, 5 };
```

Wat je ermee kan

Dit zijn de methodes en eigenschappen die op een `HashSet` zelf zitten.

Toevoegen en verwijderen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Add</code>	voeg toe, als het er nog niet in zit	<code>steden.Add("Mechelen")</code>	<code>true</code> ; <code>steden.Add("Gent")</code> geeft <code>false</code>

Methode	Wat het doet	Voorbeeld	Resultaat
Remove	haal dit item weg	<code>steden.Remove("Brugge")</code>	true ; iets wat er niet in zit geeft false
RemoveWhere	haal alles weg dat voldoet	<code>steden.RemoveWhere(s => s.Length > 5)</code>	3 : enkel Gent blijft over
Clear	maak leeg	<code>steden.Clear()</code>	Count is 0

Opzoeken

Methode	Wat het doet	Voorbeeld	Resultaat
Count	hoeveel zitten erin; een eigenschap	<code>steden.Count</code>	4
Contains	zit dit erin; snel, hoe groot de set ook is	<code>steden.Contains("Gent")</code>	true ; "gent" geeft false
TryGetValue	geef het exemplaar dat al in de set zit	<code>namen.TryGetValue("BRAM", out string? gevonden)</code>	true , gevonden is Bram (bij een set die hoofdletters negeert)
Comparer	hoe items vergeleken worden	<code>steden.Comparer</code>	de standaardvergelijking, of wat je bij het aanmaken meegaf

Rekenen met verzamelingen: de set zelf veranderen

Methode	Wat het doet	Voorbeeld	a wordt
UnionWith	voeg alles uit de tweede toe	<code>a.UnionWith(b)</code>	1, 2, 3, 4, 5
IntersectWith	hou enkel wat ook in de tweede zit	<code>a.IntersectWith(b)</code>	3, 4
ExceptWith	haal weg wat in de tweede zit	<code>a.ExceptWith(b)</code>	1, 2
SymmetricExceptWith	hou wat in precies één van beide zit	<code>a.SymmetricExceptWith(b)</code>	1, 2, 5

Deze vier veranderen **a** en geven niets terug. **b** blijft zoals het was. De tweede mag elke reeks zijn, ook een lijst met dubbels: die dubbels komen er niet bij.

Rekenen met verzamelingen: enkel een vraag stellen

Methode	Vraag	Voorbeeld	Resultaat
IsSubsetOf	zit alles van mij ook in de tweede	<code>b.IsSubsetOf(a)</code>	false : de 5 zit niet in a
IsSupersetOf	zit alles van de tweede ook in mij	<code>a.IsSupersetOf(new[] { 1, 2 })</code>	true
IsProperSubsetOf	deelverzameling, én de tweede heeft meer	<code>a.IsProperSubsetOf(a)</code>	false : even groot
IsProperSupersetOf	ik heb alles van de tweede, én meer	<code>a.IsProperSupersetOf(new[] { 1, 2 })</code>	true
Overlaps	hebben we minstens één gemeen	<code>a.Overlaps(b)</code>	true
SetEquals	dezelfde inhoud, volgorde en dubbels tellen niet	<code>a.SetEquals(new[] { 4, 3, 2, 1, 1 })</code>	true

Aanmaken, kopiëren en plaats reserveren

Schrijfwijze	Wat het doet	Resultaat
<code>new HashSet<int>(getallen)</code>	een set uit een reeks: dubbels vallen weg	uit 5, 8, 3, 8, 1 wordt 5, 8, 3, 1
<code>new HashSet<string>(StringComparer.OrdinalIgnoreCase)</code>	een set die hoofdletters negeert	"Gent" en "GENT" zijn hetzelfde
<code>steden.CopyTo(doel)</code>	kopieer naar een bestaande array	<code>doel</code> bevat de vier steden
<code>steden.Capacity</code>	hoeveel er passen voor de set moet groeien	minstens <code>Count</code>
<code>steden.EnsureCapacity(100)</code>	reserveer alvast plaats	er passen er minstens 100
<code>steden.TrimExcess()</code>	geef de vrije plaats terug	zelden nodig

Wat het ook kan, via iets anders

Een `HashSet` is ook een **reeks**: iets wat je één voor één kan doorlopen (`IEnumerable`, [hoofdstuk 12](#)). Alles wat met een reeks werkt, werkt dus ook met een set — met één belangrijke kanttekening: een set heeft **geen gegarandeerde volgorde**.

Bron	Waarom het op een set werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
<code>foreach</code>	<code>foreach</code> doorloopt elke reeks	hoofdstuk 4
<code>string.Join</code>	wil een reeks	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 5 tot 10
wat elk object kan	een set is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je set nooit**. Ze geeft een nieuw resultaat.
- Dat resultaat is **geen** `HashSet` maar een reeks. Wil je er weer een set van, zet er dan `.ToHashSet()` achter.
- LINQ werkt enkel met `using System.Linq;` bovenaan. In een nieuw project staat die er automatisch bij.

OPMERKING

`Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken.

Eén item nemen

Komt van LINQ. Alles in deze tabel hangt af van de volgorde, en die is bij een set niet gegarandeerd. De resultaten zijn die van vandaag.

Methode	Wat het doet	Voorbeeld	Resultaat
First, FirstOrDefault	het eerste dat voldoet, of niets	<code>steden.FirstOrDefault(s => s.StartsWith("Z"))</code>	<code>null</code>
Single, SingleOrDefault	het enige dat voldoet	<code>steden.Single(s => s.Length == 9)</code>	<code>Antwerpen</code>
Last, LastOrDefault	het laatste dat voldoet	<code>steden.Last(s => s.Length == 6)</code>	<code>Leuven</code>
ElementAt, ElementAtOrDefault	het item op een positie	<code>steden.ElementAt(0)</code>	<code>Gent, vandaag</code>

`Single` is bij een set het enige item uit deze tabel dat niet van de volgorde afhangt: er is er maar één die voldoet, of het crasht.

Weten of iets erin zit

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
Any	is er minstens één (die voldoet)	<code>steden.Any(s => s.Length > 8)</code>	<code>true</code>	<code>Count > 0</code>
All	voldoen ze allemaal	<code>steden.All(s => s.Length > 4)</code>	<code>false</code> : Gent heeft er 4	—
Contains	zit dit erin	<code>steden.Contains("Gent")</code>	<code>true</code>	Contains: de set gebruikt de eigen, snelle versie
SequenceEqual	dezelfde inhoud in dezelfde volgorde	<code>a.SequenceEqual(new[] { 4, 3, 2, 1 })</code>	<code>false</code>	SetEquals geeft <code>true</code>

Een deel nemen

Komt van LINQ. Het resultaat is een reeks, geen set.

Methode	Wat het doet	Voorbeeld	Resultaat
Where	enkel wat voldoet	<code>steden.Where(s => s.Length == 6)</code>	<code>Brugge, Leuven</code>
Select	maak van elk item iets anders	<code>steden.Select(s => s.Length)</code>	<code>4, 6, 9, 6</code> : dubbels kunnen terug opduiken
SelectMany	plat een lijst van lijsten	<code>sets.SelectMany(s => s)</code>	alles samen, mét dubbels
Distinct, DistinctBy	gooi dubbels weg (op iets)	<code>steden.DistinctBy(s => s.Length)</code>	<code>Gent, Brugge, Antwerpen</code>
Take, Skip, TakeLast, SkipLast	de eerste of laatste n, volgens de huidige volgorde	<code>steden.Order().Take(2)</code>	<code>Antwerpen, Brugge</code>
TakeWhile, SkipWhile, Chunk, Index	zoals bij een lijst	<code>steden.Chunk(2)</code>	twee stukken van twee
OfType	enkel de items van een bepaald type	<code>onderdelen.OfType<Processor>()</code>	enkel de processors

Wil je een vaste volgorde, zet er dan eerst `Order()` of `OrderBy(...)` voor, zoals in het voorbeeld bij `Take`.

Tellen en rekenen

Komt van *LINQ*.

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Count</code>	hoeveel er voldoen	<code>steden.Count(s => s.Length == 6)</code>	2
<code>Sum</code>	tel op	<code>a.Sum()</code>	10
<code>Min</code> , <code>Max</code> , <code>Average</code>	kleinste, grootste, gemiddelde	<code>steden.Average(s => s.Length)</code>	6.25
<code>MinBy</code> , <code>MaxBy</code>	het item met de kleinste of grootste waarde	<code>steden.MaxBy(s => s.Length)</code>	Antwerpen
<code>CountBy</code>	tel per waarde van iets	<code>steden.CountBy(s => s.Length)</code>	[4, 1], [6, 2], [9, 1]
<code>Aggregate</code> , <code>AggregateBy</code>	voeg samen met een eigen regel	<code>steden.Aggregate((x, y) => x + "-" + y)</code>	de vier steden aan elkaar

LET OP

`Average`, `Min` en `Max` op een lege set van getallen geven een `InvalidOperationException`. `Sum` geeft dan 0.

Ordenen en groeperen

Komt van *LINQ*. Een set zelf heeft geen volgorde om te sorteren; je krijgt een geordende reeks.

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Order</code> , <code>OrderDescending</code>	sorteer de waarden	<code>steden.Order()</code>	Antwerpen, Brugge, Gent, Leuven
<code>OrderBy</code> , <code>OrderByDescending</code>	sorteer op iets	<code>steden.OrderBy(s => s.Length)</code>	Gent, Brugge, Leuven, Antwerpen
<code>ThenBy</code> , <code>ThenByDescending</code>	tweede sorteersleutel	<code>steden.OrderBy(s => s.Length).ThenByDescending(s => s)</code>	Gent, Leuven, Brugge, Antwerpen
<code>Reverse</code> , <code>Shuffle</code>	omgekeerde of willekeurige volgorde	<code>steden.Reverse()</code>	de huidige volgorde omgekeerd
<code>GroupBy</code> , <code>ToLookup</code>	maak groepen	<code>steden.GroupBy(s => s.Length)</code>	4: Gent; 6: Brugge en Leuven; 9: Antwerpen

Twee reeksen combineren

Komt van *LINQ*. Dit zijn de LINQ-tegenhangers van `UnionWith` en zijn familie. Ze geven een nieuwe reeks en laten `a` en `b` zoals ze zijn.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
Union	alles uit beide, zonder dubbels	<code>a.Union(b)</code>	1, 2, 3, 4, 5	<code>UnionWith</code>
Intersect	wat in beide zit	<code>a.Intersect(b)</code>	3, 4	<code>IntersectWith</code>
Except	wat in de eerste zit en niet in de tweede	<code>a.Except(b)</code>	1, 2	<code>ExceptWith</code>
Concat	plak achter elkaar, mét dubbels	<code>a.Concat(b)</code>	1, 2, 3, 4, 3, 4, 5	—
UnionBy, IntersectBy, ExceptBy	hetzelfde, vergeleken op één eigenschap	<code>steden.ExceptBy(new[] { 6 }, s => s.Length)</code>	Gent, Antwerpen	—
Append, Prepend	één item erbij, in het resultaat	<code>a.Append(3)</code>	1, 2, 3, 4, 3: het resultaat is geen set meer	<code>Add</code>
Zip	koppel per positie	<code>steden.Zip(a)</code>	(Gent, 1), (Brugge, 2), ...	—
Join, LeftJoin, RightJoin, GroupJoin	koppel items met dezelfde sleutel	<code>bezoekers.Join(steden, ...)</code>	zoals bij een lijst (hoofdstuk 6)	—

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
ToHashSet	maak er (weer) een set van	<code>steden.Select(s => s.Length).ToHashSet()</code>	4, 6, 9
ToList	maak er een lijst van, met posities	<code>steden.Order().ToList()</code>	lijst[0] is Antwerpen
ToArray	maak er een array van	<code>a.ToArray()</code>	1, 2, 3, 4
ToDictionary, ToLookup	maak er een opzoektabel of groepen van	<code>steden.ToDictionary(s => s, s => s.Length)</code>	tabel["Gent"] is 4

LET OP

`ToHashSet()` zonder meer neemt een `StringComparer.OrdinalIgnoreCase` **niet** mee. Geef hem opnieuw mee: `namen.ToHashSet(StringComparer.OrdinalIgnoreCase)`.

Doorlopen

Komt van de taal C# zelf.

Schrijfwijze	Wat het doet	Resultaat
--------------	--------------	-----------

Schrijfwijze	Wat het doet	Resultaat
<code>foreach (string s in steden)</code>	elk item één keer	de vier steden, in de huidige volgorde
<code>HashSet<int> samen = [.. a, .. b];</code>	een nieuwe set uit twee reeksen	1, 2, 3, 4, 5
<code>HashSet<string> s = ["Gent", "Gent"];</code>	korte schrijfwijze; dubbels vallen weg	Count is 1

Een set heeft **geen positie**: `steden[0]` compileert niet, en een `for` met een teller kan dus niet. Wil je toch posities, maak er dan eerst een lijst van met `ToList()`.

BELANGRIJK

Tijdens een `foreach` mag je **niets bijvoegen**: dat crasht met een `InvalidOperationException`. Wegnemen tijdens het doorlopen doe je beter met `RemoveWhere`.

Omzetten naar iets anders

Komt van de andere verzamelingen en van `string`.

Schrijfwijze	Wat het doet	Resultaat
<code>string.Join(", ", steden)</code>	alles in één tekst	Gent, Brugge, Antwerpen, Leuven
<code>new List<string>(steden)</code>	een lijst	met posities en <code>Sort</code>
<code>new Queue<string>(steden)</code>	een wachtrij, in de huidige volgorde	<code>Dequeue</code> geeft Gent
<code>new Stack<string>(steden)</code>	een stapel	<code>Pop</code> geeft Leuven

Wat elk object kan

Komt van `object`: elke set is ook een object. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToString</code>	het object als tekst	<code>steden.ToString()</code>	<code>System.Collections.Generic.HashSet`1[System.String]</code>
<code>Equals</code> / <code>==</code>	is dit dezelfde set	twee sets met dezelfde inhoud	<code>false</code> ; vergelijk de inhoud met <code>SetEquals</code>
<code>GetType</code> , <code>GetHashCode</code>	type en hashcode	<code>steden.GetType().Name</code>	<code>HashSet`1</code>

Bestaat ook, maar kom je in deze cursus niet tegen

`SortedSet<T>` (een set die altijd gesorteerd blijft, met `Min` en `Max` als eigenschap), `HashSet.CreateSetComparer`, `GetAlternateLookup`, `ToFrozenSet`, `ToImmutableHashSet`, en uit LINQ `AsParallel` en `Cast`.

Let op bij een HashSet

Op de set zelf	Via LINQ	Het verschil
UnionWith	Union	UnionWith verandert de set en geeft niets terug. Union geeft een nieuwe reeks en laat beide sets zoals ze zijn. Hetzelfde voor IntersectWith/Intersect en ExceptWith/Except.
SetEquals	SequenceEqual	SetEquals kijkt enkel naar de inhoud. SequenceEqual kijkt ook naar de volgorde, en die is bij een set niet gegarandeerd. Bij een set wil je dus bijna altijd SetEquals.
Contains	Any(s => s == ...)	Hetzelfde resultaat, maar Contains is snel en volgt de vergelijking van de set (bv. zonder hoofdletters). Any loopt alles af en vergelijkt met ==: met hoofdletters.
Add	Append	Add voegt toe als het nog niet bestaat. Append zet het er gewoon achter, in een reeks die geen set meer is: dubbels zijn dan mogelijk.
RemoveWhere	Where	RemoveWhere haalt echt weg. Where geeft een nieuwe reeks met wat overblijft.
Count	Count()	Hetzelfde getal. Een voorwaarde kan enkel met LINQ.

LET OP

- **De volgorde is niet gegarandeerd.** Vandaag krijg je de items meestal terug in de volgorde van toevoegen, maar dat is geen belofte. First, ElementAt, Take en Reverse hangen daarvan af.
- Een set van objecten van een class vergelijkt op **referentie**: twee verschillende objecten met dezelfde inhoud zijn voor de set twee items, en beide Adds geven true. Een record vergelijkt op inhoud: de tweede Add geeft false. Zie hoofdstuk 13 en hoofdstuk 14.
- Select op een set kan **dubbels** opleveren: vier steden geven vier lengtes, waarvan twee keer 6.

Wanneer wel, wanneer niet

Wel — dubbels mogen niet voorkomen, of je moet vaak en snel nagaan of iets er al in zit.

Niet — wanneer de volgorde ertoe doet, of wanneer je iets op een positie wil opvragen. Een HashSet heeft geen [0].

Veelgemaakte fouten

LET OP

- Een dubbele Add crasht niet, hij geeft gewoon false terug. Controleer je dat niet, dan merk je nooit dat er iets dubbel binnenkwam.
- Er bestaat geen eerste, tweede of derde. Wil je op positie opvragen, dan is dit de verkeerde bouwsteen.
- "Gent" en "gent" zijn twee verschillende items, tenzij je bij het aanmaken StringComparer.OrdinalIgnoreCase meegeeft.

Voorspel de uitkomst

Vraag 1

```
var s = new HashSet<int> { 1, 2, 2, 3, 3, 3 };  
Console.WriteLine(s.Count);
```

[Toon het antwoord ›](#)

Vraag 2

```
var s = new HashSet<string>();  
Console.WriteLine(s.Add("a"));  
Console.WriteLine(s.Add("a"));
```

[Toon het antwoord ›](#)

Vraag 3

```
var a = new HashSet<int> { 1, 2, 3 };  
var b = new HashSet<int> { 2, 3, 4 };  
a.IntersectWith(b);  
Console.WriteLine(a.Count);
```

[Toon het antwoord ›](#)

Vraag 4

```
var a = new HashSet<int> { 1, 2, 3 };  
var b = new HashSet<int> { 2, 3, 4 };  
var samen = a.Union(b);  
Console.WriteLine(a.Count);  
Console.WriteLine(samen.Count());
```

[Toon het antwoord ›](#)

Vraag 5

```
var a = new HashSet<int> { 1, 2, 3 };  
Console.WriteLine(a.SetEquals(new[] { 3, 2, 1, 1 }));  
Console.WriteLine(a.SequenceEqual(new[] { 3, 2, 1 }));
```

[Toon het antwoord ›](#)

Vraag 6

```
var namen = new HashSet<string>(StringComparer.OrdinalIgnoreCase) { "Anna" };  
Console.WriteLine(namen.Contains("anna"));  
Console.WriteLine(namen.Any(n => n == "anna"));
```

[Toon het antwoord ›](#)

Vraag 7

```
var s = new HashSet<string> { "Gent", "Brugge", "Leuven" };  
var lengtes = s.Select(x => x.Length);  
Console.WriteLine(lengtes.Count());  
Console.WriteLine(lengtes.ToHashSet().Count);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

3

De dubbels worden stilzwijgend genegeerd. Er blijven drie verschillende getallen over.

[Terug naar de vraag](#)

VRAAG 2

True en daarna False

De eerste keer is "a" nieuw. De tweede keer niet meer.

[Terug naar de vraag](#)

VRAAG 3

2

Enkel wat in beide zit, blijft over in a: 2 en 3.

[Terug naar de vraag](#)

VRAAG 4

3 en daarna 4

Union is LINQ: a blijft zoals het was, het resultaat zit in samen. Met `a.UnionWith(b)` was a zelf vier groot geworden.

[Terug naar de vraag](#)

VRAAG 5

True en daarna False

`SetEquals` kijkt enkel of dezelfde getallen erin zitten. `SequenceEqual` kijkt ook naar de volgorde, en die is anders.

[Terug naar de vraag](#)

VRAAG 6

True en daarna False

`Contains` gebruikt de vergelijking van de set, die hoofdletters negeert. `Any` vergelijkt met `==`, en voor `==` zijn "Anna" en "anna" verschillend.

[Terug naar de vraag](#)

VRAAG 7

3 en daarna 2

`Select` geeft drie lengtes: 4, 6 en 6. Dat resultaat is geen set meer, dus de dubbele 6 blijft staan. Pas `ToHashSet` gooit hem weg.

[Terug naar de vraag](#)

9. Queue

VERZAMELING

Wat het is

Een wachtrij. Wie eerst komt, wordt eerst geholpen. Je kan niet voorsteken, en je kan niemand uit het midden halen.

Hoe het eruitziet

```
Queue<string> wachtrij = new Queue<string>();

wachtrij.Enqueue("Anna");      // achteraan aansluiten
wachtrij.Enqueue("Bram");
wachtrij.Enqueue("Cleo");

Console.WriteLine(wachtrij.Peek());    // Anna – kijken wie vooraan staat
string volgende = wachtrij.Dequeue();  // Anna – en nu is ze weg
Console.WriteLine(wachtrij.Count);     // 2
```

De voorbeelden in dit hoofdstuk

De tabellen hieronder vertrekken telkens van deze wachtrij: Anna staat vooraan, Cleo achteraan.

```
Queue<string> wachtrij = new Queue<string>();
wachtrij.Enqueue("Anna");
wachtrij.Enqueue("Bram");
wachtrij.Enqueue("Cleo");
```

Wat je ermee kan

Dit zijn alle methodes en eigenschappen die op een `Queue` zelf zitten.

Methode	Wat het doet	Voorbeeld	Resultaat
Enqueue	sluit achteraan aan	<code>wachtrij.Enqueue("Dirk")</code>	Dirk staat achter Cleo
Dequeue	haal de eerste eruit en geef hem terug	<code>wachtrij.Dequeue()</code>	Anna , en Anna is weg; leeg geeft een InvalidOperationException
Peek	kijk wie vooraan staat, zonder weg te halen	<code>wachtrij.Peek()</code>	Anna ; leeg geeft een InvalidOperationException
TryDequeue	haal de eerste eruit als er iemand is	<code>wachtrij.TryDequeue(out string? eerste)</code>	true en eerste is Anna; leeg geeft false

Methode	Wat het doet	Voorbeeld	Resultaat
TryPeek	kijk vooraan als er iemand is	wachtrij.TryPeek(out string? eerste)	true en eerste is Anna; leeg geeft false
Count	hoeveel staan er; een eigenschap	wachtrij.Count	3
Contains	staat deze persoon in de rij	wachtrij.Contains("Bram")	true
Clear	maak leeg	wachtrij.Clear()	Count is 0
ToArray	een array, van voor naar achter	wachtrij.ToArray()	Anna, Bram, Cleo
CopyTo	kopieer naar een bestaande array	wachtrij.CopyTo(doel, 0)	doel begint met Anna, Bram, Cleo
Capacity	hoeveel er passen voor de rij moet groeien; een eigenschap	wachtrij.Capacity	minstens 3
EnsureCapacity, TrimExcess	reserveer plaats, of geef ze terug	wachtrij.EnsureCapacity(100)	zelden nodig

Er is **geen** `Remove` voor iemand uit het midden, en **geen** `[0]` om op positie te kijken. Dat is geen gemis maar de bedoeling: een wachtrij heeft maar twee plaatsen die tellen, vooraan en achteraan.

Een rij volledig afwerken doe je zo:

```
while (wachtrij.Count > 0)
{
    string klant = wachtrij.Dequeue();
    Console.WriteLine($"{klant} is geholpen");
}
```

Of korter, zonder `Count`: `while (wachtrij.TryDequeue(out string? klant)) { ... }`.

Wat het ook kan, via iets anders

Een wachtrij is ook een **reeks** (`IEnumerable`, [hoofdstuk 12](#)): je kan ze van voor naar achter doorlopen zonder iemand weg te halen. Daardoor werkt alles wat met een reeks werkt ook op een wachtrij.

Bron	Waarom het op een wachtrij werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
<code>foreach</code>	doorloopt van voor naar achter, zonder iemand weg te halen	hoofdstuk 4
<code>string.Join</code>	wil een reeks	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 5 tot 10
wat elk object kan	een wachtrij is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je wachtrij nooit**: ze haalt niemand weg en zet niemand erbij. `wachtrij.First()` kijkt, zoals `Peek`.
- LINQ overloopt de rij **van voor naar achter**: Anna, Bram, Cleo.
- Het resultaat is **geen wachtrij** maar een reeks. Wil je er weer een wachtrij van, geef het dan mee aan `new Queue<string>(...)`.

OPMERKING

`Index`, `CountBy` en `AggregateBy` bestaan sinds .NET 9, `Shuffle`, `LeftJoin` en `RightJoin` sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken.

Eén persoon nemen of iets nagaan

Komt van *LINQ*. Niemand verlaat de rij.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
First , FirstOrDefault	de eerste (die voldoet)	<code>wachtrij.First()</code>	Anna; Count blijft 3	<code>Peek</code> , <code>TryPeek</code>
Last , LastOrDefault	de laatste (die voldoet)	<code>wachtrij.Last()</code>	Cleo	—
Single , SingleOrDefault	de enige die voldoet	<code>wachtrij.Single(n => n.StartsWith("B"))</code>	Bram	—
ElementAt , ElementAtOrDefault	wie staat er op plaats n	<code>wachtrij.ElementAt(1)</code>	Bram	—
Any , All	staat er iemand (die voldoet), voldoen ze allemaal	<code>wachtrij.Any(n => n.Length > 4)</code>	false	<code>Count > 0</code>
Contains	staat deze persoon erin	<code>wachtrij.Contains("Bram")</code>	true	<code>Contains</code> : dezelfde
SequenceEqual	dezelfde mensen in dezelfde volgorde	<code>wachtrij.SequenceEqual(new[] { "Anna", "Bram", "Cleo" })</code>	true	—

Een deel nemen, tellen en rekenen

Komt van *LINQ*. Het resultaat is een reeks, geen wachtrij.

Methode	Wat het doet	Voorbeeld	Resultaat
Where	enkel wie voldoet	<code>wachtrij.Where(n => n.Contains("a"))</code>	Anna, Bram
Select , SelectMany	maak van elk iets anders	<code>wachtrij.Select(n => n.Length)</code>	4, 4, 4

Methode	Wat het doet	Voorbeeld	Resultaat
Distinct, DistinctBy	gooi dubbels weg	<code>wachtrij.Distinct()</code>	iemand die twee keer aansloot, telt één keer
Take, Skip, TakeLast, SkipLast	de eerste of laatste n	<code>wachtrij.Take(2)</code>	Anna, Bram : wie straks aan de beurt komt
TakeWhile, SkipWhile, Chunk, Index, OfType	zoals bij een lijst	<code>wachtrij.Index()</code>	(0, Anna), (1, Bram), (2, Cleo)
Count	hoeveel er voldoen	<code>wachtrij.Count(n => n.Contains("a"))</code>	2
Sum, Min, Max, Average	rekenen	<code>wachtrij.Max()</code>	Cleo : alfabetisch de laatste
MinBy, MaxBy, CountBy, Aggregate, AggregateBy	zoals bij een lijst	<code>wachtrij.Aggregate((a, b) => a + " > " + b)</code>	Anna > Bram > Cleo

Ordenen, groeperen en combineren

Komt van LINQ. De wachtrij zelf blijft in haar volgorde: wie eerst kwam, blijft eerst.

Methode	Wat het doet	Voorbeeld	Resultaat
Order, OrderDescending, OrderBy, OrderByDescending, ThenBy, ThenByDescending	een gesorteerde kopie	<code>wachtrij.OrderByDescending(n => n)</code>	Cleo, Bram, Anna
Reverse	van achter naar voor	<code>wachtrij.Reverse()</code>	Cleo, Bram, Anna ; de rij blijft Anna, Bram, Cleo
Shuffle	willekeurige volgorde	<code>wachtrij.Shuffle()</code>	elke keer anders
GroupBy, ToLookup	maak groepen	<code>wachtrij.GroupBy(n => n[0])</code>	een groep per beginletter
Concat, Union, Intersect, Except en de By-varianten	combineer met een andere reeks	<code>wachtrij.Concat(tweedeRij)</code>	eerst deze rij, dan de tweede
Append, Prepend	één persoon achteraan of vooraan, in het resultaat	<code>wachtrij.Prepend("Zoë")</code>	Zoë, Anna, Bram, Cleo ; in de rij zelf steekt niemand voor
Zip, Join, LeftJoin, RightJoin, GroupJoin	koppel aan een andere reeks	<code>wachtrij.Zip(loketten)</code>	elke klant met zijn loket

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
ToList	een lijst, van voor naar achter	<code>wachtrij.ToList()</code>	Anna, Bram, Cleo , met posities

Methode	Wat het doet	Voorbeeld	Resultaat
ToArray	een array; de wachtrij heeft ook een eigen ToArray	<code>wachtrij.ToArray()</code>	Anna, Bram, Cleo
ToHashSet, ToDictionary, ToLookup	een set, opzoektabel of groepen	<code>wachtrij.ToHashSet()</code>	elke naam één keer

Doorlopen en omzetten

Komt van de taal C# zelf en van de andere verzamelingen.

Schrijfwijze	Wat het doet	Resultaat
<code>foreach (string n in wachtrij)</code>	iedereen van voor naar achter, zonder weg te halen	Anna, Bram, Cleo; Count blijft 3
<code>while (wachtrij.Count > 0) { wachtrij.Dequeue(); }</code>	iedereen afhandelen, en uit de rij halen	daarna is de rij leeg
<code>string.Join(", ", wachtrij)</code>	iedereen in één tekst	Anna, Bram, Cleo
<code>new Queue<string>(lijst)</code>	een wachtrij uit een lijst of array	de eerste van de lijst staat vooraan
<code>new Stack<string>(wachtrij)</code>	een stapel uit de wachtrij	de laatste, Cleo, ligt bovenop
<code>new Queue<string>(wachtrij.Where(n => n != "Bram"))</code>	een nieuwe rij zonder Bram	Anna, Cleo

Die laatste regel is hoe je toch iemand uit het midden haalt: je maakt een nieuwe rij zonder die persoon.

BELANGRIJK

In een foreach over de wachtrij mag je **niemand** bijzetten of weghalen: Enqueue en Dequeue crashen dan allebei met een **InvalidOperationException**. Wie de rij wil afwerken, gebruikt while met Dequeue of TryDequeue.

Wat elk object kan

Komt van `object`. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
ToString	het object als tekst	<code>wachtrij.ToString()</code>	<code>System.Collections.Generic.Queue`1[System.String]</code>
Equals / <code>==</code>	is dit dezelfde wachtrij	twee rijen met dezelfde mensen	<code>false</code> ; vergelijk de inhoud met <code>SequenceEqual</code>
GetType, GetHashCode	type en hashcode	<code>wachtrij.GetType().Name</code>	<code>Queue`1</code>

Bestaat ook, maar kom je in deze cursus niet tegen

`PriorityQueue<TElement, TPriority>` (een wachtrij waarin wie de hoogste prioriteit heeft eerst aan de beurt komt, niet wie eerst kwam), `ConcurrentQueue` (voor meerdere taken tegelijk), en uit LINQ `AsParallel` en `Cast`.

Let op bij een wachtrij

Op de wachtrij zelf	Via LINQ	Het verschil
Peek	<code>First()</code>	Allebei kijken ze vooraan zonder weg te halen. Bij een lege rij crashen ze allebei.
TryPeek	<code>FirstOrDefault()</code>	Allebei crashen ze niet bij een lege rij. <code>FirstOrDefault</code> geeft dan <code>null</code> (of <code>0</code> bij getallen); <code>TryPeek</code> zegt met <code>false</code> dat er niemand is.
Dequeue	<code>First()</code>	<code>Dequeue</code> haalt de eerste weg . <code>First()</code> kijkt enkel: wie dat verwacht, verwerkt dezelfde persoon eindeloos opnieuw.
Count	<code>Count()</code>	Hetzelfde getal. Een voorwaarde kan enkel met LINQ.
—	<code>ElementAt(1)</code>	Een wachtrij heeft geen <code>[1]</code> . Via LINQ kan je wel kijken wie op plaats 1 staat.

LET OP

- LINQ en `foreach` veranderen de rij **niet**. Na `wachtrij.Take(2)` staan Anna en Bram er nog: ze zijn niet geholpen.
- `Reverse()` op een wachtrij geeft een omgekeerde **reeks**. De rij zelf blijft staan zoals ze stond.
- `Contains` bij een wachtrij van objecten van een `class` vergelijkt op referentie. Zie [hoofdstuk 13](#).

Wanneer wel, wanneer niet

Wel — dingen afhandelen in de volgorde waarin ze binnenkwamen: printopdrachten, berichten, klanten aan een loket.

Niet — zodra je iets in het midden wil bereiken of wil overslaan.

Veelgemaakte fouten

LET OP

- `Dequeue` of `Peek` op een lege wachtrij crasht. Controleer eerst `Count`, of gebruik `TryDequeue`.
- `Peek` kijkt enkel, `Dequeue` haalt weg. Wie ze verwisselt, verwerkt de eerste persoon eindeloos opnieuw.
- `Dequeue` binnen een `foreach` over dezelfde wachtrij crasht. Gebruik `while`.

Voorspel de uitkomst

Vraag 1

```
var q = new Queue<int>();
q.Enqueue(1);
q.Enqueue(2);
q.Enqueue(3);
q.Dequeue();
Console.WriteLine(q.Peek());
```

[Toon het antwoord ›](#)

Vraag 2

```
var q = new Queue<int>();
q.Dequeue();
```

[Toon het antwoord ›](#)

Vraag 3

```
var q = new Queue<string>();
q.Enqueue("Anna");
q.Enqueue("Bram");
Console.WriteLine(q.First());
Console.WriteLine(q.Count);
```

[Toon het antwoord ›](#)

Vraag 4

```
var q = new Queue<string>(new[] { "Anna", "Bram", "Cleo" });
var omgekeerd = q.Reverse();
Console.WriteLine(q.Peek());
Console.WriteLine(omgekeerd.First());
```

[Toon het antwoord ›](#)

Vraag 5

```
var q = new Queue<string>(new[] { "Anna", "Bram", "Cleo" });
foreach (string n in q)
{
    q.Dequeue();
}
```

[Toon het antwoord ›](#)

Vraag 6

```
var q = new Queue<string>(new[] { "Anna", "Bram", "Cleo" });
q = new Queue<string>(q.Where(n => n != "Bram"));
Console.WriteLine(string.Join(", ", q));
```

Toon het antwoord ›

Antwoorden

VRAAG 1

2

De 1 stond vooraan en is weggehaald. Nu staat de 2 vooraan.

[Terug naar de vraag](#)

VRAAG 2

InvalidOperationException — het programma crasht.

Er staat niemand in de rij.

[Terug naar de vraag](#)

VRAAG 3

Anna en daarna 2

`First()` is LINQ: het kijkt vooraan, zoals `Peek`, maar haalt niemand weg.

[Terug naar de vraag](#)

VRAAG 4

Anna en daarna Cleo

`Reverse` geeft een omgekeerde reeks. De wachtrij zelf verandert niet: Anna staat nog altijd vooraan.

[Terug naar de vraag](#)

VRAAG 5

InvalidOperationException — het programma crasht.

De rij verandert terwijl `foreach` erdoor loopt. Wie iedereen wil afhandelen, schrijft `while (q.Count > 0) { q.Dequeue(); }`.

[Terug naar de vraag](#)

VRAAG 6

Anna, Cleo

Een wachtrij kan niemand uit het midden halen, maar je kan een nieuwe rij maken zonder Bram. De volgorde van de anderen blijft.

[Terug naar de vraag](#)

10. Stack

VERZAMELING

Wat het is

Een stapel borden. Het bord dat je laatst neerlegt, neem je als eerste weer op. Het onderste bord raak je pas als alles erboven weg is.

Hoe het eruitziet

```
Stack<string> stappen = new Stack<string>();

stappen.Push("typ tekst");    // bovenop leggen
stappen.Push("maak vet");
stappen.Push("wis regel");

Console.WriteLine(stappen.Pop()); // wis regel – bovenste eraf
Console.WriteLine(stappen.Peek()); // maak vet – kijken, niet wegnemen
```

De voorbeelden in dit hoofdstuk

De tabellen hieronder vertrekken telkens van deze stapel. *wis regel* werd als laatste neergelegd en ligt dus bovenop; *typ tekst* ligt onderaan.

```
Stack<string> stappen = new Stack<string>();
stappen.Push("typ tekst");
stappen.Push("maak vet");
stappen.Push("wis regel");
```

Wat je ermee kan

Dit zijn alle methodes en eigenschappen die op een `Stack` zelf zitten.

Methode	Wat het doet	Voorbeeld	Resultaat
Push	leg bovenop	<code>stappen.Push("kopieer")</code>	<code>kopieer</code> ligt nu bovenop
Pop	neem de bovenste eraf en geef hem terug	<code>stappen.Pop()</code>	<code>wis regel</code> , en die is weg; leeg geeft een <code>InvalidOperationException</code>
Peek	kijk wat bovenop ligt, zonder weg te nemen	<code>stappen.Peek()</code>	<code>wis regel</code> ; leeg geeft een <code>InvalidOperationException</code>
TryPop	neem de bovenste als er iets ligt	<code>stappen.TryPop(out string? bovenste)</code>	<code>true</code> en <code>bovenste</code> is <code>wis regel</code> ; leeg geeft <code>false</code>
TryPeek	kijk bovenop als er iets ligt	<code>stappen.TryPeek(out string? bovenste)</code>	<code>true</code> ; leeg geeft <code>false</code>

Methode	Wat het doet	Voorbeeld	Resultaat
Count	hoeveel liggen er; een eigenschap	<code>stappen.Count</code>	3
Contains	ligt dit ergens op de stapel	<code>stappen.Contains("maak vet")</code>	true
Clear	maak leeg	<code>stappen.Clear()</code>	Count is 0
ToArray	een array, van boven naar onder	<code>stappen.ToArray()</code>	wis regel, maak vet, typ tekst
CopyTo	kopieer naar een bestaande array, van boven naar onder	<code>stappen.CopyTo(doe1, 0)</code>	doe1 begint met wis regel
Capacity	hoeveel er passen voor de stapel moet groeien; een eigenschap	<code>stappen.Capacity</code>	minstens 3
EnsureCapacity, TrimExcess	reserveer plaats, of geef ze terug	<code>stappen.EnsureCapacity(100)</code>	zelden nodig

Er is **geen** `[0]` en **geen** manier om een bord uit het midden te trekken. Een stapel afbouwen doe je zo:

```
while (stappen.TryPop(out string? stap))
{
    Console.WriteLine($"ongedaan: {stap}");
}
```

Wat het ook kan, via iets anders

Een stapel is ook een **reeks** (`IEnumerable`, [hoofdstuk 12](#)). Alles wat met een reeks werkt, werkt dus ook op een stapel — en altijd **van boven naar onder**: eerst wat laatst werd neergelegd.

Bron	Waarom het op een stapel werkt	Volledige uitleg
LINQ	LINQ werkt op elke reeks	hoofdstuk 11
<code>foreach</code>	doorloopt van boven naar onder, zonder iets weg te nemen	hoofdstuk 4
<code>string.Join</code>	wil een reeks	hoofdstuk 1
andere verzamelingen	die kan je vullen met een reeks	hoofdstukken 5 tot 9
wat elk object kan	een stapel is ook een object	hoofdstuk 13

Wat je over LINQ moet weten om de tabellen te lezen:

- Een LINQ-methode **verandert je stapel nooit**: ze neemt niets weg en legt niets bij. `stappen.First()` kijkt, zoals `Peek`.
- LINQ overloopt de stapel **van boven naar onder**: wis regel, maak vet, typ tekst. Dat is het omgekeerde van de volgorde waarin je ze neerlegde.

- Het resultaat is **geen stapel** maar een reeks.

OPMERKING

Index, CountBy en AggregateBy bestaan sinds .NET 9, Shuffle, LeftJoin en RightJoin sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken.

Eén item nemen of iets nagaan

Komt van LINQ. Er gaat niets van de stapel af.

Methode	Wat het doet	Voorbeeld	Resultaat	Eigen tegenhanger
First, FirstOrDefault	de bovenste (die voldoet)	<code>stappen.First()</code>	wis regel; Count blijft 3	Peek, TryPeek
Last, LastOrDefault	de onderste (die voldoet)	<code>stappen.Last()</code>	typ tekst	—
Single, SingleOrDefault	de enige die voldoet	<code>stappen.Single(s => s.StartsWith("m"))</code>	maak vet	—
ElementAt, ElementAtOrDefault	het item op plaats n, van boven geteld	<code>stappen.ElementAt(0)</code>	wis regel	—
Any, All	ligt er iets (dat voldoet), voldoen ze allemaal	<code>stappen.Any()</code>	true	Count > 0
Contains	ligt dit op de stapel	<code>stappen.Contains("maak vet")</code>	true	Contains: dezelfde
SequenceEqual	dezelfde inhoud in dezelfde volgorde	<code>stappen.SequenceEqual(new[] { "wis regel", "maak vet", "typ tekst" })</code>	true: van boven naar onder	—

Een deel nemen, tellen en rekenen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
Where	enkel wat voldoet	<code>stappen.Where(s => s.Contains("e"))</code>	wis regel, maak vet, typ tekst
Select, SelectMany	maak van elk item iets anders	<code>stappen.Select(s => s.Length)</code>	9, 8, 9
Take, Skip, TakeLast, SkipLast	de bovenste of onderste n	<code>stappen.Take(2)</code>	wis regel, maak vet: wat straks eerst ongedaan wordt

Methode	Wat het doet	Voorbeeld	Resultaat
Distinct , DistinctBy , TakeWhile , SkipWhile , Chunk , Index , OfType	zoals bij een lijst, van boven naar onder	<code>stappen.Index()</code>	(0, wis regel), (1, maak vet), (2, typ tekst)
Count	hoeveel er voldoen	<code>stappen.Count(s => s.Length == 9)</code>	2
Sum , Min , Max , Average	rekenen	een stapel getallen 1, 2, 3: <code>getallen.Sum()</code>	6
MinBy , MaxBy , CountBy , Aggregate , AggregateBy	zoals bij een lijst	<code>stappen.Aggregate((a, b) => a + " > " + b)</code>	wis regel > maak vet > typ tekst

Ordenen, groeperen en combineren

Komt van LINQ. De stapel zelf verandert niet.

Methode	Wat het doet	Voorbeeld	Resultaat
Reverse	van onder naar boven: de volgorde waarin je ze neerlegde	<code>stappen.Reverse()</code>	typ tekst, maak vet, wis regel
Order , OrderBy en hun familie	een gesorteerde kopie	<code>stappen.Order()</code>	maak vet, typ tekst, wis regel
Shuffle	willekeurige volgorde	<code>stappen.Shuffle()</code>	elke keer anders
GroupBy , ToLookup	maak groepen	<code>stappen.GroupBy(s => s.Length)</code>	9: wis regel, typ tekst; 8: maak vet
Concat , Union , Intersect , Except , Append , Prepend , Zip , Join en familie	combineer met een andere reeks	<code>stappen.Append("start")</code>	vier items in het resultaat, de stapel blijft drie

Het resultaat vastleggen

Komt van LINQ.

Methode	Wat het doet	Voorbeeld	Resultaat
ToList	een lijst, van boven naar onder	<code>stappen.ToList()</code>	lijst[0] is wis regel
ToArray	een array; de stapel heeft ook een eigen <code>ToArray</code>	<code>stappen.ToArray()</code>	wis regel, maak vet, typ tekst
ToHashSet , ToDictionary , ToLookup	een set, opzoektabel of groepen	<code>stappen.ToHashSet()</code>	elke stap één keer

Doorlopen en omzetten

Komt van de taal C# zelf en van de andere verzamelingen.

Schrijfwijze	Wat het doet	Resultaat
<code>foreach (string s in stappen)</code>	alles van boven naar onder, zonder weg te nemen	wis regel, maak vet, typ tekst

Schrijfwijze	Wat het doet	Resultaat
<code>while (stappen.TryPop(out string? s)) { ... }</code>	alles afbouwen	daarna is de stapel leeg
<code>string.Join(", ", stappen)</code>	alles in één tekst, van boven naar onder	wis regel, maak vet, typ tekst
<code>new Stack<string>(lijst)</code>	een stapel uit een lijst: het laatste van de lijst ligt bovenop	uit a, b, c wordt een stapel met c bovenop
<code>new Stack<string>(stappen)</code>	een "kopie" van een stapel	de volgorde is omgekeerd : typ tekst ligt nu bovenop
<code>new Stack<string>(stappen.Reverse())</code>	een echte kopie, in dezelfde volgorde	wis regel ligt weer bovenop
<code>new Queue<string>(stappen)</code>	een wachtrij uit de stapel	wis regel staat vooraan

BELANGRIJK

`new Stack<string>(stappen)` maakt geen kopie in dezelfde volgorde. Hij doorloopt de oude stapel van boven naar onder en legt alles één voor één op de nieuwe stapel. Wat bovenaan lag, komt zo onderaan terecht.

BELANGRIJK

In een `foreach` over de stapel mag je **niets** bijleggen of wegnemen: `Push` en `Pop` crashen dan allebei met een **`InvalidOperationException`**.

Wat elk object kan

Komt van `object`. Zie hoofdstuk 13.

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToString</code>	het object als tekst	<code>stappen.ToString()</code>	<code>System.Collections.Generic.Stack`1[System.String]</code>
<code>Equals</code> / <code>==</code>	is dit dezelfde stapel	twee stapels met dezelfde inhoud	<code>false</code> ; vergelijk de inhoud met <code>SequenceEqual</code>
<code>GetType</code> , <code>GetHashCode</code>	type en hashcode	<code>stappen.GetType().Name</code>	<code>Stack`1</code>

Bestaat ook, maar kom je in deze cursus niet tegen

`ConcurrentStack` (voor meerdere taken tegelijk), `ImmutableStack`, en uit LINQ `AsParallel` en `Cast`.

Let op bij een stapel

Op de stapel zelf	Via LINQ	Het verschil
-------------------	----------	--------------

Op de stapel zelf	Via LINQ	Het verschil
Peek	First()	Allebei kijken ze bovenop zonder weg te nemen.
Pop	First()	Pop neemt de bovenste weg , First() kijkt enkel.
—	Last()	Wat onderaan ligt, bereik je enkel via LINQ (of door alles erboven weg te nemen).
ToArray()	Reverse()	ToArray geeft van boven naar onder. Wil je de volgorde waarin je ze neerlegde, gebruik dan Reverse().
Count	Count()	Hetzelfde getal. Een voorwaarde kan enkel met LINQ.

LET OP

- Alles wat een stapel overloopt — `foreach`, LINQ, `ToArray`, `ToList`, `string.Join` — begint **bovenaan**. Een lijst die je met `Push` vult en daarna met `ToList()` omzet, staat dus omgekeerd.
- `new Stack<T>(stapel)` keert de volgorde om. Een kopie in dezelfde volgorde is `new Stack<T>(stapel.Reverse())`.
- `ElementAt(0)` is de **bovenste**, niet de eerste die je neerlegde.
- Een `Stack` en een `Queue` met dezelfde inhoud worden in omgekeerde volgorde overlopen: de wachtrij van de eerste naar de laatste, de stapel van de laatste naar de eerste.

Wanneer wel, wanneer niet

Wel — ongedaan maken, de terug-knop van een browser, nagaan of haakjes correct openen en sluiten. Overal waar het laatste het eerst weer aan de beurt komt.

Niet — wanneer iedereen eerlijk op zijn beurt moet wachten. Dan is het een `Queue`.

Veelgemaakte fouten

LET OP

- `Pop` of `Peek` op een lege stapel crasht, net als bij een wachtrij.
- Een `Stack` en een `Queue` met dezelfde inhoud geven hun items in omgekeerde volgorde terug. Dat is het hele verschil, en het is groot.
- `Pop` binnen een `foreach` over dezelfde stapel crasht. Gebruik `while` met `TryPop`.

Voorspel de uitkomst

Vraag 1

```
var s = new Stack<int>();
s.Push(1);
s.Push(2);
s.Push(3);
Console.WriteLine(s.Pop() + s.Pop());
```

[Toon het antwoord ›](#)

Vraag 2

```
var q = new Queue<char>();
var s = new Stack<char>();

foreach (char c in "abc")
{
    q.Enqueue(c);
    s.Push(c);
}

Console.WriteLine(q.Dequeue());
Console.WriteLine(s.Pop());
```

[Toon het antwoord ›](#)

Vraag 3

```
var s = new Stack<string>();
s.Push("a");
s.Push("b");
s.Push("c");
Console.WriteLine(string.Join("", s));
Console.WriteLine(s.First());
Console.WriteLine(s.Count);
```

[Toon het antwoord ›](#)

Vraag 4

```
var s = new Stack<string>();
s.Push("onder");
s.Push("boven");
var kopie = new Stack<string>(s);
Console.WriteLine(kopie.Peek());
```

[Toon het antwoord ›](#)

Vraag 5

```
var lijst = new List<int> { 1, 2, 3 };
var s = new Stack<int>(lijst);
Console.WriteLine(s.Pop());
Console.WriteLine(s.ToList()[0]);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

5

De eerste `Pop` geeft 3, de tweede geeft 2. Samen 5. De 1 ligt nog onderaan.

[Terug naar de vraag](#)

VRAAG 2

a en daarna c

Dezelfde drie letters erin, in dezelfde volgorde. De wachtrij geeft de eerste terug, de stapel de laatste.

[Terug naar de vraag](#)

VRAAG 3

cba, daarna c, en dan 3

Alles wat een stapel overloopt, begint bovenaan. `First()` kijkt enkel; er gaat niets van de stapel af.

[Terug naar de vraag](#)

VRAAG 4

onder

De nieuwe stapel overloopt de oude van boven naar onder en legt alles één voor één neer. "boven" gaat eerst, "onder" komt er bovenop. De volgorde is omgekeerd.

[Terug naar de vraag](#)

VRAAG 5

3 en daarna 2

De lijst wordt van voor naar achter op de stapel gelegd: de 3 ligt bovenop. Na `Pop` ligt de 2 bovenop, en `ToList` begint bovenaan.

[Terug naar de vraag](#)

Deel 3 — Zoeken en rekenen

Nu je de verzamelingen kent: hoe haal je er iets uit, en wat krijg je dan eigenlijk terug?

11. LINQ

METHODES

Dit is LINQ. Deze methodes werken op elke verzameling — een lijst, een array, een resultaat van een andere LINQ-methode, en later ook op een databasetabel. Ze veranderen nooit iets aan de bron; ze geven een nieuw resultaat.

Bijna allemaal willen ze een voorwaarde in de vorm `f => f.Prijs > 50`. Lees de pijl als "geef mij, voor elke f, het antwoord op".

LINQ werkt enkel met `using System.Linq`; bovenaan. In een nieuw project staat die er automatisch bij.

De voorbeelden in dit hoofdstuk

De tabellen hieronder gebruiken telkens deze lijsten, zodat je elk resultaat kan narekenen. Dezelfde voorbeelden staan in de hoofdstukken over de verzamelingen.

```
List<string> fruit    = new List<string> { "appel", "peer", "kiwi" };
List<int>    getallen = new List<int> { 5, 8, 3, 8, 1 };
List<Fan>    fans     = new List<Fan>
{
    new Fan { Id = 1, Merk = "NZXT", Prijs = 24.99m },
    new Fan { Id = 2, Merk = "Noctua", Prijs = 29.99m },
    new Fan { Id = 3, Merk = "NZXT", Prijs = 89.99m },
    new Fan { Id = 4, Merk = "Arctic", Prijs = 19.99m }
};
```

OPMERKING

Index, CountBy en AggregateBy bestaan sinds .NET 9, Shuffle, LeftJoin, RightJoin, Enumerable.Sequence en Enumerable.InfiniteSequence sinds .NET 10. Je cursus werkt met .NET 10, dus je kan ze gebruiken. In oudere voorbeelden op internet kom je ze niet tegen.

Ik wil één item

Methode	Wat het doet	Als er niets is	Als er meerdere zijn
First	de eerste die voldoet	uitzondering	neemt de eerste
FirstOrDefault	de eerste die voldoet	null	neemt de eerste
Single	de enige die voldoet	uitzondering	uitzondering
SingleOrDefault	de enige die voldoet	null	uitzondering
Last	de laatste die voldoet	uitzondering	neemt de laatste
LastOrDefault	de laatste die voldoet	null	neemt de laatste
ElementAt	het item op een positie	uitzondering	—
ElementAtOrDefault	het item op een positie	null	—

`fans.FirstOrDefault(f => f.Id == 3)` geeft de fan met Id 3, of `null`. `fans.ElementAt(1)` geeft fan 2, en `fans.ElementAtOrDefault(9)` geeft `null`.

Aan de `...OrDefault`-varianten kan je ook zelf zeggen wat er moet komen als er niets gevonden wordt: `getallen.FirstOrDefault(g => g > 10, -1)` geeft `-1`.

BELANGRIJK

Neem `FirstOrDefault` wanneer het normaal is dat er niets gevonden wordt — een Id dat niet bestaat, bijvoorbeeld. Dan controleer je op `null` en geef je een nette melding.

Neem `SingleOrDefault` wanneer er hoogstens één mag zijn en twee een fout in je gegevens betekent. De uitzondering die je dan krijgt is geen hinder maar een alarm: er staan twee items met hetzelfde Id in je lijst.

Neem `First` alleen wanneer je zeker weet dat er iets is. Anders crasht je API met een `500` op een geval dat eigenlijk een `404` moest zijn.

LET OP

`FirstOrDefault` geeft niet altijd `null`. Bij een lijst van getallen geeft het `0`, en bij een lijst van waarheidswaarden `false`. De standaardwaarde van het type dus, niet noodzakelijk niets. Bij een lijst van objecten is het wel `null`. Bij een `Dictionary` krijg je een leeg paar, en bij een tekst het lege teken `'\0'`.

Ik wil weten of iets bestaat

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Any</code>	is er minstens één	<code>fans.Any()</code>	<code>true</code> als de lijst niet leeg is
<code>Any</code>	met voorwaarde	<code>fans.Any(f => f.Prijs > 50)</code>	<code>true</code> of <code>false</code>
<code>All</code>	voldoen ze allemaal	<code>fans.All(f => f.Prijs > 0)</code>	<code>true</code> of <code>false</code>
<code>Contains</code>	zit deze waarde erin	<code>maten.Contains(120)</code>	<code>true</code> of <code>false</code>
<code>SequenceEqual</code>	dezelfde inhoud in dezelfde volgorde	<code>getallen.SequenceEqual(new[] { 5, 8, 3, 8, 1 })</code>	<code>true</code>

Wil je enkel weten óf er iets is, gebruik dan `Any` en niet `Count() > 0`. `Any` stopt bij het eerste item dat voldoet, `Count` loopt de hele verzameling af.

LET OP

`All` op een lege verzameling geeft `true`. Wiskundig klopt dat — er is geen enkel item dat níet voldoet — maar het is zelden wat je bedoelde. Controleer eerst of er wel iets in zit.

Ik wil een deel van de verzameling

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Where</code>	enkel wat voldoet	<code>fans.Where(f => f.Merk == "NZXT")</code>	de NZXT-fans

Methode	Wat het doet	Voorbeeld	Resultaat
Select	maak van elk item iets anders	<code>fans.Select(f => f.Merk)</code>	enkel de merken
Select	met de positie erbij	<code>fruit.Select((f, i) => \$"{i}:{f}")</code>	0: appel, 1: peer, 2: kiwi
Distinct	gooi dubbels weg	<code>getallen.Distinct()</code>	5, 8, 3, 1
DistinctBy	één item per waarde van iets	<code>fans.DistinctBy(f => f.Merk)</code>	fan 1, 2 en 4
Take	de eerste n	<code>fans.Take(3)</code>	drie items
Skip	sla de eerste n over	<code>fans.Skip(3)</code>	de rest
TakeLast, SkipLast	de laatste n, of alles behalve de laatste n	<code>getallen.TakeLast(2)</code>	8, 1
TakeWhile	neem zolang de voorwaarde klopt	<code>getallen.TakeWhile(g => g > 2)</code>	5, 8, 3, 8
SkipWhile	sla over zolang de voorwaarde klopt	<code>getallen.SkipWhile(g => g > 2)</code>	1
SelectMany	plat een lijst van lijsten	<code>borden.SelectMany(b => b.Sloten)</code>	alle sloten samen
Chunk	verdeel in stukken van n	<code>getallen.Chunk(2)</code>	[5, 8], [3, 8], [1]
OfType	enkel de items van een bepaald type	<code>onderdelen.OfType<Processor>()</code>	enkel de processors
Index	elk item samen met zijn positie	<code>fruit.Index()</code>	(0, appel), (1, peer), (2, kiwi)

`Where` filtert, `Select` verbouwt. Dat is het hele verschil. Wil je de dure fans, dan is dat `Where`. Wil je van elke fan enkel het merk, dan is dat `Select`. Wil je de merken van de dure fans, dan is dat allebei achter elkaar.

`TakeWhile` stopt bij het eerste item dat níet voldoet, ook al voldoen er verderop nog. `Where` kijkt naar alles. `getallen.Where(g => g > 2)` geeft 5, 8, 3, 8; `getallen.TakeWhile(g => g > 6)` geeft niets, want de eerste, 5, voldoet al niet.

`OfType` hoort bij overerving: zie [hoofdstuk 20](#).

Ik wil tellen of rekenen

Methode	Wat het doet	Voorbeeld	Resultaat
Count	hoeveel er zijn	<code>fans.Count()</code>	4
Count	hoeveel er voldoen	<code>fans.Count(f => f.Prijs > 25)</code>	2
Sum	tel op	<code>fans.Sum(f => f.Prijs)</code>	164.96
Min	de kleinste waarde	<code>fans.Min(f => f.Prijs)</code>	19.99
Max	de grootste waarde	<code>fans.Max(f => f.Prijs)</code>	89.99
Average	het gemiddelde	<code>fans.Average(f => f.Prijs)</code>	41.24

Methode	Wat het doet	Voorbeeld	Resultaat
MinBy	het item met de kleinste waarde	<code>fans.MinBy(f => f.Prijs)</code>	de goedkoopste fan
MaxBy	het item met de grootste waarde	<code>fans.MaxBy(f => f.Prijs)</code>	de duurste fan
CountBy	tel per waarde van iets	<code>fans.CountBy(f => f.Merk)</code>	NZXT 2, Noctua 1, Arctic 1
Aggregate	voeg alles samen met een eigen regel	<code>fruit.Aggregate((a, b) => a + "-" + b)</code>	appel-peer-kiwi
AggregateBy	voeg samen per waarde van iets	<code>fans.AggregateBy(f => f.Merk, 0m, (som, f) => som + f.Prijs)</code>	NZXT 114.98, Noctua 29.99, Arctic 19.99
LongCount	zoals Count, voor meer dan twee miljard items	<code>fans.LongCount()</code>	4

`Max` geeft het getal, `MaxBy` geeft het item. Wil je weten wat de hoogste prijs is, dan is het `Max`. Wil je weten welke fan dat is, dan is het `MaxBy`.

Op een lijst getallen heb je geen voorwaarde nodig: `getallen.Sum()` geeft 25 en `getallen.Max()` geeft 8.

LET OP

Average, Min en Max op een lege reeks getallen geven een **InvalidOperationException**. Sum en Count geven dan 0. Bij een lege reeks teksten of objecten geven Min en Max null.

Ik wil ordenen of groeperen

Methode	Wat het doet	Voorbeeld	Resultaat
Order	sorteer de waarden zelf, oplopend	<code>getallen.Order()</code>	1, 3, 5, 8, 8
OrderDescending	sorteer de waarden zelf, aflopend	<code>getallen.OrderDescending()</code>	8, 8, 5, 3, 1
OrderBy	sorteer oplopend op iets	<code>fans.OrderBy(f => f.Prijs)</code>	goedkoopste eerst
OrderByDescending	sorteer aflopend op iets	<code>fans.OrderByDescending(f => f.Prijs)</code>	duurste eerst
ThenBy	tweede sorteersleutel	<code>fans.OrderBy(f => f.Merk).ThenBy(f => f.Prijs)</code>	per merk, dan op prijs
ThenByDescending	tweede sorteersleutel, aflopend	<code>fans.OrderBy(f => f.Merk).ThenByDescending(f => f.Prijs)</code>	per merk, duurste eerst
Reverse	omgekeerde volgorde	<code>getallen.AsEnumerable().Reverse()</code>	1, 8, 3, 8, 5
Shuffle	willekeurige volgorde	<code>fruit.Shuffle()</code>	elke keer anders

Methode	Wat het doet	Voorbeeld	Resultaat
GroupBy	maak groepen	<code>fans.GroupBy(f => f.Merk)</code>	een groep per merk
ToLookup	maak groepen, meteen vastgelegd	<code>fans.ToLookup(f => f.Merk)</code>	opzoek["NZXT"] geeft fan 1 en 3

Een groep uit `GroupBy` heeft een `.Key` (het merk) en is zelf een reeks (de fans van dat merk):

`fans.GroupBy(f => f.Merk).Select(g => $"{g.Key}: {g.Count()}")` geeft **NZXT: 2, Noctua: 1, Arctic: 1**.

LET OP

`OrderBy(...).OrderBy(...)` sorteert twee keer, en de tweede keer wint. Voor een tweede sleutel gebruik je `ThenBy`.

Ik wil twee reeksen combineren

In de voorbeelden is `a` de lijst 1, 2, 3 en `b` de lijst 3, 4.

Methode	Wat het doet	Voorbeeld	Resultaat
Concat	plak de tweede achter de eerste	<code>a.Concat(b)</code>	1, 2, 3, 3, 4
Union	alles uit beide, zonder dubbels	<code>a.Union(b)</code>	1, 2, 3, 4
Intersect	wat in beide zit	<code>a.Intersect(b)</code>	3
Except	wat in de eerste zit en niet in de tweede	<code>a.Except(b)</code>	1, 2
UnionBy, IntersectBy, ExceptBy	hetzelfde, vergeleken op één eigenschap	<code>fans.ExceptBy(new[] { "NZXT" }, f => f.Merk)</code>	fan 2 en 4
Append	één item achteraan, in het resultaat	<code>a.Append(9)</code>	1, 2, 3, 9
Prepend	één item vooraan, in het resultaat	<code>a.Prepend(0)</code>	0, 1, 2, 3
Zip	koppel per positie	<code>fruit.Zip(getallen)</code>	(appel, 5), (peer, 8), (kiwi, 3)
Join	koppel items met dezelfde sleutel	<code>fans.Join(merken, f => f.Merk, m => m.Naam, (f, m) => ...)</code>	enkel de paren die bij elkaar horen
LeftJoin	zoals <code>Join</code> , elk item uit de eerste blijft	<code>fans.LeftJoin(merken, ...)</code>	alle fans, met of zonder merk
RightJoin	zoals <code>Join</code> , elk item uit de tweede blijft	<code>fans.RightJoin(merken, ...)</code>	alle merken, met of zonder fan
GroupJoin	koppel aan elk item de groep die erbij hoort	<code>merken.GroupJoin(fans, ...)</code>	elk merk met zijn fans

`Zip` stopt bij de kortste van de twee. `Union`, `Intersect` en `Except` gooien ook de dubbels binnen één reeks weg: `new[] { 1, 1 }.Union(new[] { 2 })` geeft `1, 2`.

Ik wil het resultaat vastleggen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToList</code>	maak er een echte lijst van	<code>fans.Where(f => f.Prijs > 50).ToList()</code>	een <code>List</code>
<code>ToArray</code>	maak er een array van	<code>fans.ToArray()</code>	een array
<code>ToDictionary</code>	maak er een opzoektabel van	<code>fans.ToDictionary(f => f.Id)</code>	een <code>Dictionary</code>
<code>ToHashSet</code>	maak er een set van: dubbels vallen weg	<code>getallen.ToHashSet()</code>	<code>5, 8, 3, 1</code>
<code>ToLookup</code>	maak er groepen van	<code>fans.ToLookup(f => f.Merk)</code>	zie hierboven

BELANGRIJK

`Where`, `Select` en `OrderBy` voeren op het moment dat je ze schrijft nog niets uit. Wat dat betekent, en waarom het je kan verrassen, is het onderwerp van het volgende hoofdstuk. De vuistregel voor nu: wat je aan de client van een API teruggeeft, geef je pas terug na een `ToList()`.

LET OP

`ToDictionary` crasht met een `ArgumentException` als twee items dezelfde sleutel hebben. `fans.ToDictionary(f => f.Merk)` crasht: `NZXT` komt twee keer voor.

Ik wil zelf een reeks maken

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Enumerable.Range</code>	getallen van een begin, zoveel stuks	<code>Enumerable.Range(1, 5)</code>	<code>1, 2, 3, 4, 5</code>
<code>Enumerable.Sequence</code>	getallen van een begin tot en met een einde, met een stap	<code>Enumerable.Sequence(1, 10, 3)</code>	<code>1, 4, 7, 10</code>
<code>Enumerable.InfiniteSequence</code>	getallen zonder einde, met een stap; altijd samen met <code>Take</code>	<code>Enumerable.InfiniteSequence(0, 5).Take(4)</code>	<code>0, 5, 10, 15</code>
<code>Enumerable.Repeat</code>	dezelfde waarde een aantal keer	<code>Enumerable.Repeat("x", 3)</code>	<code>x, x, x</code>
<code>Enumerable.Empty</code>	een lege reeks	<code>Enumerable.Empty<int>()</code>	niets
<code>DefaultIfEmpty</code>	een lege reeks vervangen door één standaardwaarde	<code>new List<int>().DefaultIfEmpty()</code>	<code>0</code>
<code>Cast</code>	elk item omzetten naar een type, of crashen	<code>rooster.Cast<int>()</code>	een reeks <code>int</code> uit een rooster <code>int[,]</code>

`Enumerable.Range(1, 5)` lees je als *begin bij 1, geef er 5*, niet als *van 1 tot 5*. `Enumerable.Range(3, 2)` geeft dus 3 en 4.

`Enumerable.Sequence` werkt wel met een einde, en dat einde telt mee. Gaat de stap de verkeerde kant op, zoals `Enumerable.Sequence(1, 10, -1)`, dan krijg je een `ArgumentOutOfRangeException`.

Waar LINQ op werkt

LINQ werkt op alles wat je één voor één kan doorlopen (`IEnumerable`, hoofdstuk 12). Dat is meer dan een lijst. Per soort is er iets anders om op te letten.

Waarop	Wat elk item is	Waar je op let	Hoofdstuk
<code>List</code>	een item	De lijst heeft eigen tegenhangers (<code>Find</code> , <code>Exists</code> , <code>Sort</code> , <code>Reverse</code>). <code>lijst.Reverse()</code> draait de lijst zelf om.	6
<code>array</code>	een item	<code>Length</code> in plaats van <code>Count</code> ; <code>array.Reverse()</code> is wél LINQ en geeft een nieuwe reeks.	5
<code>Dictionary</code>	een paar (Key, Value)	<code>...OrDefault</code> geeft een leeg paar, nooit null. Werk op <code>p.Value</code> , of op <code>Keys</code> of <code>Values</code> .	7
<code>HashSet</code>	een item	De volgorde is niet gegarandeerd. <code>Union</code> en zijn familie veranderen de set niet; <code>UnionWith</code> wel.	8
<code>Queue</code>	een item, van voor naar achter	LINQ haalt niemand uit de rij: <code>First()</code> is zoals <code>Peek</code> .	9
<code>Stack</code>	een item, van boven naar onder	Alles begint bij wat laatst werd neergelegd.	10
<code>string</code>	een teken (<code>char</code>)	Het resultaat is geen tekst. Maak er weer een van met <code>string.Concat</code> .	1
een LINQ-resultaat	een item	Rekent pas als je het gebruikt, en telkens opnieuw.	12
een <code>DbSet</code> (EF Core)	een rij uit de database	Wordt een SQL-query. Gebruik de async-varianten (<code>ToListAsync</code> , <code>FirstOrDefaultAsync</code> , ...), en niet elke C#-methode kan naar SQL.	32

Let op bij LINQ

Eigen methode van de verzameling	LINQ	Het verschil
<code>lijst.Count</code> , <code>array.Length</code>	<code>Count()</code>	Hetzelfde getal. Zonder haakjes is meteen bekend; <code>Count()</code> met een voorwaarde kan enkel met LINQ.
<code>Find</code> , <code>Array.Find</code>	<code>FirstOrDefault</code>	Hetzelfde resultaat. <code>FirstOrDefault</code> werkt op elke reeks, ook op een databasequery.
<code>Exists</code> , <code>TrueForAll</code>	<code>Any</code> , <code>All</code>	Hetzelfde resultaat.
<code>FindAll</code>	<code>Where</code>	<code>FindAll</code> geeft meteen een nieuwe lijst. <code>Where</code> rekent pas als je het resultaat gebruikt.
<code>Sort</code>	<code>Order</code> , <code>OrderBy</code>	<code>Sort</code> verandert de lijst zelf en geeft niets terug. <code>OrderBy</code> laat de lijst zoals ze is.
<code>lijst.Reverse()</code>	<code>Reverse()</code>	Op een <code>List</code> wint altijd de eigen <code>Reverse</code> : die draait de lijst om. Op een <code>array</code> en de andere verzamelingen is <code>Reverse()</code> LINQ.

Eigen methode van de verzameling	LINQ	Het verschil
UnionWith	Union	UnionWith verandert de set, Union geeft een nieuwe reeks.

LET OP

- Het resultaat van Where, Select of OrderBy is geen lijst. `List<Fan> duur = fans.Where(f => f.Prijs > 50);` compileert niet. Zet er `.ToList()` achter.
- Contains, Distinct, Union, Intersect en Except vergelijken met Equals. Bij objecten van een class betekent dat: **hetzelfde object**, niet een object met dezelfde inhoud. Een record vergelijkt op inhoud. Zie hoofdstuk 13 en hoofdstuk 14. Wil je op één eigenschap vergelijken, gebruik dan de By-varianten: DistinctBy, ExceptBy.
- Order() zonder sleutel op objecten die C# niet kan vergelijken (een eigen klasse, of een paar uit een Dictionary) crasht met een **InvalidOperationException**. Zeg waarop: `OrderBy(f => f.Prijs)`.
- LINQ op een lege reeks: First, Single, Min, Max en Average crashen, FirstOrDefault, Sum, Count, Any en All niet.

Bestaat ook, maar kom je in deze cursus niet tegen

AsParallel (verwerken op meerdere processorkernen), AsQueryable, TryGetNonEnumeratedCount, ToImmutableList, ToFrozenSet en hun familie.

Er bestaat ook een tweede schrijfwijze voor LINQ, die op SQL lijkt. Je zal ze tegenkomen in oudere voorbeelden:

```
var duur = from f in fans
            where f.Prijs > 25
            orderby f.Prijs
            select f.Merk;
```

Dat is precies hetzelfde als `fans.Where(f => f.Prijs > 25).OrderBy(f => f.Prijs).Select(f => f.Merk)`. De cursus gebruikt de schrijfwijze met de methodes.

Voorspel de uitkomst

Vraag 1

```
var getallen = new List<int> { 5, 8, 3, 8 };
Console.WriteLine(getallen.First(g => g > 4));
Console.WriteLine(getallen.Last(g => g > 4));
```

[Toon het antwoord ›](#)

Vraag 2

```
var getallen = new List<int> { 5, 8, 3 };  
Console.WriteLine(getallen.FirstOrDefault(g => g > 10));
```

[Toon het antwoord ›](#)

Vraag 3

```
var getallen = new List<int> { 5, 8, 3, 8 };  
var x = getallen.SingleOrDefault(g => g == 8);
```

[Toon het antwoord ›](#)

Vraag 4

```
var leeg = new List<int>();  
Console.WriteLine(leeg.All(g => g > 100));  
Console.WriteLine(leeg.Any());
```

[Toon het antwoord ›](#)

Vraag 5

```
var namen = new List<string> { "Bram", "Anna", "Cleo" };  
var gesorteerd = namen.OrderBy(n => n);  
Console.WriteLine(namen[0]);
```

[Toon het antwoord ›](#)

Vraag 6

```
var getallen = new List<int> { 1, 2, 3, 4 };  
Console.WriteLine(getallen.Where(g => g > 2).Count());  
Console.WriteLine(getallen.Select(g => g > 2).Count());
```

[Toon het antwoord ›](#)

Vraag 7

```
var woorden = new List<string> { "kat", "olifant", "muis" };  
Console.WriteLine(woorden.Max(w => w.Length));  
Console.WriteLine(woorden.MaxBy(w => w.Length));
```

[Toon het antwoord ›](#)

Vraag 8

```
var namen = new List<string> { "Bram", "Anna", "Bert", "Axel" };
Console.WriteLine(string.Join(", ", namen.OrderBy(n => n[0])));
Console.WriteLine(string.Join(", ", namen.OrderBy(n => n[0]).ThenBy(n => n)));
```

[Toon het antwoord ›](#)

Vraag 9

```
var getallen = new List<int> { 5, 8, 3, 8, 1 };
Console.WriteLine(getallen.Where(g => g > 4).Count());
Console.WriteLine(getallen.TakeWhile(g => g > 4).Count());
```

[Toon het antwoord ›](#)

Vraag 10

```
var r = Enumerable.Range(3, 4);
Console.WriteLine(string.Join(", ", r));
```

[Toon het antwoord ›](#)

Vraag 11

```
var woorden = new List<string> { "kat", "hond", "kat", "vis", "kat" };
var top = woorden.GroupBy(w => w).MaxBy(g => g.Count());
Console.WriteLine(top.Key);
Console.WriteLine(top.Count());
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

5 en daarna 8

Groter dan 4 zijn 5, 8 en 8. `First` neemt de eerste daarvan, `Last` de laatste.

[◀ Terug naar de vraag](#)

VRAAG 2

0

Er is geen getal groter dan 10. `FirstOrDefault` geeft dan de standaardwaarde van het type terug, en bij een `int` is dat nul, niet `null`. Pas op: een 0 kan hier dus zowel "niets gevonden" als "ik vond een nul" betekenen.

[◀ Terug naar de vraag](#)

VRAAG 3

`InvalidOperationException` — het programma crasht.

Er zijn twee achten. `SingleOrDefault` mag er nul of één vinden, twee is voor hem een fout. Dat is precies waarvoor je hem gebruikt: als alarm wanneer iets dat uniek moet zijn het niet is.

[◀ Terug naar de vraag](#)

VRAAG 4

`True` en daarna `False`

In een lege lijst is er geen enkel getal dat níet groter is dan 100, dus `All` zegt `true`. Tegelijk zit er niets in, dus `Any` zegt `false`. Twee antwoorden die elkaar lijken tegen te spreken, en allebei correct.

[◀ Terug naar de vraag](#)

VRAAG 5

Bram

`OrderBy` laat de oorspronkelijke lijst met rust. Het gesorteerde resultaat zit in `gesorteerd`, niet in `namen`. Vergelijk met `Sort` in hoofdstuk 6, die de lijst zelf verandert.

[◀ Terug naar de vraag](#)

VRAAG 6

2 en daarna 4

`Where` houdt enkel 3 en 4 over. `Select` houdt alles over, maar maakt van elk getal een ja of nee: `false`, `false`, `true`, `true`. Vier antwoorden, dus 4.

[◀ Terug naar de vraag](#)

VRAAG 7

7 en daarna `olifant`

`Max` geeft de grootste lengte. `MaxBy` geeft het woord dat die lengte heeft.

[Terug naar de vraag](#)

VRAAG 8

Anna, Axel, Bram, Bert en daarna Anna, Axel, Bert, Bram

`OrderBy(n => n[0])` sorteert enkel op de eerste letter. Namen met dezelfde eerste letter blijven in hun oorspronkelijke volgorde: Bram stond vóór Bert. `ThenBy` sorteert binnen elke letter verder op de hele naam.

[Terug naar de vraag](#)

VRAAG 9

3 en daarna 2

`Where` kijkt naar alle getallen: 5, 8 en 8 zijn groter dan 4. `TakeWhile` stopt bij het eerste getal dat niet voldoet, de 3, en houdt dus enkel 5 en 8.

[Terug naar de vraag](#)

VRAAG 10

3, 4, 5, 6

`Range(3, 4)` betekent: begin bij 3 en geef er 4. Niet: van 3 tot 4.

[Terug naar de vraag](#)

VRAAG 11

kat en daarna 3

`GroupBy` maakt een groep per woord. `MaxBy` kiest de groep met de meeste items. `.Key` is het woord van die groep, `Count()` het aantal.

[Terug naar de vraag](#)

12. IEnumerable

VERZAMELING

Wat het is

Geen verzameling, maar een belofte: "je kan mij één voor één doorlopen". Zoals een playlist: je weet dat je nummer na nummer kan afspelen, maar niet noodzakelijk hoeveel er zijn of wat het vijfde is.

Elke verzameling uit deel 2 is óók een `IEnumerable`. Een array, een `List`, een `HashSet` — ze kunnen allemaal doorlopen worden. En alles wat LINQ teruggeeft, zoals het resultaat van `Where`, is een `IEnumerable`.

Hoe het eruitziet

```
List<int> getallen = new List<int> { 1, 2, 3, 4 };

IEnumerable<int> even = getallen.Where(g => g % 2 == 0);

getallen.Add(6);

Console.WriteLine(even.Count());    // 3 – de 6 telt mee!
```

Dat laatste verrast iedereen de eerste keer. `even` is geen lijst met 2 en 4. Het is een vraag — *geef mij de even getallen uit die lijst* — die pas gesteld wordt op het moment dat je het antwoord nodig hebt. En op dat moment zit de 6 er al in.

Wat je ermee kan

Een kale `IEnumerable<T>` belooft maar één ding: dat je hem van voor naar achter kan doorlopen. Daarvoor heeft hij één methode, `GetEnumerator`, en die gebruik je nooit zelf: `foreach` doet het voor jou.

Wat je wil	Op een <code>IEnumerable</code>	Wat je in de plaats doet
alles doorlopen	<code>foreach (int g in reeks)</code>	dit is waarvoor hij bestaat
weten hoeveel	Niet: <code>reeks.Count</code> compileert niet	<code>reeks.Count()</code> , met haakjes: LINQ telt ze
het item op positie 0	Niet: <code>reeks[0]</code> compileert niet	<code>reeks.First()</code> of <code>reeks.ElementAt(0)</code>
iets bijvoegen	Niet: er is geen <code>Add</code>	<code>reeks.Append(9)</code> geeft een nieuwe reeks; of eerst <code>ToList()</code> en dan <code>Add</code>
iets weghalen	Niet: er is geen <code>Remove</code>	<code>reeks.Where(g => g != 3)</code> geeft een nieuwe reeks zonder 3
sorteren	Niet: er is geen <code>Sort</code>	<code>reeks.Order()</code> of <code>reeks.OrderBy(...)</code>

Alles in de rechterkolom komt van LINQ. Zonder LINQ kan je met een kale reeks enkel `foreach` doen.

Welke types een IEnumerable zijn

Alles wat je kan doorlopen. Veel types beloven ook méér. Die extra beloftes hebben een eigen naam, en je komt ze tegen als parameter of als returntype.

Interface	Belooft bovenop het doorlopen	Wie het belooft
<code>IEnumerable<T></code>	niets: enkel doorlopen	alles hieronder, plus <code>string</code> (een reeks <code>char</code>) en elk LINQ-resultaat
<code>ICollection<T></code>	<code>Count</code>	<code>List</code> , <code>array</code> , <code>Dictionary</code> , <code>HashSet</code> , <code>Queue</code> , <code>Stack</code>
<code>IReadOnlyList<T></code>	<code>Count</code> en <code>[i]</code> om te lezen	<code>List</code> , <code>array</code>
<code>ICollection<T></code>	<code>Count</code> , <code>Add</code> , <code>Remove</code> , <code>Contains</code> , <code>Clear</code>	<code>List</code> , <code>array</code> , <code>Dictionary</code> (van paren), <code>HashSet</code>
<code>IList<T></code>	alles van <code>ICollection<T></code> , plus <code>[i]</code> , <code>Insert</code> , <code>RemoveAt</code> , <code>IndexOf</code>	<code>List</code> , <code>array</code>
<code>IQueryable<T></code>	wordt een databasequery in plaats van in het geheugen te lopen	een <code>DbSet</code> in EF Core: zie hoofdstuk 32

LET OP

Een `array` belooft `ICollection<T>` en `IList<T>`, maar kan niet groeien. Roep je via die belofte `Add` of `Insert` op, dan crasht het met een `NotSupportedException`. Vervangen met `[i]` = mag wel.

Een `Queue` en een `Stack` beloven geen `ICollection<T>`: je kan er niets in steken via `Add`, enkel via hun eigen `Enqueue` en `Push`.

Wat het ook kan, via iets anders

Bron	Wat het geeft	Volledige uitleg
LINQ	alle methodes om te zoeken, kiezen, tellen, ordenen en combineren	hoofdstuk 11
<code>foreach</code>	alles één keer doorlopen	hoofdstuk 4
<code>string.Join</code>	alles in één tekst	hoofdstuk 1
de constructors van de verzamelingen	een echte verzameling uit de reeks	hoofdstukken 5 tot 10
<code>[.. reeks]</code>	een nieuwe lijst of array uit de reeks	hoofdstuk 6

De voorbeelden hieronder vertrekken van `IEnumerable<int> groot = getallen.Where(g => g > 4);` met `getallen` gelijk aan 5, 8, 3, 8, 1. `groot` is dus 5, 8, 8.

LINQ: alles uit hoofdstuk 11

Groep	Methodes	Voorbeeld	Resultaat
één item	First, FirstOrDefault, Single, SingleOrDefault, Last, LastOrDefault, ElementAt, ElementAtOrDefault	<code>groot.First()</code>	5
nagaan	Any, All, Contains, SequenceEqual	<code>groot.Any(g => g > 7)</code>	true
een deel	Where, Select, SelectMany, Distinct, DistinctBy, Take, Skip, TakeLast, SkipLast, TakeWhile, SkipWhile, Chunk, OfType, Index	<code>groot.Distinct()</code>	5, 8
tellen en rekenen	Count, Sum, Min, Max, Average, MinBy, MaxBy, CountBy, Aggregate, AggregateBy	<code>groot.Max()</code>	8
ordenen en groeperen	Order, OrderDescending, OrderBy, OrderByDescending, ThenBy, ThenByDescending, Reverse, Shuffle, GroupBy, ToLookup	<code>getallen.GroupBy(g => g % 2 == 0)</code>	oneven: 5, 3, 1; even: 8, 8
combineren	Concat, Union, Intersect, Except en hun By-varianten, Append, Prepend, Zip, Join, LeftJoin, RightJoin, GroupJoin	<code>groot.Append(0)</code>	5, 8, 8, 0
vastleggen	ToList, ToArray, ToHashSet, ToDictionary, ToLookup	<code>groot.ToList()</code>	een List<int> met 5, 8, 8

Een echte verzameling maken

Schrijfwijze	Resultaat
<code>groot.ToList()</code> of <code>new List<int>(groot)</code> of <code>List<int> l = [.. groot];</code>	een lijst met 5, 8, 8
<code>groot.ToArray()</code>	een array met 5, 8, 8
<code>groot.ToHashSet()</code> of <code>new HashSet<int>(groot)</code>	een set met 5, 8
<code>new Queue<int>(groot), new Stack<int>(groot)</code>	een wachtrij of stapel
<code>string.Join(", ", groot)</code>	de tekst "5, 8, 8"

Wanneer rekent een reeks?

Sommige LINQ-methodes stellen enkel de vraag; andere vragen meteen het antwoord. Dat verschil verklaart het verrassende voorbeeld bovenaan.

Stelt de vraag, rekent nog niet	Vraagt het antwoord, rekent meteen
Where, Select, SelectMany, OrderBy, ThenBy, Reverse, Take, Skip, Distinct, Concat, Union, Append, Zip, Chunk, GroupBy, Join	ToList, ToArray, ToHashSet, ToDictionary, ToLookup, Count, Sum, Min, Max, Average, First, Single, Last, ElementAt, Any, All, Contains, Aggregate, en foreach

Een reeks die nog niet gerekend heeft, rekent **elke keer opnieuw** als je ze gebruikt: twee keer `Count()` is twee keer het werk, en tussendoor kan de bron veranderd zijn. Wil je één vast antwoord, zet er dan `ToList()` achter.

Wat elk object kan

Komt van `object`. Zie hoofdstuk 13.

Methode	Voorbeeld	Resultaat
<code>ToString</code>	<code>groot.ToString()</code>	de naam van een intern type, zoals <code>System.Linq.Enumerable+WhereListIterator`1[System.Int32]</code> , niet de inhoud
<code>Equals / ==</code>	twee keer dezelfde <code>Where</code>	<code>false</code> : twee verschillende vragen
<code>GetType</code>	<code>groot.GetType().Name</code>	<code>WhereListIterator`1</code> : geen <code>List</code>

Een LINQ-resultaat is geen lijst, ook al lijkt het er soms op. `(List<int>)groot` crasht met een `InvalidCastException`. `ToList()` maakt er wel een.

Zelf een reeks maken: yield return

Een methode kan zelf een reeks teruggeven, item per item, met `yield return`.

```
IEnumerable<int> Even(int tot)
{
    for (int i = 0; i <= tot; i += 2)
    {
        yield return i;
    }
}

Console.WriteLine(string.Join(", ", Even(6))); // 0, 2, 4, 6
```

De methode loopt pas als iemand de reeks doorloopt, en elke keer opnieuw — net als een `Where`. `yield break`; stopt de reeks vroegtijdig. In de cursus kom je dit zelden tegen; je herkent het aan het woord `yield`.

Let op bij een IEnumerable

Je schrijft	Wat er gebeurt
<code>reeks.Count</code>	compileert niet; het is <code>reeks.Count()</code>
<code>reeks[0]</code>	compileert niet; het is <code>reeks.First()</code> of <code>reeks.ElementAt(0)</code>
<code>reeks.Count()</code> twee keer	rekent twee keer
<code>List<int> l = getallen.Where(...)</code>	compileert niet; zet er <code>.ToList()</code> achter
de bron veranderen tijdens een <code>foreach</code> over de reeks	crasht met een <code>InvalidOperationException</code>
<code>Select</code> met een <code>Console.WriteLine</code> erin, zonder het resultaat te gebruiken	er wordt niets getoond: de vraag werd nooit gesteld

Wanneer wel, wanneer niet

Wel — als parameter van een methode die enkel moet doorlopen. Wie die methode aanroept, mag dan een array, een `List` of een `HashSet` meegeven.

Niet — zodra je `Count`, een positie of `Add` nodig hebt. Maak er dan eerst met `ToList()` een echte lijst van.

Veelgemaakte fouten

LET OP

- Elke keer dat je een `IEnumerable` doorloopt, wordt de vraag opnieuw gesteld. Twee keer `Count()` op hetzelfde resultaat is twee keer het werk.
- Verandert de bron tussendoor, dan verandert het antwoord mee.
- Er is geen `[0]` en geen `.Count` zonder haakjes. Dat zijn dingen van een `List`, niet van een belofte.

Voorspel de uitkomst

Vraag 1

```
var getallen = new List<int> { 1, 2, 3 };  
var groot = getallen.Where(g => g > 1);  
getallen.Add(10);  
Console.WriteLine(groot.Count());
```

[Toon het antwoord ›](#)

Vraag 2

```
var getallen = new List<int> { 1, 2, 3 };  
var groot = getallen.Where(g => g > 1).ToList();  
getallen.Add(10);  
Console.WriteLine(groot.Count);
```

[Toon het antwoord ›](#)

Vraag 3

```
IEnumerable<int> reeks = new List<int> { 3, 1, 2 };  
Console.WriteLine(reeks.Count());  
Console.WriteLine(reeks.First());
```

[Toon het antwoord ›](#)

Vraag 4

```
var fruit = new List<string> { "appel", "peer" };
var luid = fruit.Select(f =>
{
    Console.WriteLine(" bezig met " + f);
    return f.ToUpper();
});
Console.WriteLine("klaar");
```

[Toon het antwoord ›](#)

Vraag 5

```
int[] getallen = { 1, 2, 3 };
ICollection<int> verzameling = getallen;
verzameling.Add(4);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

3

2, 3 en 10. De vraag werd pas gesteld bij `Count()`, en toen zat de 10 er al in.

[Terug naar de vraag](#)

VRAAG 2

2

Enkel 2 en 3. `ToList()` stelde de vraag meteen en legde het antwoord vast. Wat er nadien bijkomt, telt niet meer mee. Vergelijk met vraag 1: één woord verschil, een ander antwoord.

[Terug naar de vraag](#)

VRAAG 3

3 en daarna 3

`Count()` en `First()` zijn LINQ en werken op elke reeks. `reeks.Count` zonder haakjes en `reeks[0]` hadden niet gecompileerd: de variabele is een `IEnumerable`, ook al zit er een lijst achter.

[Terug naar de vraag](#)

VRAAG 4

klaar

Enkel dat. `Select` stelt de vraag, maar niemand gebruikt het resultaat. Er wordt dus niets gerekend, en de `Console.WriteLine` binnen `Select` loopt nooit. Zet er `.ToList()` achter, en je ziet eerst de twee " bezig met"-regels.

[Terug naar de vraag](#)

VRAAG 5

`NotSupportedException` — het programma crasht.

Het compileert, want een array belooft `ICollection<int>`. Maar een array kan niet groeien, dus `Add` weigert bij het uitvoeren.

[Terug naar de vraag](#)

Deel 4 — Eigen types

Tot hier ging het over wat C# je geeft. Nu over de dingen die je zelf maakt — en over het idee dat bepaalt hoe ze zich gedragen.

13. class

EIGEN TYPE

Wat het is

Een bouwplan. De klasse is het plan, het object is het huis dat ernaar gebouwd is. Met één plan bouw je zoveel huizen als je wil, en elk huis heeft zijn eigen bewoners, zijn eigen kleur deur.

Hoe het eruitziet

```
public class Leerling
{
    public string Naam { get; set; } = "";
    public int Leeftijd { get; set; }

    public bool IsVolwassen()
    {
        return Leeftijd >= 18;
    }
}
```

En zo gebruik je het:

```
Leerling anna = new Leerling { Naam = "Anna", Leeftijd = 17 };

Console.WriteLine(anna.Naam);           // Anna
Console.WriteLine(anna.IsVolwassen());  // False

anna.Leeftijd = 18;                      // aanpassen mag
Console.WriteLine(anna.IsVolwassen());  // True
```

Wil je afdwingen dat een leerling altijd met een naam gemaakt wordt, dan geef je de klasse een **constructor** — een methode met dezelfde naam als de klasse, die draait op het moment van `new`:

```
public class Leerling
{
    public string Naam { get; set; }

    public Leerling(string naam)
    {
        Naam = naam;
    }
}

Leerling bram = new Leerling("Bram");
```

Eigenschappen die zichzelf uitrekenen: =>

Een eigenschap hoeft geen waarde te bewaren. Met `=>` rekent ze haar waarde uit, telkens opnieuw wanneer je erom vraagt:

```
public class Persoon
{
    public string Voornaam { get; set; } = "";
    public string Achternaam { get; set; } = "";

    public string VolledigeNaam => Voornaam + " " + Achternaam;
}
```

`VolledigeNaam` heeft geen `set`: je kan ze lezen, maar niet invullen. Verander je de voornaam, dan verandert de volledige naam vanzelf mee. Dezelfde pijl kan ook een korte methode schrijven: `public bool IsVolwassen() => Leeftijd >= 18;` doet hetzelfde als de versie met `return` hierboven.

In les 07 kom je deze schrijfwijze tegen bij de eigenschappen van de Unit of Work.

Waar een klasse woont: namespace en using

Elke klasse staat in een **namespace**, een soort map voor code. Zo kunnen twee klassen dezelfde naam hebben, zolang ze in een andere namespace staan.

```
namespace School.Models;

public class Leerling
{
    // ...
}
```

Wil je die klasse in een ander bestand gebruiken, dan zeg je bovenaan met `using` in welke map C# moet kijken:

```
using School.Models;

var anna = new Leerling();
```

Je zal twee schrijfwijzen tegenkomen. `namespace School.Models;` met een puntkomma geldt voor het hele bestand. De oudere vorm zet de klasse tussen accolades: `namespace School.Models { ... }`. Ze doen hetzelfde.

Een paar namespaces staan in een nieuw project al klaar zonder dat je ze typt, zoals `System`, `System.Linq` en `System.Collections.Generic`. Daarom kan je `List` en `Where` gebruiken zonder één `using`.

Een object zonder klasse: anonieme objecten

Soms wil je even een paar waarden samen doorgeven zonder er een klasse voor te schrijven:

```
var p = new { Naam = "Anna", Leeftijd = 17 };

Console.WriteLine(p.Naam);           // Anna
// p.Naam = "Bram";                  compileert niet: enkel lezen
```

Het type heeft geen naam, daarom moet de variabele `var` zijn. In les 02 zie je dit bij `CreatedAtAction(..., new { id = laptop.Id }, ...)`: dat kleine object zegt welke waarde er in de route moet komen.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>{ get; set; }</code>	een eigenschap die je kan lezen en invullen	<code>public int Leeftijd { get; set; }</code>	<code>anna.Leeftijd = 18;</code> mag
<code>{ get; }</code>	enkel lezen; invullen kan alleen in de constructor	<code>public string Naam { get; }</code>	<code>anna.Naam = "x";</code> compileert niet
<code>=></code> (eigenschap)	een eigenschap die zichzelf uitrekent	<code>public string VolledigeNaam => ...</code>	rekent telkens opnieuw
<code>=></code> (methode)	een methode van één regel	<code>public bool IsVolwassen() => Leeftijd >= 18;</code>	zoals met <code>return</code>
<code>new Klasse { ... }</code>	maak een object en vul meteen eigenschappen in	<code>new Leerling { Naam = "Anna" }</code>	een leerling met een naam
constructor	draait bij <code>new</code> en kan iets afdwingen	<code>public Leerling(string naam) { ... }</code>	<code>new Leerling()</code> compileert niet meer
<code>namespace</code>	de map waarin een klasse woont	<code>namespace School.Models;</code>	volledige naam <code>School.Models.Leerling</code>
<code>using</code>	zeg waar C# moet zoeken	<code>using School.Models;</code>	<code>Leerling</code> is bruikbaar
<code>new { ... }</code>	een anoniem object	<code>new { id = 5 }</code>	enkel te lezen

LET OP

Krijg je de fout dat een type of namespace niet gevonden wordt, dan ontbreekt meestal een `using`. Visual Studio stelt die zelf voor: zet je cursor op de rode naam en druk op `Ctrl+.`

Wat het ook kan, via iets anders

Elke klasse die je schrijft, krijgt er een paar dingen gratis bij, ook al staan ze niet in je code. Ze komen van `object`, de klasse waar elke klasse in C# van erft ([hoofdstuk 20](#)). Daarnaast bepaalt je klasse hoe ze zich gedraagt in een verzameling en in LINQ.

In de voorbeelden staat `Leerling` in de namespace `School.Models`, met de eigenschappen `Naam` en `Leeftijd`, en zijn `anna` en kopie twee aparte objecten met dezelfde inhoud: Anna, 18 jaar.

Wat elk object kan

Komt van `object`.

Methode	Wat het doet	Voorbeeld	Resultaat
ToString	het object als tekst	<code>anna.ToString()</code>	<code>School.Models.Leerling</code> : de naam van de klasse, niet de inhoud
Equals / <code>==</code>	is dit hetzelfde object	<code>anna == kopie</code>	<code>false</code> , ook al is de inhoud gelijk
GetHashCode	een getal per object, voor opzoektabellen	<code>anna.GetHashCode() == kopie.GetHashCode()</code>	<code>false</code>
GetType	welk type	<code>anna.GetType().Name</code>	<code>Leerling</code>
ReferenceEquals	zijn het echt dezelfde	<code>ReferenceEquals(anna, anna)</code>	<code>true</code>

`ToString` kan je zelf invullen. Dan toont `Console.WriteLine(anna)` wat jij wil:

```
public override string ToString() => $"{Naam} ({Leeftijd})";
```

Met `Equals` en `GetHashCode` kan dat ook, maar dat is zelden nodig: wil je vergelijken op inhoud, neem dan een `record` ([hoofdstuk 14](#)).

In een verzameling en in LINQ

Een lijst, een `HashSet` en een `Dictionary` vergelijken objecten met `Equals`. Bij een klasse is dat: **hetzelfde object**. `anna` staat in `lijst`, `kopie` niet.

Methode	Voorbeeld	Resultaat	Wat je dan doet
Contains, IndexOf, Remove	<code>lijst.Contains(kopie)</code>	<code>false</code>	zoek op een eigenschap: <code>lijst.Any(l => l.Naam == "Anna")</code>
Distinct	<code>new[] { anna, kopie }.Distinct().Count()</code>	<code>2</code>	<code>DistinctBy(l => l.Naam)</code> geeft <code>1</code>
<code>HashSet</code>	<code>new HashSet<Leerling> { anna, kopie }.Count</code>	<code>2</code>	een set van de namen, of een record
<code>Dictionary-sleutel</code>	<code>tabel.ContainsKey(kopie)</code> als <code>anna</code> de sleutel is	<code>false</code>	gebruik een eigenschap als sleutel: <code>Dictionary<string, Leerling></code>
Sort, Order	<code>leerlingen.Sort()</code>	InvalidOperationException : C# weet niet waarop	<code>OrderBy(l => l.Naam)</code> of <code>Sort((a, b) => a.Naam.CompareTo(b.Naam))</code>
Max, Min zonder sleutel	<code>leerlingen.Max()</code>	ArgumentException	<code>Max(l => l.Leeftijd)</code> of <code>MaxBy(l => l.Leeftijd)</code>

Alle andere LINQ-methodes werken gewoon op een lijst van objecten, telkens met een eigenschap in de voorwaarde: `leerlingen.Where(l => l.Leeftijd >= 18)`, `leerlingen.Select(l => l.Naam)`. Zie [hoofdstuk 11](#).

Nog meer via iets anders

Bron	Wat het geeft	Volledige uitleg
------	---------------	------------------

Bron	Wat het geeft	Volledige uitleg
een interface	een belofte van methodes, zodat je klasse ergens past	hoofdstuk 19
overerving	eigenschappen en methodes van een basisklasse	hoofdstuk 20
is, as, switch op type	nagaan van welke klasse een object is	hoofdstuk 22
null	een variabele van een klasse kan leeg zijn	hoofdstuk 17
JSON	een object als tekst bewaren en terug inlezen; enkel publieke eigenschappen tellen	hoofdstuk 30

LET OP

- `Console.WriteLine(anna)` toont de naam van de klasse, tot je zelf `ToString` invult.
- Twee objecten met dezelfde inhoud zijn voor `==`, `Contains`, `Distinct`, `HashSet` en `Dictionary` **niet** gelijk. Dat is de bedoeling bij een klasse: twee leerlingen die toevallig even oud zijn en dezelfde naam hebben, blijven twee leerlingen.

Wanneer wel, wanneer niet

Wel — de standaardkeuze. Iets met gegevens die in de loop van de tijd veranderen, en met gedrag: een leerling die ouder wordt, een bankrekening waarop je stort.

Niet — wanneer het object enkel gegevens draagt en twee objecten met dezelfde inhoud als gelijk moeten gelden. Dan is een `record` handiger.

Veelgemaakte fouten

LET OP

- `Leerling b = a;` maakt geen kopie. Het geeft hetzelfde object een tweede naam, net als bij een array.
- `a == b` zegt of het **hetzelfde** object is, niet of ze **dezelfde inhoud** hebben. Twee leerlingen met dezelfde naam zijn voor een klasse niet gelijk.
- Een klassevariabele zonder `new` is `null`. Ernaar vragen crasht.

Voorspel de uitkomst

Deze vragen gebruiken de eerste versie van `Leerling`, zonder constructor.

Vraag 1

```
var a = new Leerling { Naam = "Anna" };
var b = a;
b.Naam = "Bram";
Console.WriteLine(a.Naam);
```

[Toon het antwoord ›](#)

Vraag 2

```
var a = new Leerling { Naam = "Anna" };
var b = new Leerling { Naam = "Anna" };
Console.WriteLine(a == b);
```

[Toon het antwoord ›](#)

Vraag 3

Deze vraag gebruikt de klasse `Persoon` van hierboven.

```
var p = new Persoon { Voornaam = "Anna", Achternaam = "Peeters" };
p.Voornaam = "Bram";
Console.WriteLine(p.VolledigeNaam);
```

[Toon het antwoord ›](#)

Vraag 4

```
var p = new { Naam = "Anna" };
p.Naam = "Bram";
```

[Toon het antwoord ›](#)

Vraag 5

```
var a = new Leerling { Naam = "Anna" };
var b = new Leerling { Naam = "Anna" };
var lijst = new List<Leerling> { a };
Console.WriteLine(lijst.Contains(b));
Console.WriteLine(lijst.Any(l => l.Naam == b.Naam));
```

[Toon het antwoord ›](#)

Vraag 6

```
var leerlingen = new List<Leerling>
{
    new Leerling { Naam = "Cleo" },
    new Leerling { Naam = "Anna" }
};
leerlingen.Sort();
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

Bram

a en b zijn twee namen voor één en dezelfde leerling. Hernoem je haar via b, dan heet ze ook via a anders.

[← Terug naar de vraag](#)

VRAAG 2

False

Twee keer `new` is twee objecten. Dezelfde naam maakt ze niet tot dezelfde leerling — net zoals twee echte mensen die Anna heten niet dezelfde persoon zijn.

[← Terug naar de vraag](#)

VRAAG 3

Bram Peeters

`VolledigeNaam` bewaart niets. Ze wordt pas samengesteld op het moment dat je erom vraagt, en dan is de voornaam al Bram.

[← Terug naar de vraag](#)

VRAAG 4

Compileert niet.

De eigenschappen van een anoniem object kan je enkel lezen. Moet een waarde kunnen veranderen, schrijf dan een echte klasse.

[← Terug naar de vraag](#)

VRAAG 5

False en daarna True

`Contains` vraagt of b zelf in de lijst staat, en dat is niet zo: b is een ander object. `Any` vergelijkt de namen, en die zijn gelijk.

[← Terug naar de vraag](#)

VRAAG 6

`InvalidOperationException` — het programma crasht.

C# weet niet hoe het twee leerlingen moet vergelijken: op naam, op leeftijd? Zeg het zelf:

`leerlingen.Sort((x, y) => x.Naam.CompareTo(y.Naam))`, of gebruik `OrderBy(1 => 1.Naam)`.

[← Terug naar de vraag](#)

14. record

EIGEN TYPE

Wat het is

Een klasse die draait om haar inhoud. Twee records met exact dezelfde gegevens zijn gelijk. Zoals twee briefjes van tien euro: het maakt niet uit welk exemplaar je hebt, enkel wat erop staat.

Hoe het eruitziet

```
public record Punt(int X, int Y);
```

Die ene regel maakt een type met twee eigenschappen, een constructor, en een nette weergave. En zo gebruik je het:

```
var p1 = new Punt(1, 2);
var p2 = new Punt(1, 2);

Console.WriteLine(p1 == p2);    // True – zelfde inhoud, dus gelijk
Console.WriteLine(p1);          // Punt { X = 1, Y = 2 }

var p3 = p1 with { Y = 5 };     // een kopie met één waarde anders
Console.WriteLine(p3);          // Punt { X = 1, Y = 5 }
Console.WriteLine(p1);          // Punt { X = 1, Y = 2 } – ongewijzigd
```

Wat je ermee kan

Uit die ene regel `public record Punt(int X, int Y);` maakt C# zelf:

Wat C# maakt	Wat het doet	Voorbeeld	Resultaat
eigenschappen	X en Y, enkel te lezen	<code>p1.X</code>	1
constructor	maakt een record met alle waarden	<code>new Punt(1, 2)</code>	een punt
<code>ToString</code>	toont de naam én de inhoud	<code>p1.ToString()</code>	<code>Punt { X = 1, Y = 2 }</code>
<code>Equals</code> , <code>==</code> , <code>!=</code>	vergelijkt de inhoud	<code>p1 == p2</code>	true
<code>GetHashCode</code>	gelijke inhoud geeft hetzelfde getal	<code>p1.GetHashCode() == p2.GetHashCode()</code>	true
<code>with</code>	een kopie met een paar waarden anders	<code>p1 with { Y = 5 }</code>	<code>Punt { X = 1, Y = 5 }</code>
<code>Deconstruct</code>	de waarden in losse variabelen	<code>var (x, y) = p1;</code>	x is 1, y is 2

Een record kan ook gewone eigenschappen en methodes hebben, net als een klasse: `public record Punt(int X, int Y) { public int Som => X + Y; }.`

`record struct` maakt een record dat zich als een struct gedraagt ([hoofdstuk 15](#)): hij wordt gekopieerd in plaats van gedeeld, en heeft toch `==` en een nette `ToString`.

Wat het ook kan, via iets anders

Een record is een klasse ([hoofdstuk 13](#)): alles van daar geldt ook hier. Het grote verschil zit in hoe hij zich gedraagt in een verzameling.

In de voorbeelden is `Leerling` nu een record: `public record Leerling(string Naam, int Leeftijd);`. `a` en `b` zijn twee aparte records `new Leerling("Anna", 18).`

Methode	Voorbeeld	Resultaat	Bij een klasse
Contains , IndexOf , Remove	<code>new List<Leerling> { a }.Contains(b)</code>	<code>true</code>	<code>false</code>
Distinct	<code>new[] { a, b }.Distinct().Count()</code>	<code>1</code>	<code>2</code>
<code>HashSet</code>	<code>new HashSet<Leerling> { a, b }.Count</code>	<code>1</code>	<code>2</code>
Dictionary-sleutel	<code>tabel.ContainsKey(b)</code> als <code>a</code> de sleutel is	<code>true</code>	<code>false</code>
<code>Union</code> , <code>Intersect</code> , <code>Except</code>	vergelijken op inhoud	zoals hierboven	op referentie
Sort , Order	<code>leerlingen.Sort()</code>	<code>InvalidOperationException</code>	ook

Sorteren gaat ook bij een record niet vanzelf: zeg waarop, met `OrderBy(l => l.Naam)`.

Voor JSON, LINQ en `is/switch` gedraagt een record zich als een klasse. Les 09 toont records ook als DTO ([hoofdstuk 33](#)).

Let op bij een record

LET OP

- Een record vergelijkt de inhoud **eigenschap per eigenschap**, met de `Equals` van elke eigenschap. Zit er een `List` in je record, dan wordt die lijst op referentie vergeleken: twee records met elk een eigen lijst met dezelfde namen zijn **niet** gelijk.
- `ToString` toont zo'n lijst ook niet: je ziet `Leerlingen = System.Collections.Generic.List`1[System.String]`.
- `with` maakt een **ondiepe** kopie. De nieuwe record wijst naar dezelfde lijst: voeg je iets toe via de kopie, dan zie je het ook in het origineel.

Wanneer wel, wanneer niet

Wel — een pakketje gegevens dat je doorgeeft en niet meer verandert. In les 09 van Programming Advanced kom je records tegen als een van de twee manieren om DTO's te schrijven; zie [hoofdstuk 33](#).

Niet — wanneer het object in de loop van de tijd verandert en zijn eigen identiteit heeft. Een leerling die ouder wordt blijft dezelfde leerling; dat is een `class`.

Veelgemaakte fouten

LET OP

- De eigenschappen van `record Punt(int X, int Y)` kan je na het aanmaken niet meer aanpassen. `p1.X = 5`; compileert niet.
- `with` verandert niets aan het origineel. Het maakt een nieuw record. Wie het resultaat niet opvangt, heeft niets gedaan.

Voorspel de uitkomst

Vraag 1

```
var p1 = new Punt(1, 2);  
var p2 = p1 with { X = 9 };  
Console.WriteLine(p1.X);
```

[Toon het antwoord ›](#)

Vraag 2

```
var p1 = new Punt(1, 2);  
p1.X = 5;
```

[Toon het antwoord ›](#)

Vraag 3

```
Console.WriteLine(new Punt(3, 4));
```

[Toon het antwoord ›](#)

Vraag 4

```
public record Leerling(string Naam, int Leeftijd);  
  
var a = new Leerling("Anna", 18);  
var b = new Leerling("Anna", 18);
```

```
var set = new HashSet<Leerling> { a, b };  
Console.WriteLine(set.Count);
```

[Toon het antwoord ›](#)

Vraag 5

```
public record Klas(string Naam, List<string> Leerlingen);  
  
var k1 = new Klas("1A", new List<string> { "Anna" });  
var k2 = new Klas("1A", new List<string> { "Anna" });  
Console.WriteLine(k1 == k2);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

1

`with` maakte een kopie waarin `x` 9 is. Het origineel bleef zoals het was.

[Terug naar de vraag](#)

VRAAG 2

Compileert niet.

Bij deze korte schrijfwijze kan je de eigenschappen enkel invullen bij het aanmaken. Wil je een ander punt, maak dan een nieuw — met `with` of met `new`.

[Terug naar de vraag](#)

VRAAG 3

Punt { `X = 3`, `Y = 4` }

Een record weet uit zichzelf hoe het zich moet tonen. Een gewone klasse toont enkel haar naam — dat is een van de redenen waarom records zo handig zijn om mee te testen.

[Terug naar de vraag](#)

VRAAG 4

1

Een record vergelijkt op inhoud. Voor de `HashSet` is `b` dus een dubbel van `a`. Met een gewone klasse was het 2 geweest.

[Terug naar de vraag](#)

VRAAG 5

False

De namen zijn gelijk, maar de lijsten zijn twee verschillende objecten. Een record vergelijkt elke eigenschap met haar eigen `Equals`, en een lijst vergelijkt op referentie.

[Terug naar de vraag](#)

15. struct

EIGEN TYPE

Wat het is

Een klein pakketje waarden dat bij kopiëren écht gekopieerd wordt. Zoals een fotokopie: wat je op de kopie schrijft, staat niet op het origineel.

Je gebruikt ze al zonder het te weten. Een `int`, een `double` en een `bool` zijn allemaal structs.

Hoe het eruitziet

```
public struct Kleur
{
    public int R;
    public int G;
    public int B;
}

Kleur a = new Kleur { R = 255, G = 0, B = 0 };
Kleur b = a;           // een echte kopie
b.R = 0;

Console.WriteLine(a.R); // 255 – a is niet veranderd
Console.WriteLine(b.R); // 0
```

Vergelijk dit met vraag 1 in hoofdstuk 13. Dezelfde drie regels, een tegengesteld resultaat.

Wat je ermee kan

Een struct heeft wat je er zelf in zet: velden, eigenschappen, methodes en constructors, net als een klasse. Wat anders is:

Wat	Bij een struct	Bij een klasse
<code>Kleur b = a;</code>	een kopie	een tweede naam voor hetzelfde object
<code>new Kleur()</code> zonder waarden	alle velden op hun standaardwaarde: R is 0	een object met de standaardwaarden
een variabele zonder waarde	kan niet <code>null</code> zijn	kan <code>null</code> zijn
erven	kan niet	kan wel

Wat het ook kan, via iets anders

Komt van `object`, via `ValueType`. In de voorbeelden zijn `a` en `b` twee `Kleur`-structs met `R = 255`.

Methode	Voorbeeld	Resultaat	Bij een klasse
<code>Equals</code>	<code>a.Equals(b)</code>	<code>true</code> : veld per veld vergeleken	<code>false</code>

Methode	Voorbeeld	Resultaat	Bij een klasse
<code>==</code>	<code>a == b</code>	compileert niet	vergelijkt de referentie
<code>ToString</code>	<code>a.ToString()</code>	de naam van het type, bv. <code>School.Models.Kleur</code>	ook
<code>GetType</code>	<code>a.GetType().Name</code>	<code>Kleur</code>	ook
<code>Contains</code> , <code>Distinct</code> , <code>HashSet</code>	<code>new[] { a, b }.Distinct().Count()</code>	1 : vergelijken via <code>Equals</code>	2
LINQ	<code>kleuren.Where(k => k.R > 100)</code>	werkt zoals bij elk object	ook

`int`, `double`, `decimal`, `bool`, `char`, `DateTime` en `TimeSpan` zijn allemaal structs. Daarom gedragen ze zich als één waarde: zie [hoofdstuk 16](#).

Let op bij een struct

LET OP

- `==` bestaat niet vanzelf voor je eigen struct. Gebruik `Equals`, of maak er een `record` struct van ([hoofdstuk 14](#)): die krijgt `==` en een nette `ToString`.
- `lijst[0].R = 0;` op een lijst van structs **compileert niet**. `lijst[0]` geeft een kopie, en een kopie aanpassen heeft geen zin. Haal hem eruit, pas hem aan, en zet hem terug: `var k = lijst[0]; k.R = 0; lijst[0] = k;`
- In een `foreach` over structs kan je de variabele niet aanpassen.

Wanneer wel, wanneer niet

Wel — zelden zelf nodig. Een klein ding van een paar getallen dat zich gedraagt als één waarde: een kleur, een coördinaat.

Niet — bijna altijd. Twijfel je, neem een `class` of een `record`.

Veelgemaakte fouten

LET OP

- Een struct in een lijst aanpassen past de kopie aan, niet wat in de lijst staat. Dat is de klassieke valkuil, en de reden waarom je ze best klein en onveranderlijk houdt.

Voorspel de uitkomst

Vraag 1

```
int x = 5;
int y = x;
```

```
y = 10;  
Console.WriteLine(x);
```

[Toon het antwoord ›](#)

Vraag 2

```
var kleuren = new List<Kleur> { new Kleur { R = 255 } };  
var eerste = kleuren[0];  
eerste.R = 0;  
Console.WriteLine(kleuren[0].R);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

5

Een `int` is een struct. `y = x` kopieerde de waarde 5. Wat je daarna met `y` doet, raakt `x` niet. Dit gedrag ken je al sinds je eerste les — het heeft nu gewoon een naam.

[◀ Terug naar de vraag](#)

VRAAG 2

255

`kleuren[0]` gaf een kopie. `eerste` aanpassen raakt de struct in de lijst niet. Wil je hem echt aanpassen, zet `eerste` dan terug met `kleuren[0] = eerste;`.

[◀ Terug naar de vraag](#)

16. Referentie tegenover waarde

CONCEPT

Dit is het belangrijkste idee uit dit hele leerboek. Als je één ding onthoudt, laat het dit zijn.

Er zijn twee soorten types in C#, en ze gedragen zich tegengesteld wanneer je ze aan een tweede variabele geeft.

	Referentietype	Waardetype
Voorbeelden	<code>class</code> , <code>record</code> , <code>array</code> , <code>List</code> , <code>Dictionary</code> , alle verzamelingen	<code>int</code> , <code>double</code> , <code>bool</code> , <code>struct</code> , <code>enum</code>
<code>b = a</code> doet	geeft hetzelfde ding een tweede naam	maakt een echte kopie
Via <code>b</code> iets aanpassen	zie je ook via <code>a</code>	raakt <code>a</code> niet
Vergelijking	een huissleutel: twee sleutels, één huis	een fotokopie: twee vellen, elk apart
Kan <code>null</code> zijn	ja	nee, tenzij je <code>?</code> achter het type zet

OPMERKING

`string` is technisch een referentietype, maar gedraagt zich als een waarde: je kan een tekst nooit aanpassen, enkel vervangen door een nieuwe. Daarom verrast hij je nooit op de manier waarop een lijst dat doet.

Test jezelf: kijk terug naar vraag 4 in hoofdstuk 5, vraag 1 in hoofdstuk 13 en de vraag in hoofdstuk 15. Drie keer dezelfde drie regels, en het antwoord hangt enkel af van de soort type. De vraag onderaan dit hoofdstuk brengt het samen.

class, record en struct naast elkaar

	<code>class</code>	<code>record</code>	<code>struct</code>
Soort	referentie	referentie	waarde
<code>a == b</code> vergelijkt	is het hetzelfde object	is de inhoud gelijk	compileert niet; <code>Equals</code> vergelijkt de inhoud
Aanpassen na het maken	ja	met de korte schrijfwijze: nee	ja, maar liever niet
Zichzelf tonen	enkel de naam van het type	alle eigenschappen	enkel de naam van het type
Typisch gebruik	iets met gedrag en een identiteit	een pakketje gegevens	een klein getalachtig ding

Een verzameling kopiëren

Een verzameling is een referentietype. `var b = a;` geeft dus geen kopie. Wil je er wel een, dan zijn er een paar wegen, maar ze kopiëren enkel de verzameling, niet de objecten erin.

Je schrijft	Een nieuwe verzameling?	Dezelfde objecten erin?
-------------	-------------------------	-------------------------

Je schrijft	Een nieuwe verzameling?	Dezelfde objecten erin?
<code>var b = a;</code>	nee: een tweede naam voor dezelfde	ja
<code>new List<Leerling>(a), a.ToList(), a.ToArray()</code>	ja: <code>b.Add(...)</code> raakt <code>a</code> niet	ja: <code>b[0].Naam = "X"</code> zie je ook in <code>a</code>
<code>(int[])getallen.Clone(), getallen.ToArray()</code>	ja	bij getallen, tekst en structs: elke waarde is een kopie

Hetzelfde zie je bij `with` van een record ([hoofdstuk 14](#)): een lijst in de record wordt niet mee gekopieerd. En bij structs in een lijst ([hoofdstuk 15](#)): `lijst[0]` geeft een kopie.

Voorspel de uitkomst

Vraag 1

Dit is de vraag die alles samenbrengt. Twee methodes: de ene krijgt een lijst mee, de andere een getal.

```
void VoegToe(List<int> lijst)
{
    lijst.Add(99);
}

void Verhoog(int getal)
{
    getal = getal + 1;
}

var l = new List<int> { 1 };
int n = 1;

VoegToe(l);
Verhoog(n);

Console.WriteLine(l.Count);
Console.WriteLine(n);
```

[Toon het antwoord >](#)

Vraag 2

```
var a = new List<Leerling> { new Leerling { Naam = "Anna" } };
var b = a.ToList();
b.Add(new Leerling { Naam = "Bert" });
b[0].Naam = "X";
Console.WriteLine(a.Count);
Console.WriteLine(a[0].Naam);
```

[Toon het antwoord >](#)

Antwoorden

VRAAG 1

2 en daarna 1

Een lijst is een referentietype. `VoegToe` krijgt geen kopie van de lijst, maar een tweede sleutel tot dezelfde lijst. Wat de methode erin stopt, zit er daarna nog in.

Een `int` is een waardetype. `Verhoog` krijgt een fotokopie van het getal. Het verhoogt die kopie, en de kopie verdwijnt zodra de methode klaar is. `n` blijft 1.

Dit is de reden waarom een lijst die je aan een methode meegeeft na afloop soms "vanzelf" veranderd is.

[Terug naar de vraag](#)

VRAAG 2

1 en daarna X

`ToList` maakt een nieuwe lijst, dus Bert komt enkel in `b`. Maar de leerling op plaats 0 is in beide lijsten hetzelfde object. Wie zijn naam via `b` aanpast, ziet dat ook via `a`.

[Terug naar de vraag](#)

17. Omgaan met niets

CONCEPT

In het vorige hoofdstuk zag je dat een referentietype `null` kan zijn en een waardetype niet — tenzij je er een vraagteken achter zet. Dit hoofdstuk gaat over hoe je daar veilig mee omgaat.

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>??</code>	neem dit, tenzij het null is	<code>naam ?? "onbekend"</code>	"onbekend" als <code>naam</code> null is
<code>??=</code>	zet enkel als het nog null is	<code>_repo ??= new Repository()</code>	vult één keer in
<code>?.</code>	roep enkel aan als het niet null is	<code>fan?.Merk</code>	<code>null</code> in plaats van een crash
<code>is null</code>	is het niets	<code>if (fan is null)</code>	<code>true</code> of <code>false</code>
<code>is not null</code>	is het iets	<code>if (fan is not null)</code>	<code>true</code> of <code>false</code>
<code>int?</code>	een getal dat ook niets mag zijn	<code>int? hoogte = null</code>	mag leeg blijven
<code>HasValue</code>	zit er een waarde in	<code>hoogte.HasValue</code>	<code>false</code>
<code>GetValueOrDefault</code>	de waarde, of een terugval	<code>hoogte.GetValueOrDefault(0)</code>	<code>0</code>
<code>!</code> achter een waarde	zeg tegen de compiler: dit is niet null	<code>naam!.Length</code>	geen waarschuwing; is het tóch null, dan crasht het
<code>= default!</code>	begin met niets, zonder waarschuwing	<code>public Stad Stad { get; set; } = default!;</code>	de eigenschap is null tot iemand ze invult

Het vraagteken achter een type betekent: deze waarde mag ontbreken. Dat is de manier waarop je in code het verschil vastlegt tussen "nul" en "niet ingevuld" — precies het onderscheid waar een controle op stuk loopt als je het niet maakt.

Het uitroepteken: vertrouw me

In een nieuw project waarschuwt de compiler zodra een referentietype zonder vraagteken `null` zou kunnen zijn, zoals een `string`-eigenschap zonder beginwaarde. Het uitroepteken zet die waarschuwing uit. Aan wat er gebeurt terwijl het programma draait, verandert het niets: het is een belofte aan de compiler, geen controle.

In de cursus zie je het op twee plaatsen:

- `= default!` achter een eigenschap, vanaf les 06. De eigenschap begint als null, maar wordt later ingevuld, bijvoorbeeld door EF Core bij het ophalen. Zie [hoofdstuk 32](#). `= null!` betekent hetzelfde.
- `src.Product!.Naam` in een regel voor Mapster, in les 09. Daar zet het enkel de waarschuwing uit. Mapster kijkt zelf na of het product er is, en laat de naam leeg als het ontbreekt. Zie [hoofdstuk 33](#).

In gewone code geldt: klopt de belofte niet, dan krijg je dezelfde `NullReferenceException` als zonder uitroepteken, alleen zonder waarschuwing vooraf.

Waar null vandaan komt, via iets anders

Methodes die niets teruggeven in plaats van te crashen

Methode	Bij een klasse of tekst	Bij een getal	Hoofdstuk
<code>FirstOrDefault</code> , <code>LastOrDefault</code> , <code>SingleOrDefault</code> , <code>ElementAtOrDefault</code>	<code>null</code>	<code>0</code>	11
<code>FirstOrDefault(-1)</code>	—	je eigen terugval: <code>-1</code>	11
<code>List.Find</code> , <code>Array.Find</code>	<code>null</code>	<code>0</code>	5, 6
<code>GetValueOrDefault</code>	<code>null</code>	<code>0</code> , of je eigen terugval	7
<code>TryGetValue</code> , <code>TryParse</code>	de out-variabele wordt <code>null</code>	de out-variabele wordt <code>0</code>	2, 7
<code>Max</code> , <code>Min</code> , <code>MaxBy</code> , <code>MinBy</code> op een lege verzameling	<code>null</code>	<code>InvalidOperationException</code>	11
<code>as</code>	<code>null</code> als het type niet klopt	—	22
<code>FindAsync</code> , <code>FirstOrDefaultAsync</code>	<code>null</code>	—	32
<code>QueryFirstOrDefaultAsync</code>	<code>null</code>	—	34

LINQ op getallen met een vraagteken

In een `List<int>` slaan `Sum`, `Average`, `Max` en `Min` de lege plaatsen over. Met `{ 1, null, 3 }`:

Methode	Resultaat	Waarom
<code>Sum()</code>	4	1 + 3
<code>Average()</code>	2	(1 + 3) / 2: de lege telt niet mee
<code>Max()</code> , <code>Min()</code>	3, 1	
<code>Count()</code>	3	tellen telt wél alle plaatsen
<code>Max()</code> op een lege <code>List<int></code>	<code>null</code>	geen crash, anders dan bij <code>List<int></code>
<code>Average()</code> als alles leeg is	<code>null</code>	<code>Sum()</code> geeft dan <code>0</code>

Tekst en null

Je schrijft	Resultaat
<code>"a" + naam + "b"</code> met naam gelijk aan null	<code>"ab"</code> : null telt als lege tekst
<code>string.Join(",", ...)</code> op <code>"a"</code> , null, <code>"b"</code>	<code>"a,,b"</code>
<code>namen.OfType<string>()</code>	enkel de teksten die niet null zijn
<code>namen.Contains(null)</code>	<code>true</code> als er een null in zit
<code>string.IsNullOrEmpty(naam)</code>	null of leeg in één keer; zie hoofdstuk 1

LET OP

- `int? x = null;` en dan `x.Value`: **`InvalidOperationException`**. Gebruik `GetValueOrDefault` of `??`.
- `lijst?.Count` geeft een `int?`: null als de lijst null is.
- `int.Parse(null)` crasht met **`ArgumentNullException`**, maar `Convert.ToInt32(null)` geeft gewoon `0`.

Voorspel de uitkomst

Vraag 1

```
string? naam = null;  
Console.WriteLine(naam ?? "onbekend");
```

[Toon het antwoord ›](#)

Vraag 2

```
string? naam = null;  
Console.WriteLine(naam?.Length);
```

[Toon het antwoord ›](#)

Vraag 3

```
string? naam = null;  
Console.WriteLine(naam.Length);
```

[Toon het antwoord ›](#)

Vraag 4

```
int? score = null;  
Console.WriteLine(score.HasValue);  
Console.WriteLine(score.GetValueOrDefault());
```

[Toon het antwoord ›](#)

Vraag 5

```
string? a = null;  
a ??= "eerste";  
a ??= "tweede";  
Console.WriteLine(a);
```

[Toon het antwoord ›](#)

Vraag 6

```
string? naam = null;  
Console.WriteLine(naam!.Length);
```

[Toon het antwoord ›](#)

Vraag 7

```
var hoogtes = new List<int?> { 4, null, 2 };  
Console.WriteLine(hoogtes.Average());  
Console.WriteLine(hoogtes.Count());
```

[Toon het antwoord ›](#)

Vraag 8

```
var namen = new List<string>();  
var getallen = new List<int>();  
Console.WriteLine(namen.Max() is null);  
Console.WriteLine(getallen.Max());
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

onbekend

naam is null, dus ?? neemt wat rechts staat.

[← Terug naar de vraag](#)

VRAAG 2

Een lege regel.

naam?.Length vraagt de lengte enkel op als naam niet null is. Anders is het resultaat zelf null, en Console.WriteLine toont null als niets. Geen crash.

[← Terug naar de vraag](#)

VRAAG 3

NullReferenceException — het programma crasht.

Het enige verschil met vraag 2 is het vraagteken. Zonder ?. vraag je de lengte op van iets dat er niet is. Visual Studio waarschuwt je hier wel met een groene kronkellijn, maar compileren gaat.

[← Terug naar de vraag](#)

VRAAG 4

False en daarna 0

Er zit geen waarde in. GetValueOrDefault geeft dan de standaardwaarde van een int: nul. Merk op dat je nu wél kan zien dat de score ontbrak, via HasValue. Bij een gewone int kon dat niet.

[← Terug naar de vraag](#)

VRAAG 5

eerste

??= vult enkel in als er nog niets staat. De eerste keer was a null, de tweede keer niet meer.

[← Terug naar de vraag](#)

VRAAG 6

NullReferenceException — het programma crasht.

Hetzelfde als bij vraag 3, maar nu zonder groene kronkellijn. Het uitroepteken zegt de compiler dat naam niet null is. De compiler gelooft het, en zwijgt. Het programma zelf kijkt niet na, en loopt vast op de lengte van iets dat er niet is.

[← Terug naar de vraag](#)

VRAAG 7

3 en daarna 3

Average slaat de lege plaats over: $(4 + 2) / 2$. Count telt alle plaatsen, ook de lege.

[Terug naar de vraag](#)

VRAAG 8

True, en daarna **InvalidOperationException** — het programma crasht.

Bij een lege verzameling teksten geeft Max gewoon null. Een int kan niet null zijn, dus daar crasht het.

[Terug naar de vraag](#)

18. enum

EIGEN TYPE

Wat het is

Een vaste lijst benoemde keuzes. Een verkeerslicht staat op rood, oranje of groen. Niet op paars, niet op "rood" met drie o's.

Hoe het eruitziet

```
public enum Verkeerslicht
{
    Rood,
    Oranje,
    Groen
}
```

En zo gebruik je het:

```
Verkeerslicht licht = Verkeerslicht.Rood;

if (licht == Verkeerslicht.Rood)
{
    Console.WriteLine("Stop");
}

switch (licht)
{
    case Verkeerslicht.Rood:
        Console.WriteLine("Wachten");
        break;
    case Verkeerslicht.Groen:
        Console.WriteLine("Rijden");
        break;
}

Console.WriteLine(licht);           // Rood
Console.WriteLine((int)licht);     // 0
```

Tekst omzetten naar een enum gaat met `Enum.TryParse`, net als bij getallen:

```
if (Enum.TryParse("Groen", out Verkeerslicht gekozen))
{
    Console.WriteLine(gekozen);     // Groen
}
```

Wat je ermee kan

Een enum heeft zelf geen methodes die je schrijft. Wat je ermee doet, komt van de klasse `Enum` en van de taal. In de voorbeelden is het `Verkeerslicht` van hierboven: Rood (0), Oranje (1), Groen (2).

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToString</code>	de naam als tekst	<code>Verkeerslicht.Oranje.ToString()</code>	"Oranje"
<code>(int)</code>	het getal erachter	<code>(int)Verkeerslicht.Groen</code>	2
<code>(Verkeerslicht)</code>	van een getal naar een keuze	<code>(Verkeerslicht)1</code>	Oranje
<code>Enum.GetValues</code>	alle keuzes	<code>Enum.GetValues<Verkeerslicht>()</code>	Rood, Oranje, Groen
<code>Enum.GetNames</code>	alle namen, als tekst	<code>Enum.GetNames<Verkeerslicht>()</code>	"Rood", "Oranje", "Groen"
<code>Enum.TryParse</code>	tekst omzetten, zonder te crashen	<code>Enum.TryParse("groen", true, out Verkeerslicht v)</code>	true: met true negeert het hoofdletters
<code>Enum.Parse</code>	tekst omzetten, of crashen	<code>Enum.Parse<Verkeerslicht>("Paars")</code>	ArgumentException
<code>Enum.IsDefined</code>	bestaat deze keuze echt	<code>Enum.IsDefined((Verkeerslicht)7)</code>	false
<code>==, !=, <, ></code>	vergelijken, op het getal erachter	<code>Verkeerslicht.Rood < Verkeerslicht.Groen</code>	true
<code>CompareTo, Equals, GetType</code>	wat elk object kan	<code>Verkeerslicht.Rood.GetType().Name</code>	Verkeerslicht
switch	per keuze iets anders doen	<code>licht switch { Verkeerslicht.Rood => "stop", _ => "rijden" }</code>	"stop" bij rood

Wat het ook kan, via iets anders

Bron	Wat het geeft	Voorbeeld	Resultaat
LINQ op <code>Enum.GetValues</code>	werken met alle keuzes	<code>Enum.GetValues<Verkeerslicht>().Where(v => v != Verkeerslicht.Oranje)</code>	Rood, Groen
LINQ op een lijst keuzes	tellen, sorteren, groeperen	<code>lichten.Count(v => v == Verkeerslicht.Groen)</code>	het aantal groene
<code>Order</code>	sorteert op het getal erachter, niet op de naam	<code>lichten.Order()</code>	eerst alle Rood, dan Oranje, dan Groen
een tekst	alles van hoofdstuk 1 op de naam	<code>licht.ToString().ToLower()</code>	"groen"
default	de keuze met getal 0	<code>default(Verkeerslicht)</code>	Rood

Een enum is een waarde, zoals een getal: `==` vergelijkt de keuze, en `Contains`, `Distinct` en `HashSet` werken zoals je verwacht.

Bestaat ook, maar kom je in deze cursus niet tegen

[Flags]: een enum waarvan je meerdere keuzes tegelijk kan aanzetten, zoals `Rechten.Lezen` | `Rechten.Schrijven`, met `HasFlag` om er één na te gaan. En `Enum.GetUnderlyingType`, `Enum.Format`.

Let op bij een enum

LET OP

- `Enum.TryParse` aanvaardt ook getallen: `Enum.TryParse("7", out Verkeerslicht v)` geeft `true`, met `v` gelijk aan 7 — een keuze die niet bestaat. Controleer achteraf met `Enum.IsDefined`.
- Een eigenschap van een enum-type die je niet invult, staat op de keuze met getal 0. Bij `Verkeerslicht` is dat `Rood`, zonder dat iemand het koos.
- `Order` sorteert op het getal, niet alfabetisch op de naam.

Wanneer wel, wanneer niet

Wel — een eigenschap heeft een klein, vast aantal mogelijke waarden die niet veranderen terwijl je programma draait: dagen van de week, een status, een richting. Het grote voordeel tegenover een `string`: een tikfout wordt een compileerfout in plaats van een fout die pas opduikt als het te laat is.

Niet — wanneer de lijst van keuzes groeit terwijl het programma draait, of door gebruikers aangevuld wordt. Een enum wijzig je enkel door je code te herschrijven.

Veelgemaakte fouten

LET OP

- Onder de motorkap is elke keuze een getal: `Rood` is 0, `Oranje` 1, `Groen` 2. Voeg je vooraan een keuze toe, dan schuiven alle getallen op. Staan die getallen ergens opgeslagen, dan betekenen ze plots iets anders.
- `(Verkeerslicht)7` compileert, ook al bestaat er geen zevende kleur.
- `Enum.TryParse` is standaard hoofdlettergevoelig.

Voorspel de uitkomst

Vraag 1

```
Console.WriteLine((int)Verkeerslicht.Groen);
```

[Toon het antwoord ›](#)

Vraag 2

```
Verkeerslicht l = (Verkeerslicht)7;  
Console.WriteLine(l);
```

[Toon het antwoord ›](#)

Vraag 3

```
bool ok = Enum.TryParse("groen", out Verkeerslicht l);  
Console.WriteLine(ok);
```

[Toon het antwoord ›](#)

Vraag 4

```
bool gelukt = Enum.TryParse("1", out Verkeerslicht v);  
Console.WriteLine(gelukt);  
Console.WriteLine(v);
```

[Toon het antwoord ›](#)

Vraag 5

```
var licht = (Verkeerslicht)7;  
Console.WriteLine(licht);  
Console.WriteLine(Enum.IsDefined(licht));
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

2

Tellen begint bij nul: Rood is 0, Oranje 1, Groen 2.

[← Terug naar de vraag](#)

VRAAG 2

7

C# laat het toe, en omdat er geen naam bij 7 hoort, toont het gewoon het getal. Een enum beschermt je dus tegen tikfouten in je code, maar niet tegen elk getal dat van buiten binnenkomt.

[← Terug naar de vraag](#)

VRAAG 3

False

"groen" met kleine letter is niet Groen. Wil je dat het wel lukt, dan geef je een extra `true` mee om hoofdletters te negeren.

[← Terug naar de vraag](#)

VRAAG 4

True en daarna Oranje

`TryParse` aanvaardt ook het getal achter een keuze. 1 is Oranje.

[← Terug naar de vraag](#)

VRAAG 5

7 en daarna False

Omzetten van een getal lukt altijd, ook als er geen keuze bij hoort. `ToString` toont dan gewoon het getal. `IsDefined` zegt dat de keuze niet bestaat.

[← Terug naar de vraag](#)

19. interface

EIGEN TYPE

Wat het is

Een contract. Het zegt wát iets moet kunnen, niet hóe. Zoals een stopcontact: elk toestel met de juiste stekker past erin, of het nu een lamp is of een waterkoker. Het stopcontact hoeft niet te weten wat je aansluit.

Hoe het eruitziet

```
public interface IGeluid
{
    string MaakGeluid();
}
```

Het contract zegt enkel: *wie mij belooft, kan een geluid maken*. Geen uitwerking. Die komt in de klassen die de belofte doen:

```
public class Hond : IGeluid
{
    public string MaakGeluid()
    {
        return "Woef";
    }
}

public class Kat : IGeluid
{
    public string MaakGeluid()
    {
        return "Miauw";
    }
}
```

En nu het nut ervan:

```
List<IGeluid> dieren = new List<IGeluid> { new Hond(), new Kat() };

foreach (IGeluid dier in dieren)
{
    Console.WriteLine(dier.MaakGeluid());
}
```

De lus weet niet of hij een hond of een kat voor zich heeft. Dat hoeft ook niet: hij weet dat ze allebei een geluid kunnen maken, en meer heeft hij niet nodig.

Waarom dit in je cursus zo belangrijk is

In les 03 van Programming Advanced maak je een repository met een interface ervoor. Dat is precies dit principe. De controller praat met het contract, niet met de klasse. Daardoor kan de klasse achter het contract later vervangen worden — een lijst in het geheugen wordt een database — zonder dat de controller één letter verandert.

Wat het ook kan, via iets anders

.NET kent zelf een paar interfaces. Beloofd je klasse er een, dan werken er methodes uit andere hoofdstukken die anders niet werken.

Interface	Je belooft	Wat je er gratis bij krijgt	Hoofdstuk
<code>IComparable<T></code>	<code>int CompareTo(T ander)</code>	<code>Sort()</code> , <code>Order()</code> , <code>Max()</code> en <code>Min()</code> zonder sleutel	6, 11, 13
<code>IEnumerable<T></code>	een manier om je items één voor één te geven	<code>foreach</code> en heel LINQ	12

```
public class Leerling : IComparable<Leerling>
{
    public string Naam { get; set; } = "";

    public int CompareTo(Leerling? ander) => Naam.CompareTo(ander?.Naam);
}
```

Zonder die belofte crasht `leerlingen.Sort()` (hoofdstuk 13). Met de belofte sorteert het op naam, en geeft `leerlingen.Max()` de leerling met de naam die alfabetisch het laatst komt.

Een `List<IGeluid>` is een gewone lijst: alles van hoofdstuk 6 en LINQ werkt erop, zoals `dieren.Select(d => d.MaakGeluid())`. Wil je enkel de katten, gebruik dan `OfType<Kat>()` uit hoofdstuk 22.

Bestaat ook, maar kom je in deze cursus niet tegen

`IEquatable<T>` (zelf bepalen wanneer twee objecten gelijk zijn), `IDisposable` (opruimen met `using`), `IComparer<T>` (een aparte vergelijker die je aan `Sort` geeft).

Wanneer wel, wanneer niet

Wel — wanneer verschillende klassen hetzelfde moeten kunnen, of wanneer je wil dat een deel van je programma later vervangbaar is zonder de rest aan te passen.

Niet — voor een klasse waarvan er maar één soort ooit zal bestaan en die je nooit zal vervangen. Een contract met maar één ondertekenaar voegt weinig toe.

Moeten verwante klassen niet alleen hetzelfde kunnen, maar ook gegevens of uitgewerkte methodes delen, dan is een abstracte klasse het betere antwoord. Zie hoofdstuk 20.

Veelgemaakte fouten

LET OP

- Van een interface kan je geen object maken. `new IGeluid()` compileert niet — je kan geen stopcontact aansluiten op een ander stopcontact.
- Beloofd een klasse het contract, dan moet ze **alles** uitwerken wat erin staat. Eén methode vergeten en het compileert niet.
- Volgens de afspraak begint de naam met een hoofdletter **I**.

Voorspel de uitkomst

Vraag 1

Wat verschijnt er op het scherm bij de lus hierboven?

[Toon het antwoord ›](#)

Vraag 2

```
IGeluid g = new IGeluid();
```

[Toon het antwoord ›](#)

Vraag 3

```
IGeluid g = new Kat();  
Console.WriteLine(g is Kat);
```

[Toon het antwoord ›](#)

Vraag 4

`Leerling` belooft `IComparable<Leerling>` zoals hierboven.

```
var leerlingen = new List<Leerling>  
{  
    new Leerling { Naam = "Cleo" },  
    new Leerling { Naam = "Anna" },  
    new Leerling { Naam = "Bert" }  
};  
leerlingen.Sort();  
Console.WriteLine(leerlingen[0].Naam);  
Console.WriteLine(leerlingen.Max().Naam);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

Woef en daarna Miauw

Elke klasse gebruikt haar eigen uitwerking, ook al weet de lus enkel dat ze `IGeluid` beloven.

[◀ Terug naar de vraag](#)

VRAAG 2

Compileert niet.

Een interface is een belofte zonder uitwerking. Er valt niets te bouwen.

[◀ Terug naar de vraag](#)

VRAAG 3

True

De variabele zegt enkel "iets dat een geluid maakt", maar het object dat erin zit is nog altijd een kat. Met `is` kan je dat nagaan.

[◀ Terug naar de vraag](#)

VRAAG 4

Anna en daarna Cleo

`Sort` en `Max` gebruiken jouw `CompareTo`, en die vergelijkt de namen. Zonder de interface was `Sort` gecrasht met een `InvalidOperationException`.

[◀ Terug naar de vraag](#)

20. Overerving en abstracte klassen

EIGEN TYPE

Wat het is

Een klasse die verder bouwt op een andere. Zoals een basisrecept voor deeg: pizza en brood beginnen allebei met hetzelfde deeg, en voegen er elk hun eigen stappen aan toe. Wie het basisrecept aanpast, past het voor allebei aan.

De klasse waarop je verder bouwt, heet de **basisklasse**. De klasse die erop verder bouwt, heet de **afgeleide klasse**. Die krijgt alles mee wat de basisklasse heeft, en voegt er haar eigen dingen aan toe.

Hoe het eruitziet

```
public class Voertuig
{
    public int Wielen { get; set; }

    public string Toeter()
    {
        return "Tuut";
    }
}

public class Fiets : Voertuig
{
    public bool HeeftBel { get; set; }
}
```

Het dubbelpunt na `Fiets` betekent: *een fiets is een voertuig*. Een fiets heeft dus ook `Wielen` en kan ook `Toeter()`, zonder dat die er opnieuw in staan:

```
Fiets f = new Fiets { Wielen = 2, HeeftBel = true };

Console.WriteLine(f.Wielen);    // 2 – komt van Voertuig
Console.WriteLine(f.Toeter());  // Tuut – komt van Voertuig
Console.WriteLine(f.HeeftBel);  // True – van de fiets zelf

Voertuig v = f;                 // mag: een fiets is een voertuig
Console.WriteLine(v.Wielen);    // 2
// v.HeeftBel compileert niet: via v zie je enkel wat elk voertuig heeft
```

Een basisklasse die zelf nooit bestaat: abstract

Een "voertuig" op zich rijdt nergens rond. Wat op de weg staat, is altijd een fiets, een bus of iets anders. Met `abstract` leg je dat vast: van de basisklasse zelf kan je geen object meer maken, enkel van wat ervan erft.


```

public abstract class Voertuig
{
    public int Wielen { get; }
    protected int Kilometers { get; set; }

    protected Voertuig(int wielen)
    {
        Wielen = wielen;
    }

    public void Rij(int km)
    {
        Kilometers += km;
    }

    public virtual string Toeter()
    {
        return "Tuut";
    }

    public abstract int MaxPassagiers();
}

public class Fiets : Voertuig
{
    public Fiets() : base(2) { }

    public override string Toeter()
    {
        return "Tring";
    }

    public override int MaxPassagiers()
    {
        return 1;
    }
}

public class Bus : Voertuig
{
    public Bus() : base(6) { }

    public override int MaxPassagiers()
    {
        return 50;
    }
}

```

Wat daar gebeurt:

- `abstract` vóór `class`: een object maken van `Voertuig` zelf compileert niet meer.
- `protected Voertuig(int wielen)` is de constructor van de basisklasse. Met `: base(2)` roept de fiets hem aan met twee wielen.
- `protected int Kilometers` is zichtbaar in `Voertuig` en in elke klasse die ervan erft, maar niet daarbuiten.

- `virtual` geeft `Toeter` een standaardversie die een afgeleide klasse mag vervangen. De fiets doet dat met `override`, de bus niet.
- `abstract` vóór een methode betekent: geen uitwerking. Elke afgeleide klasse moet ze zelf uitwerken, met `override`.

En het nut ervan:

```
List<Voertuig> vloot = new List<Voertuig> { new Fiets(), new Bus() };

foreach (Voertuig v in vloot)
{
    Console.WriteLine($"{v.Wielen} wielen, {v.Toeter()}, max {v.MaxPassagiers()}");
}
// 2 wielen, Tring, max 1
// 6 wielen, Tuut, max 50
```

De lus weet niet of ze een fiets of een bus voor zich heeft. Dat hoeft ook niet: elk voertuig heeft wielen, kan toeteren en weet hoeveel passagiers erin passen. Het is hetzelfde idee als bij de interface in [hoofdstuk 19](#), met één verschil: de basisklasse geeft ook gegevens en uitgewerkte methodes mee.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>: Basisklasse</code>	erf van een klasse	<code>class Fiets : Voertuig</code>	een fiets heeft alles van een voertuig
<code>: Basisklasse, IContract</code>	erf van één klasse en belooft daarnaast contracten	<code>class Fiets : Voertuig, IGeluid</code>	eerst de klasse, dan de interfaces
<code>base(...)</code>	roep de constructor van de basisklasse aan	<code>public Fiets() : base(2) { }</code>	Wielen is 2
<code>protected</code>	zichtbaar in de klasse en in wie ervan erft, niet daarbuiten	<code>protected int Kilometers</code>	<code>fiets.Kilometers</code> compileert niet
<code>virtual</code>	deze methode mag vervangen worden	<code>public virtual string Toeter()</code>	de bus gebruikt de standaardversie
<code>override</code>	vervang een methode die <code>virtual</code> of <code>abstract</code> is	<code>public override string Toeter()</code>	"Tring"
<code>base.Methode()</code>	gebruik de versie van de basisklasse, binnen je eigen versie	<code>return base.Toeter() + "!"</code>	"Tuut!"
<code>abstract class</code>	een klasse waarvan je geen object kan maken	<code>new Voertuig(4)</code>	compileert niet
<code>abstract</code> (methode)	een methode zonder uitwerking; elke afgeleide klasse moet ze uitwerken	<code>public abstract int MaxPassagiers();</code>	vergeet je ze, dan compileert het niet
<code>sealed</code>	van deze klasse mag niemand meer erven	<code>public sealed class Bus : Voertuig</code>	<code>class Dubbeldekker : Bus</code> compileert niet

Vorm	Wat het doet	Voorbeeld	Resultaat
is	is dit object van dat type, of ervan afgeleid	<code>new Fiets() is Voertuig</code>	<code>true</code>
GetType	het echte type van het object, niet dat van de variabele	<code>Voertuig v = new Fiets();</code> en dan <code>v.GetType().Name</code>	<code>"Fiets"</code>

Wat het ook kan, via iets anders

Bron	Wat het geeft	Voorbeeld	Resultaat
een lijst van de basisklasse	fietsen en bussen samen in één lijst	<code>var voertuigen = new List<Voertuig> { new Fiets(), new Bus() };</code>	—
LINQ op die lijst	werken met wat alle voertuigen hebben	<code>voertuigen.Sum(v => v.Wielen)</code>	<code>8</code>
OfType	enkel één soort	<code>voertuigen.OfType<Fiets>()</code>	enkel de fietsen, als <code>Fiets</code>
is in LINQ	tellen per soort	<code>voertuigen.Count(v => v is Bus)</code>	<code>1</code>
object	elke klasse erft er uiteindelijk van: <code>ToString</code> , <code>Equals</code> , <code>GetType</code>	<code>override string ToString()</code>	<code>Console.WriteLine(v)</code> en <code>(\$"{v}")</code> tonen jouw tekst

Zie [hoofdstuk 13](#) voor wat je van `object` krijgt, en [hoofdstuk 22](#) voor `is`, `as` en `OfType`.

Abstracte klasse tegenover interface

	Abstracte klasse	Interface
Zegt	wat iets is	wat iets kan
Bewaart gegevens	ja	nee — ze kan enkel zeggen dát er een eigenschap moet zijn
Uitgewerkte methodes	ja, naast de abstracte	nee, op een zeldzame uitzondering na die je in deze cursus niet tegenkomt
Constructor	ja	nee
Hoeveel per klasse	één	zoveel je wil
Een object van maken	nee	nee
Typisch gebruik	verwante klassen die gegevens en werk delen: een fiets en een bus	klassen die hetzelfde moeten kunnen, ook als ze niets met elkaar te maken hebben: een hond en een deurbel maken allebei geluid

Waar je dit in de cursus tegenkomt

Les	Wat je ziet	Wat er gebeurt
01	een controller die erft van <code>ControllerBase</code>	je controller erft <code>Ok</code> , <code>NotFound</code> en de rest uit hoofdstuk 26
04	<code>: base(options)</code> in je context	de opties gaan door naar de constructor van <code>DbContext</code> ; zie hoofdstuk 31

Les	Wat je ziet	Wat er gebeurt
04	<code>protected override void OnModelCreating(...)</code>	je vervangt een methode die <code>DbContext</code> daarvoor openzet
07	een specifieke repository die erft van de generieke	ze krijgt alle gewone methodes mee; <code>protected</code> maakt de context ook voor haar zichtbaar

Wanneer wel, wanneer niet

Wel — verwante klassen die echt hetzelfde zijn en gegevens of uitgewerkte methodes delen. De test: klinkt *een X is een Y* juist? Een fiets is een voertuig: ja. Maak de basisklasse `abstract` zodra ze op zichzelf niets betekent.

Niet — enkel om een paar methodes te hergebruiken tussen klassen die niets met elkaar te maken hebben. Een leerling is geen klaslokaal, ook al hebben ze allebei een naam. En gaat het enkel om wat iets kan, dan is een interface lichter.

Veelgemaakte fouten

LET OP

- Wat `private` is in de basisklasse, kan de afgeleide klasse niet zien. Heeft ze het nodig, maak het dan `protected` — niet `public`, want dan kan iedereen erbij.
- Heeft de basisklasse enkel een constructor met parameters, dan moet elke afgeleide klasse : `base(...)` aanroepen. Vergeet je dat, dan compileert het niet.
- `override` kan enkel op een methode die `virtual` of `abstract` is. Schrijf je in de afgeleide klasse gewoon een methode met dezelfde naam, dan vervang je niets: je verstopt de oude alleen. Visual Studio waarschuwt je, maar het compileert. Zie vraag 4.
- Een klasse erft van hoogstens één klasse. Interfaces mag je er zoveel bij zetten als je wil.
- Via een variabele van de basisklasse zie je enkel wat de basisklasse heeft, ook als het object meer kan. Met `is` uit [hoofdstuk 22](#) ga je na wat het echt is.

Voorspel de uitkomst

Vragen 1, 2, 3 en 6 gebruiken de tweede versie van `Voertuig`, met `abstract`, en de klassen `Fiets` en `Bus` die erbij horen.

Vraag 1

```
Voertuig v = new Fiets();
Console.WriteLine(v.Toeter());
```

[Toon het antwoord ›](#)

Vraag 2

```
Voertuig v = new Bus();  
Console.WriteLine(v.Toeter());
```

[Toon het antwoord ›](#)

Vraag 3

```
Voertuig v = new Voertuig(4);
```

[Toon het antwoord ›](#)

Vraag 4

```
public class Basis  
{  
    public string Wie()  
    {  
        return "basis";  
    }  
}  
  
public class Afgeleid : Basis  
{  
    public string Wie()  
    {  
        return "afgeleid";  
    }  
}  
  
Basis b = new Afgeleid();  
Afgeleid a = new Afgeleid();  
Console.WriteLine(b.Wie());  
Console.WriteLine(a.Wie());
```

[Toon het antwoord ›](#)

Vraag 5

```
public class A  
{  
    public A()  
    {  
        Console.WriteLine("A");  
    }  
}  
  
public class B : A  
{  
    public B()  
    {  
        Console.WriteLine("B");  
    }  
}
```

```
new B();
```

[Toon het antwoord ›](#)

Vraag 6

```
Voertuig v = new Fiets();  
Console.WriteLine(v is Voertuig);  
Console.WriteLine(v.GetType() == typeof(Voertuig));  
Console.WriteLine(v.GetType().Name);
```

[Toon het antwoord ›](#)

Vraag 7

```
var voertuigen = new List<Voertuig> { new Fiets(), new Bus(), new Fiets() };  
Console.WriteLine(voertuigen.OfType<Fiets>().Count());  
Console.WriteLine(voertuigen.Sum(v => v.Wielen));
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

Tring

De variabele zegt enkel "een voertuig", maar het object is een fiets. Omdat `Toeter` virtual is en de fiets ze met `override` vervangt, telt de versie van het object, niet die van de variabele.

[Terug naar de vraag](#)

VRAAG 2

Tuut

De bus vervangt `Toeter` niet. Ze gebruikt dus de standaardversie uit `Voertuig`. Een virtual methode mag je vervangen, je moet niet.

[Terug naar de vraag](#)

VRAAG 3

Compileert niet.

`Voertuig` is abstract. Van de basisklasse zelf kan je geen object maken, enkel van een klasse die ervan erft: een fiets of een bus.

[Terug naar de vraag](#)

VRAAG 4

`basis` en daarna `afgeleid`

Twee keer hetzelfde object, twee verschillende antwoorden. Zonder `virtual` en `override` vervangt `Afgeleid` de methode niet, maar verstopt ze haar enkel. Welke versie je krijgt, hangt dan af van de variabele, niet van het object. Zet `virtual` bij `Basis` en `override` bij `Afgeleid`, en je krijgt twee keer `afgeleid`.

[Terug naar de vraag](#)

VRAAG 5

A en daarna B

Eerst de fundering, dan de muren. Vóór de constructor van B draait die van de basisklasse. Staat er geen `: base(...)`, dan roept C# zelf de constructor zonder parameters van A aan.

[Terug naar de vraag](#)

VRAAG 6

`True`, `False` en daarna `Fiets`

`is` vraagt: *is dit een voertuig, of iets dat ervan erft?* Een fiets is dat. `GetType()` geeft het echte type van het object, en dat is precies `Fiets`, niet `Voertuig`. Wil je weten of iets tot een familie behoort, gebruik dan `is`.

[Terug naar de vraag](#)

VRAAG 7

2 en daarna 10

OfType houdt enkel de fietsen over. Sum mag Wielen gebruiken, want elk voertuig heeft wielen: $2 + 6 + 2$.

[Terug naar de vraag](#)

21. Generics

EIGEN TYPE

OPMERKING

Zelf generics schrijven komt aan bod in les 07 van Programming Advanced. Het staat hier al, zodat je het klaar hebt wanneer je het nodig hebt.

Wat het is

Een type met een open plaats. Zoals een doos met een etiket *inhoud: ____*. De doos werkt hetzelfde voor schoenen, boeken of speelgoed; pas wanneer je hem gebruikt, vul je in wat erin gaat.

Je gebruikt het al sinds [hoofdstuk 6](#). `List<T>` is één klasse die werkt voor `List<string>`, `List<int>` en `List<Leerling>`. De `T` is de open plaats. In dit hoofdstuk maak je zelf zo'n type.

Hoe het eruitziet

```
public class Doos<T>
{
    private T _inhoud;

    public Doos(T inhoud)
    {
        _inhoud = inhoud;
    }

    public T Open()
    {
        return _inhoud;
    }
}
```

Overal waar `T` staat, komt straks het type dat je invult:

```
Doos<string> brief = new Doos<string>("Proficiat!");
Doos<int> getal = new Doos<int>(42);

string tekst = brief.Open();           // Proficiat!
int n = getal.Open();                  // 42 – een echt getal, je kan ermee rekenen

// new Doos<int>("42") compileert niet: dat is een tekst, geen getal
```

Ook een losse methode kan een open plaats hebben:

```
T Eerste<T>(List<T> lijst)
{
    return lijst[0];
}
```

```

}

string s = Eerste(new List<string> { "kiwi", "peer" }); // kiwi
int g = Eerste(new List<int> { 7, 3 }); // 7

```

C# leidt zelf af wat `T` is uit wat je meegeeft. Je hoeft `Eerste<string>(...)` niet uit te schrijven, al mag het.

De open plaats beperken: where

Zonder meer weet C# niets over `T`. Het kan een getal zijn, een tekst of een leerling, dus je mag er niets mee doen wat niet voor *alle* types werkt. Met `where` beperk je wat er ingevuld mag worden — en daar krijg je iets voor terug:

```

public class Garage<T> where T : Voertuig
{
    private List<T> _voertuigen = new List<T>();

    public void Zet(T voertuig)
    {
        _voertuigen.Add(voertuig);
    }

    public int TotaalWielen()
    {
        return _voertuigen.Sum(v => v.Wielen); // mag: elke T is een voertuig
    }
}

Garage<Fiets> stalling = new Garage<Fiets>(); // mag: een fiets is een voertuig

```

`Voertuig` en `Fiets` komen uit [hoofdstuk 20](#). Zonder `where T : Voertuig` zou `v.Wielen` niet compileren, want dan kan `T` ook een tekst zijn.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>class Naam<T></code>	een klasse met een open plaats	<code>class Doos<T></code>	<code>Doos<int></code> , <code>Doos<string></code> , ...
<code>Methode<T>(...)</code>	een methode met een open plaats	<code>T Eerste<T>(List<T> lijst)</code>	<code>Eerste(namen)</code> geeft een tekst
<code>interface INaam<T></code>	een contract met een open plaats	<code>interface IHouder<T></code>	elk type belooft het voor zijn eigen <code>T</code>
<code>class X<T> : IY<T></code>	geef de open plaats door aan het contract	<code>class Doos<T> : IHouder<T></code>	de doos belooft het contract, voor elke <code>T</code>
<code>class X : IY<int></code>	vul de open plaats meteen in	<code>class Spaarpot : IHouder<int></code>	enkel voor getallen
<code><T1, T2></code>	meerdere open plaatsen	<code>class Paar<TLinks, TRechts></code>	zoals <code>Dictionary<TKey, TValue></code>

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>where T : class</code>	T moet een referentietype zijn	<code>class Doos<T> where T : class</code>	<code>Doos<int></code> compileert niet
<code>where T : struct</code>	T moet een waardetype zijn	<code>class Doos<T> where T : struct</code>	<code>Doos<string></code> compileert niet
<code>where T : Basisklasse</code>	T moet die klasse zijn, of ervan erven	<code>where T : Voertuig</code>	binnenin mag je <code>Wielen</code> gebruiken
<code>where T : IContract</code>	T moet dat contract beloven	<code>where T : IGeluid</code>	binnenin mag je <code>MaakGeluid()</code> aanroepen
<code>where T : new()</code>	T moet een constructor zonder parameters hebben	<code>where T : new()</code>	binnenin mag je <code>new T()</code> schrijven
<code>default(T)</code>	de standaardwaarde van T	<code>default(int)</code>	<code>0</code> ; bij een klasse <code>null</code> , bij een <code>bool</code> <code>false</code>
<code>typeof(T)</code>	het type dat ingevuld werd	<code>typeof(T).Name</code> in een <code>Doos<int></code>	<code>"Int32"</code>

Meerdere beperkingen zet je achter elkaar, gescheiden door een komma: `where T : class, new()`.

OPMERKING

`string` is een klasse, en dus een referentietype. Bij `where T : class` mag je `string` dus wél invullen. Enkel waardetypes zoals `int`, `double` en `bool` vallen erbuiten.

Generics die je al kent

Bijna elke verzameling en veel methodes uit dit leerboek hebben een open plaats.

Waar	Voorbeeld	Hoofdstuk
verzamelingen	<code>List<T></code> , <code>Dictionary<TKey, TValue></code> , <code>HashSet<T></code> , <code>Queue<T></code> , <code>Stack<T></code>	6 tot 10
reeksen	<code>IEnumerable<T></code>	12
LINQ	elke methode, zoals <code>Select<TSource, TResult></code> : C# vult de types zelf in	11
omzetten	<code>Enum.Parse<T></code> , <code>JsonSerializer.Deserialize<T></code>	18, 30
asynchroon	<code>Task<T></code>	29
in je API	<code>ActionResult<T></code> , <code>DbSet<T></code> , <code>Set<T>()</code> , <code>Adapt<T>()</code> , <code>QueryAsync<T></code>	26, 31, 32, 33, 34

Met `where T : IComparable<T>` mag je binnenin `CompareTo` gebruiken: zie [hoofdstuk 19](#).

Waarom dit in je cursus zo belangrijk is

In les 07 van Programming Advanced schrijf je één repository die werkt voor al je entiteiten, in plaats van één per entiteit. De open plaats is de entiteit; de cursus noemt ze `TEntity` in plaats van `T`. De beperking `where TEntity : class` staat erbij omdat EF Core enkel met klassen werkt.

Specifieke repositories erven daarna van die generieke, zoals in [hoofdstuk 20](#). Hoe zo'n repository aan de juiste tabel komt, staat in [hoofdstuk 32](#).

Wanneer wel, wanneer niet

Wel — dezelfde code werkt voor meerdere types, en enkel het type verschilt. Zonder generics zou je dezelfde klasse per type kopiëren.

Niet — wanneer de code per type iets anders moet doen. Schrijf je in een generieke methode `if (typeof(T) == typeof(int))`, dan heb je geen open plaats nodig maar aparte methodes, of overerving.

Veelgemaakte fouten

LET OP

- Zonder `where` kan je binnen de klasse niets specifiek met `T` doen. `voertuig.Wielen` compileert pas met `where T : Voertuig`, ook al stop je er in de praktijk enkel voertuigen in.
- `return null;` compileert niet als `T` ook een getal kan zijn. Gebruik `default`, of beperk `T` met `where T : class`.
- `Doos<Fiets>` is geen `Doos<Voertuig>`, ook al is een fiets een voertuig. Hetzelfde geldt voor `List<Fiets>` en `List<Voertuig>`. Zie vraag 4.
- Volgens de afspraak begint de naam van een open plaats met een `T`: `T`, `TSleutel`, `TEntity`.

Voorspel de uitkomst

De vragen gebruiken `Doos<T>` en `Garage<T>` van hierboven, en `Voertuig` en `Fiets` uit hoofdstuk 20.

Vraag 1

```
var a = new Doos<int>(5);
var b = new Doos<string>("5");
Console.WriteLine(a.Open() + 1);
Console.WriteLine(b.Open() + 1);
```

[Toon het antwoord ›](#)

Vraag 2

```
Console.WriteLine(default(int));
Console.WriteLine(default(bool));
Console.WriteLine(default(string) == null);
```

[Toon het antwoord ›](#)

Vraag 3

```
var g = new Garage<string>();
```

[Toon het antwoord ›](#)

Vraag 4

```
Doos<Voertuig> d = new Doos<Fiets>(new Fiets());
```

[Toon het antwoord ›](#)

Vraag 5

```
string Soort<T>(T waarde)
{
    return typeof(T).Name;
}

Console.WriteLine(Soort(42));
Console.WriteLine(Soort("hallo"));
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

6 en daarna 51

Dezelfde klasse, twee verschillende types. De eerste doos geeft een echt getal terug, en $5 + 1$ is 6. De tweede geeft een tekst terug, en een tekst + 1 plakt de 1 erachter.

[Terug naar de vraag](#)

VRAAG 2

0, False en daarna True

Elk type heeft een standaardwaarde: nul voor getallen, false voor een bool en null voor referentietypes. Het is dezelfde waarde die FirstOrDefault uit hoofdstuk 11 teruggeeft wanneer het niets vindt.

[Terug naar de vraag](#)

VRAAG 3

Compileert niet.

Garage<T> eist met where T : Voertuig dat T een voertuig is. Een tekst is dat niet. De fout valt bij het bouwen, niet pas wanneer het programma draait.

[Terug naar de vraag](#)

VRAAG 4

Compileert niet.

Voor C# zijn Doos<Fiets> en Doos<Voertuig> twee aparte types, zonder verband. Dat een fiets een voertuig is, maakt een doos met een fiets nog geen doos met een voertuig. Wat wel mag: een Doos<Voertuig> maken en daar een fiets in stoppen, met new Doos<Voertuig>(new Fiets()).

[Terug naar de vraag](#)

VRAAG 5

Int32 en daarna String

C# leidt T af uit wat je meegeeft: eerst een int, daarna een string. Dat de namen er anders uitzien, komt doordat int en string korte namen zijn voor Int32 en String. Het zijn dezelfde types.

[Terug naar de vraag](#)

22. Types herkennen

METHODES

De voorbeelden gebruiken `IGeluid`, `Hond` en `Kat` uit hoofdstuk 19.

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>typeof</code>	het type zelf, als je de naam kent	<code>typeof(Kat)</code>	het type
<code>GetType</code>	het type van een bestaand object	<code>dier.GetType()</code>	het type
<code>is</code>	is dit object van dat type	<code>if (dier is Kat)</code>	<code>true</code> of <code>false</code>
<code>is</code> met naam	idem, en geef het meteen die naam	<code>if (dier is Kat k)</code>	<code>k</code> is bruikbaar
<code>as</code>	behandel als dat type, of <code>null</code>	<code>dier as Kat</code>	het object of <code>null</code>
<code>nameof</code>	de naam van iets als tekst	<code>nameof(GetFanById)</code>	<code>"GetFanById"</code>

`nameof` lijkt onnozel tot je het nodig hebt. Het voordeel tegenover `"GetFanById"` tussen aanhalingstekens is dat je code niet meer compileert wanneer je die methode hernoemt. Een typfout in een naam vind je zo bij het bouwen in plaats van bij het testen.

Types herkennen in een verzameling, via iets anders

Komt van LINQ. In de voorbeelden is `dieren` een `List<IGeluid>` met een hond en twee katten.

Methode	Voorbeeld	Resultaat
<code>OfType</code>	<code>dieren.OfType<Kat>()</code>	de twee katten, als <code>Kat</code> ; ook <code>null</code> wordt overgeslagen
<code>Cast</code>	<code>dieren.Cast<Kat>().ToList()</code>	InvalidCastException : de hond is geen kat
Where met <code>is</code>	<code>dieren.Where(d => d is Kat)</code>	de twee katten, maar nog als <code>IGeluid</code>
Count met <code>is</code>	<code>dieren.Count(d => d is Kat)</code>	2
GroupBy op het type	<code>dieren.GroupBy(d => d.GetType().Name)</code>	Hond: 1, Kat: 2

LET OP

`Cast` controleert pas als je de items echt gebruikt. `dieren.Cast<Kat>().Count()` geeft 3, zonder fout: tellen kan zonder om te zetten. `ToList()` of een `foreach` crasht wel. Wil je enkel de katten, neem dan `OfType`.

Voorspel de uitkomst

Vraag 1

```
IGeluid dier = new Hond();
Console.WriteLine(dier is Kat);
```

[Toon het antwoord >](#)

Vraag 2

```
IGeluid dier = new Hond();  
Kat? k = dier as Kat;  
Console.WriteLine(k is null);
```

[Toon het antwoord ›](#)

Vraag 3

```
Console.WriteLine(nameof(Kat));  
Console.WriteLine(typeof(Kat).Name);
```

[Toon het antwoord ›](#)

Vraag 4

```
var dieren = new List<IGeluid> { new Hond(), new Kat(), new Kat() };  
Console.WriteLine(dieren.OfType<Kat>().Count());  
var katten = dieren.Cast<Kat>().ToList();
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

False

De variabele zegt enkel "iets dat een geluid maakt". Wat erin zit, is een hond.

[Terug naar de vraag](#)

VRAAG 2

True

as probeert het dier als kat te behandelen. Lukt dat niet, dan krijg je null in plaats van een crash. Daarom controleer je na een as altijd op null.

[Terug naar de vraag](#)

VRAAG 3

Kat en daarna Kat

Twee wegen naar dezelfde tekst. nameof wordt al bij het compileren ingevuld, typeof(...).Name pas als het programma draait. Voor een naam in een melding of in CreatedAtAction is nameof de gewoonte.

[Terug naar de vraag](#)

VRAAG 4

2, en daarna **InvalidCastException** — het programma crasht.

OfType slaat de hond over. Cast probeert elk dier om te zetten naar een kat, ook de hond, en dat lukt niet.

[Terug naar de vraag](#)

23. Uitzonderingen

CONCEPT

Wat het is

Een noodrem. Wanneer een methode iets gevraagd wordt dat ze niet kan — een tekst omzetten die geen getal is, een sleutel opzoeken die er niet is — dan stopt ze meteen en gooit ze een **uitzondering**. Die gaat terug naar wie de methode aanriep, en naar wie die aanriep, tot iemand ze opvangt. Vangt niemand ze op, dan stopt het hele programma.

Je bent ze in dit leerboek al vaak tegengekomen, in het rood. Dit hoofdstuk gaat over wat je ermee doet.

Hoe het eruitziet

```
try
{
    int leeftijd = int.Parse("twaalf");
    Console.WriteLine($"Leeftijd: {leeftijd}");
}
catch (FormatException)
{
    Console.WriteLine("Dat is geen getal.");
}
finally
{
    Console.WriteLine("Klaar.");
}
```

`int.Parse` gooit een `FormatException`. De rest van het `try`-blok wordt overgeslagen, het `catch`-blok vangt de uitzondering op, en `finally` draait altijd — fout of geen fout. Op het scherm komt `Dat is geen getal.` en daarna `Klaar.`

Je kan ook zelf een uitzondering gooien, wanneer je methode iets gevraagd wordt dat ze niet kan of mag doen:

```
void ZetThermostaat(int graden)
{
    if (graden < 5 || graden > 30)
    {
        throw new ArgumentException("Kies tussen 5 en 30 graden.");
    }

    Console.WriteLine($"Thermostaat op {graden} graden.");
}

try
{
    ZetThermostaat(40);
}
```

```
catch (ArgumentException ex)
{
    Console.WriteLine(ex.Message);    // Kies tussen 5 en 30 graden.
}
```

`throw` werkt zoals `return`: de methode stopt daar. Het verschil is dat er geen waarde teruggaat maar een uitzondering, en dat wie de methode aanriep ze moet opvangen — of zelf ook stopt.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>try</code>	probeer dit blok	<code>try { ... }</code>	loopt tot de eerste uitzondering
<code>catch (Soort)</code>	vang deze soort op	<code>catch (FormatException)</code>	het programma loopt verder na het blok
<code>catch (Soort ex)</code>	vang op en geef ze een naam	<code>catch (FormatException ex)</code>	<code>ex</code> is bruikbaar in het blok
<code>catch (Exception)</code>	vang elke soort op	<code>catch (Exception ex)</code>	vangt alles, ook wat je niet verwachtte
<code>finally</code>	draait altijd, fout of geen fout	<code>finally { ... }</code>	ook na een <code>return</code> in het <code>try</code> -blok
<code>throw new</code>	gooi zelf een uitzondering	<code>throw new ArgumentException("Te warm.")</code>	de methode stopt hier
<code>throw;</code>	gooi dezelfde uitzondering verder, binnen een <code>catch</code>	<code>catch (Exception) { ...; throw; }</code>	wie hoger staat, krijgt ze alsnog
<code>Message</code>	de uitleg bij de uitzondering	<code>ex.Message</code>	"Te warm."
<code>GetType</code>	welke soort het is	<code>ex.GetType().Name</code>	"FormatException"

De uitzonderingen die je al kent

Uitzondering	Wanneer	Hoofdstuk
FormatException	<code>int.Parse</code> , <code>Convert.ToInt32</code> of <code>DateTime.Parse</code> op een tekst die niet past	2, 3
ArgumentNullException	<code>int.Parse(null)</code> , of <code>null</code> als sleutel in een <code>Dictionary</code>	2, 7
DivideByZeroException	een <code>int</code> delen door 0; bij een <code>double</code> krijg je ∞ of <code>NaN</code> , zonder fout	2
OverflowException	een berekening in <code>checked</code> , of <code>Sum</code> in LINQ die boven <code>int.MaxValue</code> gaat	2, 11
IndexOutOfRangeException	een positie buiten een array, of buiten een tekst: <code>"abc"[5]</code>	1, 5
ArgumentOutOfRangeException	een positie buiten een <code>List</code> , <code>ElementAt</code> buiten de reeks, of <code>Substring</code> buiten de tekst	1, 6, 11
ArgumentException	<code>Add</code> of <code>ToDictionary</code> met een sleutel die al bestaat, <code>Enum.Parse</code> met een onbekende naam, <code>Max()</code> op objecten die niet vergelijkbaar zijn	7, 11, 13, 18
KeyNotFoundException	<code>[sleutel]</code> die niet bestaat	7

Uitzondering	Wanneer	Hoofdstuk
InvalidOperationException	een verzameling veranderen in een <code>foreach</code> , <code>Dequeue</code> , <code>Pop</code> of <code>Peek</code> op iets leegs, <code>First</code> zonder resultaat, <code>Single</code> met twee, <code>Max</code> of <code>Average</code> op een lege lijst getallen, <code>Sort()</code> op objecten die niet vergelijkbaar zijn, <code>.Value</code> op een lege <code>int?</code> , een vraag die EF Core niet kan vertalen	4, 6, 9, 10, 11, 13, 17, 32
InvalidCastException	<code>(int)</code> op iets dat geen getal is, of <code>Cast<T></code> met een item van een ander type	22
NullReferenceException	iets opvragen van <code>null</code>	17
FileNotFoundException	<code>File.ReadAllText</code> op een bestand dat niet bestaat	30
JsonException	<code>JsonSerializer.Deserialize</code> op tekst die geen geldige JSON is, of met een waarde van het verkeerde type; een kring in je objecten	30, 32
NotImplementedException	een methode die nog niet uitgewerkt is	les 05 van de cursus

Ontbreekt niet alleen het bestand maar ook de map waarin het moet staan, dan krijg je een `DirectoryNotFoundException` in plaats van een `FileNotFoundException`.

Voorkomen tegenover opvangen

Veel uitzonderingen kan je voorkomen door eerst te kijken. Dat is bijna altijd beter: het zegt duidelijker wat je bedoelt, en een `catch` die te veel opvangt, verbergt fouten die je had moeten zien.

In plaats van opvangen	Kijk vooraf met	Hoofdstuk
<code>FormatException</code> van <code>Parse</code>	<code>TryParse</code>	2
<code>KeyNotFoundException</code>	<code>TryGetValue</code> of <code>ContainsKey</code>	7
<code>InvalidOperationException</code> van <code>First</code>	<code>FirstOrDefault</code> en een controle op <code>null</code>	11
<code>NullReferenceException</code>	<code>?.of is null</code>	17
<code>ArgumentException</code> van <code>Add</code> in een <code>Dictionary</code>	<code>TryAdd</code> of <code>ContainsKey</code>	7
<code>InvalidOperationException</code> van <code>Max</code> op een lege lijst	<code>Any()</code> vooraf, of <code>MaxBy</code> op objecten	11
<code>InvalidCastException</code>	<code>is</code> , <code>as</code> of <code>OfType</code>	22
<code>DivideByZeroException</code>	kijk of de deler 0 is	2

Opvangen blijft nodig bij alles wat van buiten je programma komt: een bestand, een netwerk, een database. Of een bestand er nog staat op het moment dat je het leest, en of de inhoud geldige JSON is, kan je vooraf niet zeker weten.

De volgorde van je catch-blokken

Alle uitzonderingen erven van `Exception` — overerving uit hoofdstuk 20. Daarom vangt `catch (Exception)` elke soort op. Zet je meerdere `catch`-blokken onder elkaar, dan loopt C# ze van boven naar onder af en neemt het het eerste dat past. De specifieke soorten komen dus eerst, de algemene als laatste:

```

try
{
    int aantal = int.Parse(invoer);
    Console.WriteLine(100 / aantal);
}
catch (FormatException)
{
    Console.WriteLine("Geef een getal.");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Nul mag niet.");
}
catch (Exception ex)
{
    Console.WriteLine($"Onverwacht: {ex.Message}");
}

```

In je API

Een uitzondering die in een controller niemand opvangt, legt niet de hele server plat. ASP.NET Core vangt ze op het laatste moment op en stuurt **500 Internal Server Error** terug — dat zag je in vraag 3 van [hoofdstuk 26](#). Voor de client betekent 500: *de server heeft een fout gemaakt*. Lag de fout eigenlijk bij de client — een Id dat niet bestaat, een ongeldige waarde — dan hoort daar een 404 of een 400 bij.

Wanneer wel, wanneer niet

Wel — `try` en `catch` bij alles wat van buiten je programma komt en wat je vooraf niet volledig kan nakijken: bestanden, netwerk, een database, JSON van elders. En `throw` wanneer je methode iets gevraagd wordt dat niet kan, en ze zelf niet weet wat ze dan moet doen.

Niet — als vervanger van een controle die je vooraf kan doen. En nooit om een fout stil te laten verdwijnen.

Veelgemaakte fouten

LET OP

- Een lege `catch { }` slikt de fout in. Het programma loopt verder alsof er niets gebeurd is, met verkeerde of ontbrekende gegevens, en niemand weet waarom.
- `catch (Exception)` vangt ook de fouten die je niet verwachtte, zoals een `NullReferenceException` uit je eigen code. Vang op wat je verwacht en waar je iets zinnigs mee kan doen.
- Een variabele die je in het `try`-blok maakt, bestaat niet meer na het blok. Heb je hem nadien nodig, maak hem dan vóór de `try`.
- Code die in hetzelfde blok na een `throw` staat, wordt nooit uitgevoerd.

Voorspel de uitkomst

Vraag 1

```
try
{
    Console.WriteLine("A");
    int.Parse("x");
    Console.WriteLine("B");
}
catch (FormatException)
{
    Console.WriteLine("C");
}
finally
{
    Console.WriteLine("D");
}
Console.WriteLine("E");
```

[Toon het antwoord ›](#)

Vraag 2

```
try
{
    var d = new Dictionary<string, int>();
    Console.WriteLine(d["x"]);
}
catch (FormatException)
{
    Console.WriteLine("opgevangen");
}
```

[Toon het antwoord ›](#)

Vraag 3

```
try
{
    int.Parse("x");
}
catch (Exception)
{
    Console.WriteLine("algemeen");
}
catch (FormatException)
{
    Console.WriteLine("formaat");
}
```

[Toon het antwoord ›](#)

Vraag 4

```
try
{
    throw new InvalidOperationException("Rij is leeg");
}
catch (Exception ex)
{
    Console.WriteLine(ex.GetType().Name);
    Console.WriteLine(ex.Message);
}
```

[Toon het antwoord ›](#)

Vraag 5

```
string Test()
{
    try
    {
        return "uit try";
    }
    finally
    {
        Console.WriteLine("finally");
    }
}

Console.WriteLine(Test());
```

[Toon het antwoord ›](#)

Vraag 6

```
void Binnen()
{
    throw new ArgumentException("mis");
}

void Midden()
{
    Binnen();
    Console.WriteLine("Midden klaar");
}

try
{
    Midden();
}
catch (ArgumentException ex)
{
    Console.WriteLine($"Opgevangen: {ex.Message}");
}
```

[Toon het antwoord ›](#)

Vraag 7

```
double a = 10, b = 0;  
int c = 10, d = 0;  
Console.WriteLine(a / b);  
Console.WriteLine(c / d);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

A, C, D en daarna E

A komt nog op het scherm. Daarna gooit `int.Parse` een uitzondering, en de rest van het `try`-blok — de B — wordt overgeslagen. Het `catch`-blok toont C, `finally` toont D, en omdat de uitzondering opgevangen is, loopt het programma gewoon verder met E.

[Terug naar de vraag](#)

VRAAG 2

KeyNotFoundException — het programma crasht.

De sleutel "x" bestaat niet, en dat geeft een `KeyNotFoundException`. Het `catch`-blok vangt enkel een `FormatException` op. Een andere soort gaat er gewoon langs.

[Terug naar de vraag](#)

VRAAG 3

Compileert niet.

`FormatException` erft van `Exception`. Het eerste blok vangt dus al alles op, en het tweede kan nooit aan de beurt komen. C# weigert dat. Draai de volgorde om: de specifieke soort eerst, de algemene als laatste.

[Terug naar de vraag](#)

VRAAG 4

`InvalidOperationException` en daarna `Rij is leeg`

`catch (Exception ex)` vangt elke soort op, maar `ex` blijft wat het echt is. Net als bij de fiets in hoofdstuk 20 geeft `GetType()` het echte type, niet dat van de variabele.

[Terug naar de vraag](#)

VRAAG 5

`finally` en daarna `uit try`

De `return` legt het antwoord klaar, maar vóór de methode echt stopt, draait `finally` nog. Pas daarna komt het antwoord bij `Console.WriteLine` aan. Dat is wat *altijd* betekent.

[Terug naar de vraag](#)

VRAAG 6

Opgevangen: `mis`

Binnen `gooit`, en `Midden` vangt niets op. De uitzondering gaat dus meteen verder naar boven, en `Midden` klaar komt nooit op het scherm. Pas het `catch`-blok helemaal bovenaan vangt ze op.

[Terug naar de vraag](#)

VRAAG 7

∞ , en daarna **DivideByZeroException** — het programma crasht.

Een `double` kent een waarde voor oneindig, dus delen door 0 lukt. Een `int` kent die niet, en crasht.

[Terug naar de vraag](#)

24. static, const en readonly

CONCEPT

Wat het is

Wat bij de klasse hoort, en wat bij één object. Denk aan een school. Elk klaslokaal heeft zijn eigen bord: wat erop staat, verschilt per lokaal. Maar er is maar één schoolbel, en als die gaat, hoort iedereen hem.

Een gewoon veld is het bord: elk object heeft er zijn eigen exemplaar van. Een `static` veld is de bel: er is er één, en die hoort bij de klasse zelf.

Daarnaast zijn er twee manieren om te zeggen dat iets niet meer mag veranderen: `const` en `readonly`.

Hoe het eruitziet

```
public class Teller
{
    public static int Totaal = 0;    // één voor alle tellers samen
    public int Eigen = 0;           // één per teller

    public void Tik()
    {
        Totaal++;
        Eigen++;
    }
}

var a = new Teller();
var b = new Teller();

a.Tik();
a.Tik();
b.Tik();

Console.WriteLine(Teller.Totaal); // 3 – via de klasse, niet via a of b
Console.WriteLine(a.Eigen);       // 2
Console.WriteLine(b.Eigen);       // 1
```

Je kent `static` al zonder het te weten. `Math.Max`, `int.Parse` en `Console.WriteLine` roep je op via de naam van de klasse, zonder eerst een object te maken. Dat zijn `static` methodes.

En zo zien `const` en `readonly` eruit:

```
public class Bestelling
{
    public const int MaxAantal = 10; // ligt vast bij het compileren

    private readonly DateTime _aangemaakt; // ligt vast na de constructor
```

```

public Bestelling()
{
    _aangemaakt = DateTime.Now;           // hier mag het nog
}

public void Wijzig()
{
    // _aangemaakt = DateTime.Now;        compileert niet
}
}

```

Waarom dit in je cursus zo belangrijk is

Les 02. De lijst in de controller is daar `private static List<Laptop>`. ASP.NET Core maakt voor elk verzoek een nieuw controller-object. Zonder `static` hoort de lijst bij dat ene object en begint elk verzoek met een verse lijst: wat je met een POST toevoegde, is bij de volgende GET verdwenen. Met `static` hoort de lijst bij de klasse, en blijft ze bestaan zolang de applicatie draait.

Vanaf les 03. Een repository of logger die je via de constructor krijgt, bewaar je in een `private readonly` veld. Zo kan niemand dat veld later in de klasse nog vervangen. Hoe dat precies werkt, staat in [hoofdstuk 27](#).

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
static veld	één waarde voor de hele klasse	<code>public static int Totaal;</code>	alle objecten delen dezelfde
static methode	oproepen zonder object	<code>Teller.Reset()</code>	zoals <code>Math.Max</code>
static class	een klasse met enkel static leden	<code>public static class Omrekenen</code>	<code>new Omrekenen()</code> compileert niet
Klasse.Lid	een <code>static</code> lid gebruik je via de klassenaam	<code>Teller.Totaal</code>	3
const	een vaste waarde, al bekend bij het compileren	<code>const int MaxAantal = 10;</code>	kan nooit veranderen
readonly	invullen bij de declaratie of in de constructor, daarna nooit meer	<code>private readonly ILogger _logger;</code>	toewijzen in een andere methode compileert niet
static readonly	één vaste waarde voor de hele klasse, die geen <code>const</code> kan zijn	<code>static readonly DateTime Start = DateTime.Now;</code>	wordt één keer ingevuld

Wat het ook kan, via iets anders

static, const en readonly die je al kent

Klasse	Voorbeelden	Soort	Hoofdstuk
<code>string</code>	<code>string.Join</code> , <code>string.IsNullOrEmpty</code> , <code>string.Empty</code>	static methodes; <code>Empty</code> is static readonly	1

Klasse	Voorbeelden	Soort	Hoofdstuk
int, double	int.Parse, int.TryParse, int.MaxValue	static methodes; MaxValue is een const	2
Math, Convert	Math.Round, Math.PI, Convert.ToInt32	static class; PI is een const	2
DateTime	DateTime.Now, DateTime.Parse	static	3
Array	Array.Sort, Array.IndexOf	static methodes	5
Enumerable	Enumerable.Range, en elke LINQ-methode	static: getallen.Where(...) is een kortere schrijfwijze voor Enumerable.Where(getallen, ...)	11
Enum	Enum.Parse, Enum.GetValues	static methodes	18
Task	Task.Delay	static methode	29
File, Path, JsonSerializer	File.ReadAllText, Path.Combine, JsonSerializer.Serialize	static class	30

Een lijst echt alleen-lezen maken

`readonly` beschermt het veld, niet wat erin zit. Wil je dat niemand nog iets toevoegt, geef dan een alleen-lezen kijk op de lijst:

Je schrijft	Wat het doet
<code>lijst.AsReadOnly()</code>	een <code>ReadOnlyCollection</code> zonder <code>Add</code> en <code>Remove</code> (hoofdstuk 6)
een eigenschap van het type <code>IReadOnlyList<T></code>	wie de eigenschap gebruikt, ziet enkel lezende methodes (hoofdstuk 12)

Het blijft een kijk op dezelfde lijst: voeg je iets toe aan de originele lijst, dan zie je het ook in de alleen-lezen versie.

Wanneer wel, wanneer niet

Wel — `static` voor wat echt bij de klasse hoort: een teller over alle objecten, een hulpmethode die geen gegevens van een object nodig heeft. `const` voor een waarde die nooit verandert, zoals een maximum. `readonly` voor een veld dat je één keer invult en daarna met rust laat, zoals alles wat je via de constructor krijgt.

Niet — `static` voor gegevens die bij één object horen. De `static` lijst uit les 02 is een tijdelijke oplossing: zodra er een database is, verdwijnt ze.

Veelgemaakte fouten

LET OP

- Een `static` methode kan niet aan de gewone velden van de klasse. Ze hoort bij de klasse, niet bij één object, dus ze weet niet wélk object je bedoelt.
- `readonly` beschermt het veld, niet wat erin zit. Een `readonly` lijst kan je niet vervangen door een andere lijst, maar je kan er wel nog items aan toevoegen.
- `const` werkt enkel met waarden die al vastliggen bij het compileren: getallen, tekst, `true` en `false`. `const List<int>` of `const DateTime` compileert niet. Gebruik dan `static readonly`.
- Een `static` lid roep je op via de klassenaam. `a.Totaal` compileert niet, `Teller.Totaal` wel.

Voorspel de uitkomst

Vraag 1

Deze vraag gebruikt de klasse `Teller` van hierboven.

```
var a = new Teller();
var b = new Teller();
var c = new Teller();

a.Tik();
b.Tik();
b.Tik();

Console.WriteLine(Teller.Totaal);
Console.WriteLine(c.Eigen);
```

[Toon het antwoord ›](#)

Vraag 2

```
public class Mand
{
    private List<string> _items = new List<string>();
    private static List<string> _gedeeld = new List<string>();

    public void Voeg(string item)
    {
        _items.Add(item);
        _gedeeld.Add(item);
    }

    public string Tel()
    {
        return _items.Count + " en " + _gedeeld.Count;
    }
}

new Mand().Voeg("appel");
```

```
new Mand().Voeg("peer");  
Console.WriteLine(new Mand().Tel());
```

[Toon het antwoord ›](#)

Vraag 3

```
public class Klas  
{  
    private readonly List<string> _namen = new List<string>();  
  
    public void Voeg(string naam)  
    {  
        _namen.Add(naam);  
    }  
  
    public int Aantal()  
    {  
        return _namen.Count;  
    }  
}  
  
var k = new Klas();  
k.Voeg("Anna");  
k.Voeg("Bram");  
Console.WriteLine(k.Aantal());
```

[Toon het antwoord ›](#)

Vraag 4

```
public class Rekenhulp  
{  
    private int _laatste = 0;  
  
    public static int Dubbel(int x)  
    {  
        _laatste = x * 2;  
        return _laatste;  
    }  
}
```

[Toon het antwoord ›](#)

Vraag 5

```
var basis = new List<int> { 1, 2, 3 };  
var alleenLezen = basis.AsReadOnly();  
basis.Add(4);  
Console.WriteLine(alleenLezen.Count);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

3 en daarna 0

Er werd drie keer getikt, en het totaal is van iedereen samen. c heeft zelf nooit getikt, dus zijn eigen teller staat nog op nul.

[Terug naar de vraag](#)

VRAAG 2

0 en 2

Elke `new Mand()` is een nieuwe mand met een lege eigen lijst. De gedeelde lijst hoort bij de klasse en onthoudt alles. Dit is precies wat er in een controller gebeurt: elk verzoek krijgt een nieuw object, en enkel wat `static` is, blijft.

[Terug naar de vraag](#)

VRAAG 3

2

`readonly` zegt enkel dat `_namen` altijd naar dezelfde lijst wijst. Wat er in die lijst zit, mag gewoon veranderen. `_namen = new List<string>();` in `Voeg` had niet gecompileerd.

[Terug naar de vraag](#)

VRAAG 4

Compileert niet.

`Dubbel` is `static` en hoort bij de klasse. `_laatste` is een gewoon veld en hoort bij één object. Als je `Rekenhulp.Dubbel(5)` oproept zonder object, is er geen `_laatste` om in te schrijven.

[Terug naar de vraag](#)

VRAAG 5

4

`AsReadOnly` maakt geen kopie, maar een kijk op dezelfde lijst. Via `alleenLezen` kan je niets toevoegen, via `basis` wel, en dat zie je overal.

[Terug naar de vraag](#)

Deel 5 — In je API

Wat je nodig hebt zodra je van een consoletoepassing naar een API overstapt.

25. Attributen en routes

IN JE API

Wat het is

Een etiket op je code. Een attribuut staat tussen vierkante haken boven een klasse, een methode of een parameter. Het verandert niets aan wat de code doet, maar het vertelt het framework iets erover. `[HttpGet]` boven een methode zegt: *deze methode antwoordt op een GET-verzoek*.

Voor een API is het belangrijkste wat attributen regelen de **route**: welke URL bij welke methode uitkomt.

Hoe het eruitziet

```
[ApiController]
[Route("api/[controller]")]
public class ReceptController : ControllerBase
{
    [HttpGet]
    public ActionResult<List<Recept>> GetAll() { /* ... */ }

    [HttpGet("{id}")]
    public ActionResult<Recept> GetById(int id) { /* ... */ }

    [HttpGet("keuken/{keuken}")]
    public ActionResult<List<Recept>> GetPerKeuken(string keuken) { /* ... */ }

    [HttpPost]
    public ActionResult<Recept> VoegToe(Recept recept) { /* ... */ }

    [HttpPut("{id}")]
    public IActionResult Wijzig(int id, Recept recept) { /* ... */ }

    [HttpDelete("{id}")]
    public IActionResult Verwijder(int id) { /* ... */ }
}
```

De route van de klasse en die van de methode worden aan elkaar geplakt:

Verzoek	Komt uit bij	Met
GET api/recept	GetAll	—
GET api/recept/5	GetById	id is 5
GET api/recept/keuken/italiaans	GetPerKeuken	keuken is "italiaans"
POST api/recept	VoegToe	recept komt uit de JSON in de body
PUT api/recept/5	Wijzig	id uit de route, recept uit de body
DELETE api/recept/5	Verwijder	id is 5

[controller] wordt vervangen door de naam van de klasse zonder het woord *Controller*: van ReceptController blijft Recept over. Hoofdletters maken in een route niet uit: api/Recept en api/recept komen bij dezelfde methode uit. De waarde zelf behoudt wel haar hoofdletters: bij keuken/Italiaans is keuken gelijk aan "Italiaans".

Waar een parameter vandaan komt

Met [ApiController] op de klasse beslist ASP.NET Core zelf waar elke parameter vandaan komt:

1. Staat de naam tussen accolades in de route, dan komt hij uit de route.
2. Is het een object, zoals een Recept, dan komt het uit de body.
3. Al de rest komt uit de querystring: het stuk na het vraagteken, zoals ?zoek=pasta.

Je kan het ook zelf zeggen met [FromRoute], [FromBody] of [FromQuery] vóór de parameter. Met [ApiController] heb je dat zelden nodig.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
[ApiController]	zet het API-gedrag aan: parameters herkennen, automatisch 400 bij ongeldige invoer	boven de klasse	—
[Route("api/[controller]")]	de basisroute van de klasse	op ReceptController	api/recept
[Route("api/keuken")]	een vaste route, los van de klassenaam	op ReceptController	api/keuken
[HttpGet]	antwoordt op GET, op de basisroute	GetAlle()	GET api/recept
[HttpGet("{id}")]	met een routeparameter; de naam tussen accolades moet die van de parameter zijn	GetById(int id)	GET api/recept/5
[HttpGet("{id:int}")]	routeparameter met een beperking: enkel een geheel getal	GetById(int id)	GET api/recept/abc geeft 404
[HttpGet("vast/{x}")]	een vast stuk en een parameter	keuken/{keuken}	GET api/recept/keuken/thais
[HttpGet("{a}/{b}")]	twee parameters in één route	Bereken(int a, int b)	GET api/recept/2/3
[HttpPost]	antwoordt op POST	VoegToe(Recept r)	POST api/recept
[HttpPut("{id}")]	antwoordt op PUT	Wijzig(int id, Recept r)	PUT api/recept/5
[HttpDelete("{id}")]	antwoordt op DELETE	Verwijder(int id)	DELETE api/recept/5

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>[FromBody]</code>	haal deze parameter uit de body	<code>VoegToe([FromBody] Recept r)</code>	de JSON wordt een Recept
<code>[FromQuery]</code>	haal deze parameter uit de querystring	<code>Zoek([FromQuery] string zoek)</code>	<code>api/recept?zoek=pasta</code>
<code>[FromRoute]</code>	haal deze parameter uit de route	<code>GetById([FromRoute] int id)</code>	zoals zonder attribuut

Veelgemaakte fouten

LET OP

- Twee methodes met hetzelfde verb en dezelfde route compileren, maar zodra een verzoek binnenkomt, weet ASP.NET Core niet welke het moet nemen. Dat geeft **500** met een `AmbiguousMatchException`. `{id}` en `{naam}` op dezelfde plaats tellen als dezelfde route.
- De naam tussen accolades moet exact die van de parameter zijn. `[HttpGet("{id}")]` met `GetById(int nummer)` vult `nummer` niet uit de route, en dan blijft het op 0.
- Een route in een methode-attribuut die begint met `/` negeert de route van de klasse. `[HttpGet("/zoek")]` is gewoon `zoek`, zonder `api/recept` ervoor.
- Een waarde uit de URL behoudt haar hoofdletters. Vergelijk je ze met een vaste tekst, denk dan aan `Equals` met `OrdinalIgnoreCase` uit hoofdstuk 1.

Voorspel de uitkomst

Bij een API voorspel je wat de client terugkrijgt. Deze vragen gebruiken de `ReceptController` van hierboven.

Vraag 1

```
GET api/recept/abc
```

[Toon het antwoord ›](#)

Vraag 2

Dezelfde controller, maar nu met `[HttpGet("{id:int}")]` boven `GetById`.

```
GET api/recept/abc
```

[Toon het antwoord ›](#)

Vraag 3

Iemand voegt deze methode toe aan de `ReceptController`:

```
[HttpGet("{naam}")]  
public ActionResult<Recept> GetByNaam(string naam) { /* ... */ }  
  
GET api/recept/5
```

[Toon het antwoord ›](#)

Vraag 4

```
GET api/RECEPT/keuken/Thais
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

400 Bad Request

De route `{id}` past: er staat iets na `api/recept/`. Maar `abc` kan geen `int` worden. `[ApiController]` merkt dat en stuurt zelf een 400 terug, nog voor `GetById` draait.

[Terug naar de vraag](#)

VRAAG 2

404 Not Found

Door de beperking `:int` past de route niet meer bij `abc`. Er is dus geen enkele methode voor dit adres, en dat is een 404. Vergelijk met vraag 1: een kleine beperking, een andere statuscode.

[Terug naar de vraag](#)

VRAAG 3

500 Internal Server Error

`api/recept/{id}` en `api/recept/{naam}` zijn voor de route hetzelfde: een stuk tekst na `api/recept/`. Er passen nu twee methodes, en ASP.NET Core weigert te kiezen. Het compileerde nochtans zonder fout.

[Terug naar de vraag](#)

VRAAG 4

`GetPerKeuken` draait, met `keuken` gelijk aan `"Thais"`.

De hoofdletters in `RECEPT` maken voor de route niets uit. De waarde `Thais` komt wel letterlijk in de parameter, met hoofdletter. Vergelijk je die met `"thais"`, dan geeft `==` vals.

[Terug naar de vraag](#)

26. De helpers van een controller

IN JE API

Deze erf je van `ControllerBase`. Ze maken een antwoord met de juiste statuscode.

Methode	Statuscode	Voorbeeld
<code>Ok</code>	200 OK	<code>Ok(fans)</code>
<code>CreatedAtAction</code>	201 Created	<code>CreatedAtAction(nameof(GetFanById), new { id = fan.Id }, fan)</code>
<code>NoContent</code>	204 No Content	<code>NoContent()</code>
<code>BadRequest</code>	400 Bad Request	<code>BadRequest("De prijs moet groter zijn dan 0.")</code>
<code>NotFound</code>	404 Not Found	<code>NotFound(\$"Geen fan met Id {id}.")</code>
<code>StatusCode</code>	wat je zelf opgeeft	<code>StatusCode(500)</code>

Ze nemen allemaal een tekst of een object mee, en dat is wat de client in de body ziet. Een `NotFound()` zonder tekst is geldig maar zegt niets; een `NotFound()` met een zin erbij scheelt de volgende persoon tien minuten zoeken.

OPMERKING

Geef je een verzameling mee, zoals `Ok(fans)`, dan wordt die JSON zoals in [hoofdstuk 30](#): een `List`, array of `HashSet` wordt een rij tussen `[ ]`, een `Dictionary` een object tussen `{ }`. ASP.NET Core schrijft de namen van eigenschappen met een kleine beginletter.

`ActionResult<T>` **tegenover** `ActionResult`

Een methode die een van deze helpers teruggeeft, heeft een terugkeertype nodig dat zowel een statuscode als gegevens kan dragen. In de cursus kom je er twee tegen: `ActionResult<T>` vanaf les 02, en `ActionResult` vanaf les 06, bij PUT en DELETE.

	<code>ActionResult<T></code>	<code>ActionResult</code>
Zegt wat er terugkomt	ja: een <code>T</code>	nee
<code>return recept;</code>	mag, wordt 200 OK	compileert niet — schrijf <code>return Ok(recept);</code>
<code>return NotFound();</code>	mag	mag
Scalar toont het type van het antwoord	ja	nee
Typisch gebruik	GET en POST, die iets teruggeven	PUT en DELETE, die enkel een statuscode teruggeven

LET OP

`ActionResult<T>` kan een waarde enkel rechtstreeks teruggeven als `T` een klasse is, geen interface. Bij `ActionResult<IEnumerable<Recept>>` compileert `return resultaat;` niet als `resultaat` zelf als `IEnumerable` getypt is. Maak er eerst met `ToList()` een lijst van, of kies meteen `ActionResult<List<Recept>>`.

Voorspel de uitkomst

Bij een API voorspel je geen tekst op het scherm, maar wat de client terugkrijgt: een statuscode en een body.

Vraag 1

Een methode eindigt met:

```
return NotFound("Onbekend");
```

[Toon het antwoord ›](#)

Vraag 2

Na een geslaagde DELETE eindigt de methode met:

```
return NoContent();
```

[Toon het antwoord ›](#)

Vraag 3

Een methode zoekt met `First` en het gevraagde `Id` bestaat niet:

```
var dier = dieren.First(d => d.Id == id);  
return Ok(dier);
```

[Toon het antwoord ›](#)

Vraag 4

Twee methodes, allebei met een recept `r` dat gevonden werd:

```
public ActionResult<Recept> GetEen(int id)  
{  
    // ...  
    return r;  
}  
  
public IActionResult GetAnder(int id)  
{  
    // ...
```

```
    return n;  
}
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

404 Not Found, met in de body de tekst **Onbekend**.

[← Terug naar de vraag](#)

VRAAG 2

204 No Content, met een lege body.

Dat is geen vergetelheid. Er valt na een verwijdering niets meer te tonen, en de statuscode zegt al dat het gelukt is.

[← Terug naar de vraag](#)

VRAAG 3

500 Internal Server Error

`First` crasht zodra er niets gevonden wordt, en die crash wordt nergens opgevangen. De server meldt dan een fout aan zijn kant — terwijl de fout eigenlijk bij de client lag, die een `Id` vroeg dat niet bestaat. Met `FirstOrDefault`, een controle op null en een `NotFound` was dit een nette 404 geworden. Kijk nog eens naar hoofdstuk 11.

[← Terug naar de vraag](#)

VRAAG 4

De eerste geeft **200 OK** met het recept. De tweede **compileert niet**.

`ActionResult<Recept>` weet dat er een recept terug mag komen en maakt er zelf een 200 van.

`ActionResult` kent geen recepten, enkel antwoorden. Daar moet je het recept zelf inpakken met `Ok(r)`.

[← Terug naar de vraag](#)

27. Program.cs en dependency injection

IN JE API

Wat het is

Het startpunt en de schakelkast van je API. `Program.cs` draait één keer, bij het opstarten. Het doet vier dingen, altijd in deze volgorde:

1. een **bouwer** maken;
2. bij die bouwer **registreren** wat je API nodig heeft: de diensten;
3. de app **bouwen**;
4. vastleggen welke weg elk verzoek aflegt, en de app **starten**.

Hoe het eruitziet

```
var builder = WebApplication.CreateBuilder(args);

// 2. Diensten registreren
builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddScoped<IReceptRepository, ReceptRepository>();

// 3. Bouwen
var app = builder.Build();

// 4. De weg van een verzoek, en starten
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Alles wat met `builder.Services` begint, staat vóór `builder.Build()`. Alles wat met `app` begint, staat erna.

Dependency injection: vragen in plaats van zelf maken

Een controller heeft vaak iets anders nodig, zoals een repository. Hij maakt dat niet zelf met `new`. Hij **vraagt** erom in zijn constructor:

```
public class ReceptController : ControllerBase
{
    private readonly IReceptRepository _repository;
```

```

public ReceptController(IReceptRepository repository)
{
    _repository = repository;
}
}

```

Komt er een verzoek binnen, dan ziet ASP.NET Core dat de controller een `IReceptRepository` nodig heeft. Het zoekt in de registraties welke klasse daarbij hoort, maakt er een object van en geeft dat mee. De controller kent enkel het contract uit [hoofdstuk 19](#). Zo kan je later een andere klasse registreren, bijvoorbeeld een die een database gebruikt, zonder één letter in de controller te veranderen.

Het `private readonly` veld uit [hoofdstuk 24](#) bewaart wat je kreeg, zodat geen andere methode het nog kan vervangen.

Hoe lang een dienst leeft

Registratie	Hoe vaak er een nieuw object komt	Typisch voor
<code>AddTransient<I, K>()</code>	elke keer dat iemand erom vraagt	kleine diensten zonder eigen gegevens
<code>AddScoped<I, K>()</code>	één per verzoek	repositories en de databasecontext; de cursus gebruikt dit
<code>AddSingleton<I, K>()</code>	één voor de hele levensduur van de app	iets dat iedereen moet delen

BELANGRIJK

Bij `AddScoped` krijgt elk verzoek een nieuwe repository. Bewaart die repository haar gegevens in een gewone lijst, dan begint elk verzoek met een verse lijst: wat je met een POST toevoegde, is bij de volgende GET weg. Dat is hetzelfde probleem als bij de controller in [hoofdstuk 24](#). Met een database speelt het niet, want die onthoudt de gegevens zelf.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>WebApplication.CreateBuilder</code>	maak de bouwer	<code>var builder = WebApplication.CreateBuilder(args);</code>	de bouwstandaard
<code>AddControllers</code>	zet controllers aan	<code>builder.Services.AddControllers();</code>	zonder controller
<code>AddOpenApi</code>	beschrijf je API voor Scalar	<code>builder.Services.AddOpenApi();</code>	—
<code>AddScoped</code>	registreer: één object per verzoek	<code>AddScoped<IReceptRepository, ReceptRepository>()</code>	wie het vraagt
<code>AddTransient</code>	registreer: elke keer een nieuw object	<code>AddTransient<IKlok, Klok>()</code>	—

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>AddSingleton</code>	registreer: één object voor de hele app	<code>AddSingleton<ITeller, Teller>()</code>	—
<code>GetConnectionString</code>	lees een verbinding uit <code>appsettings.json</code>	<code>builder.Configuration.GetConnectionString("Standaard")</code>	de tek... Conne...
<code>Build</code>	bouw de app; daarna liggen de registraties vast	<code>var app = builder.Build();</code>	de app...
<code>IsDevelopment</code>	draait de app op je eigen computer	<code>if (app.Environment.IsDevelopment())</code>	true... ontwik...
<code>MapOpenApi, MapScalarApiReference</code>	maak de beschrijving en de Scalar-pagina bereikbaar	<code>app.MapScalarApiReference();</code>	de Sca... bereik...
<code>UseHttpsRedirection</code>	stuur http door naar https	<code>app.UseHttpsRedirection();</code>	—
<code>UseAuthorization</code>	controleer wie wat mag	<code>app.UseAuthorization();</code>	belang... beveili...
<code>MapControllers</code>	koppel de routes van je controllers	<code>app.MapControllers();</code>	zonder... route...
<code>Run</code>	start de app en blijf luisteren	<code>app.Run();</code>	wat er... pas als...

OPMERKING

In nieuwere code zie je soms een kortere schrijfwijze, een *primary constructor*: `public class ReceptController(IReceptRepository repository) : ControllerBase`. De parameter staat dan meteen achter de klassenaam, en het veld en de constructor vallen weg. Het werkt op dezelfde manier.

Veelgemaakte fouten

LET OP

- Vraagt een controller iets dat niet geregistreerd is, dan compileert alles. Pas wanneer er een verzoek binnenkomt, krijg je een `InvalidOperationException: Unable to resolve service for type...` De client ziet **500**.
- Een registratie na `builder.Build()` komt te laat. Zet ze er altijd vóór.
- Vergeet je `app.MapControllers()`, dan start de app gewoon, maar geeft elke route 404.
- `new ReceptRepository()` in een controller werkt, maar breekt het hele idee. De controller hangt dan weer vast aan één klasse.

Voorspel de uitkomst

Vraag 1

Een repository bewaart recepten in een gewone lijst, geen `static`. Ze is geregistreerd met `AddScoped`. Je doet een POST die één recept toevoegt, en daarna een GET die alle recepten opvraagt. De lijst begon leeg.

Hoeveel recepten geeft de GET terug? En als de repository met `AddSingleton` geregistreerd was?

[Toon het antwoord ›](#)

Vraag 2

In `Program.cs` ontbreekt de regel met `AddScoped<IReceptRepository, ReceptRepository>()`. De `ReceptController` vraagt in zijn constructor een `IReceptRepository`. De app start, en je doet GET `api/recept`.

[Toon het antwoord ›](#)

Vraag 3

```
app.MapControllers();  
  
app.Run();  
  
Console.WriteLine("De API draait.");
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

Met `AddScoped`: 0. Met `AddSingleton`: 1.

`AddScoped` geeft elk verzoek een nieuwe repository, met een nieuwe, lege lijst. Het recept van de POST zat in de repository van dat ene verzoek. Bij `AddSingleton` is er één repository voor de hele app, en die onthoudt het recept.

[◀ Terug naar de vraag](#)

VRAAG 2

500 Internal Server Error

De app start gewoon, want niemand heeft de controller al nodig. Bij het eerste verzoek wil ASP.NET Core er een maken, vindt geen klasse voor `IREceptRepository`, en geeft een `InvalidOperationException`.

[◀ Terug naar de vraag](#)

VRAAG 3

Zolang de API draait, verschijnt er niets.

`app.Run()` keert pas terug wanneer de app stopt. Wil je iets melden bij het opstarten, dan moet het vóór `app.Run()` staan.

[◀ Terug naar de vraag](#)

28. Logging

IN JE API

Wat het is

Een logboek dat je API zelf bijhoudt. Terwijl de API draait, schrijft hij in het terminalvenster wat er gebeurt: welk verzoek er binnenkwam, wat er niet gevonden werd, wat er misliep. Zo zie je achteraf wat er gebeurd is, ook als niemand op dat moment keek.

Elke melding krijgt een **niveau**, van een detail dat enkel tijdens het ontwikkelen nuttig is, tot een fout die meteen aandacht vraagt.

Hoe het eruitziet

```
public class ReceptController : ControllerBase
{
    private readonly ILogger<ReceptController> _logger;

    public ReceptController(ILogger<ReceptController> logger)
    {
        _logger = logger;
    }

    [HttpGet("{id}")]
    public ActionResult<Recept> GetById(int id)
    {
        _logger.LogInformation("Recept {Id} opgevraagd", id);

        // ...

        _logger.LogWarning("Recept {Id} niet gevonden", id);
        return NotFound();
    }
}
```

De logger vraag je in de constructor, net als een repository in [hoofdstuk 27](#). Registreren hoeft niet: ASP.NET Core heeft hem al klaarstaan. Tussen de punthaken staat de klasse die logt. Die naam komt mee in het logboek, zodat je ziet waar een melding vandaan komt.

De niveaus

Methode	Wanneer	Standaard zichtbaar
LogTrace	de kleinste details	nee
LogDebug	details die enkel tijdens het ontwikkelen helpen	nee
LogInformation	de normale werking: een verzoek, een geslaagde actie	ja
LogWarning	iets onverwachts, maar de app werkt verder: een Id dat niet bestaat	ja
LogError	iets is misgelopen: een actie lukte niet	ja

Methode	Wanneer	Standaard zichtbaar
<code>LogCritical</code>	de app zelf zit in de problemen	ja

Welke niveaus je ziet, staat in `appsettings.json`, onder `Logging` en `LogLevel`. Een nieuw project zet `Default` op `Information`: alles vanaf dat niveau verschijnt, `Debug` en `Trace` niet.

Twee schrijfwijzen

In les 03 schrijft de cursus de melding met een `$` ervoor, zoals in hoofdstuk 1: de waarden staan meteen in de tekst. Dat werkt.

Microsoft raadt een andere vorm aan, met een **sjabloon**: de tekst bevat plaatshouders tussen accolades, en de waarden komen erachter, gescheiden door komma's. Bijvoorbeeld `_logger.LogInformation("Recept {Id} opgevraagd", id);`. In het terminalvenster zie je hetzelfde. Het verschil is dat de logger de waarden apart bewaart, zodat een loggingprogramma later kan zoeken op *alle meldingen over recept 5*. Visual Studio stelt daarom soms voor om de eerste vorm om te zetten.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>ILogger<T></code>	een logger voor klasse <code>T</code> , via de constructor	<code>ILogger<ReceptController> logger</code>	de logger staat klaar
<code>LogInformation</code>	meld de normale werking	<code>_logger.LogInformation("Alle recepten opgevraagd");</code>	<code>info: ... Alle recepten opgevraagd</code>
<code>LogWarning</code>	meld iets onverwachts	<code>_logger.LogWarning("Recept {Id} niet gevonden", id);</code>	<code>warn: ... Recept 5 niet gevonden</code>
<code>LogError</code>	meld een fout	<code>_logger.LogError("Verwijderen van {Id} mislukt", id);</code>	<code>fail: ...</code>
<code>LogError</code> met uitzondering	meld een fout en geef de uitzondering mee	<code>_logger.LogError(ex, "Bestand niet gelezen");</code>	de melding, met de details van <code>ex</code> eronder
<code>{Naam}</code>	plaatshouder in een sjabloon	<code>"{Aantal} recepten", aantal</code>	<code>12 recepten</code>

Veelgemaakte fouten

LET OP

- In een sjabloon telt de **volgorde** van de waarden, niet de naam tussen de accolades. De eerste waarde komt in de eerste plaatshouder, wat er ook tussen de accolades staat.
- Een `LogDebug` die je nooit ziet, is meestal geen fout in je code. Het niveau staat in `appsettings.json` hoger ingesteld.
- Zet nooit wachtwoorden of andere geheime gegevens in een logboek. Wie het logboek kan lezen, kan ze dan ook lezen.

Voorspel de uitkomst

Vraag 1

```
_logger.LogInformation("{Peren} peren en {Appels} appels", 3, 8);
```

[Toon het antwoord ›](#)

Vraag 2

In `appsettings.json` staat `"Default": "Information"`. Een methode bevat:

```
_logger.LogDebug("Begin van de methode");  
_logger.LogWarning("Geen recepten gevonden");
```

Wat verschijnt er in het terminalvenster?

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

3 peren en 8 appels

De eerste waarde gaat naar de eerste plaatshouder, de tweede naar de tweede. Dat de namen `Peren` en `Appels` heten, speelt geen rol bij welke waarde waar komt.

[← Terug naar de vraag](#)

VRAAG 2

Enkel `Geen recepten gevonden`, als waarschuwing.

`Debug` ligt lager dan `Information`, dus die melding valt weg. `Warning` ligt hoger en komt erdoor.

[← Terug naar de vraag](#)

29. Asynchroon werken

IN JE API

Vorm	Wat het doet
<code>async</code>	markeert een methode die mag wachten
<code>await</code>	wacht op het resultaat zonder de server te blokkeren
<code>Task</code>	een belofte dat er ooit iets klaar is
<code>Task<T></code>	een belofte dat er ooit een <code>T</code> klaar is

De asynchrone tegenhangers die je in de latere lessen tegenkomt dragen dezelfde naam met Async erachter: `ToListAsync`, `FirstOrDefaultAsync`, `FindAsync`, `SaveChangesAsync`, `AnyAsync`, `CountAsync`. Ze doen hetzelfde als hun gewone versie; het verschil zit in wat de server intussen mag doen.

OPMERKING

Deze komen pas aan bod in les 05 van Programming Advanced. Ze staan hier zodat je ze herkent wanneer je ze in een voorbeeld ziet staan, niet om ze nu al te gebruiken. Hoe je ze met een database gebruikt, staat in hoofdstuk 32.

Hoe het eruitziet

```
async Task<int> TellLangzaamAsync()
{
    await Task.Delay(1000);    // wacht een seconde, zonder de thread vast te houden
    return 42;
}

int n = await TellLangzaamAsync();
Console.WriteLine(n);        // 42, na een seconde
```

In een controller ziet dat er zo uit:

```
[HttpGet]
public async Task<ActionResult<List<Recept>>> GetAll()
{
    List<Recept> recepten = await _repository.GetAllAsync();
    return recepten;
}
```

Drie dingen veranderen tegenover de gewone versie: `async` voor het terugkeertype, het terugkeertype zelf in een `Task<...>`, en `await` voor elke aanroep die tijd kost.

Async is besmettelijk

Les 05 noemt het het domino-effect. Gebruik je ergens `await`, dan moet die methode `async` worden en een `Task` teruggeven. Wie die methode oproept, moet dan zelf ook `await` gebruiken,

en dus ook `async` worden. Zo schuift het naar boven: van de repository, via de interface, tot in de controller.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>async Task</code>	een asynchrone methode die niets teruggeeft	<code>async Task BewaarAsync()</code>	—
<code>async Task<T></code>	een asynchrone methode die een <code>T</code> teruggeeft	<code>async Task<int> TelAsync()</code>	<code>await TelAsync()</code> geeft een <code>int</code>
<code>await</code>	wacht op een <code>Task</code> en pak het resultaat uit	<code>int n = await TelAsync();</code>	<code>n</code> is een getal
<code>Task.Delay</code>	wacht een aantal milliseconden, zonder te blokkeren	<code>await Task.Delay(500);</code>	een halve seconde later gaat het verder
<code>Async</code> achter de naam	de afspraak voor asynchrone methodes	<code>GetAllAsync, SaveChangesAsync</code>	je ziet meteen dat er een <code>await</code> bij hoort
<code>.Result, .Wait()</code>	wacht door de thread vast te houden	<code>TelAsync().Result</code>	werkt, maar blokkeert; niet doen
<code>async void</code>	een asynchrone methode zonder <code>Task</code>	<code>async void Bewaar()</code>	fouten erin kan niemand opvangen; niet doen

Wat het ook kan, via iets anders

Asynchrone methodes die je tegenkomt

Methode	Geeft na <code>await</code>	Hoofdstuk
<code>ToListAsync, FirstOrDefaultAsync, CountAsync, ...</code>	een <code>List<T></code> , een object of null, een getal	32
<code>SaveChangesAsync</code>	het aantal geschreven rijen	32
<code>QueryAsync<T>, ExecuteAsync</code>	een <code>IEnumerable<T></code> , het aantal geraakte rijen	34
<code>File.ReadAllTextAsync, File.WriteAllTextAsync</code>	de tekst, of niets	30
<code>Task.Delay</code>	niets	hierboven

Pas na `await` heb je wat erin zit. Zonder `await` heb je een `Task<List<Recept>>`, en die heeft geen `Count`, geen `[0]` en geen LINQ. Met `await` heb je een gewone `List<Recept>` met alles van hoofdstuk 6.

Bestaat ook, maar kom je in deze cursus niet tegen

`Task.WhenAll` (wacht op meerdere taken tegelijk), `Task.WhenAny`, `await foreach` over een `IAsyncEnumerable<T>`, `CancellationToken` (een taak afbreken).

Veelgemaakte fouten

LET OP

- Een vergeten `await` compileert vaak gewoon, met een waarschuwing. Je code gaat dan verder voor het werk klaar is.
- `.Result` en `.Wait()` maken van asynchrone code weer gewone, wachtende code. Alle winst is weg, en in sommige toepassingen loopt het programma vast, omdat twee stukken op elkaar wachten (een *deadlock*). Les 05 zegt het kort: hou het bij `async` en `await`.
- `async void` geeft geen `Task` terug. Wie de methode oproept, kan er dus niet op wachten en geen fout opvangen. Gebruik altijd `async Task` of `async Task<T>`.

Voorspel de uitkomst

Stel dat er een methode bestaat met deze kop:

```
Task<int> TelAsync()
```

Vraag 1

```
var x = await TelAsync();
```

Van welk type is `x`?

[Toon het antwoord ›](#)

Vraag 2

```
var x = TelAsync();
```

Van welk type is `x` nu?

[Toon het antwoord ›](#)

Vraag 3

```
async Task SchrijfAsync()
{
    Console.WriteLine("B");
    await Task.Delay(100);
    Console.WriteLine("D");
}

Console.WriteLine("A");
Task t = SchrijfAsync();
Console.WriteLine("C");
await t;
Console.WriteLine("E");
```

[Toon het antwoord ›](#)

Vraag 4

```
async Task<int> GeefAsync()  
{  
    await Task.Delay(10);  
    return 5;  
}  
  
int x = GeefAsync() + 1;
```

[Toon het antwoord ›](#)

Vraag 5

GetAllAsync geeft een Task<List<Recept>>.

```
var recepten = _repository.GetAllAsync();  
Console.WriteLine(recepten.Count);
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

`int`

`await` wacht tot de belofte ingelost is en geeft je wat erin zit: het getal.

[Terug naar de vraag](#)

VRAAG 2

`Task<int>`

Zonder `await` krijg je de belofte zelf, niet het getal. Probeer je er daarna mee te rekenen, dan compileert het niet. Een vergeten `await` is een van de meest voorkomende fouten zodra je asynchroon begint te werken.

[Terug naar de vraag](#)

VRAAG 3

A, B, C, D en daarna E

`SchrijfAsync` begint meteen en schrijft B. Bij de eerste `await` die echt moet wachten, geeft de methode de beurt terug. Daardoor komt C vóór D. Pas bij `await t` wacht het hoofdprogramma tot de methode helemaal klaar is, en dus komt E als laatste.

[Terug naar de vraag](#)

VRAAG 4

Compileert niet.

Zonder `await` is `GeefAsync()` een `Task<int>`, een belofte, en een belofte plus 1 bestaat niet. Met `await` `GeefAsync() + 1` lukt het wel: eerst het getal uitpakken, dan optellen.

[Terug naar de vraag](#)

VRAAG 5

Compileert niet

Zonder `await` is `recepten` een `Task<List<Recept>>`: een belofte, nog geen lijst. Een `Task` heeft geen `Count`. Met `var recepten = await _repository.GetAllAsync();` lukt het wel.

[Terug naar de vraag](#)

30. JSON en bestanden

IN JE API

Methode	Wat het doet
<code>JsonSerializer.Serialize</code>	maak van een object een JSON-tekst
<code>JsonSerializer.Deserialize</code>	maak van een JSON-tekst een object
<code>File.ReadAllText</code>	lees een heel bestand als tekst
<code>File.WriteAllText</code>	schrijf een tekst naar een bestand
<code>File.Exists</code>	bestaat dat bestand
<code>Path.Combine</code>	plak padstukken correct aan elkaar

Ontbreekt het bestand, of staat er geen geldige JSON in, dan krijg je een uitzondering. Hoe je die opvangt, staat in [hoofdstuk 23](#).

LET OP

`JsonSerializer.Deserialize` is standaard hoofdlettergevoelig. Staat er "krakenId" in je bestand en heet je eigenschap `KrakenId`, dan blijft die leeg zonder één foutmelding. Dat gedrag zet je om met `PropertyNameCaseInsensitive`. Dat ASP.NET Core zelf wél soepel is met hoofdletters geldt enkel voor verzoeken die het framework zelf uitleest, niet voor een bestand dat je zelf inleest.

Wat het ook kan, via iets anders

Wat je terugkrijgt

Methode	Geeft	Daarmee werk je verder in
<code>File.ReadAllText</code>	één <code>string</code>	hoofdstuk 1
<code>File.ReadAllLines</code>	een <code>string[]</code> , één regel per vakje	hoofdstuk 5
<code>File.ReadLines</code>	een <code>IEnumerable<string></code> : leest de regels pas terwijl je ze overloopt	hoofdstuk 12
<code>File.WriteAllLines</code>	schrijft een verzameling teksten, één per regel	—
<code>File.AppendAllText</code>	voegt tekst toe aan het einde, in plaats van te vervangen	—
<code>JsonSerializer.Deserialize<List<T>></code>	een <code>List<T></code> , of null als er null staat	hoofdstuk 6
<code>JsonSerializer.Deserialize<Dictionary<string, int>></code>	een <code>Dictionary</code>	hoofdstuk 7

Hoe verzamelingen en types in JSON staan

In C#	In JSON
<code>List</code> , <code>array</code> , <code>HashSet</code> , <code>Queue</code>	een rij: <code>[1, 2]</code>

In C#	In JSON
Stack	een rij, van boven naar onder: <code>[3,2,1]</code>
Dictionary<string, int>	een object: <code>{"a":1,"b":2}</code>
een enum	het getal erachter: <code>"Licht":2</code>
een publiek veld (zonder { get; set; })	niets: enkel eigenschappen komen mee
een anoniem object, een klasse, een record	een object met de eigenschappen

LET OP

- Een Stack die je bewaart en terug inleest, staat omgekeerd: wat bovenaan lag, ligt nu onderaan.
- Een enum wordt een getal. Verander je later de volgorde van de keuzes, dan betekenen oude bestanden iets anders.

Bestaat ook, maar kom je in deze cursus niet tegen

File.ReadAllTextAsync en File.WriteAllTextAsync (asynchroon, zie [hoofdstuk 29](#)),
 JsonSerializer.ConvertToEnum (een enum als naam in plaats van getal), JsonSerializer.Defaults.Web
 (kleine beginletters, zoals ASP.NET Core), Directory (mappen).

Voorspel de uitkomst

Vraag 1

```
var p = new { Naam = "Anna", Leeftijd = 17 };
Console.WriteLine(JsonSerializer.Serialize(p));
```

[Toon het antwoord >](#)

Vraag 2

```
public class Persoon
{
    public string Naam { get; set; } = "";
}

var p = JsonSerializer.Deserialize<Persoon>("{\"naam\":\"Anna\"}");
Console.WriteLine("[ " + p!.Naam + " ]");
```

[Toon het antwoord >](#)

Vraag 3

```
var stapel = new Stack<int>();
stapel.Push(1);
stapel.Push(2);
stapel.Push(3);
```

```
string json = JsonSerializer.Serialize(stapel);  
var terug = JsonSerializer.Deserialize<Stack<int>>(json);  
Console.WriteLine(json);  
Console.WriteLine(terug!.Peek());
```

[Toon het antwoord ›](#)

Vraag 4

```
public enum Licht { Rood, Oranje, Groen }  
  
public class Lamp  
{  
    public int Nummer = 5;  
    public string Naam { get; set; } = "";  
    public Licht Licht { get; set; }  
}  
  
Console.WriteLine(JsonSerializer.Serialize(new Lamp { Naam = "Gang", Licht = Licht.Groen }));
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

```
{"Naam": "Anna", "Leeftijd": 17}
```

Alles op één regel, en de namen precies zoals in je code, met hoofdletter. Dat een API zijn JSON met kleine letters teruggeeft, komt doordat ASP.NET Core dat voor jou instelt; `JsonSerializer` zelf doet het niet.

[← Terug naar de vraag](#)

VRAAG 2

```
[]
```

In de JSON staat `naam` met kleine letter, in de klasse `Naam` met hoofdletter. Standaard is dat voor `JsonSerializer` een andere naam, dus `Naam` blijft leeg. Er komt geen enkele foutmelding. Met `PropertyNameCaseInsensitive = true` in de opties lost je dit op.

[← Terug naar de vraag](#)

VRAAG 3

```
[3, 2, 1]
```

 en daarna `1`

Een stapel wordt van boven naar onder geschreven. Bij het inlezen gaat elk getal er opnieuw met `Push` op, dus de `1` eindigt bovenaan.

[← Terug naar de vraag](#)

VRAAG 4

```
{"Naam": "Gang", "Licht": 2}
```

`Nummer` is een veld, geen eigenschap, en komt niet mee. De enum wordt het getal erachter.

[← Terug naar de vraag](#)

Deel 6 — Data

Van gegevens in het geheugen naar gegevens in een database: bewaren, opvragen, en enkel doorgeven wat nodig is.

31. EF Core: model en database

DATA

Wat het is

Een tolk tussen je klassen en je tabellen. Entity Framework Core, kort EF Core, zet je C#-klassen om naar tabellen in een database, en de rijen uit die tabellen weer naar objecten. Je schrijft zelf geen SQL. Je beschrijft je klassen, en EF Core maakt de tabellen erbij. De cursus noemt die werkwijze *code first*: eerst de code, dan de database.

Drie stukken werken samen:

1. je **entiteiten**: gewone klassen, één per tabel;
2. de **context**: één klasse die erft van `DbContext`, met per tabel een `DbSet`;
3. de **migraties**: scripts die je database gelijkzetten met je klassen.

Dit hoofdstuk gaat over het model: hoe je tabellen en relaties vastlegt. Hoe je gegevens opvraagt en bewaart, staat in [hoofdstuk 32](#).

Hoe het eruitziet

Twee entiteiten: een keuken heeft veel recepten, een recept hoort bij één keuken.

```
public class Keuken
{
    public int Id { get; set; }
    public string Naam { get; set; } = default!;

    public List<Recept> Recepten { get; set; } = new();
}

public class Recept
{
    public int Id { get; set; }
    public string Titel { get; set; } = default!;
    public decimal Kostprijs { get; set; }

    public int KeukenId { get; set; }           // de vreemde sleutel
    public Keuken Keuken { get; set; } = default!; // de navigatie-eigenschap
}
```

De context, met de verfijning in `OnModelCreating`:

```
public class ReceptContext : DbContext
{
    public ReceptContext(DbContextOptions<ReceptContext> options) : base(options) { }

    public DbSet<Keuken> Keukens { get; set; }
    public DbSet<Recept> Recepten { get; set; }
}
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Recept>(entity =>
    {
        entity.Property(r => r.Titel).IsRequired().HasMaxLength(100);
        entity.Property(r => r.Kostprijs).HasPrecision(8, 2);

        entity.HasOne(r => r.Keuken)
            .WithMany(k => k.Recepten)
            .HasForeignKey(r => r.KeukenId)
            .OnDelete(DeleteBehavior.Restrict);
    });
}
```

De registratie in `Program.cs`, met de verbinding uit `appsettings.json`:

```
var connectionString = builder.Configuration.GetConnectionString("Standaard");
builder.Services.AddDbContext<ReceptContext>(options =>
options.UseNpgsql(connectionString));

"ConnectionStrings": {
  "Standaard":
  "Host=localhost;Port=5432;Database=ReceptenDb;Username=postgres;Password=..."
}
```

`AddDbContext` registreert de context zoals `AddScoped` uit [hoofdstuk 27](#): één context per verzoek. Een repository vraagt hem gewoon in haar constructor.

Wat EF Core zelf afleidt

Veel hoef je niet te zeggen. EF Core volgt **conventies**: vaste afspraken over namen en types.

Je schrijft	EF Core maakt ervan
<code>DbSet<Recept></code> <code>Recepten</code> in de context	een tabel <code>Recepten</code>
een eigenschap <code>Id</code> of <code>ReceptId</code>	de primaire sleutel; bij een <code>int</code> telt de database zelf op
<code>string</code>	een verplichte kolom van onbeperkte lengte
<code>string?, int?</code>	een kolom die leeg mag blijven
<code>decimal</code>	een kolom voor kommagetallen, exact, zonder afrondingsfouten
<code>int KeukenId</code> naast <code>Keuken Keuken</code>	een vreemde sleutel naar de tabel <code>Keukens</code>
<code>int KeukenId</code>	een verplichte relatie: elk recept heeft een keuken
<code>int? KeukenId</code>	een optionele relatie: een recept mag zonder keuken

Alles wat afwijkt van die afspraken, of wat je strenger wil, zet je in `OnModelCreating`. Die schrijfwijze, met methodes achter elkaar, heet de **Fluent API**. `override` en `base` ken je uit [hoofdstuk 20](#).

Relaties

Een **navigatie-eigenschap** is een eigenschap die naar een ander object wijst, zoals `Recept.Keuken`, of naar een lijst objecten, zoals `Keuken.Recepten`. De **vreemde sleutel** is het getal in de tabel dat die koppeling bewaart, zoals `KeukenId`.

Soort	Voorbeeld	Waar de vreemde sleutel staat	Navigatie-eigenschappen
één op één	een leerling en zijn kluisje	aan één kant, die je zelf kiest	aan elke kant één object
één op veel	een keuken en haar recepten	aan de <i>veel</i> -kant: <code>Recept.KeukenId</code>	<code>Recept.Keuken</code> en de lijst <code>Keuken.Recepten</code>
veel op veel	recepten en ingrediënten	in een aparte koppeltabel, met twee vreemde sleutels	elke kant een lijst koppelobjecten

Bij veel op veel maakt de cursus de koppeltabel als een eigen klasse, met twee één-op-veel-relaties. Zo kan de koppeling zelf ook gegevens dragen, zoals de hoeveelheid van een ingrediënt in een recept.

Een relatie in de Fluent API lees je vanuit de entiteit die je configureert:

Regel	Lees als
<code>HasOne(r => r.Keuken)</code>	een recept heeft één keuken
<code>.WithMany(k => k.Recepten)</code>	die keuken heeft veel recepten
<code>.HasForeignKey(r => r.KeukenId)</code>	de koppeling zit in <code>KeukenId</code>
<code>.onDelete(DeleteBehavior.Restrict)</code>	een keuken met recepten mag niet weg

Verwijderen: Cascade of Restrict

	Cascade	Restrict
Standaard bij	een verplichte relatie, zoals <code>int KeukenId</code>	nooit, je kiest het zelf
Een keuken met recepten verwijderen	de recepten verdwijnen mee	<code>SaveChanges</code> weigert, met een uitzondering
Het gevaar	gegevens verdwijnen zonder dat iemand het merkt	geen; je ruimt eerst zelf op

Les 06 vertelt het verhaal van klant Gerrit, die verwijderd wil worden. Met `Cascade` gaan al zijn bestellingen stilletjes mee. Met `Restrict` houdt de database tegen.

Migraties

Een migratie is een script dat beschrijft hoe de database moet veranderen om weer bij je klassen te passen. Elk script heeft een `Up`, die de wijziging uitvoert, en een `Down`, die ze terugdraait.

Package Manager Console (Visual Studio)	Terminal	Wat het doet
Add-Migration Naam	<code>dotnet ef migrations add Naam</code>	maak een nieuw script in de map <code>Migrations</code>
Update-Database	<code>dotnet ef database update</code>	voer de scripts uit die de database nog niet kent
Remove-Migration	<code>dotnet ef migrations remove</code>	verwijder het laatste script, zolang de database het nog niet uitvoerde

De vaste cyclus, telkens je een entiteit toevoegt of aanpast:

1. pas je klasse aan, en voeg indien nodig een `DbSet` toe;
2. `Add-Migration` met een naam die zegt wat er verandert;
3. kijk het script na: heeft EF Core begrepen wat je bedoelde?
4. `Update-Database`, met Docker aan.

Je hebt drie packages nodig: `Npgsql.EntityFrameworkCore.PostgreSQL`, `Microsoft.EntityFrameworkCore.Design` en, voor de Package Manager Console, `Microsoft.EntityFrameworkCore.Tools`. EF Core zelf komt mee met het eerste. Les 04 installeert het ook apart, en dat kan geen kwaad. In de terminal installeer je eenmalig ook het programma zelf, met `dotnet tool install --global dotnet-ef`.

LET OP

In les 04 staat op een paar plaatsen `dotnet Entity Framework migrations add` en `dotnet Entity Framework database update`. De commando's heten `dotnet ef migrations add` en `dotnet ef database update`. De lange vorm herkent de terminal niet.

Startgegevens met HasData

Met `HasData` geef je rijen mee die de migratie meteen in de tabel zet. Zo heeft je database na `Update-Database` al gegevens om mee te testen. Twee regels: je vult het `Id` zelf in, ook al telt de database het anders zelf op, en een relatie leg je via de vreemde sleutel, niet via de navigatie-eigenschap.

```
modelBuilder.Entity<Keuken>().HasData(
    new Keuken { Id = 1, Naam = "Italiaans" },
    new Keuken { Id = 2, Naam = "Thais" });
```

LET OP

PostgreSQL telt de `Ids` met een teller die niet weet dat jouw startgegevens 1 en 2 al gebruiken. De eerste keuken die je daarna via de API toevoegt, krijgt ook `Id 1`, en dat geeft een fout over een dubbele sleutel: een **500**. De documentatie van `Npgsql` noemt twee uitwegen: de teller laten beginnen boven je startgegevens, met `HasIdentityOptions`, of je startgegevens negatieve `Ids` geven.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
DbContext	de basisklasse van je context	<code>class ReceptContext : DbContext</code>	de brug naar de database
DbSet<T>	één tabel	<code>public DbSet<Recept> Recepten { get; set; }</code>	de tabel Recepten
: base(options)	geef de opties door aan DbContext	in de constructor van de context	de context weet welke database
AddDbContext	registreer de context, één per verzoek	<code>builder.Services.AddDbContext<ReceptContext>(...)</code>	repositories kunnen hem vragen
UseNpgsql	gebruik PostgreSQL, met deze verbinding	<code>options.UseNpgsql(connectionString)</code>	—
OnModelCreating	de plaats voor de Fluent API	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code>	—
Entity<T>	configureer één entiteit	<code>modelBuilder.Entity<Recept>(entity => { ... })</code>	—
ToTable	kies zelf de naam van de tabel	<code>entity.ToTable("Recepten")</code>	—
HasKey	kies zelf de primaire sleutel	<code>entity.HasKey(r => r.Code)</code>	nodig als de sleutel geen Id heet
Property	configureer één eigenschap	<code>entity.Property(r => r.Titel)</code>	—
IsRequired	de kolom mag niet leeg zijn	<code>.IsRequired()</code>	—
HasMaxLength	een maximale lengte voor tekst	<code>.HasMaxLength(100)</code>	hoogstens 100 tekens
HasPrecision	het aantal cijfers van een kommagetal	<code>.HasPrecision(8, 2)</code>	8 cijfers, waarvan 2 na de komma
HasOne + WithMany	één op veel, gelezen vanaf de <i>veel</i> -kant	<code>entity.HasOne(r => r.Keuken).WithMany(k => k.Recepten)</code>	—
HasOne + WithOne	één op één	<code>.HasOne(l => l.Kluisje).WithOne(k => k.Leerling)</code>	—

Vorm	Wat het doet	Voorbeeld	Resultaat
HasForeignKey	welke eigenschap de vreemde sleutel is	<code>.HasForeignKey(r => r.KeukenId)</code>	bij één op één met het type erbij: <code>HasForeignKey<Kluisje>(k => k.LeerlingId)</code>
onDelete	wat er gebeurt als de andere kant verdwijnt	<code>.onDelete(DeleteBehavior.Restrict)</code>	—
HasData	startgegevens via een migratie	<code>modelBuilder.Entity<Keuken>().HasData(...)</code>	rijen in de tabel na <code>Update-Database</code>

Wat het ook kan, via iets anders

Wat	Waar het vandaan komt	Hoofdstuk
een navigatie-eigenschap <code>List<Recept></code> Recepten	een gewone <code>List</code> : <code>Count</code> , <code>Any</code> en LINQ werken, zodra ze geladen is	6, 32
<code>DbSet<T></code>	een <code>IQueryable<T></code> : LINQ wordt SQL	32
een enum als eigenschap	wordt standaard het getal erachter in de database	18
<code>DateTime</code> als eigenschap	PostgreSQL wil UTC; zie de waarschuwing in hoofdstuk 3	3

Veelgemaakte fouten

LET OP

- Een klasse aangepast, maar geen migratie gemaakt of `Update-Database` vergeten? Alles compileert, maar bij het eerste verzoek meldt PostgreSQL dat een kolom of tabel niet bestaat. De client ziet **500**.
- Heet de sleutel geen `Id`, en ook niet de naam van de klasse met `Id` erachter, en staat er geen `HasKey`, dan stopt `Add-Migration` met *The entity type '...' requires a primary key to be defined.*
- Pas de database nooit met de hand aan. De volgende migratie gaat ervan uit dat de database er nog uitziet zoals de vorige migratie hem achterliet.
- Draait Docker niet, dan kan `Update-Database` de database niet bereiken.
- Zet `base.OnModelCreating(modelBuilder)` altijd bovenaan. Bij een gewone `DbContext` merk je het verschil niet, maar erft je context later van een klasse die zelf tabellen configureert, zoals bij de beveiliging, dan krijg je zonder die regel fouten over ontbrekende sleutels.

Voorspel de uitkomst

Vraag 1

De context bevat `public DbSet<Ingredient> Ingredienten { get; set; }`, en verder niets over ingrediënten.

```
public class Ingredient
{
    public int Nummer { get; set; }
    public string Naam { get; set; } = default!;
}
```

Add-Migration Ingredienten

[Toon het antwoord ›](#)

Vraag 2

Recept heeft `public int KeukenId`, en in `OnModelCreating` staat niets over `onDelete`. De keuken met Id 1 heeft drie recepten. Je verwijdert die keuken en roept `SaveChangesAsync` op.

Hoeveel recepten van die keuken staan er daarna nog in de database?

[Toon het antwoord ›](#)

Vraag 3

Hetzelfde, maar nu met `.onDelete(DeleteBehavior.Restrict)` op de relatie. Het verwijderen gebeurt in een DELETE-methode van een controller, zonder `try` en `catch`.

[Toon het antwoord ›](#)

Vraag 4

Je voegt `public int Bereidingstijd { get; set; }` toe aan `Recept`. Je maakt een migratie met `Add-Migration`, maar vergeet `Update-Database`. Dan start je de API en doe je `GET api/recept`.

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

Er komt geen migratie. EF Core meldt dat `Ingredient` een primaire sleutel nodig heeft.

EF Core zoekt een eigenschap die `Id` of `IngredientId` heet. `Nummer` past bij geen van beide. Hernoem de eigenschap, of wijs ze aan met `HasKey`.

[Terug naar de vraag](#)

VRAAG 2

`0`

`int KeukenId` kan niet leeg zijn, dus de relatie is verplicht. Bij een verplichte relatie kiest EF Core standaard `Cascade`: de recepten gaan mee met hun keuken. Geen foutmelding, geen waarschuwing.

[Terug naar de vraag](#)

VRAAG 3

500 Internal Server Error, en de keuken staat nog in de database, met haar drie recepten.

`SaveChangesAsync` gooit een uitzondering, omdat er nog recepten aan de keuken hangen. Er wordt dus niets verwijderd. Wil je de client een duidelijker antwoord geven, kijk dan vóór het verwijderen of er nog recepten zijn, en geef een **400 Bad Request** met uitleg.

[Terug naar de vraag](#)

VRAAG 4

500 Internal Server Error

De migratie staat klaar in je project, maar de database kent ze nog niet. EF Core vraagt de kolom `Bereidingstijd` op, en PostgreSQL meldt dat die niet bestaat. Het script maken en het script uitvoeren zijn twee aparte stappen.

[Terug naar de vraag](#)

32. EF Core: opvragen en bewaren

DATA

Wat het is

Een winkelmandje. Wat je erin legt, eruit haalt of omruilt, is nog niet betaald. Pas aan de kassa gaat alles in één keer door. Bij EF Core is de context het mandje: hij noteert wat je toevoegt, wijzigt en verwijdert. Dat noteren heet de **change tracker**. De kassa is `SaveChangesAsync`: pas dan maakt EF Core er SQL van en stuurt hij alles naar de database.

Opvragen werkt anders. Een vraag gaat meteen naar de database, op het moment dat je het antwoord opeist.

De voorbeelden gebruiken de `ReceptContext` uit [hoofdstuk 31](#), bewaard in een veld `_context`.

Hoe het eruitziet

```
// opvragen
List<Recept> alle = await _context.Recepten.ToListAsync();
Recept? vijf = await _context.Recepten.FindAsync(5);
List<Recept> goedkoop = await _context.Recepten
    .Where(r => r.Kostprijs < 5)
    .OrderBy(r => r.Titel)
    .ToListAsync();

// bewaren
_context.Recepten.Add(nieuwRecept);
await _context.SaveChangesAsync();
```

`Where`, `OrderBy` en `Select` ken je uit [hoofdstuk 11](#). Ze werken hier net zo, met één verschil: EF Core vertaalt ze naar SQL, en de database doet het filteren. De asynchrone methodes zoals `ToListAsync` staan in de namespace `Microsoft.EntityFrameworkCore`. Zonder die `using` bovenaan kent je code ze niet.

Opvragen: de vraag vertrekt pas als je het antwoord opeist

`_context.Recepten.Where(...)` geeft een `IQueryable<Recept>`. Net als de `IEnumerable` uit [hoofdstuk 12](#) is dat een vraag die nog niet gesteld is. Het verschil: een `IQueryable` wordt vertaald naar SQL, zodat de database het werk doet in plaats van je programma.

Bouwt de vraag verder	Stuurt de vraag naar de database
<code>Where</code> , <code>OrderBy</code> , <code>Select</code> , <code>Include</code> , <code>Take</code> , <code>Skip</code>	<code>ToListAsync</code> , <code>FirstOrDefaultAsync</code> , <code>FindAsync</code> , <code>CountAsync</code> , <code>AnyAsync</code> , een <code>foreach</code>

Elke keer dat je dezelfde `IQueryable` uitvoert, vertrekt er een nieuwe vraag naar de database. Les 05 noemt dit *deferred execution*: uitgestelde uitvoering.

OPMERKING

Geeft een repository een lijst terug, en filter je daarna in de controller met `Where`, dan gebeurt dat filteren in het geheugen, op alles wat de database stuurde. Zet een filter dus in de vraag zelf, vóór `ToListAsync`.

Bewaren: wat de context noteert

Je doet	De context noteert	Bij <code>SaveChangesAsync</code>
<code>Add(recept)</code>	nieuw	een INSERT
een opgevraagd object aanpassen	gewijzigd, vanzelf	een UPDATE van de gewijzigde kolommen
<code>Update(recept)</code> , of <code>Attach</code> met <code>State = EntityState.Modified</code>	gewijzigd	een UPDATE van alle kolommen
<code>Remove(recept)</code>	te verwijderen	een DELETE
niets	ongewijzigd	niets

`SaveChangesAsync` geeft het aantal rijen terug dat het schreef. Een nieuw object heeft na `Add` nog geen `Id`: de database kiest dat getal pas bij het bewaren. Daarom staat een `CreatedAtAction` met het nieuwe `Id` altijd ná `SaveChangesAsync`.

`FindAsync` zoekt op de primaire sleutel, en kijkt eerst of de context dat object al kent. Is dat zo, dan vraagt hij niets aan de database.

Gerelateerde gegevens: Include en ThenInclude

Vraag je een recept op, dan komt alleen het recept mee. `recept.Keuken` blijft `null`. EF Core laadt gerelateerde gegevens niet vanzelf: je zegt het met `Include`.

Stel dat een recept ook een lijst `ReceptIngredienten` heeft, de koppelklasse uit hoofdstuk 31, en dat elk koppelobject zijn `Ingredient` kent:

```
Recept? recept = await _context.Recepten
    .Include(r => r.Keuken)
    .Include(r => r.ReceptIngredienten)
    .ThenInclude(ri => ri.Ingredient)
    .FirstOrDefaultAsync(r => r.Id == id);
```

`Include` vertrekt altijd van het recept. `ThenInclude` gaat één stap verder vanaf de vorige `Include`: van het koppelobject naar zijn ingrediënt. Alles samen wordt één vraag aan de database.

Zonder `Include` zou je per recept een aparte vraag moeten stellen om de keuken op te halen: één vraag voor de lijst, en dan nog eens N vragen, één per recept. Les 08 noemt dat het **N+1-probleem**. Het gebeurt ongemerkt bij *lazy loading*, waarbij gegevens pas geladen worden zodra je de eigenschap gebruikt. In EF Core staat lazy loading standaard uit.

OPMERKING

Bovenaan les 08 staat dat EF Core standaard werkt "via het Lazy Loading principe". Daar bedoelt de les: gelinkte gegevens komen niet vanzelf mee. Verderop in de les, en in de documentatie van Microsoft, betekent *lazy loading* iets anders: vanzelf laden zodra je de eigenschap aanraakt. Dat doet EF Core standaard niet. Zonder `Include` blijft de eigenschap gewoon `null`.

Kringlopen in JSON

Een recept wijst naar zijn keuken, en die keuken heeft een lijst met haar recepten, waaronder hetzelfde recept. Geef je zo'n object rechtstreeks terug uit een controller, dan probeert de JSON-omzetter die kring eindeloos te volgen. Hij stopt met een `JsonException: A possible object cycle was detected`.

Les 08 lost dat snel op met één regel in `Program.cs`: `.AddJsonOptions(o => o.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles)` achter `AddControllers()`, met `using System.Text.Json.Serialization;` bovenaan. Les 08 schrijft de volledige naam uit, dan heb je die `using` niet nodig. Waar de kring zich sluit, komt dan `null`.

BELANGRIJK

Les 08 zegt het zelf: `IgnoreCycles` is een snelle oplossing, maar op het examen verwacht men DTO's. Die staan in hoofdstuk 33.

In een generieke repository en een Unit of Work

Les 07 maakt één repository voor alle entiteiten, met de open plaats uit hoofdstuk 21. Omdat de repository niet weet welke tabel ze krijgt, schrijft ze niet `_context.Recepten` maar `_context.Set<TEntity>()`: de `DbSet` van het type dat ingevuld werd.

De repositories roepen zelf geen `SaveChangesAsync` meer op. Dat doet de **Unit of Work**: één klasse die alle repositories aanbiedt en ze allemaal dezelfde context meegeeft. Omdat ze één context delen, noteert die ene context alle wijzigingen. Eén `SaveChangesAsync` op de Unit of Work bewaart ze dan samen, of geen enkele.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>ToListAsync</code>	voer de vraag uit, alles in een lijst	<code>await _context.Recepten.ToListAsync()</code>	een <code>List<Recept></code>
<code>FindAsync</code>	zoek op de primaire sleutel	<code>await _context.Recepten.FindAsync(5)</code>	het recept, of <code>null</code>
<code>FirstOrDefaultAsync</code>	de eerste die voldoet	<code>.FirstOrDefaultAsync(r => r.Titel == "Soep")</code>	het recept, of <code>null</code>
<code>AnyAsync</code>	is er minstens één	<code>.AnyAsync(r => r.KeukenId == 3)</code>	<code>true</code> of <code>false</code>

Vorm	Wat het doet	Voorbeeld	Resultaat
CountAsync	hoeveel zijn er	<code>.CountAsync(r => r.Kostprijs > 10)</code>	een getal
Add	noteer als nieuw	<code>_context.Recepten.Add(recept)</code>	het Id is nog 0
Update	noteer als gewijzigd	<code>_context.Recepten.Update(recept)</code>	—
Attach	laat de context een object volgen	<code>_context.Set<TEntity>().Attach(entity)</code>	gevolgd als ongewijzigd
Entry(...).State	zet zelf de toestand van een object	<code>_context.Entry(entity).State = EntityState.Modified;</code>	gevolgd als gewijzigd
Remove	noteer om te verwijderen	<code>_context.Recepten.Remove(recept)</code>	—
SaveChangesAsync	stuur alle genoteerde wijzigingen naar de database	<code>int n = await _context.SaveChangesAsync();</code>	het aantal geschreven rijen
Include	laad een navigatie-eigenschap mee	<code>.Include(r => r.Keuken)</code>	<code>recept.Keuken</code> is ingevuld
ThenInclude	ga één stap dieper vanaf de vorige Include	<code>.ThenInclude(ri => ri.Ingredient)</code>	—
Set<T>	de DbSet van een type dat je pas later kent	<code>_context.Set<TEntity>()</code>	in een generieke repository
IQueryable<T>	een vraag die naar SQL vertaald wordt	<code>var vraag = _context.Recepten.Where(...);</code>	nog niets uitgevoerd

Wat het ook kan, via iets anders

Een `DbSet` is meer dan de tabel hierboven laat zien. Hij heeft eigen methodes, hij is een `IQueryable` waarop bijna heel LINQ werkt, en EF Core voegt nog een reeks asynchrone methodes toe. Wat je na `ToListAsync` overhoudt, is een gewone lijst.

Wat een DbSet zelf nog kan

Methode	Wat het doet	Voorbeeld	Resultaat
AddRange	noteer meerdere als nieuw	<code>_context.Recepten.AddRange(r1, r2)</code>	twee INSERTs bij het bewaren
UpdateRange, RemoveRange, AttachRange	hetzelfde als Update, Remove, Attach, voor meerdere	<code>_context.Recepten.RemoveRange(oude)</code>	—
AddAsync, AddRangeAsync	zoals Add; enkel nodig bij speciale sleutels	<code>await _context.Recepten.AddAsync(r)</code>	—
Find	<code>FindAsync</code> zonder <code>await</code>	<code>_context.Recepten.Find(5)</code>	het recept, of <code>null</code>
Entry	de notities van de context over één object	<code>_context.Entry(r).State</code>	bv. <code>Unchanged</code> , <code>Modified</code>

Methode	Wat het doet	Voorbeeld	Resultaat
Local	wat de context nu al kent, zonder de database te vragen	<code>_context.Recepten.Local.Count</code>	het aantal gevolgde recepten

Een vraag opbouwen

Komt van LINQ. Deze methodes geven op een `IQueryable` opnieuw een `IQueryable`: er vertrekt nog niets. EF Core vertaalt ze naar SQL.

Methode	Voorbeeld	Wordt in SQL
Where	<code>.Where(r => r.Kostprijs < 5)</code>	WHERE
OrderBy, OrderByDescending, ThenBy, ThenByDescending	<code>.OrderBy(r => r.Titel)</code>	ORDER BY
Order, OrderDescending	<code>.Order()</code> : sorteert op de primaire sleutel (EF Core 9 en later)	ORDER BY "Id"
Select	<code>.Select(r => r.Titel)</code>	enkel die kolommen
Skip, Take	<code>.Skip(10).Take(10)</code> : de tweede pagina van tien	OFFSET en LIMIT
Distinct	<code>.Select(r => r.KeukenId).Distinct()</code>	SELECT DISTINCT
GroupBy	<code>.GroupBy(r => r.KeukenId).Select(g => new { g.Key, Aantal = g.Count() })</code>	GROUP BY
Include, ThenInclude	zie hierboven	een JOIN
AsNoTracking	<code>.AsNoTracking()</code> : de context volgt de resultaten niet	dezelfde SQL, maar sneller en enkel om te lezen

Binnen een voorwaarde mag je ook tekst- en datummethodes gebruiken die EF Core kent. Met PostgreSQL wordt `r.Titel.Contains("soep")` een `LIKE`, `StartsWith` en `EndsWith` ook, en `ToLower()` wordt `lower()`.

Een vraag uitvoeren

Komt van EF Core, in `Microsoft.EntityFrameworkCore`. Elke methode van LINQ die één antwoord of een lijst geeft, heeft hier een versie met `Async`. Die stuurt de vraag naar de database.

Methode	Wat het geeft	Als er niets is
ToListAsync, ToArrayAsync	een <code>List<T></code> of een <code>T[]</code>	een lege lijst
ToDictionaryAsync	een <code>Dictionary</code> : <code>.ToDictionaryAsync(r => r.Id)</code>	een lege tabel
ToHashSetAsync	een <code>HashSet<T></code>	een lege set
FirstAsync, FirstOrDefaultAsync	het eerste dat voldoet	InvalidOperationException of <code>null</code>
SingleAsync, SingleOrDefaultAsync	het enige dat voldoet; meer dan één geeft InvalidOperationException	InvalidOperationException of <code>null</code>

Methode	Wat het geeft	Als er niets is
LastAsync , LastOrDefaultAsync	het laatste; enkel na een <code>OrderBy</code>	<code>InvalidOperationException</code> of <code>null</code>
ElementAtAsync , ElementAtOrDefaultAsync	het element op een plaats, na een <code>OrderBy</code>	een uitzondering of <code>null</code>
AnyAsync , AllAsync	true of false	<code>false</code> en <code>true</code>
ContainsAsync	zit deze waarde erin: <code>.Select(r => r.Titel).ContainsAsync("Soep")</code>	<code>false</code>
CountAsync , LongCountAsync	hoeveel	0
SumAsync	de som: <code>.SumAsync(r => r.Kostprijs)</code>	0
MinAsync , MaxAsync , AverageAsync	kleinste, grootste, gemiddelde	<code>InvalidOperationException</code>
ForEachAsync	voer iets uit per resultaat	er gebeurt niets
LoadAsync	laad de resultaten in de context, zonder ze terug te geven	—

De versies zonder `Async`, zoals `ToList()`, `First()` en `Count()`, werken ook op een `IQueryable`. Ze laten je programma wachten tot de database klaar is. In een API gebruik je de `Async`-versie, met `await`.

Na `ToListAsync`: een gewone lijst

Wat `ToListAsync` teruggeeft, is een `List<T>` uit [hoofdstuk 6](#), en alles van daar en van [hoofdstuk 11](#) werkt erop. Maar het gebeurt dan in je programma, niet meer in de database:

Je schrijft	Waar het gebeurt
<code>(await _context.Recepten.Where(r => r.Kostprijs > 10).ToListAsync()).Count</code>	filteren in de database, tellen in je programma
<code>(await _context.Recepten.ToListAsync()).Where(r => r.Kostprijs > 10)</code>	alle recepten komen over, en je programma filtert
<code>await _context.Recepten.CountAsync(r => r.Kostprijs > 10)</code>	alles in de database; enkel het getal komt over

Bestaat ook, maar kom je in deze cursus niet tegen

Methode	Wat het doet
ExecuteUpdateAsync , ExecuteDeleteAsync	wijzig of verwijder alle rijen die voldoen, meteen , zonder ze op te halen en zonder <code>SaveChangesAsync</code> : <code>.Where(r => r.Kostprijs > 100).ExecuteDeleteAsync()</code> . Geeft het aantal rijen terug. De context merkt er niets van.
ToQueryString	de SQL die EF Core zou sturen, als tekst: handig om te kijken
FromSql , SqlQuery	zelf SQL schrijven en toch EF Core gebruiken
AsSplitQuery	een vraag met veel <code>Includes</code> in meerdere stukken sturen
AsAsyncEnumerable	resultaten één per één verwerken met <code>await foreach</code>

Let op bij EF Core

Wat	Waarom
een nieuw object na <code>Add</code>	zit nog niet in het antwoord op een vraag: de database kent het pas na <code>SaveChangesAsync</code>
<code>LastAsync</code> of <code>LastOrDefaultAsync</code> zonder <code>OrderBy</code>	InvalidOperationException : <i>must have a deterministic sort order</i>
een eigen methode in <code>Where</code> of <code>OrderBy</code>	InvalidOperationException : <i>could not be translated</i>
een eigen <code>static</code> methode in de laatste <code>Select</code>	werkt: EF Core haalt de kolommen op en roept je methode op in je programma; zonder <code>static</code> weigert EF Core ze
een object aanpassen na <code>AsNoTracking</code>	<code>SaveChangesAsync</code> bewaart niets, want de context volgt het niet
<code>Contains</code> , <code>StartsWith</code> , <code>==</code> op tekst in PostgreSQL	maken onderscheid tussen hoofdletters en kleine letters: " <code>soep</code> " vindt <code>Soep</code> niet

LET OP

- Een filter ná `ToListAsync` laat eerst de hele tabel overkomen. Zet `Where` altijd vóór `ToListAsync`.
- `ExecuteDeleteAsync` en `ExecuteUpdateAsync` wachten niet op `SaveChangesAsync`. Wat ze doen, staat meteen in de database, ook als je daarna niet bewaart.

Veelgemaakte fouten

LET OP

- `Add`, `Update` of `Remove` zonder `SaveChangesAsync` erachter: er gebeurt niets in de database, en je krijgt geen enkele foutmelding.
- Het `Id` van een nieuw object gebruiken vóór `SaveChangesAsync`. Het staat dan nog op 0.
- `recept.Kuiken.Naam` zonder `Include`: de keuken is niet geladen, en je krijgt een `NullReferenceException`.
- `Include` werkt niet samen met `FindAsync`. Wil je gerelateerde gegevens mee, gebruik dan `FirstOrDefaultAsync` met een voorwaarde op het `Id`.
- Een eigen C#-methode in een `Where` kan EF Core niet naar SQL vertalen. Het compileert, maar bij het uitvoeren krijg je *The LINQ expression ... could not be translated*.
- Een `await` vergeten bij een EF-methode, en meteen een tweede oproepen? Dan werkt de context aan twee dingen tegelijk, en dat weigert hij met een uitzondering. Zet `await` voor elke asynchrone EF-methode.

Voorspel de uitkomst

Vraag 1

De tabel `Recepten` is net aangemaakt en nog leeg.

```
var r = new Recept { Titel = "Soep", KeukenId = 1 };
_context.Recepten.Add(r);
Console.WriteLine(r.Id);
await _context.SaveChangesAsync();
Console.WriteLine(r.Id);
```

[Toon het antwoord ›](#)

Vraag 2

De tabel `Recepten` bevat vier recepten.

```
_context.Recepten.Add(new Recept { Titel = "Curry", KeukenId = 2 });
int aantal = await _context.Recepten.CountAsync();
Console.WriteLine(aantal);
```

[Toon het antwoord ›](#)

Vraag 3

```
var r = await _context.Recepten.FindAsync(2);
r!.Titel = "Tomatensoep";
await _context.SaveChangesAsync();
```

Er staat nergens `Update`. Heet recept 2 nu in de database *Tomatensoep*?

[Toon het antwoord ›](#)

Vraag 4

In een nieuwe context, waarin nog niets geladen werd:

```
var r = await _context.Recepten.FirstOrDefaultAsync(x => x.Id == 2);
Console.WriteLine(r!.Keuken.Naam);
```

[Toon het antwoord ›](#)

Vraag 5

```
var duur = _context.Recepten.Where(x => x.Kostprijs > 10);
int aantal = await duur.CountAsync();
List<Recept> lijst = await duur.ToListAsync();
```

Hoeveel keer vraagt deze code iets aan de database?

[Toon het antwoord ›](#)

Vraag 6

Een GET-methode geeft een recept terug, opgehaald met `.Include(x => x.Keuken)`. De klasse `Keuken` heeft een lijst `Recepten`. In `Program.cs` staat niets over `ReferenceHandler`.

[Toon het antwoord ›](#)

Vraag 7

```
var r = await _context.Recepten.AsNoTracking().FirstOrDefault(x => x.Id == 2);
r.Titel = "Tomatensoep";
int n = await _context.SaveChangesAsync();
Console.WriteLine(n);
```

[Toon het antwoord ›](#)

Vraag 8

```
var laatste = await _context.Recepten.LastOrDefaultAsync();
```

[Toon het antwoord ›](#)

Vraag 9

`Kort` is een eigen `static` methode die een tekst afkapt op tien tekens.

```
var a = await _context.Recepten.Select(r => Kort(r.Titel)).ToListAsync();
var b = await _context.Recepten.Where(r => Kort(r.Titel) == "Soep").ToListAsync();
```

Welke regel werkt?

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

0 en daarna 1

Na `Add` is het recept enkel genoteerd. Het `Id` kiest de database, en die heeft het recept nog niet gezien. Na `SaveChangesAsync` vult EF Core het `Id` in met wat de database koos.

[Terug naar de vraag](#)

VRAAG 2

4

`CountAsync` stelt de vraag aan de database, en daar staat de curry nog niet in. Het recept ligt nog in het mandje, want `SaveChangesAsync` is niet opgeroepen.

[Terug naar de vraag](#)

VRAAG 3

Ja.

De context heeft het recept zelf opgehaald, en volgt het sindsdien. Hij merkt dat de titel veranderd is, en `SaveChangesAsync` stuurt een `UPDATE`. Update heb je enkel nodig voor een object dat de context niet zelf ophaalde, zoals een object dat uit de body van een verzoek komt.

[Terug naar de vraag](#)

VRAAG 4

`NullReferenceException` — het programma crasht.

Het recept is gevonden, maar zijn keuken niet meegeladen. `r.Keuken` is `null`, ook al staat er = default! achter de eigenschap. Met `.Include(x => x.Keuken)` vóór `FirstOrDefaultAsync` lukt het wel.

[Terug naar de vraag](#)

VRAAG 5

2

`duur` is een vraag, geen antwoord. `CountAsync` stelt ze één keer, `ToListAsync` stelt ze opnieuw. Komt er tussen die twee een recept bij, dan kan `aantal` zelfs verschillen van `lijst.Count`. Heb je beide nodig, vraag dan de lijst op en tel die.

[Terug naar de vraag](#)

VRAAG 6

500 Internal Server Error

Het recept wijst naar de keuken, en de keuken heeft het recept in haar lijst `Recepten`: EF Core vult die lijst aan met wat hij al geladen heeft. De JSON-omzetter loopt in een kring en stopt met een `JsonException`. Met `IgnoreCycles` krijg je wel een antwoord, met `null` waar de kring zich sluit. Met een DTO heb je het probleem niet.

[Terug naar de vraag](#)

VRAAG 7

0

Met `AsNoTracking` volgt de context het recept niet. Hij merkt de nieuwe titel dus niet op, en `SaveChangesAsync` heeft niets te bewaren. Vergelijk met vraag 3.

[← Terug naar de vraag](#)

VRAAG 8

`InvalidOperationException` — het programma crasht.

Een tabel heeft geen vaste volgorde, dus "het laatste" bestaat niet. EF Core weigert de vraag zonder `OrderBy`. Met `.OrderBy(x => x.Id).LastOrDefaultAsync()` lukt het wel.

[← Terug naar de vraag](#)

VRAAG 9

De eerste. De tweede geeft een **`InvalidOperationException`**: *could not be translated*.

In de laatste `Select` mag EF Core een stuk in je programma uitvoeren: hij haalt de titels op en roept daarna `Kort` op. Dat mag enkel met een `static` methode. Een `Where` moet de database zelf doen, en die kent `Kort` niet.

[← Terug naar de vraag](#)

33. DTO's en Mapster

DATA

Wat het is

De etalage, niet het magazijn. Je entiteit is het magazijn: alles wat je bewaart, met alle relaties en velden die niemand buiten je API hoeft te zien. Een **DTO**, een *Data Transfer Object*, is de etalage: een klasse of record met enkel wat de client te zien krijgt, in de vorm die hem past.

Een DTO lost vier dingen tegelijk op:

- geen kringlopen in je JSON, want de DTO wijst niet terug naar zijn ouder (zie [hoofdstuk 32](#));
- je stuurt niet meer dan nodig;
- velden die binnen moeten blijven, komen er gewoon niet in;
- je database mag veranderen zonder dat de client het merkt.

Van een entiteit een DTO maken is veel overtypen: `Titel = recept.Titel`, en zo verder voor elke eigenschap. **Mapster** doet dat voor je.

Hoe het eruitziet

```
public class ReceptDto
{
    public int Id { get; set; }
    public string Titel { get; set; } = default!;
    public string KeukenNaam { get; set; } = default!;
}
```

Les 09 toont ook de kortere vorm als `record`, uit [hoofdstuk 14](#):

```
public record ReceptDto(int Id, string Titel, string KeukenNaam);
```

Omzetten, na `dotnet add package Mapster` en `using Mapster`:

```
ReceptDto dto = recept.Adapt<ReceptDto>();
List<ReceptDto> dtos = recepten.Adapt<List<ReceptDto>>();
```

`Adapt` is een *extension method*: een methode die Mapster toevoegt aan elk object, zodat je ze kan oproepen alsof ze er altijd al stond.

Hoe Mapster een eigenschap vult

Mapster overloopt elke eigenschap van de DTO en zoekt in de bron:

1. een eigenschap met **exact dezelfde naam**, hoofdletters inbegrepen: `Titel` uit `Titel`;

2. een naam die aan elkaar geplakt is uit een eigenschap en de eigenschap daarvan: `KeukenNaam` uit `Keuken.Naam`. Dat heet **platslaan** (*flattening*);
3. vindt hij niets, dan laat hij de eigenschap op haar standaardwaarde staan, zonder foutmelding.

Wat in de bron staat en niet in de DTO, laat hij gewoon liggen.

OPMERKING

Platslaan werkt enkel als de keuken ook geladen is. Zonder `Include` is de keuken `null`. Mapster vangt dat op, zonder crash, en laat `KeukenNaam` gewoon leeg.

Eigen regels: een `MapperProfile`

Past een naam niet, of moet een waarde ergens anders vandaan komen, dan schrijf je een regel. Les 09 zet die regels samen in één klasse die `IRegister` belooft. Stel een `ReceptDetailDto` met een tekst `Herkomst` en een lijst `Ingredienten`:

```
public class MapperProfile : IRegister
{
    public void Register(TypeAdapterConfig config)
    {
        config.NewConfig<Recept, ReceptDetailDto>()
            .Map(dest => dest.Herkomst, src => src.Keuken.Naam)
            .Map(dest => dest.Ingredienten, src => src.ReceptIngredienten);
    }
}
```

Lees `.Map(dest => dest.Herkomst, src => src.Keuken.Naam)` als: *vul in het doel `Herkomst` met de naam van de keuken uit de bron*. Wat wel overeenkomt, blijft Mapster zelf doen. Voor de lijst `Ingredienten` zoekt Mapster op zijn beurt een regel om één koppelobject om te zetten.

Die klasse doet pas iets als `Program.cs` haar laat zoeken, met één regel:

```
TypeAdapterConfig.GlobalSettings.Scan(Assembly.GetExecutingAssembly());
```

`Scan` doorzoekt je project naar klassen die `IRegister` beloven, en leest hun regels. Bovenaan heb je `using Mapster;` en `using System.Reflection;` nodig.

OPMERKING

Les 09 raadt aan om in een profiel geen berekeningen te doen, zoals een totaalprijs. Code in een profiel is moeilijk te testen en te debuggen. Hou het profiel bij overzetten; het rekenwerk hoort in een controller, een service of een hulpklasse.

Wanneer wel, wanneer niet

Wel — alles wat je API teruggeeft aan de client. Voor wat binnenkomt maak je vaak een aparte DTO zonder `Id`, want dat kiest de database.

Niet — binnen je eigen code, tussen repository en database. Daar werk je met de entiteiten zelf.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>Adapt<T></code>	maak een nieuw object van type <code>T</code>	<code>recept.Adapt<ReceptDto>()</code>	een <code>ReceptDto</code>
<code>Adapt</code> op een lijst	zet elk item om	<code>recepten.Adapt<List<ReceptDto>>()</code>	een <code>List<ReceptDto></code>
<code>bron.Adapt(doel)</code>	kopieer in een object dat al bestaat	<code>dto.Adapt(recept)</code>	<code>recept</code> krijgt de waarden van <code>dto</code>
<code>IRegister</code>	de interface voor een klasse met eigen regels	<code>class MapperProfile : IRegister</code>	—
<code>Register</code>	de methode waarin je de regels zet	<code>public void Register(TypeAdapterConfig config)</code>	—
<code>NewConfig</code>	begin een regel voor dit paar	<code>config.NewConfig<Recept, ReceptDto>()</code>	vervangt een eerdere regel voor hetzelfde paar
<code>ForType</code>	maak of vul een regel aan	<code>config.ForType<Recept, ReceptDto>()</code>	een eerdere regel blijft gelden
<code>Map</code>	vul deze doel-eigenschap met dit	<code>.Map(dest => dest.Herkomst, src => src.Keuken.Naam)</code>	—
<code>Ignore</code>	laat deze doel-eigenschap met rust	<code>.Ignore(dest => dest.Id)</code>	<code>Id</code> wordt niet overschreven
<code>Scan</code>	zoek alle <code>IRegister</code> -klassen en lees hun regels	<code>GlobalSettings.Scan(...)</code>	één regel in <code>Program.cs</code> , zie hierboven

Wat het ook kan, via iets anders

Wat	Voorbeeld	Resultaat
<code>Adapt</code> naar elke soort verzameling	<code>recepten.Adapt<ReceptDto[]>(),</code> <code>Adapt<List<ReceptDto>>()</code>	een array of een lijst, ook vanuit een <code>IEnumerable</code> van <code>Dapper</code>
een enum naar tekst	<code>Status</code> is een enum in de entiteit en een <code>string</code> in de DTO	de naam van de keuze, zoals "Groen"
tekst naar een enum	omgekeerd	de keuze met die naam
een getal naar een ander getal	<code>int</code> in de entiteit, <code>decimal</code> in de DTO	omgezet
een lijst in een lijst	<code>Ingredienten</code> met elk een eigen DTO	elk item omgezet, zie hierboven

Wat `Adapt` teruggeeft, is een gewone lijst of array: alles van [hoofdstuk 5](#), [6](#) en [11](#) werkt erop.

Veelgemaakte fouten

LET OP

- Een naam die net anders is, zoals `Naam` in de bron en `Titel` in de DTO, geeft geen fout. De eigenschap blijft gewoon leeg. Kijk bij een lege eigenschap eerst naar de namen.
- De `Scan`-regel vergeten in `Program.cs`: alles compileert en draait, maar je eigen regels worden nooit gelezen. Enkel wat vanzelf overeenkomt, wordt ingevuld.
- Twee keer `NewConfig` voor hetzelfde paar: de tweede vervangt de eerste helemaal. Wil je aanvullen, gebruik dan `ForType`, of zet alles in één regel.
- DTO's gebruiken én `IgnoreCycles` laten staan. Les 09 haalt die regel weer weg: een DTO zonder terugwijzende eigenschappen heeft hem niet nodig.

Voorspel de uitkomst

Deze vragen gebruiken gewone klassen met eigenschappen, zonder database.

Vraag 1

```
public class Dier
{
    public int Id { get; set; }
    public string Naam { get; set; } = "";
    public string Soort { get; set; } = "";
}

public class DierDto
{
    public string Naam { get; set; } = "";
    public string Ras { get; set; } = "";
}

var dto = new Dier { Id = 3, Naam = "Bo", Soort = "hond" }.Adapt<DierDto>();
Console.WriteLine "[" + dto.Naam + "] [" + dto.Ras + "]");
```

[Toon het antwoord ›](#)

Vraag 2

```
public class Stad
{
    public string Naam { get; set; } = "";
}

public class Leerling
{
    public string Naam { get; set; } = "";
    public Stad Stad { get; set; } = new();
}
```

```

}

public class LeerlingDto
{
    public string Naam { get; set; } = "";
    public string StadNaam { get; set; } = "";
}

var anna = new Leerling { Naam = "Anna", Stad = new Stad { Naam = "Gent" } };
var dto = anna.Adapt<LeerlingDto>();
Console.WriteLine(dto.StadNaam);

```

[Toon het antwoord ›](#)

Vraag 3

LeerlingDto krijgt er twee teksten bij: Woonplaats en Klas. Leerling krijgt een eigenschap Klasgroep, met daarin een Code. Het profiel bevat:

```

config.NewConfig<Leerling, LeerlingDto>()
    .Map(dest => dest.Woonplaats, src => src.Stad.Naam);

config.NewConfig<Leerling, LeerlingDto>()
    .Map(dest => dest.Klas, src => src.Klasgroep.Code);

```

Welke van de twee wordt ingevuld?

[Toon het antwoord ›](#)

Vraag 4

Het profiel van vraag 3 is verbeterd en zet beide regels achter één NewConfig. Maar in Program.cs ontbreekt de regel met Scan.

Wat staat er in Naam, Woonplaats en Klas van de DTO?

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

[Bo] []

Naam staat aan beide kanten, dus die wordt gekopieerd. Ras vindt geen bron en houdt zijn beginwaarde, een lege tekst. Id en Soort hebben geen plaats in de DTO en vallen weg. Nergens een foutmelding.

[Terug naar de vraag](#)

VRAAG 2

Gent

StadNaam bestaat niet in Leerling, maar Stad gevolgd door Naam wel. Mapster slaat dat vanzelf plat. Een eigen regel heb je pas nodig als de naam dat patroon niet volgt, zoals bij Woonplaats.

[Terug naar de vraag](#)

VRAAG 3

Enkel Klas.

De tweede NewConfig voor hetzelfde paar gooit de eerste regel weg. Woonplaats blijft leeg. Zet beide Map-regels achter één NewConfig, of begin de tweede met ForType.

[Terug naar de vraag](#)

VRAAG 4

Naam is ingevuld, Woonplaats en Klas blijven leeg.

Zonder Scan leest niemand het profiel. Mapster doet enkel wat het zelf kan: eigenschappen met dezelfde naam kopiëren, en platslaan. Er is geen enkele foutmelding die je op het spoor zet.

[Terug naar de vraag](#)

34. Dapper

DATA

OPMERKING

Les 08 zegt het duidelijk: op het examen ligt de focus op EF Core. Dapper hoeft je niet zelf te kunnen schrijven, maar je moet kunnen uitleggen wat het is, en waarom en wanneer iemand ervoor kiest. Dit hoofdstuk helpt je de code te lezen en het verschil te begrijpen.

Wat het is

Zelf de taal van de database spreken. EF Core is een tolk: jij schrijft C#, hij maakt er SQL van. Met Dapper schrijf je de SQL zelf, en Dapper doet enkel de laatste stap: de rijen die terugkomen omzetten in objecten. Zo'n kleine tussenpersoon heet een **micro-ORM**. EF Core is een volledige ORM.

	EF Core	Dapper
SQL	maakt het zelf, uit LINQ	schrijf je zelf
Wijzigingen bijhouden	ja, de change tracker	nee
Tabellen en migraties	ja	nee; de database moet al bestaan
Gerelateerde gegevens	<code>Include</code>	een <code>JOIN</code> in je SQL, en zelf de objecten koppelen
Sterk in	schrijven, en de structuur van je database	snel lezen, ingewikkelde rapporten

Grotere toepassingen combineren ze vaak. EF Core doet het schrijven, met change tracking en een Unit of Work. Dapper doet het lezen waar elke milliseconde telt. Les 08 noemt dat het hybride patroon, naar het principe *Command Query Separation*.

Hoe het eruitziet

```
using Dapper;

string sql = """
    SELECT * FROM "Recepten"
    WHERE "Id" = @Id
    """;

Recept? recept = await _dbConnection.QueryFirstOrDefaultAsync<Recept>(sql, new { Id = id
});
```

`_dbConnection` is een `IDbConnection`: een verbinding met de database, die de repository in haar constructor krijgt. Dapper voegt zijn methodes toe aan die verbinding.

De drie aanhalingstekens openen een *raw string*: een tekst, op één regel of over meerdere regels, waarin gewone aanhalingstekens gewoon mogen staan. Dat komt van pas, want

PostgreSQL wil de namen van tabellen en kolommen die EF Core maakte tussen aanhalingstekens. Zie de waarschuwing verderop.

Parameters: altijd met @

@Id in de SQL is een plaatshouder. De waarde komt uit het **anonieme object** `new { Id = id }` uit [hoofdstuk 13](#): de naam na @ moet de naam van een eigenschap in dat object zijn. Dapper geeft de waarde apart door aan de database, nooit als stuk tekst in de SQL.

Plak je een waarde zelf in de SQL, met een \$ en accolades zoals in [hoofdstuk 1](#), dan kan wie die waarde aanlevert, eigen SQL meesturen. Dat heet **SQL-injectie**. Vraag 1 toont hoe weinig daarvoor nodig is.

Een rij in twee objecten: multi-mapping

Met een `JOIN` komen een recept en zijn keuken samen op één rij. Dapper moet weten waar het recept ophoudt en de keuken begint:

```
string sql = """
    SELECT r.*, k.*
    FROM "Recepten" r
    INNER JOIN "Keukens" k ON r."KeukenId" = k."Id"
    WHERE r."Id" = @Id
    """;

var resultaat = await _dbConnection.QueryAsync<Recept, Keuken, Recept>(
    sql,
    (recept, keuken) =>
    {
        recept.Keuken = keuken;
        return recept;
    },
    param: new { Id = id },
    splitOn: "Id");
```

- Tussen de punthaken staan de types per stuk van de rij, en als laatste wat je terugkrijgt.
- `splitOn: "Id"` zegt: zodra je opnieuw een kolom `Id` tegenkomt, begint het volgende object. `Id` is ook wat Dapper aanneemt als je niets opgeeft.
- De lambda koppelt de stukken aan elkaar, zoals `Include` dat bij EF Core voor je doet.

Bij één op veel staat de ouder op elke rij opnieuw, één keer per kind. Les 08 houdt daarom een `Dictionary` bij uit [hoofdstuk 7](#), met het `Id` als sleutel, zodat elke ouder maar één keer in het resultaat komt.

Wat je ermee kan

In de voorbeelden is `db` de verbinding, `sql` de tekst van de query en `param` een anoniem object met de parameters, zoals `new { Id = 5 }`. Voor elke oproep staat nog een `await`.

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>QueryAsync<T></code>	alle rijen, elk als object	<code>db.QueryAsync<Recept>(sql)</code>	een <code>IEnumerable<Recept></code>
<code>QueryFirstOrDefaultAsync<T></code>	de eerste rij, of niets	<code>db.QueryFirstOrDefaultAsync<Recept>(sql, param)</code>	het recept, of <code>null</code>
<code>QueryFirstAsync<T></code>	de eerste rij, die er moet zijn	<code>db.QueryFirstAsync<Recept>(sql, param)</code>	geen rij: <code>InvalidOperationException</code>
<code>QuerySingleOrDefaultAsync<T></code>	precies één rij, of niets	<code>db.QuerySingleOrDefaultAsync<Recept>(sql, param)</code>	meer dan één rij: <code>InvalidOperationException</code>
<code>ExecuteAsync</code>	voer een INSERT, UPDATE of DELETE uit	<code>db.ExecuteAsync(sql, param)</code>	het aantal geraakte rijen
<code>ExecuteScalarAsync<T></code>	voer uit en geef één waarde terug	<code>db.ExecuteScalarAsync<int>(sql)</code>	één getal of tekst
<code>@Naam + new { Naam = ... }</code>	een parameter	<code>WHERE "Titel" = @Titel</code> met <code>new { Titel = titel }</code>	veilig voor SQL-injectie
<code>QueryAsync<A, B, A></code>	één rij verdelen over twee objecten	met een lambda en <code>splitOn</code>	het object A, met B eraan gekoppeld
<code>splitOn</code>	de kolom waar het volgende object begint	<code>splitOn: "Id"</code>	—

Dapper koppelt elke kolom aan de eigenschap met dezelfde naam. Anders dan bij Mapster maken hoofdletters daarbij niet uit: een kolom `titel` vult `Titel`. Vindt hij geen eigenschap, dan slaat hij de kolom over, zonder foutmelding.

Wat het ook kan, via iets anders

`QueryAsync<T>` geeft een `IEnumerable<T>` uit [hoofdstuk 12](#). Dapper leest standaard alle rijen in één keer in, en bewaart ze in een lijst. Daarna is de database niet meer betrokken: alles wat je verder doet, gebeurt in je programma.

Op het resultaat van QueryAsync

Komt van LINQ. Alles van [hoofdstuk 11](#) werkt erop. In de voorbeelden is `recepten` het resultaat van `await db.QueryAsync<Recept>(sql)`.

Methode	Voorbeeld	Resultaat
Count, Any, First, FirstOrDefault	<code>recepten.Count()</code>	het aantal rijen; de SQL loopt niet opnieuw
Where, OrderBy, Select	<code>recepten.Where(r => r.Kostprijs > 10)</code>	gefilterd in je programma, niet in de database
ToList	<code>recepten.ToList()</code>	een <code>List<Recept></code> , met alles van hoofdstuk 6
AsList	<code>recepten.AsList()</code>	ook een <code>List<Recept></code> ; Dapper's eigen versie, die geen kopie maakt als het al een lijst is
ToDictionary	<code>recepten.ToDictionary(r => r.Id)</code>	een <code>Dictionary<int, Recept></code> , zoals bij multi-mapping
<code>foreach</code>	<code>foreach (var r in recepten)</code>	overloopt de rijen

Wat **niet** werkt: `recepten.Add(...)`, `recepten[0]` en `recepten.Count` zonder haakjes. Dat zijn dingen van een lijst, en het type is `IEnumerable<Recept>`. Maak er eerst een lijst van.

Bestaat ook, maar kom je in deze cursus niet tegen

Methode	Wat het doet
QuerySingleAsync	precies één rij, anders een InvalidOperationException
QueryMultipleAsync	meerdere SELECTs in één keer, en de resultaten één voor één lezen
ExecuteReaderAsync	de rijen zelf uitlezen, zonder objecten
<code>buffered: false</code>	rijen één voor één inlezen in plaats van allemaal tegelijk, voor heel grote resultaten
een lijst als parameter	<code>new { Ids = new[] { 1, 2, 3 } }</code> : Dapper vult de waarden zelf in, veilig
<code>ExecuteAsync</code> met een lijst objecten	voert dezelfde SQL uit voor elk object, en telt alle geraakte rijen samen

Let op bij Dapper

Wat	Waarom
<code>Where</code> op het resultaat van <code>QueryAsync</code>	alle rijen kwamen al over; filter liever in de SQL, met <code>WHERE</code>
een kolom die anders heet dan de eigenschap	Dapper slaat ze over, zonder fout; gebruik <code>AS</code> in je SQL
geen change tracker	een object aanpassen doet niets in de database; schrijf zelf een <code>UPDATE</code>

Veelgemaakte fouten

LET OP

- **Een waarde in de SQL plakken met \$ en accolades.** Gebruik altijd @Naam en een anoniem object.
- **Namen zonder aanhalingstekens.** Tabellen die EF Core in PostgreSQL maakte, heten exact Recepten, met hoofdletter. PostgreSQL maakt van elke naam zonder aanhalingstekens kleine letters, en zoekt dan recepten. Die bestaat niet. Schrijf "Recepten" en "KeukenId".
- **SCOPE_IDENTITY() in PostgreSQL.** Het voorbeeld in les 08 haalt het nieuwe Id op met SELECT CAST(SCOPE_IDENTITY() as int). Dat is SQL Server. In PostgreSQL zet je RETURNING "Id" achter de INSERT, en lees je dat getal met ExecuteScalarAsync<int>.
- **Een IDbConnection vragen die niet geregistreerd is.** Les 08 toont niet hoe die in Program.cs komt. Je registreert ze met AddScoped, met een lambda die een NpgsqlConnection maakt met je connection string. Zonder registratie krijg je dezelfde fout als in hoofdstuk 27.

Voorspel de uitkomst

db is een verbinding met een database waarin EF Core de tabel "Recepten" maakte. Die bevat twaalf recepten, en geen enkel recept heeft een vreemde titel.

Vraag 1

```
string titel = "' OR '1'='1";
string sql = $"SELECT * FROM \"Recepten\" WHERE \"Titel\" = '{titel}'";
var recepten = await db.QueryAsync<Recept>(sql);
Console.WriteLine(recepten.Count());
```

[Toon het antwoord ›](#)

Vraag 2

```
int n = await db.ExecuteAsync(
    "DELETE FROM \"Recepten\" WHERE \"Id\" = @Id",
    new { Id = 999 });
Console.WriteLine(n);
```

[Toon het antwoord ›](#)

Vraag 3

```
var r = await db.QueryFirstOrDefaultAsync<Recept>(
    "SELECT \"Id\", \"Titel\" AS naam FROM \"Recepten\" WHERE \"Id\" = 1");
Console.WriteLine($"[" + r!.Titel + "]");
```

[Toon het antwoord ›](#)

Vraag 4

```
var alle = await db.QueryAsync<Recept>("SELECT * FROM Recepten");
```

[Toon het antwoord ›](#)

Vraag 5

```
var r = await db.QueryFirstOrDefaultAsync<Recept>(
    ""SELECT * FROM "Recepten" WHERE "Id" = @Id"",
    new { Id = 999 });
Console.WriteLine(r is null);
```

[Toon het antwoord ›](#)

Vraag 6

```
var recepten = await db.QueryAsync<Recept>("""SELECT * FROM "Recepten" """);
Console.WriteLine(recepten.Count());
Console.WriteLine(recepten.Count(r => r.Kostprijs > 10));
```

Hoeveel keer vraagt deze code iets aan de database?

[Toon het antwoord ›](#)

Vraag 7

```
var recepten = await db.QueryAsync<Recept>("""SELECT * FROM "Recepten" """);
recepten.Add(new Recept { Titel = "Curry" });
```

[Toon het antwoord ›](#)

Antwoorden

VRAAG 1

12

Na het invullen staat er `WHERE "Titel" = '' OR '1'='1'`. Het tweede deel is altijd waar, dus elke rij voldoet. Met `WHERE "Titel" = @Titel` en `new { Titel = titel }` zoekt de database letterlijk naar die vreemde titel, en vindt ze er 0.

[Terug naar de vraag](#)

VRAAG 2

0

Er is geen recept 999, dus er wordt niets verwijderd. Dat is geen fout: de database meldt gewoon dat nul rijen geraakt zijn. In een controller is dat het moment voor een `NotFound`.

[Terug naar de vraag](#)

VRAAG 3

[]

De kolom heet in het antwoord `naam`, en `Recept` heeft geen eigenschap met die naam. Dapper slaat de kolom over. `Titel` krijgt dus niets, en blijft `null`. Het `Id` is wel ingevuld.

[Terug naar de vraag](#)

VRAAG 4

Een uitzondering van PostgreSQL: *relation "recepten" does not exist*. In een API wordt dat een **500**.

Zonder aanhalingstekens maakt PostgreSQL er `recepten` van, met kleine letters. EF Core maakte de tabel als `"Recepten"`. Voor PostgreSQL zijn dat twee verschillende namen.

[Terug naar de vraag](#)

VRAAG 5

True

Geen rij gevonden, dus `null`, net als `FirstOrDefault` in [hoofdstuk 11](#). Met `QueryFirstAsync` was het een `InvalidOperationException` geweest.

[Terug naar de vraag](#)

VRAAG 6

1

Dapper leest alle rijen in bij `QueryAsync`. De twee `Counts` tellen in je programma. Vergelijk met vraag 5 van [hoofdstuk 32](#): daar was het een `IQueryable`, en vertrok de vraag twee keer.

[Terug naar de vraag](#)

VRAAG 7

Compileert niet

recepten is een `IEnumerable<Recept>`, en die heeft geen `Add`. Met `var lijst = recepten.ToList();` kan het wel, maar de curry komt daarmee enkel in je lijst, niet in de database.

[← Terug naar de vraag](#)

Veelgemaakte verwarringen

Tweeënveertig paren die op elkaar lijken en iets anders doen. Dit is het deel waar je het vaakst naar terug zal bladeren.

`Length`, `Count` en `Count()`

Een array en een tekst zeggen `Length`. Een `List` zegt `Count`, zonder haakjes, en weet het meteen. `Count()` met haakjes is LINQ: dat werkt op elke verzameling, maar loopt ze daarvoor helemaal af. Op een lijst neem je de eigenschap. Hoofdstukken [5](#), [6](#) en [11](#).

Array tegenover List

Een array heeft een vaste lengte, een `List` groeit mee. In de praktijk kies je bijna altijd de `List`, tenzij de hoeveelheid echt vastligt. Hoofdstukken [5](#) en [6](#).

List tegenover Dictionary

Zoek je in een `List` telkens met `Find` of `FirstOrDefault` op dezelfde eigenschap — een naam, een code — dan wil je eigenlijk een `Dictionary` met die eigenschap als sleutel. De `List` kijkt elk item na tot hij het vindt, de `Dictionary` weet meteen waar hij moet zijn. Zoek je afwisselend op verschillende eigenschappen, of doet de volgorde ertoe, dan blijft de `List` de betere keuze. Hoofdstukken [6](#) en [7](#).

Queue tegenover Stack

Allebei laten ze je enkel aan één kant bij. Stel jezelf de vraag: *wie komt er als volgende aan de beurt?* Wie het langst wacht — `Queue`. Wie het laatst kwam — `Stack`. Hoofdstukken [9](#) en [10](#).

`Dequeue` tegenover `First()`

`Dequeue` haalt de eerste uit de rij: daarna is hij weg. `First()` van LINQ kijkt enkel, net als `Peek`. Op een lege rij crashen ze alle drie. Bij een `Stack` geldt hetzelfde voor `Pop` tegenover `First()`. Hoofdstukken [9](#) en [10](#).

`UnionWith` tegenover `Union`

`a.UnionWith(b)` verandert de set `a` zelf en geeft niets terug. `a.Union(b)` komt van LINQ: `a` blijft zoals hij was, en je krijgt een nieuwe reeks zonder dubbels. Hetzelfde geldt voor `IntersectWith` tegenover `Intersect`, en `ExceptWith` tegenover `Except`. Hoofdstukken [8](#) en [11](#).

`SetEquals` tegenover `SequenceEqual`

`SetEquals` vraagt of dezelfde dingen erin zitten; volgorde en dubbels tellen niet. `SequenceEqual` vraagt of alles in dezelfde volgorde staat, met even veel items. Hoofdstukken [8](#) en [11](#).

`Remove`, `RemoveAt` en `RemoveAll`

`Remove` wil het item, `RemoveAt` wil een positie, `RemoveAll` wil een voorwaarde. Ze geven ook iets anders terug: `Remove` een ja of nee, `RemoveAll` het aantal dat weg is, `RemoveAt` niets. En `Remove` haalt enkel het eerste voorkomen weg. Hoofdstuk 6.

`Reverse` op een lijst, een array en in LINQ

`lijst.Reverse()` draait de lijst zelf om en geeft niets terug. Op een array geeft `getallen.Reverse()` in .NET 10 een nieuwe, omgekeerde reeks, en blijft de array zelf zoals hij was; `Array.Reverse(getallen)` draait de array zelf om. Wil je op een lijst de versie van LINQ, schrijf dan `lijst.AsEnumerable().Reverse()`. Hoofdstukken 5, 6 en 11.

`Exists` tegenover `Any`

Hetzelfde antwoord: is er minstens één die voldoet. `Exists` is een methode van `List` en van `Array`, `Any` komt van LINQ en werkt op elke verzameling, ook op een `Dictionary`, een `HashSet` en een vraag aan de database. Hoofdstukken 6 en 11.

`FindAll` tegenover `Where`

`FindAll` is een methode van `List` en geeft meteen een nieuwe lijst. `Where` komt van LINQ en geeft een vraag die pas rekent als je ze overloopt (hoofdstuk 12). Wil je een lijst, schrijf dan `Where(...).ToList()`. Hoofdstukken 6 en 11.

`Parse` tegenover `TryParse`

`Parse` gelooft je en crasht als je ongelijk hebt. `TryParse` vraagt het na en geeft `false`. Alles wat van buiten komt, gaat door `TryParse`. Hoofdstuk 2.

`TryParse` tegenover `try/catch`

Allebei voorkomen ze dat je programma stopt op een tekst die geen getal is. `TryParse` kijkt vooraf en geeft `false`; `try/catch` laat de fout gebeuren en vangt ze daarna op. Kan je vooraf kijken, doe dat dan: het is sneller en zegt duidelijker wat je bedoelt. Houd `try/catch` voor wat je vooraf niet kan nakijken, zoals een bestand. Hoofdstukken 2 en 23.

`=` tegenover `==`

`=` geeft een variabele een waarde, `==` vraagt of twee waarden gelijk zijn. In een `if` hoort altijd `==`. C# weigert `if (x = 5)`, want een toewijzing is geen ja of nee. Hoofdstuk 4.

`decimal` **tegenover** `double`

Allebei zijn het kommagetallen. Een `double` is snel, maar wijkt soms een heel klein beetje af. Een `decimal` rekent tot op de cent juist. Voor geld en prijzen neem je `decimal`, en dan schrijf je een `m` achter elk getal: `19.99m`. Hoofdstuk 2.

`(int)` **tegenover** `Convert.ToInt32`

`(int)2.9` kapt af: `2`. `Convert.ToInt32(2.9)` rondt af: `3`. En `Convert.ToInt32(2.5)` geeft `2`, want bij een halve rondt C# af naar het even getal. Hoofdstuk 2.

`Hours` **tegenover** `TotalHours`

`Hours` is enkel het uren-deel van een duur, van 0 tot 23. `TotalHours` is de hele duur in uren, met komma. Bij 14 dagen en 6 uur is `Hours` `6` en `TotalHours` `342`. Hoofdstuk 3.

`==` **tegenover** `Equals` **bij tekst**

Allebei vergelijken ze letterlijk, hoofdletters inbegrepen. Het verschil is dat `Equals` een tweede argument aanvaardt waarmee je die regel kan versoepelen. Zodra hoofdletters er niet toe mogen doen, heb je `Equals` nodig. Hoofdstuk 1.

`string.Join` **op een lijst tegenover op één tekst**

`string.Join("-", fruit)` zet een streepje tussen de items van de lijst. `string.Join("-", "kiwi")` geeft gewoon `"kiwi"`: één tekst is één item. Wil je tussen de tekens, geef dan de tekens mee: `string.Join("-", "kiwi".ToCharArray())`. Hoofdstuk 1.

`Find` **tegenover** `FirstOrDefault`

Op een lijst doen ze hetzelfde. `Find` bestaat enkel op een `List` en een array; `FirstOrDefault` werkt op elke verzameling, ook op een databasequery. Omdat je later met databases werkt en `Find` daar niet bestaat, went `FirstOrDefault` beter aan. Hoofdstukken 6 en 11.

`First` **tegenover** `Single`

`First` zegt: er is er minstens één, geef mij de eerste. `Single` zegt: er is er precies één, en als dat niet klopt is er iets grondig mis. Gebruik `Single` op een `Id`, en `First` op een sortering. Hoofdstuk 11.

`Where` **tegenover** `Select`

`Where` houdt items over, `Select` verbouwt ze. Krijg je minder items terug dan je erin stak, dan heb je gefilterd — dat is `Where`. Krijg je er evenveel terug maar van een ander soort, bijvoorbeeld enkel de namen, dan heb je verbouwd — dat is `Select`. Hoofdstuk 11.

Max **tegenover** MaxBy

Max geeft de waarde, MaxBy geeft het item dat die waarde draagt. Dezelfde verhouding geldt voor Min en MinBy. Hoofdstuk 11.

Any() **tegenover** Count() > 0

Hetzelfde antwoord, maar Any stopt bij het eerste item dat voldoet. Op een lijst van vijf maakt dat niets uit; op een databasetabel van vijftigduizend wel. Hoofdstuk 11.

Wat een methode teruggeeft, en wat ze verandert

lijst.Sort() verandert je lijst en geeft niets terug. lijst.OrderBy(...) laat je lijst met rust en geeft een nieuw resultaat. Hetzelfde geldt voor Trim en ToUpper bij tekst: die veranderen niets, ze geven een nieuwe tekst terug. Compileert je regel niet, of lijkt hij niets te doen, dan heb je meestal de verkeerde van de twee te pakken. Hoofdstukken 1, 6 en 11.

IEnumerable **tegenover** List

Een IEnumerable is een vraag die nog gesteld moet worden. Een List is een antwoord dat al klaarligt. Where voert niets uit tot je het resultaat doorloopt; verandert je bron intussen, dan verandert je antwoord mee. ToList() stelt de vraag meteen en legt het antwoord vast. Hoofdstuk 12.

Contains bij een class tegenover een record

Bij een class zoekt Contains **hetzelfde object**: een tweede object met dezelfde inhoud vindt hij niet. Bij een record zoekt hij **dezelfde inhoud**. Hetzelfde verschil zit in Distinct, HashSet, een sleutel van een Dictionary en ==. Hoofdstukken 13 en 14.

OfType **tegenover** Cast

OfType<Kat>() houdt enkel de katten over en slaat de rest over. Cast<Kat>() probeert alles om te zetten, en crasht met een InvalidCastException zodra er iets anders tussen zit. Hoofdstuk 22.

Abstracte klasse tegenover interface

Een abstracte klasse zegt wat iets **is**, en geeft gegevens en uitgewerkte methodes mee. Een interface zegt wat iets **kan**, en geeft niets mee. Een klasse erft van hoogstens één abstracte klasse, maar kan zoveel interfaces beloven als ze wil. Delen je klassen gegevens, dan is het een abstracte klasse; delen ze enkel een vaardigheid, dan een interface. Hoofdstukken 19 en 20.

`private tegenover protected`

Allebei verbergen ze iets voor de buitenwereld. Het verschil zit bij wie erft: `private` is enkel zichtbaar in de klasse zelf, `protected` ook in elke klasse die ervan erft. Kies `private`, tenzij een afgeleide klasse het echt nodig heeft. Hoofdstuk 20.

`static tegenover een gewoon veld`

Een gewoon veld heeft elk object apart. Een `static` veld is er één keer, voor de hele klasse. Daarom onthoudt een `static` lijst in een controller wat er toegevoegd werd, terwijl een gewone lijst bij elk verzoek opnieuw leeg begint. Hoofdstuk 24.

`const tegenover readonly`

Allebei veranderen ze niet meer. Een `const` ligt al vast bij het compileren en kan enkel een getal, een tekst of een `bool` zijn. Een `readonly` veld vul je één keer in, bij de declaratie of in de constructor. Het kan dus ook iets zijn dat pas bekend is als het programma draait, zoals een repository. Hoofdstuk 24.

`readonly tegenover AsReadOnly`

`readonly` voor een veld beschermt het veld: je kan er geen andere lijst in zetten. Aan de lijst zelf kan je nog altijd toevoegen. `AsReadOnly()` geeft een kijk op de lijst zonder `Add` en `Remove`. Hoofdstukken 6 en 24.

`{id} tegenover {id:int}`

Allebei halen ze een waarde uit de route. Zonder beperking past de route bij elke tekst, en geeft een tekst die geen getal is een 400. Met `:int` past de route enkel bij een geheel getal, en geeft al de rest een 404. Hoofdstuk 25.

`ActionResult<T> tegenover IActionResult`

`ActionResult<T>` zegt wat er terugkomt, en laat je dat ook rechtstreeks teruggeven. `IActionResult` kent enkel antwoorden, dus daar pak je je gegevens zelf in met `Ok(...)`. Geeft een methode gegevens terug, neem dan `ActionResult<T>`. Geeft ze enkel een statuscode terug, dan volstaat `IActionResult`. Hoofdstuk 26.

`AddScoped tegenover AddSingleton`

`AddScoped` maakt één object per verzoek, `AddSingleton` één voor de hele app. Bewaart een dienst zelf gegevens in het geheugen, dan is een scoped dienst ze na elk verzoek kwijt, terwijl een singleton ze onthoudt. `AddTransient` maakt elke keer een nieuw object. Hoofdstuk 27.

`await tegenover .Result`

Allebei geven ze je wat er in een `Task` zit. `await` laat de thread intussen vrij voor ander werk. `.Result` houdt hem vast tot het antwoord er is, en kan een programma zelfs laten vastlopen. In een API gebruik je altijd `await`. Hoofdstuk 29.

`IEnumerable tegenover IQueryable`

Allebei zijn het vragen die nog niet gesteld zijn. Een `IEnumerable` voert je programma zelf uit, op wat al in het geheugen zit. Een `IQueryable` wordt naar SQL vertaald, en de database voert hem uit. Filter je pas na `ToListAsync`, dan komt eerst alles uit de database, en filtert je programma daarna. Hoofdstukken 12 en 32.

`Add tegenover SaveChangesAsync`

`Add` legt iets in het mandje, `SaveChangesAsync` rekent af. Na `Add` staat er nog niets in de database, en heeft het object nog geen `Id`. Hetzelfde geldt voor `Update` en `Remove`. Hoofdstuk 32.

`Include tegenover ThenInclude`

`Include` vertrekt altijd van het object dat je opvraagt. `ThenInclude` gaat één stap verder vanaf de vorige `Include`. De keuken van een recept is een `Include`. De ingrediënten achter de koppelobjecten van een recept zijn een `Include`, gevolgd door een `ThenInclude`. Hoofdstuk 32.

`Cascade tegenover Restrict`

Allebei zeggen ze wat er gebeurt als je iets verwijdert waar nog andere rijen aan hangen. `Cascade` neemt die rijen mee, zonder iets te vragen. `Restrict` weigert, en laat jou eerst opruimen. Bij een verplichte relatie is `Cascade` de standaard. Hoofdstuk 31.

`-1, null en 0`

`IndexOf` geeft `-1` wanneer er niets gevonden is. `FirstOrDefault` geeft `null` bij objecten, maar `0` bij getallen. `TryGetValue` zet zijn uitkomst op de standaardwaarde. Drie manieren om "niets" te zeggen, en elk vraagt een andere controle. Hoofdstukken 1, 7, 11 en 17.

Index

Alfabetisch. Het **vette** nummer is het hoofdstuk met de volledige uitleg. De andere nummers zijn hoofdstukken waar je het ook tegenkomt, bijvoorbeeld als LINQ-methode op die verzameling of in een tabel "via iets anders".

Naam	Hoofdstuk
abstract	20
ActionResult<T>	26
Adapt	33
Add	3 , 6 , 7 , 8 , 32
Add-Migration	31
AddAsync	32
AddControllers	27
AddDays	3
AddDbContext	31
AddHours	3
AddMinutes	3
AddMonths	3
AddOpenApi	27
AddRange	6 , 32
AddRangeAsync	32
AddScoped	27
AddSeconds	3
AddSingleton	27
AddTransient	27
AddYears	3
Aggregate	5 , 6 , 7 , 8 , 9 , 10 , 11
AggregateBy	5 , 6 , 7 , 8 , 9 , 10 , 11
All	1 , 5 , 6 , 7 , 8 , 9 , 10 , 11
AllAsync	32
and	4
anoniem object	13
Any	1 , 5 , 6 , 7 , 8 , 9 , 10 , 11 , 34
AnyAsync	32
[ApiController]	25
Append	1 , 5 , 6 , 7 , 8 , 9 , 10 , 11

Naam	Hoofdstuk
AppendJoin	1
AppendLine	1
array	5
Array.AsReadOnly	5
Array.BinarySearch	5
Array.Clear	5
Array.ConvertAll	5
Array.Copy	5
Array.Empty	5
Array.Exists	5
Array.Fill	5
Array.Find	5
Array.FindAll	5
Array.FindIndex	5
Array.FindLast	5
Array.FindLastIndex	5
Array.ForEach	5
Array.IndexOf	5
Array.LastIndexOf	5
Array.Resize	5
Array.Reverse	5
Array.Sort	5
Array.TrueForAll	5
as	22
AsAsyncEnumerable	32
AsList	34
AsNoTracking	32
AsReadOnly	6
AsSplitQuery	32
Async	29
async	29
Attach	32
AttachRange	32
attribuut	25

Naam	Hoofdstuk
Average	2, 5, 6, 7, 8, 9, 10, 11
AverageAsync	32
await	29
BadRequest	26
base	20
basisklasse	20
BinarySearch	6
break	4
Build	27
Capacity	6, 9, 10
Cascade	31
Cast	11, 22
catch	23
change tracker	32
char.GetNumericValue	1
char.IsAsciiDigit	1
char.IsAsciiLetter	1
char.IsDigit	1
char.IsLetter	1
char.IsLetterOrDigit	1
char.IsLower	1
char.IsPunctuation	1
char.IsUpper	1
char.IsWhiteSpace	1
char.ToLower	1
char.ToUpper	1
checked	2
Chunk	1, 5, 6, 7, 8, 9, 10, 11
class	13
Clear	1, 6, 7, 8, 9, 10
Clone	5
Comparer	7, 8
CompareTo	1, 2, 3, 18
Concat	1, 5, 6, 7, 8, 9, 10, 11

Naam	Hoofdstuk
<code>const</code>	24
<code>constructor</code>	13
<code>Contains</code>	1, 3, 5, 6, 7, 8, 9, 10, 11, 13, 14
<code>ContainsAsync</code>	32
<code>ContainsKey</code>	7
<code>ContainsValue</code>	7
<code>continue</code>	4
<code>Convert.ToInt32</code>	2
<code>ConvertAll</code>	6
<code>CopyTo</code>	5, 6, 9, 10
<code>Count</code>	1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 34
<code>CountAsync</code>	32
<code>CountBy</code>	1, 5, 6, 7, 8, 9, 10, 11
<code>CreatedAtAction</code>	26
<code>CultureInfo.InvariantCulture</code>	2
<code>Dapper</code>	34
<code>Date</code>	3
<code>DateOnly</code>	3
<code>DateOnly.FromDateTime</code>	3
<code>DateTime</code>	3
<code>DateTime.DaysInMonth</code>	3
<code>DateTime.IsLeapYear</code>	3
<code>DateTime.MaxValue</code>	3
<code>DateTime.MinValue</code>	3
<code>DateTime.Now</code>	3
<code>DateTime.ParseExact</code>	3
<code>DateTime.Today</code>	3
<code>DateTime.TryParse</code>	3
<code>DateTime.TryParseExact</code>	3
<code>DateTime.UtcNow</code>	3
<code>Day</code>	3
<code>DayNumber</code>	3
<code>DayOfWeek</code>	3
<code>DayOfYear</code>	3

Naam	Hoofdstuk
Days	3
DbContext	31
DbSet	31
decimal	2
decimal.Parse	2
decimal.TryParse	2
Deconstruct	14
default	21
DefaultIfEmpty	11
dependency injection	27
Dequeue	9
Dictionary	7
Distinct	1, 2, 5, 6, 7, 8, 9, 10, 11, 13, 14, 32
DistinctBy	5, 6, 8, 9, 10, 11
do	4
dotnet ef	31
double	2
double.Parse	2
DTO	33
Duration	3
ElementAt	1, 5, 6, 7, 8, 9, 10, 11
ElementAtAsync	32
ElementAtOrDefault	5, 6, 7, 8, 9, 10, 11
ElementAtOrDefaultAsync	32
else	4
EndsWith	1
Enqueue	9
EnsureCapacity	6, 9, 10
Entity<T>	31
EntityState	32
Entry	32
enum	18
Enum.GetNames	18
Enum.GetValues	18

Naam	Hoofdstuk
<code>Enum.IsDefined</code>	18
<code>Enum.Parse</code>	18
<code>Enum.TryParse</code>	18
<code>Enumerable.Empty</code>	11
<code>Enumerable.InfiniteSequence</code>	11
<code>Enumerable.Range</code>	2, 11
<code>Enumerable.Repeat</code>	11
<code>Enumerable.Sequence</code>	11
<code>Equals</code>	1, 2, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 18
<code>Except</code>	1, 5, 6, 7, 8, 9, 10, 11
<code>ExceptBy</code>	5, 6, 7, 8, 11
<code>Exception</code>	23
<code>ExceptWith</code>	8
<code>ExecuteAsync</code>	34
<code>ExecuteDeleteAsync</code>	32
<code>ExecuteReaderAsync</code>	34
<code>ExecuteScalarAsync</code>	34
<code>ExecuteUpdateAsync</code>	32
<code>Exists</code>	6
<code>File.AppendAllText</code>	30
<code>File.Exists</code>	30
<code>File.ReadAllLines</code>	30
<code>File.ReadAllText</code>	30
<code>File.ReadLines</code>	30
<code>File.WriteAllLines</code>	30
<code>File.WriteAllText</code>	30
<code>finally</code>	23
<code>Find</code>	6, 32
<code>FindAll</code>	6
<code>FindAsync</code>	32
<code>FindIndex</code>	6
<code>FindLast</code>	6
<code>FindLastIndex</code>	6
<code>First</code>	1, 5, 6, 7, 8, 9, 10, 11, 34

Naam	Hoofdstuk
FirstAsync	32
FirstOrDefault	1, 5, 6, 7, 8, 9, 10, 11, 34
FirstOrDefaultAsync	32
Fluent API	31
for	4
ForEach	6
foreach	4
ForEachAsync	32
ForType	33
[FromBody]	25
FromDays	3
FromHours	3
[FromQuery]	25
FromSql	32
generics	21
GetConnectionString	27
GetHashCode	1, 5, 6, 7, 8, 9, 10, 13, 14
GetLength	5
GetLowerBound	5
GetRange	6
GetType	1, 2, 3, 5, 6, 7, 8, 9, 10, 12, 13, 15, 18, 20, 22, 23
GetUpperBound	5
GetValueOrDefault	7, 17
GroupBy	1, 3, 5, 6, 7, 8, 9, 10, 11, 32
GroupJoin	5, 6, 7, 8, 9, 11
HasData	31
HasForeignKey	31
HashSet	8
HasKey	31
HasMaxLength	31
HasOne	31
HasPrecision	31
HasValue	17
Hour	3

Naam	Hoofdstuk
Hours	3
[HttpDelete]	25
[HttpGet]	25
[HttpPost]	25
[HttpPut]	25
IActionResult	26
IEnumerable	12
if	4
Ignore	33
IgnoreCycles	32
ILogger	28
Include	32
Index	5, 6, 8, 9, 10, 11
IndexOf	1, 6, 13, 14
IndexOfAny	1
Insert	1, 6
InsertRange	6
int.MaxValue	2
int.MinValue	2
int.Parse	1, 2
int.TryParse	1, 2
interface	19
Intersect	1, 5, 6, 7, 8, 9, 10, 11
IntersectBy	5, 6, 7, 8, 11
IntersectWith	8
IQueryable	32
IRenderer	33
is	20, 22
IsBetween	3
IsDevelopment	27
IsNullOrEmpty	1
IsNullOrWhiteSpace	1
IsProperSubsetOf	8
IsProperSupersetOf	8

Naam	Hoofdstuk
IsRequired	31
IsSubsetOf	8
IsSupersetOf	8
Join	5, 6, 7, 8, 9, 10, 11
JsonSerializer.Deserialize	30
JsonSerializer.Serialize	30
Keys	7
Keys.Contains	7
Keys.ToList	7
KeyValuePair	7
Kind	3
Last	1, 5, 6, 7, 8, 9, 10, 11
LastAsync	32
LastIndexOf	1, 6
LastIndexOfAny	1
LastOrDefault	1, 5, 6, 7, 8, 9, 10, 11
LastOrDefaultAsync	32
lazy loading	32
LeftJoin	5, 6, 7, 8, 9, 11
Length	1, 5
List	6
LoadAsync	32
Local	32
LogCritical	28
LogDebug	28
LogError	28
LogInformation	28
LogTrace	28
LogWarning	28
long	2
LongCount	11
LongCountAsync	32
m	2
Map	33

Naam	Hoofdstuk
MapControllers	27
MapOpenApi	27
MapScalarApiReference	27
Mapster	33
Math.Abs	2
Math.Ceiling	2
Math.Clamp	2
Math.Floor	2
Math.Max	2
Math.Min	2
Math.Pow	2
Math.Round	2
Math.Sign	2
Math.Sqrt	2
Math.Truncate	2
Max	1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 13
MaxAsync	32
MaxBy	3, 5, 6, 7, 8, 9, 10, 11
Message	23
micro-ORM	34
MidpointRounding	2
migratie	31
Millisecond	3
Min	1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 13
MinAsync	32
MinBy	5, 6, 7, 8, 9, 10, 11
Minute	3
Minutes	3
Month	3
N+1-probleem	32
nameof	22
namespace	13
navigatie-eigenschap	31
new()	21

Naam	Hoofdstuk
NewConfig	33
NoContent	26
NotFound	26
OfType	5, 6, 8, 9, 10, 11, 20, 22
Ok	26
OnDelete	31
OnModelCreating	31
or	4
Order	1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 13, 14, 18, 32
OrderBy	3, 5, 6, 7, 8, 9, 10, 11, 32, 34
OrderByDescending	5, 6, 7, 8, 9, 11, 32
OrderDescending	1, 2, 3, 5, 6, 7, 8, 9, 11, 32
overerving	20
Overlaps	8
override	20
PadLeft	1
PadRight	1
Parse	2
ParseExact	3
Path.Combine	30
Peek	9, 10
platslaan	33
Pop	10
Prepend	1, 5, 6, 7, 8, 9, 10, 11
primaire sleutel	31
Property	31
protected	20
Push	10
QueryAsync	34
QueryFirstAsync<T>	34
QueryFirstOrDefaultAsync	34
QueryMultipleAsync	34
QuerySingleAsync	34
QuerySingleOrDefaultAsync<T>	34

Naam	Hoofdstuk
Queue	9
Random.Shared.Next	2
Random.Shared.NextDouble	2
Rank	5
readonly	24
record	14
ReferenceEquals	13
referentietype	16
Register	33
Remove	1, 6, 7, 8, 13, 14, 32
Remove-Migration	31
RemoveAll	6
RemoveAt	6
RemoveRange	6, 32
RemoveWhere	8
Replace	1
Restrict	31
Reverse	1, 5, 6, 7, 8, 9, 10, 11
RightJoin	5, 6, 7, 8, 9, 11
[Route]	25
routeparameter	25
Run	27
SaveChangesAsync	32
Scan	33
sealed	20
Second	3
Select	1, 5, 6, 7, 8, 9, 10, 11, 32, 34
SelectMany	5, 6, 7, 8, 9, 10, 11
SequenceEqual	1, 5, 6, 8, 9, 10, 11
Set<T>	32
SetEquals	8
Shuffle	5, 6, 7, 8, 9, 10, 11
Single	1, 5, 6, 7, 8, 9, 10, 11
SingleAsync	32

Naam	Hoofdstuk
SingleOrDefault	1, 5, 6, 7, 8, 9, 10, 11
SingleOrDefaultAsync	32
Skip	1, 5, 6, 7, 8, 9, 10, 11, 32
SkipLast	1, 5, 6, 8, 9, 10, 11
SkipWhile	1, 5, 6, 7, 8, 9, 10, 11
Slice	6
Sort	6, 13, 14
Split	1
splitOn	34
SQL-injectie	34
SqlQuery	32
Stack	10
StartsWith	1
static	24
StatusCode	26
string.Compare	1
string.Concat	1
string.Empty	1
string.Equals	1
string.Format	1
string.IsNullOrEmpty	1
string.IsNullOrEmptyOrWhiteSpace	1
string.Join	1
struct	15
Substring	1
Subtract	3
Sum	1, 2, 5, 6, 7, 8, 9, 10, 11
SumAsync	32
switch	4
SymmetricExceptWith	8
Take	1, 5, 6, 7, 8, 9, 10, 11, 32
TakeLast	1, 5, 6, 8, 9, 10, 11
TakeWhile	1, 5, 6, 7, 8, 9, 10, 11
Task	29

Naam	Hoofdstuk
<code>Task.Delay</code>	29
<code>ThenBy</code>	5, 6, 7, 8, 9, 11, 32
<code>ThenByDescending</code>	5, 6, 7, 8, 9, 11, 32
<code>ThenInclude</code>	32
<code>throw</code>	23
<code>TimeOfDay</code>	3
<code>TimeOnly</code>	3
<code>TimeOnly.AddMinutes</code>	3
<code>TimeSpan</code>	3
<code>TimeSpan.FromMinutes</code>	3
<code>TimeSpan.Zero</code>	3
<code>ToArray</code>	5, 6, 7, 8, 9, 10, 11
<code>ToArrayAsync</code>	32
<code>DateTime</code>	3
<code>ToDictionary</code>	5, 6, 7, 8, 9, 10, 11, 34
<code>ToDictionaryAsync</code>	32
<code>ToHashSet</code>	5, 6, 8, 9, 10, 11
<code>ToHashSetAsync</code>	32
<code>ToList</code>	5, 6, 7, 8, 9, 10, 11, 34
<code>ToListAsync</code>	32
<code>ToLookup</code>	5, 6, 7, 8, 9, 10, 11
<code>ToLower</code>	1
<code>ToLowerInvariant</code>	1
<code>ToQueryString</code>	32
<code>ToShortDateString</code>	3
<code>ToString</code>	1, 2, 3, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 18
<code>Table</code>	31
<code>TotalDays</code>	3
<code>TotalHours</code>	3
<code>TotalMinutes</code>	3
<code>ToUpper</code>	1
<code>ToUpperInvariant</code>	1
<code>Trim</code>	1
<code>TrimEnd</code>	1

Naam	Hoofdstuk
<code>TrimExcess</code>	6, 9, 10
<code>TrimStart</code>	1
<code>TrueForAll</code>	6
<code>try</code>	23
<code>TryAdd</code>	7
<code>TryDequeue</code>	9
<code>TryGetValue</code>	7, 8
<code>TryPeek</code>	9, 10
<code>TryPop</code>	10
<code>typeof</code>	22
uitzondering	23
<code>Union</code>	1, 5, 6, 7, 8, 9, 10, 11
<code>UnionBy</code>	5, 6, 7, 8, 11
<code>UnionWith</code>	8
Unit of Work	32
<code>Update</code>	32
<code>Update-Database</code>	31
<code>UpdateRange</code>	32
<code>UseAuthorization</code>	27
<code>UseHttpsRedirection</code>	27
<code>UseNpgsql</code>	31
<code>using</code>	13
<code>Values</code>	7
<code>Values.Contains</code>	7
<code>Values.ToHashSet</code>	7
<code>Values.ToList</code>	7
<code>virtual</code>	20
vreemde sleutel	31
waardetype	16
<code>WebApplication.CreateBuilder</code>	27
<code>Where</code>	1, 3, 5, 6, 7, 8, 9, 10, 11, 32, 34
<code>where</code>	21
<code>while</code>	4
<code>with</code>	14

Naam	Hoofdstuk
WithMany	31
WithOne	31
Year	3
Zip	1, 5, 6, 7, 8, 9, 10, 11
! en = default!	17
""	34
%	2
&& en	4
.Result	29
=>	13
? :	4
?.	17
?? en ??=	17